

BITÁCORAS ARQUITECTURA DE COMPUTADORES

2020-2021

2º curso – 2º cuatrimestre

Os seguintes apuntamentos foron recollidos en forma de bitácoras
durante cada clase do curso polas seguintes persoas:

Adriana Aurora Rodríguez Oreiro, Adrián Vidal Lorenzo, Andrea Solla Alfonsín, Andrés Balea Domínguez, Belén Valle Cao, Diana Mascareñas Sande, Elena Segade Martínez, Hugo Vázquez Docampo, Inés Quintana Raña, Javier Beiro Piñón, Manuel Cid Domínguez, Martín Campos Zamora, Miguel Méndez Martínez, Nerea Freiría Alonso, Nicolás Vilela Pérez, Pablo Gil Pérez, Pablo Martínez González, Roberto de la Iglesia, Roque Fernández Iglesias, Teresa Gutiérrez Blanco



Non son apuntamentos recollidos de forma 100% formal e pode haber erros nalgúns datos ou fallos de formato. A pesar disto, pensamos que poden ser de gran axuda para os vindeiros cursos :)



Escola
Técnica
Superior
de Enxeñaría



Grao en Enxeñaría informática

2º curso – 2º cuadrimestre

Arquitectura de Computadores

11-02-2021



Clase 1

Temas traballados: Presentación e tema 1

Índice

1. Presentación	3
1.1 Preguntas respostadas.....	3
1.2 ¿Que é a Arquitectura de Computadores?	3
1.3 Obxectivos.....	3
1.4 Contidos	3
1.5 Sesións interactivas	4
1.6 Avaliación da materia.....	4
1.7 Aviso	4
1.8 Bibliografía	4
1.9 Profesores.....	5
2 Tema 1: Fundamentos do deseño de computadores: sistemas multinúcleo.	5
2.1 Parámetros do deseño de un sistema	5
2.2 Clases de computadores	5
2.3 Definindo a arquitectura dun computador.....	6
2.4 Tipos de paralelismo	6
2.5 Clasificación das arquitecturas paralelas.....	7
2.6 Taxonomía de Flynn.....	7
2.7 Arquitecturas MIMD.....	8
2.8 Multiprocesadores	9
2.9 Multicomputadores.....	10
2.10 Sistemas MIMD: segundo latencias de acceso a memoria.....	10
2.11 Volvendo ó procesador.....	11

1. Presentación

1.1 Preguntas respostadas

Vaise ter en conta a investigación que cada persoa faga de forma autodidacta sobre as prácticas? Si, podemos investigar pola nosa conta e desta forma vai ter máis valor o noso traballo, estas prácticas non son tan acotadas como noutras materias como por exemplo FundComp.

As clases prácticas van ser presenciais? Vaise comunicar a semana que ven se as practicas son telemáticas ou presenciais, de todas formas estas comezarían a selo a partir do día 22.

Como debe ser o informe? O informe debe ser claro e conciso, explicando cada paso realizado e o seu motivo.

1.2 ¿Que é a Arquitectura de Computadores?

“A arquitectura de computadores é a ciencia e a arte de seleccionar e interconectar compoñentes hardware para crear un computador que se adapte a uns obxectivos funcionais, de rendemento e de coste”.

1.3 Obxectivos

- Dar a coñecer os principios básicos relacionados coa **arquitectura dos microprocesadores actuais** e mostrar as tendencias de evolución.
- Analizar e comprender a **relación** entre os diferentes elementos da **arquitectura do procesador** e o **rendemento** que é posible conseguir mediante unha programación axeitada.

1.4 Contidos

- **Tema 1: Fundamentos do deseño de computadores: sistemas multinúcleo.**
Faremos unha primeira aproximación ós microprocesadores multinúcleo e ó concepto de paralelismo. Avaliarase o rendemento.
- **Tema 2: Paralelismo a nivel de instrución. Segmentación.**
Conceptos básicos para entender a técnica básica para explotar o paralelismo a nivel de instrución: a segmentación ou *pipelining*.
- **Tema 3: Paralelismo a nivel de instrución. Conceptos avanzados.**
Conceptos máis avanzados, próximos ós incorporados por procesadores actuais.
- **Tema 4: Subsistema de memoria compartida en procesadores multinúcleo.**
Mecanismos de coherencia.

1.5 Sesións interactivas

- **Práctica 1 (4 ou 5 sesións):** Estudo do efecto da xerarquía de memoria e da localidade nos accesos sobre as prestacións dun programa.
- **Prácticas 2 (5 sesións):** Mellora de prestacións en programación multinúcleo e usando extensións SIMD.
- **Resolución de problemas (1 sesión):** Exercicios de tipo dos do exame que se resolverán durante as sesións.

Durante as sesións de prácticas vanse poder lanzar problemas que nos darán puntuación a maiores da nota de prácticas. A asistencia a prácticas non é obrigatoria pero si recomendable. Se asistimos a clase de prácticas, non deberemos saír das sesións. As prácticas están menos conectadas coa teoría que en FundComp.

1.6 Avaliación da materia

- **Exame** sobre os contados das clases expositivas (**5 puntos da nota final**)
- **Prácticas (5 puntos da nota final):**
 - Práctica 1: 2.2 puntos
 - Práctica 2: 2.8 puntos
- Puntos extra por actividades relacionadas con resolución de exercicios.
- Para poder superar a materia precisase obter como **mínimo un 3 sobre 10 nas prácticas e un 3 sobre 10 no exame.**
- Para superar a materia debe alcanzarse unha **puntuación total de 5 puntos ou máis.**
- Se en edición anteriores xa se superou a parte práctica, non será necesario repetilas este curso.
- **Non hai exame de prácticas en Xullo. Nesta convocatoria** poderemos volver entregar a práctica 1 ou 2 no caso de que en algunha delas se obtivese unha puntuación menor de 3 na convocatoria ordinaria.

1.7 Aviso

Espérase que tomemos apuntes. **Non hai uns apuntes da materia que cubran todo xa que se cambiaron contidos con respecto o ano pasado** e imos dar cousas a maiores das diapositivas que utilizamos na clase.

1.8 Bibliografía

[Link ós libros da materia](#)

1.9 Profesores

Dora Blanco Heras: dora.blanco@usc.es

Pablo Quesada: pablo.quesada@usc.es

César Piñeiro: cesaralfredo.pineiro@usc.es

2 Tema 1: Fundamentos do deseño de computadores: sistemas multinúcleo.

2.1 Parámetros do deseño de un sistema

- Rendemento
- Custo
- Potencia (estática + dinámica)
 - Potencia pico
 - Potencia media
- Robustez
 - Tolerancia ao ruído
 - Resistencia a la radiación
- Testabilidade
- Reconfigurabilidade: Se un nodo do sistema se estropea, este debe ser capaz de adaptarse a nova situación.
- Tempo de saída ao mercado, etc.

2.2 Clases de computadores

- **Personal Mobile Device (PMD)**
 - Exemplo: Tablet
 - Faise énfase na eficiencia enerxética debido ás limitacións da disipación de calor e á duración autónoma. Tamén se fai énfase en aumentar a capacidade de resposta rápida (tempo-real)
- **Computación de escritorio**
 - Faise énfase no custo-rendemento
- **Servidores**
 - Faise énfase na dispoñibilidade, escalabilidade e produtividade.
- **Clusters/Warehouse Scale Computers**
 - Usados para “Software as a service” (SaaS)
 - Énfase na dispoñibilidade e coste-rendemento.
 - Sub-clase: Supercomputadores. Énfase en rendemento en punto flotante e redes internas rápidas. A eficiencia enerxética é algo secundario neste tipo de sistemas. Exemplo: CESGA.

2. Tema 1: Fundamentos do deseño de computadores: sistemas multinúcleo.

- Internet das cousas/Computadores embebidos
 - Faise énfase no prezo

Característica	PMD	Sobremesa	Servidor	Clusters/WSC	Embebidos
Prezo do sistema	\$100-\$1000	\$300-\$2500	\$5000-\$10000000	\$100000-\$200000000	\$10-\$100000
Prezo do procesador	\$10-\$100	\$50-\$500	\$200-\$2000	\$50-\$250	\$0.01-\$100
Problemas críticos de deseño	Coste, eficiencia enerxética, rendemento medio, capacidade de resposta	Prezo-Rendemento, eficiencia enerxética, rendemento gráfico	Rendemento, dispoñibilidade, escalabilidade, enerxía	Prezo-rendemento, rendemento bruto, proporcionalidade enerxética	Prezo, eficiencia enerxética, Rendemento específico da súa aplicación

2.3 Definindo a arquitectura dun computador

- Visión de arquitectura dun computador “vella”:
 - Deseño da arquitectura do conxunto de instrucións (ISA).
 - Toma de decisións:
 - Rexistros, direccións de memoria, instrucións dispoñíbeis, etc.
- Arquitectura de computadores “real”:
 - Requisitos específicos da máquina
 - Restricións de deseño: custo, potencia e dispoñibilidade.
 - Inclúe ISA, microarquitectura, hardware

Hai novas restricións á hora de crear ordenadores, xa que depende do fin e do contexto no que se van usar.

2.4 Tipos de paralelismo

Explotar paralelismo: Capacidade dun ordenador para realizar varias tarefas o mesmo tempo.

Instrucións vectoriais: Consiste en agrupar as variables en vectores cando temos unha gran cantidade de datos e deste xeito facemos grandes cálculos cun número reducido de instrucións.

A explotación do paralelismo esta relacionada coa mellora do rendemento.

- **Tipos de paralelismo nas aplicacións**
 - Paralelismo de nivel datos (DLP)
 - Paralelismo de nivel tarefas (TLP)
- **Tipos de paralelismo nas arquitecturas**
 - Paralelismo de nivel instrución (ILP): Varias instrucións na CPU executándose o mesmo tempo en distintos puntos da execución, por exemplo unha instrución decodificándose mentres outra realiza cálculos. Dise que se está esgotando porque a nivel tecnolóxico non hai moitas melloras posibles e débese experimentar con novos métodos para mellorar o rendemento.

2. Tema 1: Fundamentos do deseño de computadores: sistemas multinúcleo.

- Arquitectura de Vectores / Unidades de procesamiento gráfico (GPUs): Explora o paralelismo a nivel datos.
- Paralelismo de nivel subprocesos (TLP): Usado por software como OpenMP, pode explotar tanto o paralelismo a nivel datos ou tarefas.
- Paralelismo de nivel petición: Trátase de tarefas de gran tamaño que se atopan desacopladas.

O paralelismo require reestructurar as aplicacións. Polo cal, o código debe expolo de maneira explícita.

2.5 Clasificación das arquitecturas paralelas

Diferentes criterios de clasificación:

- Taxonomía de Flynn.
- Organización do Sistema de memoria.
- Escalabilidade.
- Disponibilidade comercial das súas compoñentes.
- Cociente rendemento/custo.
- ...

2.6 Taxonomía de Flynn

Clasificación de arquitecturas paralelas baseadas en fluxos de datos e instrucións:

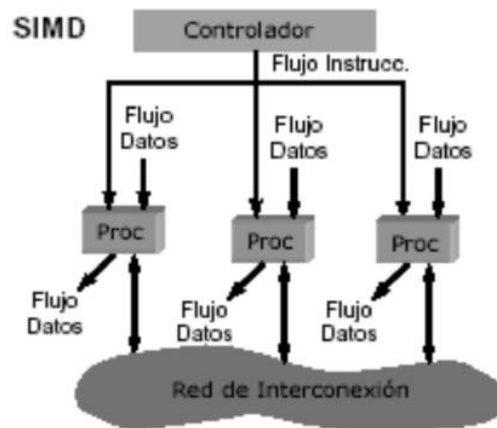
➤ Instrución simple, fluxo de datos simple (SISD):

- Computador secuencial que pode explotar paralelismo a nivel de instrución. Son computadores mononúcleos. Exemplo: Intel Pentium 4.



➤ Instrución simple, fluxo de datos múltiple (SIMD):

- Arquitecturas vectoriais.
- Extensión multimedia como as instrución SSE ou AVX.
- Unidades de procesamiento gráfico (GPUs).



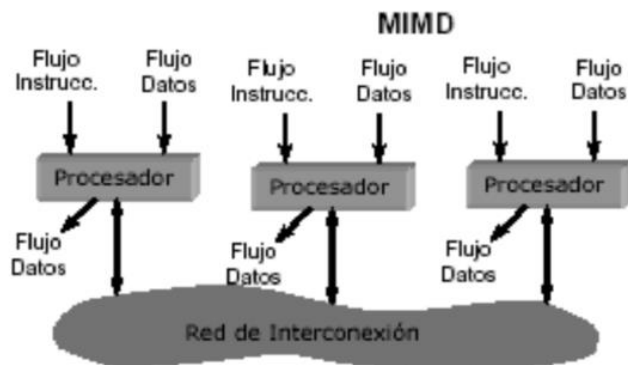
➤ **Múltiples instrucciones, flujo de datos simple (MISD):**

- Non hai implementacións comerciais.



➤ **Múltiples instrucciones, múltiples datos:**

- Cada procesador ten as súas propias instrucións e opera sobre os seus propios datos. Os clusters entran dentro desta categoría. Exemplo Intel Xeon e5345.
- Son máis flexibles que os SIMD pero máis caros.
- Sistemas multicore.



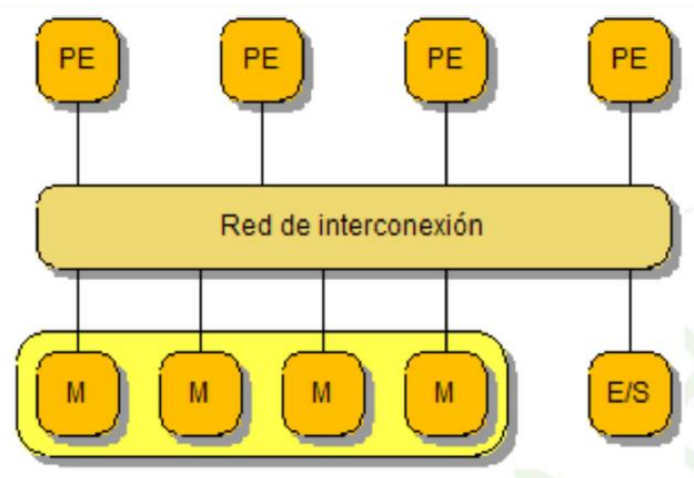
- O exemplo máis claro pode ser o uso dun clúster por parte de dous usuarios, onde cada un envía un programa para a súa execución. Ditos programas serán atendidos por cadanseu procesador propio e enviados a unha rede de interconexión.

2.7 Arquitecturas MIMD

Son as de uso máis extendido:

- **Procesadores de memoria compartida**
 - Todos os procesadores comparten o mesmo espazo de direccións
 - O programador non necesita coñecer a ubicación dos datos
- **Multiprocesadores de memoria distribuída (multicomputadores)**
 - Cada procesador ten o seu propio espazo de direccións
 - O programador necesita saber onde están situados os datos
- **Multicores: multiprocesadores nun único chip**
- **Outros subtipos como clústers, MPP's...**

2.8 Multiprocesadores

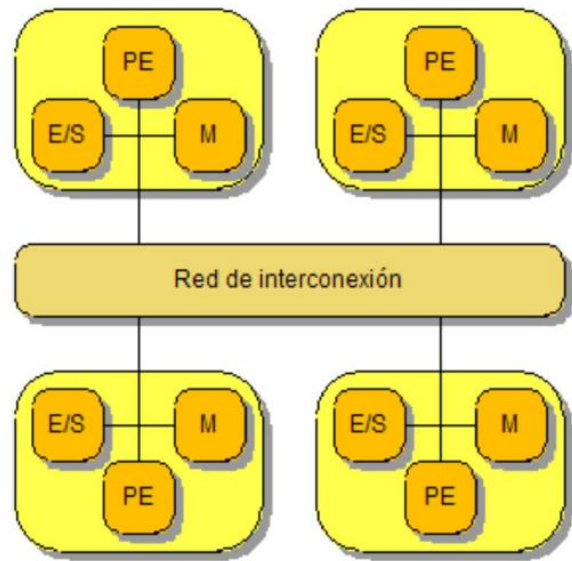


PE: Procesador

M: Memoria

- Os procesadores e a memoria atópanse separados por unha rede de interconexión.
- Cada procesador pode ter niveles propios de caché pero comparten a mesma memoria RAM.
- A entrada/saída (E/S) está centralizada.
- A rede de interconexión pode ser: barras cruzadas, multietapa...
- As comunicacións realízanse a través de escrituras/lecturas en memoria.
- As memorias atópanse nunha zona amarela para recalcar que estas son xestionadas conxuntamente.
- Un sistema é escalable se ao engadirille novos recursos o seu rendemento mellora de maneira proporcional.
- Os procesadores teñen unha baixa escalabilidade.
- Posibles solucións a esta crítica ós multiprocesadores poden ser:
 - Engadir caché os procesadores.
 - Usar redes de baixa latencia e un alto ancho de banda
 - ...

2.9 Multicomputadores



PE: Procesador

M: Memoria

- Cada procesador ten o seu propio banco de memoria e accede ó seu propio espazo de direccións.
- Os datos compartidos transfírense a través dunha rede, que pode ser de diversos tipos.
- Para executar unha aplicación:
 - Con comunicacións -> transferencia de datos
 - Requírense sincronizacións usualmente realizadas xunto as comunicacións

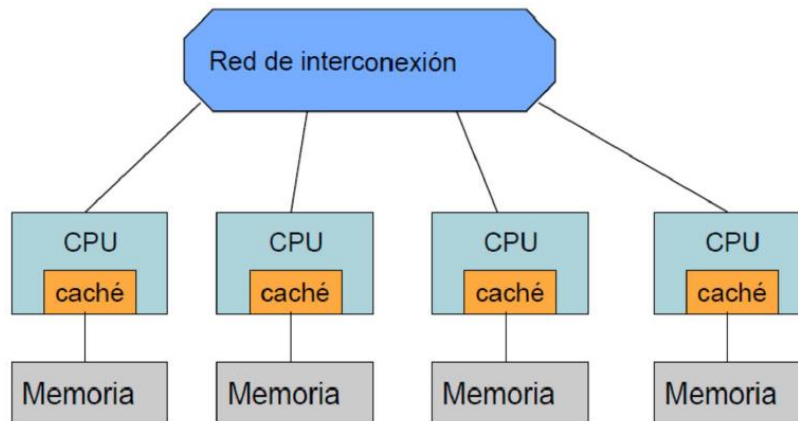
2.10 Sistemas MIMD: segundo latencias de acceso a memoria.

- **Multiprocesadores de acceso uniforme a memoria (UMA)**
 - Latencia de acceso a memoria independente do procesador.
 - Baixa escalabilidade: 128 procesadores como máximo.
- **Acceso a memoria non uniforme (NUMA)**
 - A latencia de acceso a memoria depende do procesador: máis baixa para memoria local.
 - Mellor escalabilidade: de centos de miles de procesadores.

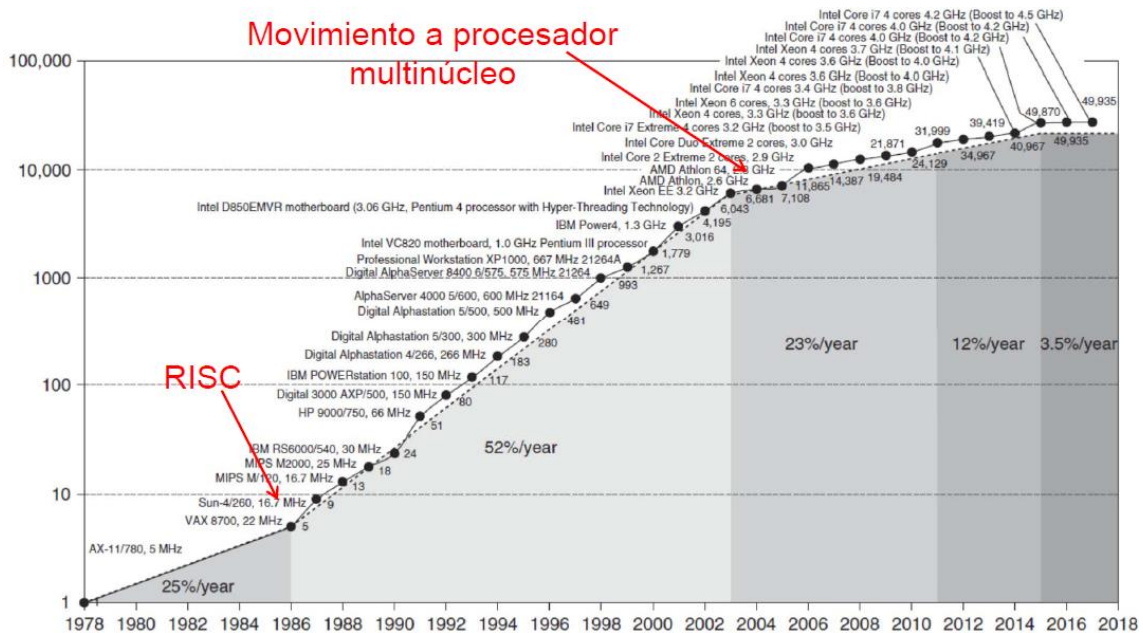
Os sistemas NUMA poden ser:

- NCC-NUMA (Non-Cache-Coherent Non-Uniform Memory Access). Marcan como sucios os datos que foron modificados e non están actualizados.
- CC-NUMA (Cache-Coherent Non-Uniform Memory Access).

A organización típica dos sistemas NUMA é a seguinte:



2.11 Volvendo ó procesador



Rendemento dos procesadores ó longo do tempo executando un SPECint comezando por sistemas dun solo procesador (eixo Y rendemento fronte un procesador estándar de 1978).

No ano 2004 aparecen as computadores multinúcleo. Tamén baixa a porcentaxe da evolución notablemente, isto débese ó mencionado anteriormente do esgotamento e a non aparición de melloras alternativas.

Agora mesmo hai que plantexarse novas técnicas para a mellora do rendemento dos procesadores porque o crecemento estase estancado con respecto á evolución tida ata o momento.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Asignatura

18-02-2021



Clase 01

Temas trabajados:

Páginas (20-36)

Índice

1. El procesador de un solo núcleo	3
El Pentium 4 (2003)	3
Arquitectura de un núcleo	4
2. El microprocesador multinúcleo	5
.....	5
3. Concepto de segmentación(pipeline)	5
4. Tecnología de fabricación CMOS	6
5. Evolución del área de lógica con la tecnología.....	7
6. Escalamiento de potencia para carga numérica	8
7. Eficiencia de microprocesadores de propósito general.....	9
8. Energía y potencia dinámica	9
9. Potencia estática	10
10. Potencia y Energía	11
11. Consumo de potencia (TDP).....	11
12. Potencia consumida	12
13. Técnicas para aumentar la eficiencia energética	12
14. Ancho de banda y latencia	13
15. Ancho de banda de memoria	13

1. El procesador de un solo núcleo

El Pentium 4 (2003)

El Pentium 4 fue uno de los últimos procesadores creados antes de la aparición del movimiento multinúcleo. Este podía ser empleado en distintos tipos de equipos, ordenadores personales, servidores...

Características:

- Aplicaciones: ordenadores de sobre mesa y servidores
- Tecnología: 90nm (1/100th del 4004)¹

- 55M transistores (20,000x)
- 101 mm² (10x)
- 3.4 GHz (10,000x)
- 1.2 Volts (1/10th)

- 32/64-bit datos (16x)
- Pipe-line de 22 etapas
- Superscalar: permite la ejecución de 3 instrucciones por ciclo
- Dos niveles de memoria caché (L1 y L2) integradas en el chip-
- Estructura SIMD: permite la ejecución de instrucciones vectoriales.
- Permite el hyperthreading: simula dos procesadores (dos núcleos) para que el procesador siempre tenga trabajo (a costa de un mayor coste energético).

Información:

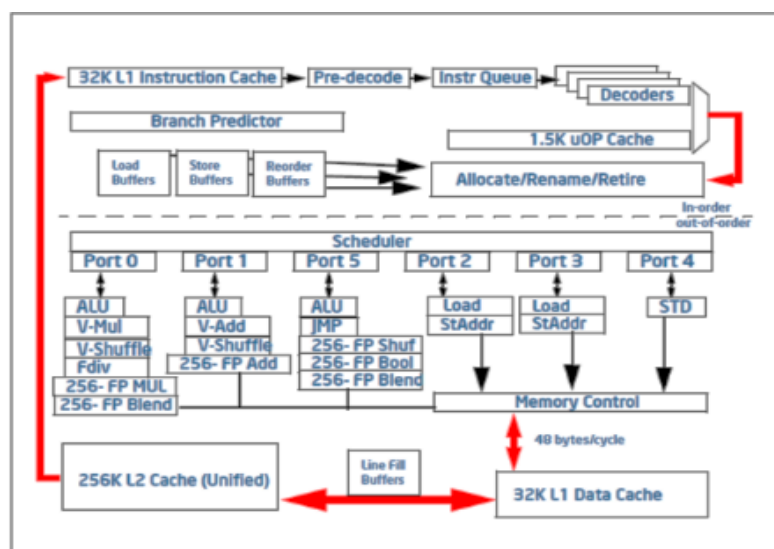
1. El Pentium 4 tenía un tamaño de transistor con una centésima parte del tamaño del Intel 4004 de 1971. (El resto de las referencias son comparaciones también con el Intel 4004)

2. Información adicional:

<https://courses.cs.washington.edu/courses/cse378/09au/lectures/cse378au09-14.pdf>

Arquitectura de un núcleo

- Varios niveles de caché.
- Nivel 1 separado en Datos e Instrucciones de 32KB cada una. Y tamaño de línea de 64 bytes.
- L1 permite 2 lecturas y 1 escritura por ciclo, esto es posible gracias a que hay más de una ALU(en este caso 3) que realizan operaciones.
- L2 unificada (datos e instrucciones juntas) con latencia de 12 ciclos (se necesitan 12 ciclos para acceder a esta memoria).
- L3 de 4-32 MB, ancho de banda de 32 bytes/ciclo, y accesos de 25-35 ciclos.
- Hardware para ejecución fuera de orden y unidades de ejecución que operan en paralelo (se utiliza el predictor de salto y el buffer de reordenación para que esto funcione).



Arquitectura de un núcleo.

Explicación de componentes:

Proceso que siguen las instrucciones desde su búsqueda el L1:

- Pasan por una etapa de predecodificación.
- Se encolan.
- Se decodifican.
- Se reordenan para una ejecución más eficiente empleando los buffer de reordenamiento. Aquí se ejecuta el Branch Predictor¹.

Puertos 0, 1, 5:

Gracias a la existencia de varias ALU's se pueden procesar operaciones aritméticas de instrucciones distintas de forma paralela.

Información:

1. Branch Predictor: permite predecir si una instrucción va a producir o no un salto, de este modo se puede cambiar el orden que posee en la ejecución para priorizar aquellas que no generan saltos.
2. La caché L3 no se encuentra representada en el dibujo.

2. El microprocesador multinúcleo

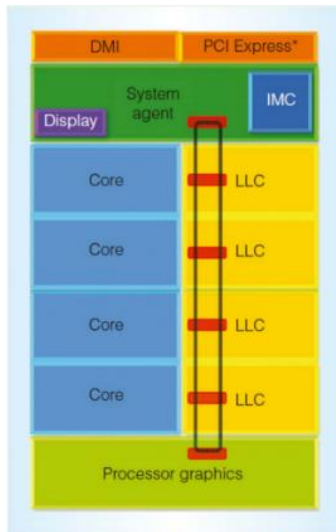


Diagrama de bloques del procesador Haswell.

➤ El nivel de caché L3 (último nivel) es compartida entre todos los procesadores mediante una conexión de anillo.

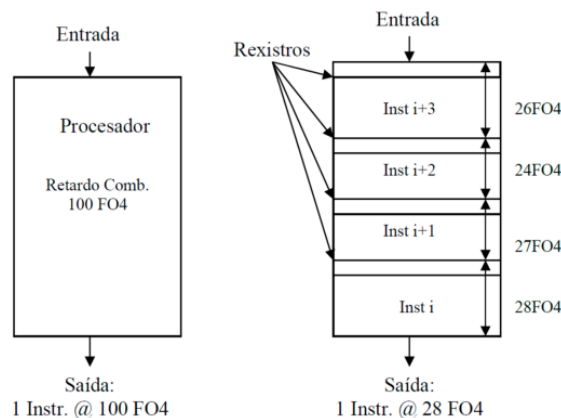
➤ Las diferentes unidades del procesador tienen un voltaje y una frecuencia independiente del resto.

- Una unidad de control decide de forma dinámica como distribuir el consumo de potencia entre las partes para mejorar el rendimiento.

Información:

1. LLC: Last level caché.

3. Concepto de segmentación(pipeline)

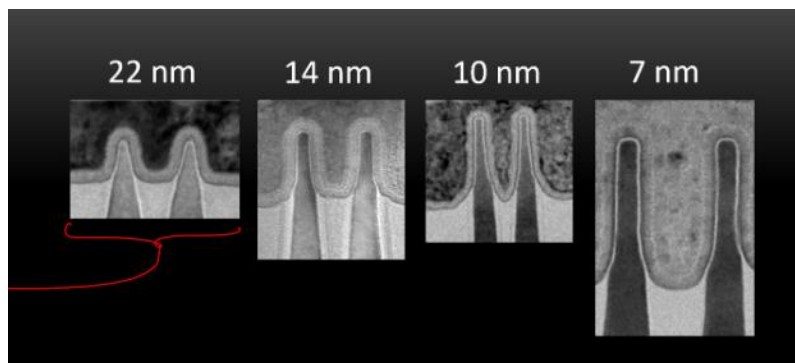


- FO4(“fan-out-of-4”): Es el retardo equivalente a un inversor al que se conectan cuatro inversores a la salida. Para poder saber el valor hay que consultar cuanto valía este retardo en el nodo tecnológico en el que se fabrica el componente.
Ejemplo: @100 FO4 → 100*(valor de FO4 para este nodo)
- Segmentación: Ejecución en etapas y cada etapa un retardo.
- Profundidad del pipeline: Es una decisión de diseño muy importante¹ y se corresponde con el número de etapas en que se divide (4 para el ejemplo de la figura).
- El retardo con segmentación es más alto que sin ella debido a la lógica adicional añadida.

Importante:

1. Cada etapa en el pipeline añade retardos debido a los registros. Además, hay que tener en cuenta que la velocidad de reloj tiene que ser mayor que la etapa más lenta debido a esto, el pipeline establece una frecuencia máxima de reloj posible.
2. La aparición de etapas permite que varias instrucciones se estén procesando de forma simultánea. No obstante, en caso de que una de las instrucciones necesite un dato para la continuación de su ejecución se parará la “continuación del pipeline” de modo que las instrucciones que se encuentren en etapas anteriores no podrán continuar generando lo denominado como *burbujas*.

4. Tecnología de fabricación CMOS

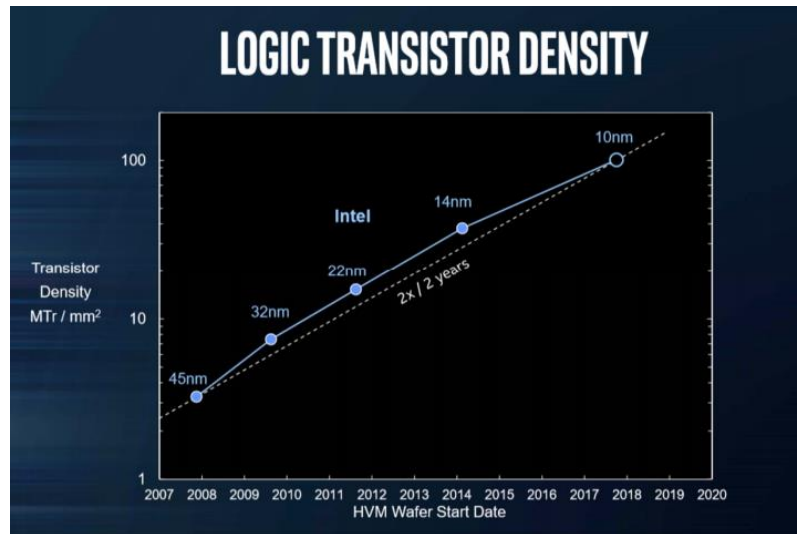


- Conexión de cobre de nivel 1: Conexión que hay entre dos transistores en la capa más baja (los chips tienen varias capas de transistores apiladas unas sobre otras para ahorrar superficie).
- El valor numérico que identifica al nodo tecnológico es el ancho mínimo de una conexión de cobre de nivel 1 y está directamente relacionado con la superficie que ocupa el transistor.

Información:

1. Límite del CMOS: actualmente la tecnología CMOS está llegando a sus límites a nivel de escalado, la reducción de la separación entre transistores se complica debido a saltos de los electrones entre ellos por efecto túnel.
2. Nodo tecnológico: cambio en el proceso de fabricación de los procesadores, SIEMPRE respetando la estructura y geometría básica de los transistores.

5. Evolución del área de lógica con la tecnología

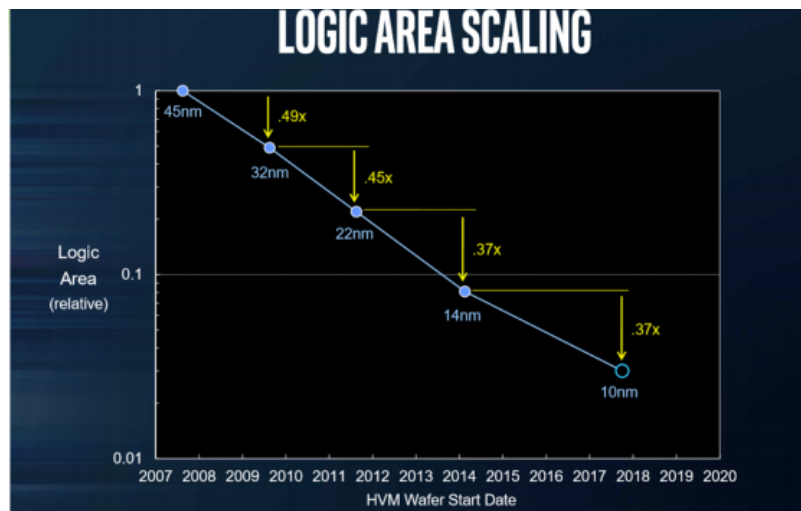


El número de transistores por unidad de área a lo largo del tiempo.

Ley de Moore → Evolución de 2x cada 2 años.

Importante:

El aumento lineal del número de transistores permite el aumento lineal de gran parte de las prestaciones de los procesadores.



Evolución del área ocupada por la misma cantidad de transistores al cambiar de un nodo al siguiente.

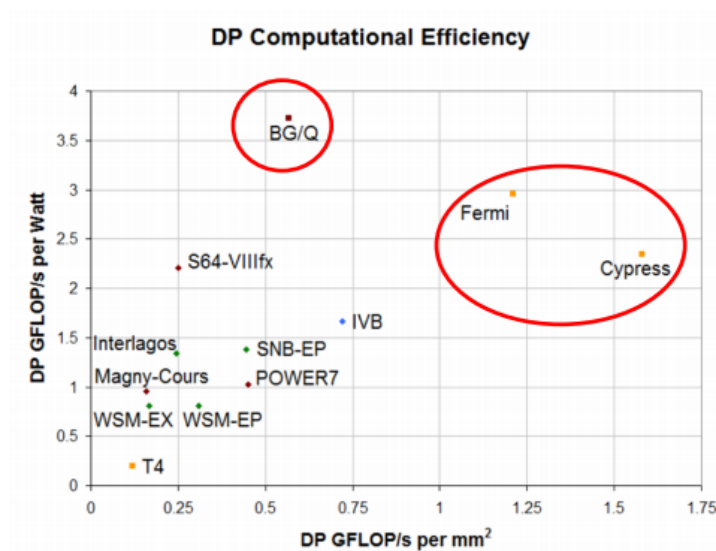
Al principio se reducía a la mitad entre nodos, pero cada vez se reduce menos.

6. Escalamiento de potencia para carga numérica

- Si un microprocesador ocupaba 2 cm^2 con tecnología de 65nm, debería ocupar 1 cm^2 (la mitad) en el siguiente nodo, 45nm.
- PROBLEMA: No todos los transistores se organizan igual en todas las partes del chip:
 - Memoria: organización regular de transistores.
 - CPU: organización irregular.

Este hecho provoca que no todos los circuitos dentro de un microprocesador escalen igual con la tecnología, impidiendo la reducción del tamaño a la mitad, aunque se reduzca a la mitad la distancia entre transistores.

6. Escalamiento de potencia para carga numérica



Eje X: Operaciones en punto flotante/mm² (Número de instrucciones ejecutables en función del área del procesador).

Eje Y: Operaciones en punto flotante/Watio (Número de instrucciones ejecutables con una potencia de un watio).

Eficiencia de microprocesadores de altas prestaciones por lo que la medida es muy adecuada.

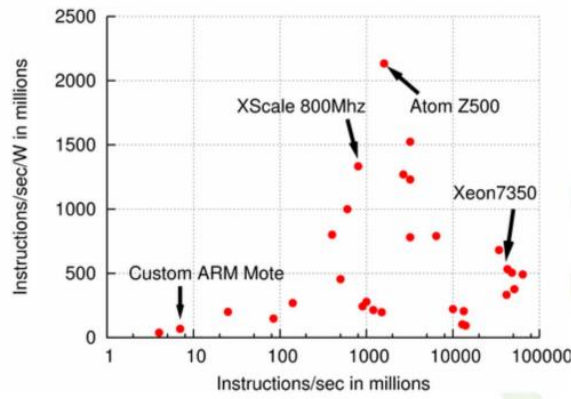
Los Procesadores que más destacan son los rodeados en rojo:

- BG/Q por su eficiencia en cuanto al consumo.
- Fermi y Cypress por su potencia computacional por espacio ocupado.

Información:

1. GPLOPS/s: 10⁹ operaciones en punto flotante por segundo.

7. Eficiencia de microprocesadores de propósito general



- La mejor posición es la esquina superior derecha.
- La posición en un cuadrante depende del propósito con el que hayan sido fabricados.
 - Xeon → Servidores de altas prestaciones. No son eficientes en consumo.
 - Atom Z500 → Es el mejor en cuanto al buen equilibrio entre eficiencia energética y eficiencia de instrucciones por segundo.

Información:

1. La grafica representa carga de trabajo no dominada por computación numérica en punto flotante.

No es necesario saber las gráficas de memoria, pero si **saber interpretarlas**.

8. Energía y potencia dinámica

- Energía dinámica
 - Paso de un transistor de 1-0 y 0-1
 - Proporcional a

$$\frac{1}{2} \cdot Carga\ Capaciva \cdot Voltaje^2$$

- Potencia dinámica
 - Proporcional a

$$\frac{1}{2} \cdot Carga\ Capaciva \cdot Voltaje^2 \cdot Frecuencia$$

Información:

1. Actualmente el voltaje es inferior a 1V

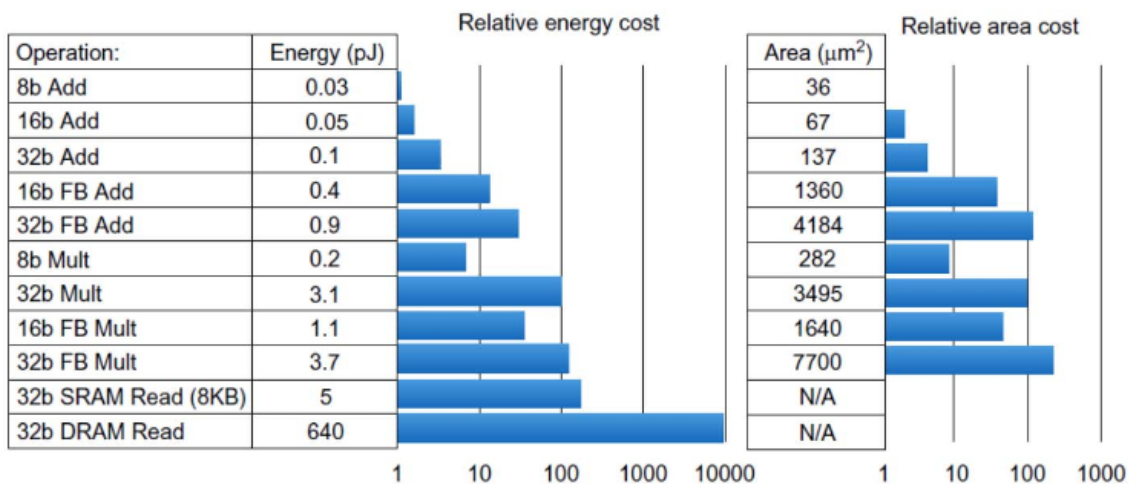
2. Potencia: ritmo de uso de energía (cantidad de trabajo por unidad de tiempo).
3. A lo largo de los años el consumo de potencia dinámica se ha mantenido constante, este hecho es debido a que, pese a la reducción del voltaje en los procesadores, se ha producido un aumento de la frecuencia.
4. Reducir la frecuencia de reloj reduce la potencia, no la energía.

9. Potencia estática

- Consumo de potencia estática:
 - 25-50% de la potencia total.
 - Es proporcional a:

$$Corriente_{estática} \cdot Voltaje$$

- Escala con el número de transistores.
- Para reducir: *power gating* o *diseños domain-specific*.



Información:

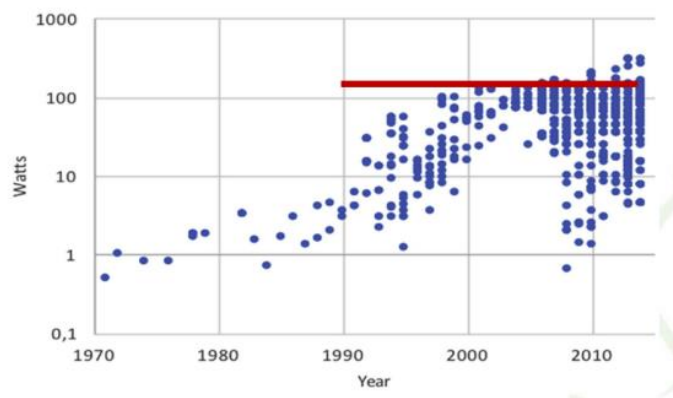
1. Domain-specific: intenta reducir el área de los circuitos y el consumo de las tareas, reduciendo las operaciones en punto flotante de 32bits y hacerlas de 8 y 16 y desconectando parte de los circuitos cuando no se estén empleado para reducir el consumo de potencia estática.
2. El consumo de potencia estática llega al 50% en caso de que las cachés sean muy grandes debido al aumento del número de transistores.
3. En la tablas se expone el coste de potencia estática en función de tipo de instrucción a ejecutar.
4. SRAM: memoria empleada en la caché
5. DRAM: memoria empleada en la memoria principal.

6. Si los transistores aumentan en número aun reduciendo el voltaje se mantiene la potencia estática (los transistores aunque estén apagados gastan energía).

10. Potencia y Energía

- Problema: entrada y salida de potencia.
- Thermal Design Power (TDP)
 - Caracteriza el consumo de potencia sostenido.
 - Se usa como objetivo en cuanto a potencia suministrada y sistema de refrigeración.
 - Tiene un valor más bajo que la que la potencia pico (1.5x más alta), y más alta que el consumo de potencia medio.
- La frecuencia de reloj se puede reducir dinámicamente para limitar el consumo de potencia.
- La energía por tarea es a menudo una mejor medida.

11. Consumo de potencia (TDP)



Evolución del consumo de potencia (TDP).

- Se observa que se ha llegado a un límite (100-200 Watt) en la disipación de potencia.
- Razones del límite: técnicas (no se puede disipar el calor con ventiladores) y económicas (sería necesario usar la refrigeración líquida).
- Mantener el límite en los nuevos microprocesadores requiere optimizar el consumo de potencia (desconectar unidades cuando no se usan, mejorar los transistores para que consuman menos, emplear operaciones de menos bits, etc).

Información:

1. La barrera de la disipación de potencia de 100-200 Watt es debido a que cuando un procesador llega a esos conjuntos de potencia las pérdidas energéticas por calor aumentan provocando que los ventiladores no sean suficiente para la disipación forzándonos a recurrir a otros métodos más sofisticados y **caros**.

12. Potencia consumida

- Intel 80386 consumía 2 W
- 3.3GHz Intel Core i7 consume 130 W
- El calor debe ser disipado desde un chip de 1.5 x 1.5 cm
- Este es el límite de lo que puede ser enfriado usando el aire.

13. Técnicas para aumentar la eficiencia energética

- No hacer nada
 - Apagar el reloj de módulos inactivos para ahorrar energía (evitando el consumo estático de transistores).
- Escalado dinámico de Voltaje-Frecuencia
 - Aprovechar periodos de inactividad para operar a frecuencias de reloj y voltajes más bajos.
- Estado de baja potencia para memorias DRAM, discos (unidades de memoria en general)
 - Válido mientras están inactivas.
- Overclocking
 - Intel desde 2008: trabajar a frecuencias más altas (hasta un 10% +) durante cortos periodos de tiempo (Turbo Boost)
 - Puede implicar apagar los núcleos para los que no se sube la frecuencia
 - Limitado por la subida de temperatura
 - Es transparente al usuario

Información:

1. Turbo Boost: nombre que recibe el overclocking en la arquitectura x64-86 de Intel.
2. Como el overcloking produce un mayor gasto de energía debido al aumento de frecuencia, en general, si el código a ejecutar posee un solo hilo se desconectan el resto de los núcleos para reducir a 0 su consumo de energía.

14. Ancho de banda y latencia

- Ancho de banda
 - Trabajo total hecho en un tiempo dado
 - Mejora de 32,000-40,000X para procesadores
 - Mejora de 300-1200X para memoria y discos
- Latencia
 - Tiempo entre el comienzo y la finalización de un evento
 - Mejora de 50-90X para procesadores
 - Mejora de 6-8X para memoria y discos

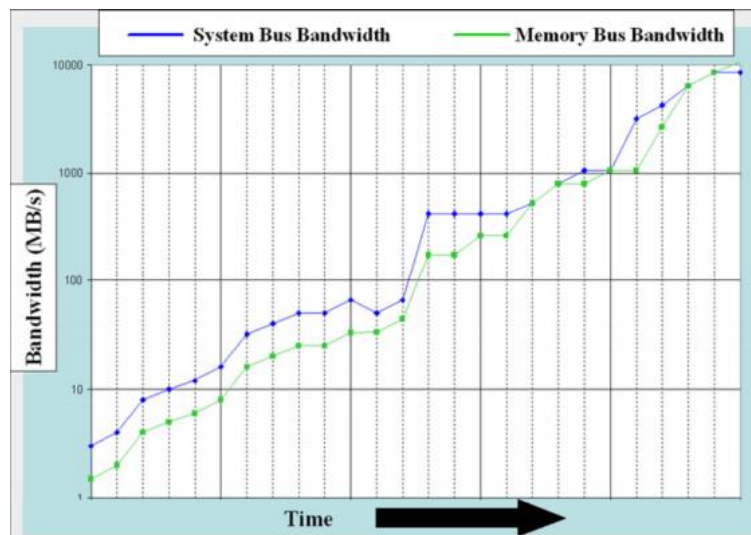
Información:

1. Ancho de banda (procesador vs memoria): Los procesadores “piden” información a mayor velocidad de la que la memoria es capaz de “dar” (permiten pasar una transmisión de datos a mayor velocidad).

2. Latencia (procesador vs memoria): el tiempo que tarda la memoria entre que recibe una orden para buscar un dato y comienza a buscarlo es mucho mayor que el tiempo que tarda el procesador entre que recibe el dato y comienza a procesarlo.

15. Ancho de banda de memoria

Tendencia histórica en el escalado del ancho de banda de memoria en los procesadores de Intel.



ATENCIÓN: En la figura vemos que no siempre se escala el ancho de banda entre generaciones. Se suele x2 entre nodos tecnológicos y mantener el mismo diseño de memoria dentro del mismo nodo.

- El ancho de banda máximo sostenido (BWM) se diseña para satisfacer cargas de trabajo representativas para el microprocesador.
- Debe escalar con el número de núcleos N , la frecuencia F y el número de instrucciones por ciclo (inverso del CPI)

$$BWM = N \times F / (IP \times CPI_{corei}) \text{ (bytes/s)}$$

- IP es la intensidad operacional para la cual fue dimensionado el procesador. Se mide en (instrucciones/byte)

Información:

1. Por lo general el producto:

$$latencia^2 \cdot potencia$$

Se mantiene constante. Este hecho no es una propiedad física sino una realidad tecnológica (al igual que la ley de Moore).



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura de Computadores

25-02-2021



Clase 03

Temas trabajados: Tema 1

Índice

1. Escalamiento del retardo en circuitos.....	3
2. Evolución de la frecuencia de reloj en los microprocesadores de Intel.....	3
3. Escalamiento del ancho de banda y la latencia	5
4. Efecto de la evolución tecnológica en prestaciones y potencia.....	6
5. Evolución del procesador Itanium 2.....	6
6. Escalamiento de prestaciones en microprocesadores.....	8
7. Evolución de procesadores de un solo núcleo en prestaciones	12
8. Final de los procesadores de un solo núcleo	13

1. Escalamiento del retardo en circuitos

La ley de Moore es una ley empírica que dice que cada 16-24 meses el número de transistores que se pueden meter en cada circuito integrado se suele doblar. Esto tiene una serie de implicaciones, una de ellas es que el retardo de los circuitos también va aumentando. A medida que va cambiando la tecnología podemos meter más transistores en un mismo circuito porque pasamos a una nueva generación de transistores que pueden estar más cerca. Por ejemplo, pasamos de 35 nanómetros a 22, es decir, que, si antes los transistores estaban a 35 nanómetros de distancia, ahora están a 22.

Cuanto más avance la tecnología, más retardo tienen los circuitos. Los retardos no se miden normalmente en tiempo absoluto (en segundos, milisegundos...), se miden en unidades que se denominan F04 (fan-out 4): el retardo que tiene un inversor que tiene 4 inversores conectados a la salida. El valor de este retardo F04 cambia con el nodo tecnológico, es decir, cambia si paso de 35 nanómetros a 22, por ejemplo. Para convertir el retardo F04 en retardos absolutos, hay que conocer cuál es el valor del F04 en el nodo tecnológico en el que estemos. Para tecnologías CMOS de altas prestaciones, una aproximación razonable para la unidad de retardo se calcula con la siguiente fórmula:

$$F04 = 0.2 \times T$$

Este valor se mide en picosegundos y T es la longitud que identifica el nodo tecnológico.

Si suponemos que tenemos un microprocesador con un ciclo de reloj de 150 F04. Si tengo una tecnología de 10 nm el retardo de un F04 será:

$$F04 = 0.2 \times 10 = 2 \text{ ps}$$

Entonces la frecuencia de reloj será 1/periodo:

$$\frac{1}{150 \times 2 \text{ ps}} = 3.33 \text{ GHz}$$

Si en lugar de 10nm, se utiliza una tecnología antigua de 65 nm, con el mismo ciclo de reloj:

$$F04 = 0.2 \times 65 = 13 \text{ ps}$$

$$\frac{1}{150 \times 13 \text{ ps}} = 513 \text{ MHz}$$

En cada momento, se puede calcular el valor del retardo típico F04 con la fórmula anterior, teniendo T en nm y obteniendo F04 en ps.

La fórmula anterior se calculó experimentalmente y normalmente nos da un retardo típico. En condiciones de operación extrema, el retardo se multiplicaría por 1.4.

2. Evolución de la frecuencia de reloj en los microprocesadores de Intel

La siguiente gráfica presenta la evolución de los ciclos de reloj a lo largo del tiempo para diferentes procesadores. Es un histórico de los procesadores de Intel. Por ejemplo, el Intel Pentium II del año 1997 tenía una frecuencia de 300 Mhz y tiene una tecnología de 350

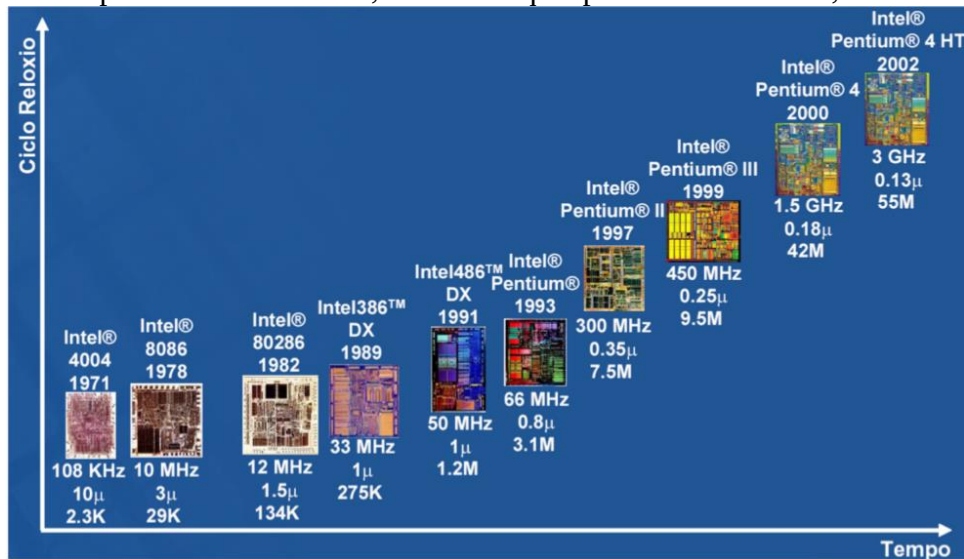
2. Evolución de la frecuencia de reloj en los microprocesadores de Intel

nm. En esta tecnología el circuito, el chip de la CPU, tenía 7.5 millones de transistores y el ciclo de reloj era de 47 F04.

Otro ejemplo es el Pentium 4 con HyperThreading del año 2002, que tenía una tecnología de 130 nm, 55 millones de transistores en el circuito y la frecuencia a la que iba era 3GHz y el ciclo de reloj era de 12.5 F04.

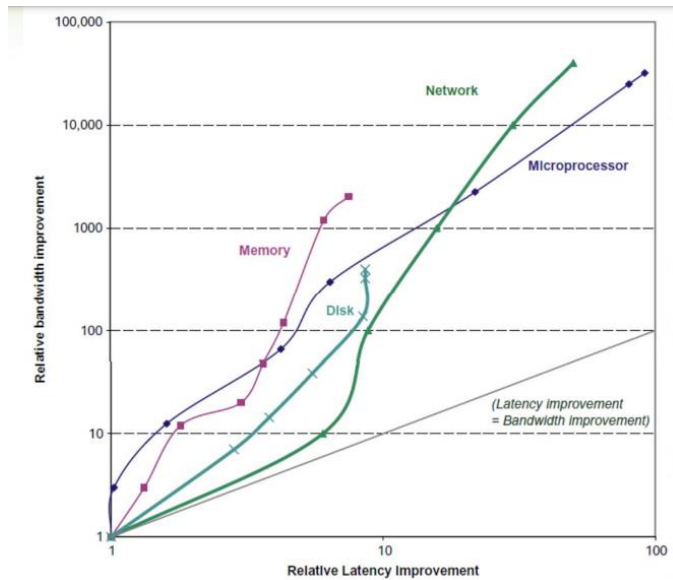
Podemos ver que a medida que fue aumentando la frecuencia, el ciclo de reloj se hace más pequeño (47 F04 en Pentium II y 12.5 F04 en Pentium 4).

Aplicando la formula explicada en el apartado anterior ($F04=0.2 \times T$), tenemos un retardo de 70 ps en el Pentium II, mientras que para el Pentium 4, un F04 es de 26 ps.



3. Escalamiento del ancho de banda y la latencia

En cuanto al ancho de banda y la latencia, han evolucionado de forma distinta en los microprocesadores y en las memorias. La siguiente gráfica logarítmica en los dos ejes representa los cambios de la latencia y el ancho de banda para la memoria, el disco, los microprocesadores, etc. Simplemente para indicar que, a lo largo del tiempo, ha ido mejorando la latencia de todos los elementos.



La relación entre latencia y consumo de potencia en un microprocesador es:

$$\text{Latencia}^3 \times \text{Potencia} = \text{cte}(T)$$

La constante T depende del nodo tecnológico.

En la fórmula anterior la latencia es mucho más importante que la potencia. Si se disminuye la latencia, se puede aumentar la frecuencia de operación de manera que el consumo se mantenga dentro de un valor que cumpla esta fórmula. Por ejemplo, si se reduce la latencia a la mitad, se puede esperar que el circuito consuma 8 veces más potencia para mantener constante el producto entre ambas.

También se cumple la siguiente fórmula:

$$\text{Latencia}^n \times \text{Area} = \text{cte}(T)$$

Estas fórmulas están determinadas experimentalmente, por ejemplo, en esta última fórmula, para $n=2$ si se pretende reducir la latencia a la mitad en un circuito, el área aumentará en un factor de 4 para poder mantener el proceso de fabricación.

Entonces en cada tecnología hay que tomar decisiones de diseño teniendo en cuenta la relación entre los parámetros y las restricciones que vienen dadas por estas fórmulas.

4. Efecto de la evolución tecnológica en prestaciones y potencia

En la siguiente gráfica el eje x representa las prestaciones del sistema en ejecución de operaciones.

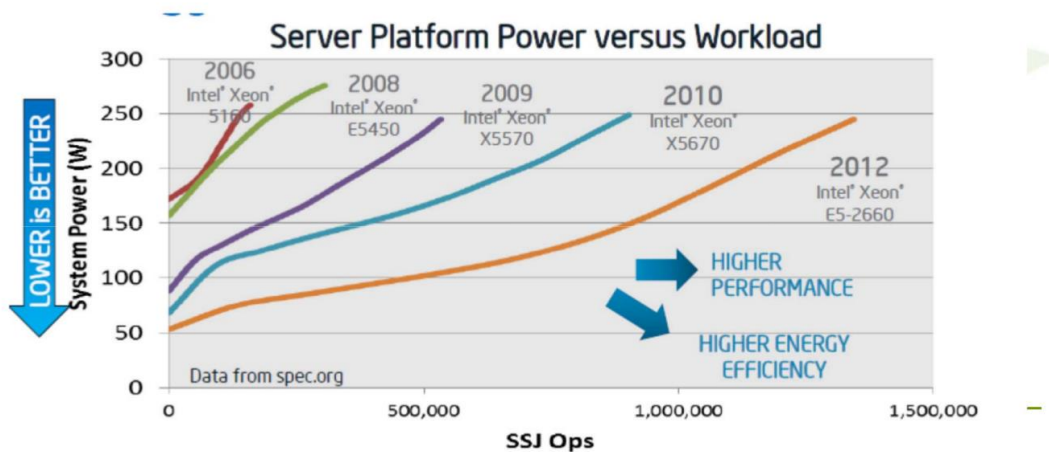
En el eje y se tiene el consumo de potencia.

En esta gráfica se presentan datos para diferentes servidores que están contruidos con procesadores Inter Xeon fabricados en diferentes nodos tecnológicos, es decir, de una línea a otra cambia la tecnología de fabricación. También cambia la arquitectura del procesador para optimizar la tecnología.

Para cada nodo tecnológico hay una línea en lugar de un solo punto porque cada sistema es escalable en prestaciones. Variando el número de núcleos o el número de procesadores activos, la frecuencia de reloj y diferentes elementos, para cada nodo tecnológico podemos tener diferentes sistemas, tal y como se ve en la gráfica.

Dado un nivel de prestaciones, es decir un punto en el eje x, vemos que cada nuevo nodo tecnológico permite reducir el consumo de potencia. Si por ejemplo se observa el punto 500.000 del eje x, se puede afirmar que, en la tecnología del año 2012, el consumo de potencia es más bajo que en la anterior (2010), es decir, dado un nivel de prestaciones cada nodo tecnológico reduce el consumo de potencia con respecto al nodo tecnológico anterior.

Por otro lado, cada nodo tecnológico permite alcanzar un incremento del nivel de prestaciones consumiendo la misma potencia.



5. Evolución del procesador Itanium 2

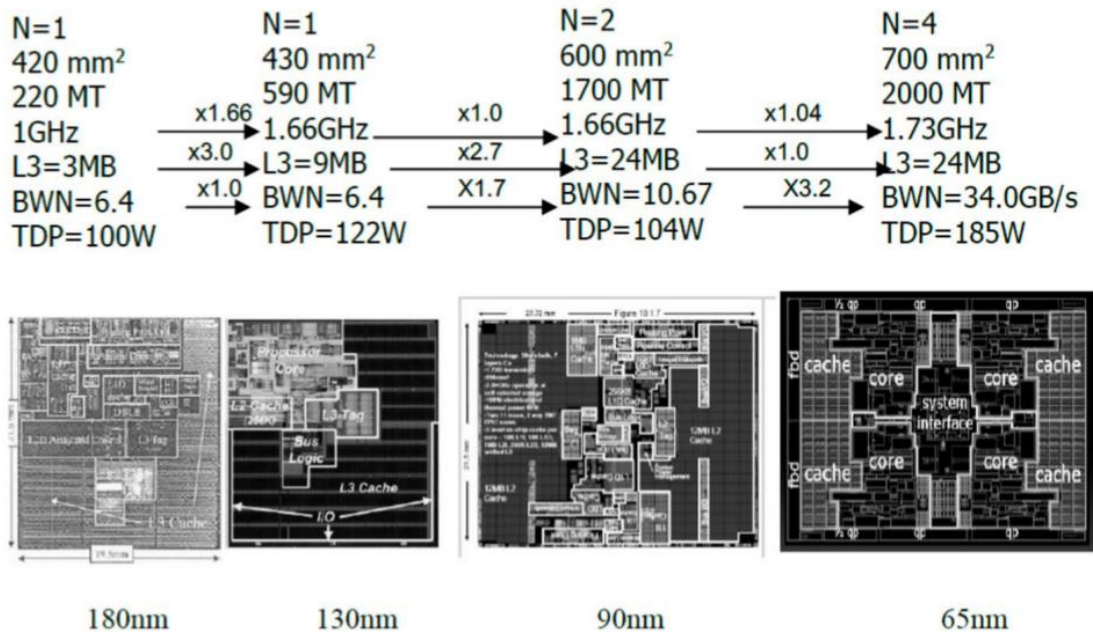
Otra cosa que interesa observar es como ha ido evolucionando el procesador Itanium 2 a lo largo de diferentes generaciones tecnológicas.

Esta gráfica representa la evolución de la familia de procesadores de altas prestaciones del Intel Itanium 2 en 4 nodos tecnológicos distintos que corresponden a diferentes años.

N indica el número de núcleos. Hasta 2005 tenían un único núcleo (una única CPU) en el procesador.

6. Evolución de la dedicación de área de chip en microprocesadores

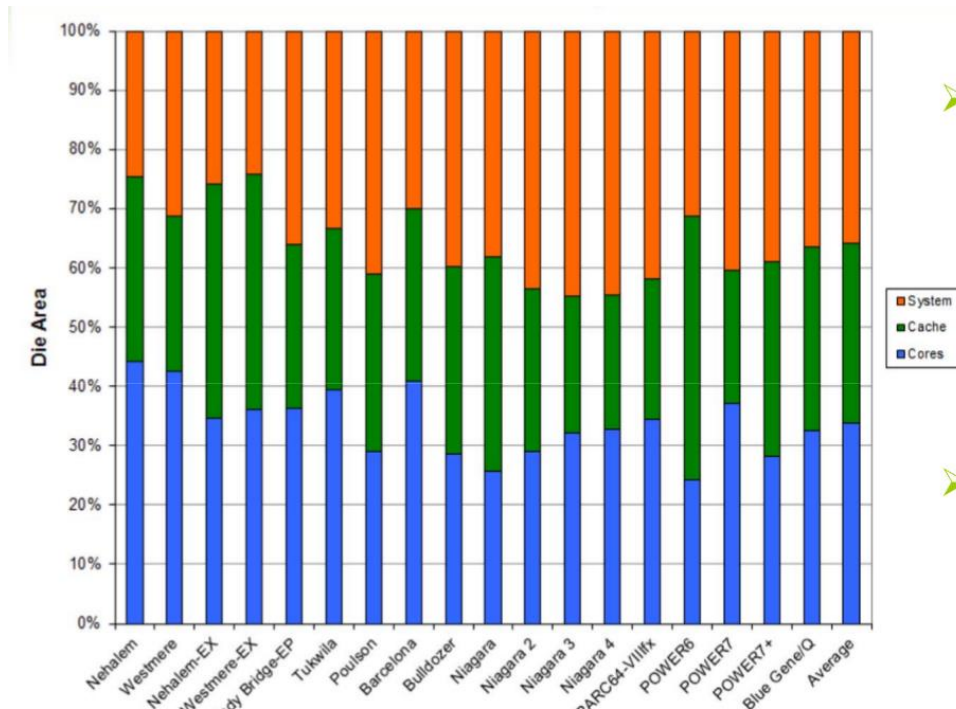
La frecuencia se multiplica por 1,66 primero y luego por 1,04, es decir la frecuencia va escalando con factores de escalado cada vez más bajos. Con respecto al ancho de banda sostenido, hasta 2005 se mantiene debido a la presencia de un único núcleo, luego se multiplica por 1,7 y luego por 3,2. Es decir, a más núcleos es necesario más ancho de banda. Este ancho de banda también se denomina tasa de transferencia nominal con la memoria (BWN). MT indica el número de transistores en millones. Podemos observar que este número aumenta considerablemente de un nodo a otro.



Gracias al dibujo anterior, se puede saber que existe una parte muy grande del procesador dedicada a la caché. En el de 180 nm el 60% del área era de cache L3, el 25% de procesador, 9% de circuitos de entrada/salida y 5% de lógica de interfaz con el bus. A la hora de realizar un diseño debemos tener en cuenta ciertas restricciones y dependencias entre parámetros.

6. Evolución de la dedicación de área de chip en microprocesadores

En general, en los servidores de grandes sistemas se puede ver dentro del área del chip a que se dedica cada parte, eso se muestra en la siguiente gráfica.



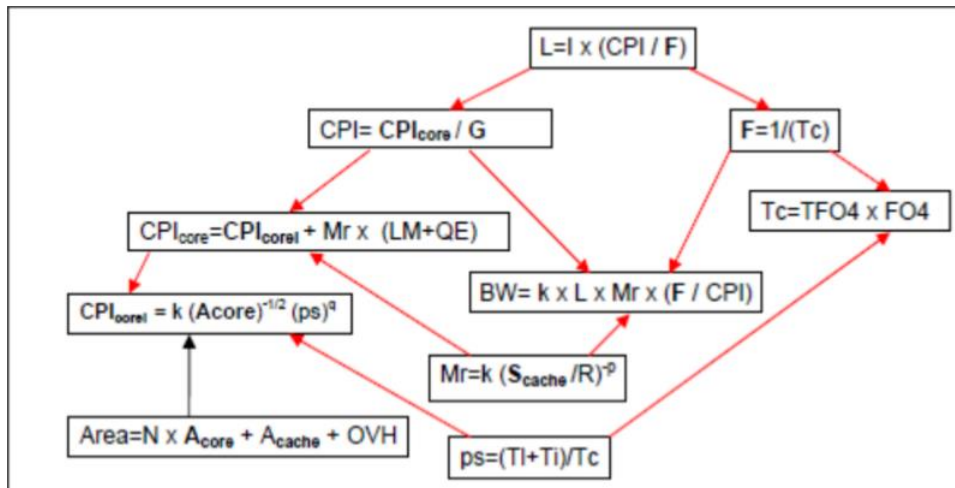
En el eje x se representan microprocesadores y servidores de grandes sistemas y en el eje y se indica el porcentaje de circuitos del microprocesador que se utiliza para los Cores (azul clarito), para la caché(verde) y para el sistema(naranja). El sistema incluye el área que se dedica al controlador de memoria integrado, el interfaz de memoria, al hardware para interconectar los sistemas multiprocesador...

La última columna de la derecha es la media de todos ellos: en general, en el área del chip el 34% se dedica a los núcleos, el 30% a la caché y el 36% al resto del hardware del sistema. Es importante ver esto, tenemos $\frac{1}{3}$ del área dedicada a la caché, siendo los circuitos de la caché muy diferentes a los del procesador. Una característica que tienen los circuitos integrados que se dedican a la caché es que tiene una densidad de transistores por unidad de área muy alta (2.1 millones de trans/mm²) mientras que para el procesador hay 250.000 trans/mm².

¿Por qué hay tantos para la caché? Porque la organización de la caché es mucho más regular. El procesador tiene la ALU para ejecutar operaciones de punto flotante, tiene unidades para ejecutar operaciones con números enteros, tiene los registros (que son otra parte completamente diferente), tiene multiplexores...lo que provoca que su circuito sea irregular, haciendo que la densidad de trans/mm² sea mucho menor que el de la caché.

7. Escalamiento de prestaciones en microprocesadores

Las siguientes fórmulas no hay que saberlas, solo hay que entender lo que está escrito en ellas.



Simplemente expresan la relación entre diferentes elementos de diseño de los procesadores que han marcado la evolución de Intel y está marcada por la relación entre los diferentes parámetros que se encuentran en las fórmulas. Son todos elementos muy simples, por ejemplo: en la fórmula de $L = I \times (CPI / F)$, L es el tamaño de la línea caché, I es el número de instrucciones que ejecuta el programa, por el CPI dividido por la frecuencia. La frecuencia es también la inversa del período.

Tc es el período en $F04$ multiplicado por el retardo de un $F04$.

La fórmula del área: $Area = N \times A_{core} + A_{cache} + OVH \rightarrow$ El área es el número de núcleos de la CPU multiplicada por el área de cada núcleo más el área de la caché más la contribución al área de otros elementos distintos del núcleo y la caché.

L en general es la latencia de un programa que tienen I instrucciones, CPI es el número medio de ciclos por instrucción que se ejecuta y Mr es la tasa de fallos de caché del último nivel para cada instrucción. Una tasa por ejemplo de 0,01 significa que de cada 1000 instrucciones ejecutadas se produce un fallo de caché de último nivel.

En general estas fórmulas contemplan la posibilidad de que el procesador tenga varios núcleos de procesamiento, en particular N núcleos de procesamiento, y las aplicaciones estén divididas en hilos para poder procesarlas en paralelo entre los diferentes núcleos y cada núcleo soporte varios hilos ejecutándose a la vez.

LM : Latencia media por penalización de acceso a memoria principal. En particular este parámetro es lo que se tarda como media en acceder a memoria principal cuando se produce un fallo en la caché del nivel más alto.

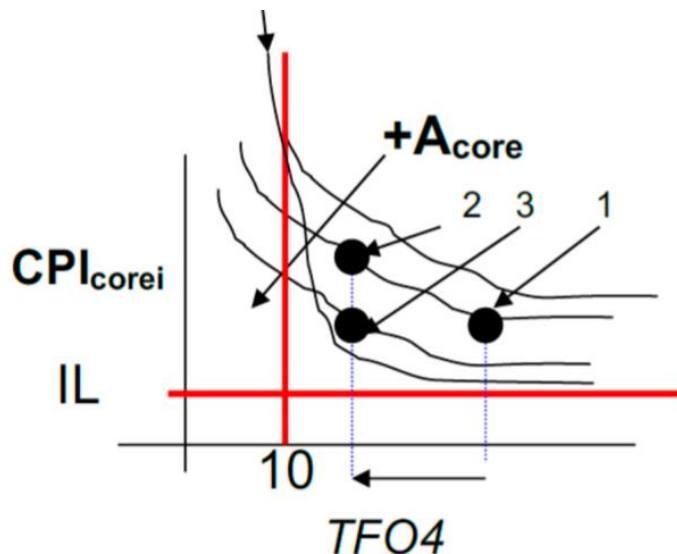
¿Qué puede afectar a esta latencia? Se ve afectada por la ejecución multihilo, es decir, si se tienen varios hilos ejecutándose a la vez, la LM se va a ver afectada porque va a haber varios hilos compitiendo. Si se pide un dato en memoria principal, el tiempo que se tarda en dar ese dato si se tienen varios hilos ejecutándose simultáneamente pidiendo datos, la LM se va a reducir porque van a estar entrando en conflicto. La LM depende de cuántas transacciones a memoria principal se puedan hacer a la vez. Normalmente en un procesador se pueden hacer hasta 10 transacciones a la vez con memoria principal debido a que la memoria está entrelazada.

Si suponemos que vamos a realizar transacciones a memoria principal que se pueden realizar hasta 10 a la vez y si la LM son 6 ns en un procesador de 3 GHz tenemos 18 ciclos

por transacción y, al tener 10 transacciones, se obtendrán $10 \times 6 \text{ ns} \times 3 \text{ GHz} = 180$ ciclos de latencia total a memoria.

Las transacciones no se hacen todas a la vez, sino que se hacen de manera segmentada. Esto quiere decir que mientras una está en una etapa, otra está en otra etapa diferente, reduciendo así la latencia total de acceso a memoria.

La siguiente gráfica relaciona el tiempo de ciclo, que es el período de la señal expresada en F04, y el CPI de un core ($\text{CPI}_{\text{corei}}$), el número de ciclos por instrucción que ejecuta sin que necesite ir a memoria principal, es decir, puede ir a caché de primer nivel, segundo nivel... pero no puede ir a memoria principal.



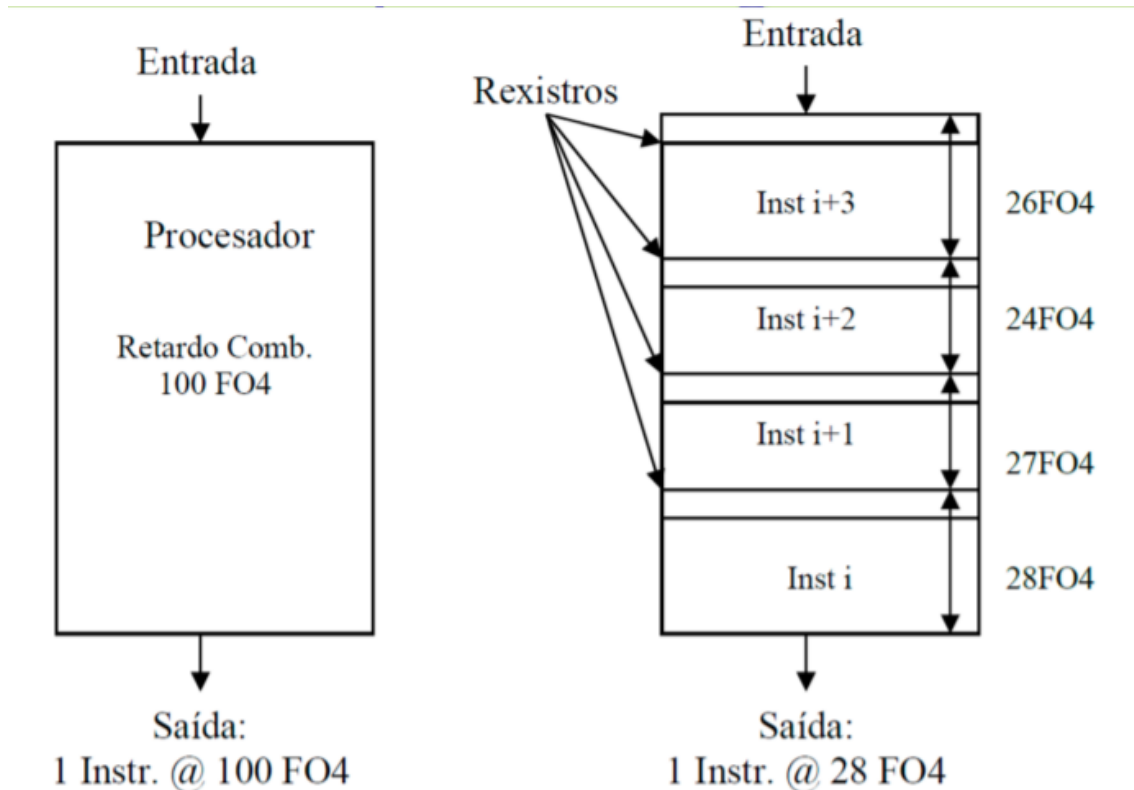
En general el CPI de un core depende de la fórmula: $\text{CPI}_{\text{corei}} = k(A_{\text{core}})^{(-1/2)}(p_s)^q$.

A_{core} hace referencia al área del chip que se dedica al core y p_s es la profundidad del pipeline, es decir, el número de etapas en que se divide la ejecución de una instrucción.

Las instrucciones se van a dividir en etapas en su ejecución, a diferencia de lo que pasaba en el MIPS. Ahora lo que va a suceder es que la instrucción se va a ejecutar en trozos independientes, en fases: una fase de buscar la instrucción en memoria, una fase de codificarla, una fase de operar y una fase de guardar en memoria... Entonces cuanto mayor sea el número de etapas, más va a aumentar el número de ciclos por instrucción, ya que cada etapa se hace en un ciclo. Normalmente se diseña la segmentación para que cada etapa de la segmentación sea realizada en un único ciclo.

Si se ponen registros entre las diferentes etapas de la ejecución (la parte de buscar instrucción, de decodificarla, coger los operandos, ejecutar y escribir en memoria) se puede obtener una instrucción que estaría escribiendo en memoria mientras que otra podría estar en ejecución, mientras otra lee los datos, mientras otra se está decodificando. Así se estaría aprovechando toda la ejecución y todas las partes para estar ejecutando instrucciones en diferentes etapas. A esto se le llama segmentación. Estas 4 etapas, son las 4 etapas del pipeline. Cada etapa se produce siempre en 1 ciclo de reloj y, después, se activan los registros, se produce un cambio de estado y se pasa a la ejecución de la siguiente etapa. Entonces, en el siguiente ciclo la instrucción i se iría, la $i+1$ pasaría a dónde estaba la i , la $i+2$ donde estaba la $i+1$, la $i+3$ en donde estaba la $i+2$ y la siguiente entraría. Si en vez de dividir esto en 4 etapas se divide en 8, cada etapa va a tardar la mitad de tiempo y la frecuencia va a subir. Si se sube la frecuencia, se sube el consumo

de potencia. Va a haber por el medio registros que van a consumir retardo y, además, el ciclo de reloj se va a hacer más pequeño, por lo que aumenta la frecuencia.



Volviendo a la fórmula del CPIcorei, que dice que el número de ciclos por instrucción que necesita una determinada CPU del procesador, un core, cuanta más área ocupe, más bajo es el pipeline y más rápido se ejecuta la instrucción y, cuantas más etapas tenga el pipeline más ciclos por instrucción necesita. Esta relación se representa en la gráfica anterior.

Volviendo a la última gráfica, en el eje x aparece el tiempo de ciclo medido en TF04. Si nos movemos en la gráfica a la izquierda, aumenta el número de etapas del pipeline, aumentando así el TF04. Al ser más profundo, tenemos un número de esperas más grande cuando alguna instrucción se quede parada en el pipeline, debido a que está calculando una dirección de salto, o porque no tiene los datos que necesita...; y, por lo tanto, el número de ciclos por instrucción va a subir, lo que explica la subida de las curvas de la gráfica. Se puede reducir este efecto aumentando el área, añadiendo unidades de predicción salto más sofisticadas o elementos que alivien este problema...

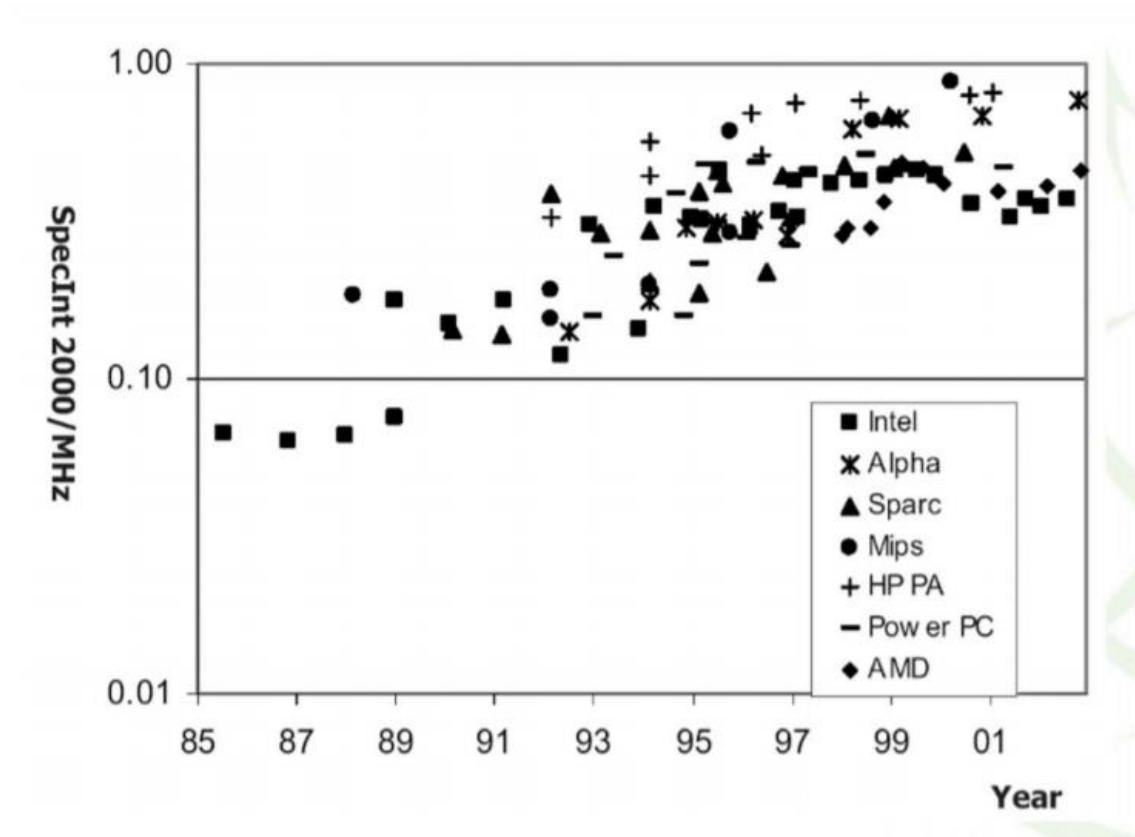
La reducción del tiempo de ciclo del punto 1 al punto 2, supone un aumento del CPI del núcleo. Para mantener el CPI original tendríamos que aumentar el área, pasando así al punto 3.

Las diferentes curvas que se ven ahí, corresponden a diferentes áreas del núcleo. Si el núcleo ocupa más área pueden meterse unidades adicionales que ayuden a aliviar problemas.

Además, el espacio de diseño de la imagen está limitado por paralelismo a nivel de instrucción. Se ve una barra en vertical de 10 retardos porque no se considera fiable tener relojes con menos retardo que este. En general el límite será 10 FO4.

Esta figura representa restricciones o dependencias entre elementos tal y como se produce en el mundo real en la implementación de los procesadores.

8. Evolución de procesadores de un solo núcleo en prestaciones



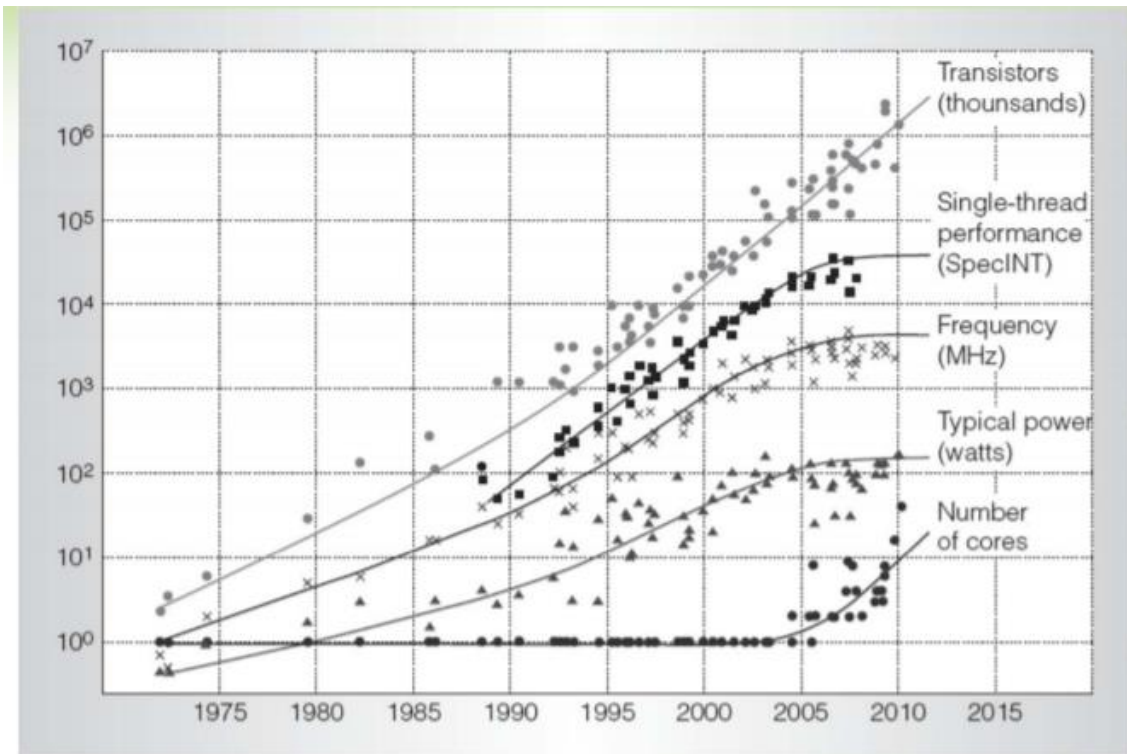
En el eje x se representa el tiempo desde el año 1985 hasta la actualidad y lo que se ve representado en la gráfica son las prestaciones de microprocesadores de un solo núcleo a lo largo del tiempo para diferentes familias de microprocesadores. Spec es un benchmark, una organización internacional que proporciona una serie de programas que miden prestaciones en relación a un sistema de referencia.

La tendencia de los procesadores de Intel fue mantener constante el número de ciclos por instrucción y en las últimas generaciones este empezó a aumentar porque el pipeline comenzó a tener cada vez más profundidad. Se intentó hacer un pipeline con más profundidad para intentar estar ejecutando muchas instrucciones a la vez. En esta gráfica se ve, para los diferentes procesadores, la ratio entre el SpecInt/MHz (proporcional al inverso del CPI). Este SpecInt va aumentando a lo largo del tiempo y en las últimas generaciones ha habido un aumento del CPI debido a la profundidad del pipeline.

Existen también otros Spec como SpecFloat o SpecDouble para evaluar las operaciones en punto flotante.

9. Final de los procesadores de un solo núcleo

Los procesadores fueron evolucionando a lo largo del tiempo según esta gráfica:



Si nos fijamos el número de cores, que es la línea de abajo, estuvo estable en un solo core hasta el año 2004, a partir de entonces comenzaron los sistemas multicore. El consumo de potencia fue aumentando hasta llegar a estabilizarse porque se podían desconectar algunos cores mientras otros estaban trabajando, aumentando la frecuencia a un core mientras desconectas los demás. La frecuencia también fue aumentando hasta que llega un momento en el que la potencia disipada empezaba a ser tal que con la ventilación era difícil disiparlo, entonces, se trató de mantener la frecuencia constante.

El número de transistores se fue duplicando cada 2 años, cumpliendo la ley de Moore, tal y como se ve en la línea de arriba, pero, las prestaciones se fueron aclimatando para adaptarse a las nuevas tecnologías.

La latencia pasó a ser la mitad de una generación de procesadores a la siguiente.

En general, el retardo medido en F04, aumenta de forma exponencial, es decir, cada vez hacen falta más ciclos de reloj para transmitir una señal por una determinada interconexión por como son los procesos de fabricación y porque las conexiones largas cada vez están más superpuestas y existen más problemas para que estas interconexiones se puedan hacer de manera eficiente.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura Computadores

04-03-2021

Clase 04

Temas trabajados:

Tema 1:

Agotamiento del modelo de escalamiento de un solo núcleo

Métricas de rendimiento de un sistema

1. Índice

1. Introducción	3
2. Agotamiento del modelo de escalamiento de un solo núcleo.....	3
3. Métricas de rendimiento de un sistema	3
Métricas de rendimiento típicas	3
Speedup de X relativo a Y.....	3
Benchmarks.....	4
Tiempo de respuesta.....	4
Tiempo de ejecución en CPU	4
Ciclo de reloj por instrucción (CPI)	4
Ecuación básica de rendimiento	5
MIPS	5
4. Técnicas de medida de rendimiento	5
Utilización de información mantenida por el sistema operativo	5
Monitores software.....	5
Monitores hardware	6
Análisis de programas (profiling)	6
Programas de prueba (benchmarks)	6
Conclusiones evaluación de rendimiento	7

1. Introducción

En esta clase, la primera parte ha sido dedicada a un repaso general a lo anterior dado, lo cual no ha sido recogido en esta bitácora ya que se profundiza en bitácoras anteriores. Lo mismo ocurre con el final de clase, donde se explica qué vamos a estudiar en el tema 2, lo cuál será profundizado en la siguiente clase.

2. Agotamiento del modelo de escalamiento de un solo núcleo

Sobre el año 2004, se agota el modelo de trabajo basado en el uso de un solo núcleo por los siguientes motivos:

- De una generación a otra el área de los componentes se reduce a la mitad siguiendo **la ley de Moore**.
- Debido al uso de componentes grandes como cachés de mayor tamaño aparecen conexiones que no reducen su longitud y acarrear consigo los siguientes problemas:
 - Necesidad de más ciclos de reloj para transmitir una señal.
 - Aumento de la profundidad del pipeline y segmentación de conexiones largas mediante registros.
 - Esto se traduce en un impacto negativo en los Ciclos Por Instrucción (CPI).

Para bajar el CPI es necesaria una mejor predicción de saltos y mejores estructuras en general (más complejidad de hardware). Todo esto, provoca que llega un momento en el que la precarga de datos, la ejecución de varios hilos... no son suficientes para mantener un rendimiento por lo que se pasó a un sistema multinúcleo (multicore).

3. Métricas de rendimiento de un sistema

Métricas de rendimiento típicas

- **Tiempo de respuesta:** es el tiempo entre el inicio y el final de una tarea. Para maximizar el rendimiento se minimiza el tiempo de ejecución de una tarea.
- **Throughput** (productividad). Cantidad de trabajo hecha por unidad de tiempo.

Speedup de X relativo a Y

Se define como:

$$\text{Tiempo ejecución}_Y / \text{Tiempo ejecución}_X$$

Por ejemplo, si mejoramos nuestro código haciendo una versión más eficiente, vamos a conseguir un SpeedUp que va a ser el: el tiempo de ejecución en el sistema antiguo / tiempo de ejecución en el sistema nuevo.

Tiempo de ejecución: medida infalible de que las cosas se están haciendo bien en un sistema. Se mide en :

- **Wall clock time:** incluye todos los sobrecostos del sistema
- **CPU time:** se refiere solo al tiempo de computación

3. Métricas de rendimiento de un sistema

Benchmarks

Conjunto de programas elaborados previamente para evaluar el rendimiento de los programas.

- Kernels (ejemplo matrix multiply)
- Programas “de juguete” (ejemplo sorting)
- Benchmarks sintéticos (ejemplo Dhrystone)
- Conjuntos de benchmarks (e.g. SPEC06fp, TPC-C)

Tiempo de respuesta

Tiempo que incluye los **accesos a disco, accesos a memoria, actividades de E/S y la carga adicional del Sistema Operativo.**

Tiempo de ejecución en CPU

Es el tiempo que se mide habitualmente. **Tiempo que la CPU dedica a ejecutar una tarea concreta:**

$$T_{cpu} = T_{usuario} + T_{Sistema\ Operativo}$$

Ejemplo:

```
./programa datos
```

real 0m11.588s -> tiempo total

user 0m3.735s -> tiempo de CPU del usuario

sys 0m3.286s -> tiempo de CPU del S.O.

Donde el tiempo real es el tiempo de respuesta y el tiempo de ejecución es el tiempo de usuario + tiempo de CPU del sistema Operativo.

$$Tiempo\ de\ CPU = ciclos\ CPU \times tiempo\ de\ ciclo\ de\ reloj$$

$$Tiempo\ de\ CPU = \frac{Ciclos\ de\ CPU}{Frecuencia\ de\ reloj}$$

El compilador genera las instrucciones para ejecutar y el procesador las ejecuta. El número de ciclos de reloj requerido por un programa es:

$$Ciclos\ de\ CPU = instrucciones \times media\ de\ ciclos\ de\ instrucción$$

Ciclo de reloj por instrucción (CPI)

Es el **número medio de ciclos de reloj que una instrucción necesita** para ejecutarse.

Instrucciones diferentes pueden necesitar diferente cantidad de ciclos de reloj.

El CPI proporciona una media:

$$CPI = \frac{\sum_{i=1}^m CPI_i \times NumeroInstrucciones_i}{NumeroMaximoInstrucciones}$$

En el mips todas las instrucciones tardan un ciclo. En el cisc algunas son cortas y otras son largas por lo que no todas tardan lo mismo.

Se puede medir el CPI de un programa solo, de un conjunto de programas...

4. Técnicas de medida de rendimiento

Ecuación básica de rendimiento

$$T_{CPU} = \text{NumeroInstrucciones}(N) \times CPI \times T(\text{Período Señal})$$
$$= \frac{\text{NumeroInstrucciones}(N) \times CPI}{\text{Frecuencia } (F)}$$

Esta fórmula se puede utilizar para comparar dos realizaciones diferentes o para evaluar un diseño alternativo si se conoce el impacto de los tres parámetros.

MIPS

$$MIPS = \frac{\text{NumeroInstrucciones } (N)}{(T_{CPU} \times 10^6)}$$

Problemas:

- No se pueden comparar computadores con diferentes repertorios de instrucciones.
- Varía entre programas en el mismo computador puede variar inversamente el rendimiento,

4. Técnicas de medida de rendimiento

Utilización de información mantenida por el sistema operativo

Los actuales sistemas operativos mantienen información sobre las diferentes tareas que se están ejecutando: utilización de la memoria y del procesador, el número de tareas en el sistema o el tiempo de inicio y de finalización de cada tarea.

Ventaja

- No hace falta desarrollar código específico para realizar las medidas de rendimiento.

Desventajas

- Sobrecarga en el acceso a esta información.
- Formato poco amigable que impone bastantes restricciones.
- No mantiene información de todos los eventos.

Monitores software

Se ejecuta una aplicación específicamente programada para recoger y tratar la información necesaria.

Tipos de monitores:

- Detección de eventos
- Muestreo
 - Cuenta de eventos
 - Traza, es un programa que imprime las direcciones de acceso a memoria en un fichero.

Ventajas

- Se adecuan a las necesidades de los usuarios y recogen la información necesaria.

Desventajas

- Sobrecarga en el acceso a esta información si se produce con una gran frecuencia.

4. Técnicas de medida de rendimiento

Monitores hardware

Los fabricantes incluyen en el hardware monitores que permiten contar con gran precisión una multitud de eventos.

Ventajas

- No se consumen recursos (tiempo del procesador y almacenamiento).
- Se pueden monitorizar eventos de muy bajo nivel que son completamente transparente para el software e incluso para el sistema operativo.
- Muy precisos.

Desventajas

- Se miden magnitudes físicas y no lógicas.
- Librerías específicas.

Análisis de programas (profiling)

Se emplean medias de tiempo y recursos. Consiste en medir el tiempo al principio y al final del programa.

Se utiliza cuando la evaluación del rendimiento es a un nivel alto y en términos de optimización de código o de un sistema.

Ventajas

- No está sujeta a errores aleatorios como el muestreo.

Desventajas

- Sobrecarga la aplicación

Programas de prueba (benchmarks)

Para evaluar el rendimiento de un sistema se usa un conjunto específico de programas de pruebas (benchmarks).

La práctica aceptada hoy es usar una selección de programas de aplicación reales.

Los programas de prueba forman una carga con la que el usuario espera predecir el rendimiento de la carga real del sistema.

Para indicar las medidas del rendimiento se debe hacer un informe con una lista detallada de forma que otra persona pueda reproducir los resultados (versión del sistema operativo, compilador, entradas, configuración de la máquina, etc...)

El rendimiento se suele dar como un único número.

El grupo de prueba más popular y completo es el SPEC:

- SPEC (Standard Performance Evaluation Corporation – www.spec.org)
- La última versión de los SPEC para el procesador es el banco de pruebas SPEC CPU2017.
- Los programas SPEC se dividen en dos grupos:
 - Con carga computacional intensa en punto flotante (SPECfp2017).
 - Con carga computacional intensa en enteros (SPECint2017).

4. Técnicas de medida de rendimiento

Los tiempos de ejecución de un sistema deben ser normalizados, para lo que se usa un sistema como referencia.

Ratio SPEC para un programa

$$velocidad\ SPEC = \frac{T_{computadorReferencia}}{T_{computadorTest}}$$

La prueba se repite para todos los programas del conjunto SPEC y se computa la media geométrica de los resultados.

Ratio SPEC para un conjunto

$$velocidad\ SPEC = \sqrt[n]{\sum_{i=1}^n (SPEC_i)}$$

Se obtiene la nota y se compara con los demás.

Conclusiones evaluación de rendimiento

La medida más importante del rendimiento de un computador es la rapidez con la que se puede ejecutar programas.

En el caso del procesador, esta rapidez vendrá determinada por:

- Organización de su hardware (CPI)
- Ciclo de reloj (T_{ciclo})
- Instrucciones de lenguaje máquina (NumeroInstrucciones(NI) y CPI)
- El compilador que se utilice (NI)

Además del tiempo de ejecución existen otras métricas del rendimiento como los MIPS (Millones Instrucciones Por Segundo) y GFLOPS. Estas dos son útiles para comparar el mismo código de computadores diferentes donde tengan el mismo conjunto de instrucciones.

Los programas de prueba (benchmarks) nos permiten comparar tiempos de ejecución entre máquinas diferentes y obtener una medida normalizada del rendimiento de los computadores.

La ley de Amdahl permite acotar la aceleración que se obtendrá al mejorar alguno de los subsistemas del conjunto. Es decir, nos permite encontrar cuanto tiempo tenemos que mejorar un parámetro (por ejemplo, el tiempo que se tarda en realizar las multiplicaciones) para reducir el tiempo de ejecución. Hay tiempos de ejecución mínimos que no se pueden reducir, esta ley nos ayuda a encontrarlos.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Asignatura: Arquitectura de Computadores

11-03-2021



Clase 05

Temas trabajados:

Tema 2: Paralelismo a nivel de instrucción: segmentación

Índice

1. Comentarios sobre la siguiente práctica.....	3
2. Ejercicio: Fundamentos de Computadores 2.0.....	3
-Ley de Amdahl:	4
3. Continuación Tema 2: paralelización a nivel de instrucción.....	5
MIPS	5
Instrucciones MIPS	6
Instrucciones aritméticas y lógicas.....	6
Instrucciones de transferencia y saltos	7
Formato de instrucción.....	7
Pequeño esquema de resumen.....	7
Etapas por instrucción	8
Implementación multiciclo y segmentación.....	8
Ejemplo arquitectura multiciclo en simulador	9
Ejemplo arquitectura segmentación en simulador.....	12
Diferencia multiciclo y segmentación	14
Diferencia monociclo y segmentación	15
Segmentación con enteros y con punto flotante	15
Tiempo de instrucciones con segmentación	16
Arquitectura MIPS segmentado	16
Registros de separación en el MIPS	17
Ejercicio de ejemplo.....	19

1. Comentarios sobre la siguiente práctica

- Instrucciones SIMD: (simple instrucción, múltiple dato) Instrucciones que se ejecutan sobre un conjunto de datos.
- Vamos a realizar: una versión secuencial, versión secuencial optimizada, una versión con instrucciones vectoriales y una programada en OpenMP.
- OpenMP: lenguaje de programación que permite paralelizar lazos de forma automática usando directivas denominadas pragma. Es decir, repartir el trabajo entre los cores.

En el enunciado aparece qué tiempos se deben comparar para conseguir aceleraciones variando los tamaños de las matrices.

Nos beneficia usar el número 8 ya que es múltiplo del tamaño de los registros, y n sea un número muy grande para cargar de trabajo.

Todos los códigos en OpenMP.

- Se utiliza un vector red y no una simple variable m (double), ya que esto optimizaría la ejecución.
- Al producirse overhead tiene que compensarse con la localidad: cuanto mayor sean las matrices, más rendimiento voy a conseguir de esas mejoras.
- Dentro de BLAS, está GEMM: producto de matrices. Optimizaciones al código.

*Revisar el manual de OpenMP.

2. Ejercicio: Fundamentos de Computadores 2.0

Enunciado: Ejercicio 1 (campus virtual)

1. Se dispone de una aplicación que permite procesar imágenes. Una fracción de dicha aplicación es paralelizable mientras que el resto debe ejecutarse secuencialmente.

a) Indicar que aceleración debe realizarse sobre la versión paralelizable para conseguir una aceleración global de 10 en la versión paralela.

b) Si se quiere conseguir una aceleración global de la versión paralela de 10 y se acelera 18 veces la fracción paralelizable, ¿cuál debe ser la fracción paralelizable?

Solución:

Tener en cuenta:

- Al ser paralelizable, utilizando un lenguaje de programación adecuado y varias cores se puede optimizar el problema.
 - Es un código simple
- Aplicación que procesa imágenes



$$\frac{T_{inicial}}{T_{paralelo}} = S = 10$$

S=speedup

PREGUNTA:

¿Cuál es la fracción de código que tiene que ser paralelizable para conseguir esta aceleración?

-Ley de Amdahl:

Otra formulación de la ley de Amdahl:

- A un código que se ejecuta en un tiempo T, le aplicamos una mejora que reduce un porcentaje F de su ejecución en un factor M, ¿cuál es el nuevo tiempo de ejecución?

$$T' = \frac{F \cdot T}{M} + (1 - F) \cdot T \Rightarrow$$

$$\frac{T'}{T} = \frac{F}{M} + (1 - F)$$

Si hay una fracción de código que puedo paralelizar (F). Se puede reducir el tiempo de ejecución en un factor, o una aceleración n.

$$S = \frac{1}{(1 - F) \frac{F}{n}}$$

**Hay que saberse la fórmula de memoria

S es lo que da el enunciado, F la fracción de código paralelizable.

-Despejamos F:

$$F = \frac{n(S - 1)}{S(n - 1)}$$

-Pasamos a sustituir por los valores

$$F = \frac{n(10 - 1)}{10(n - 1)} = \frac{9n}{10n - 10}$$

$$1 \geq F$$

Por lo que:

$$1 \geq \frac{9n}{10n - 10}$$

$$10n - 10 \geq 9n$$

$$n \geq 10$$

RESPUESTA a)

Para conseguir una aceleración de $S=10$ en el código con fracción paralelizable, necesito una aceleración en la fracción paralelizable ≥ 10

¿cuál debe ser la fracción paralelizable?

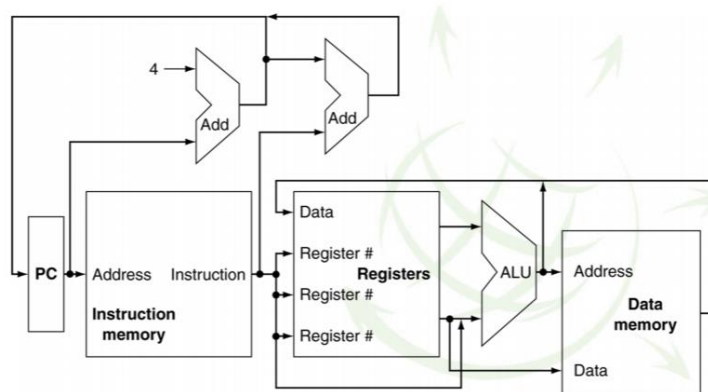
3. Continuación Tema 2: paralelización a nivel de instrucción

MIPS

Arquitectura del MIPS: arquitectura tipo RISC

Para la ejecución de una instrucción se tienen dos fases: búsqueda y ejecución (hay más etapas que solo estas dos porque cada una se divide en más).

Estructura básica (simplificada) del MIPS:



1* Correspondiente a:

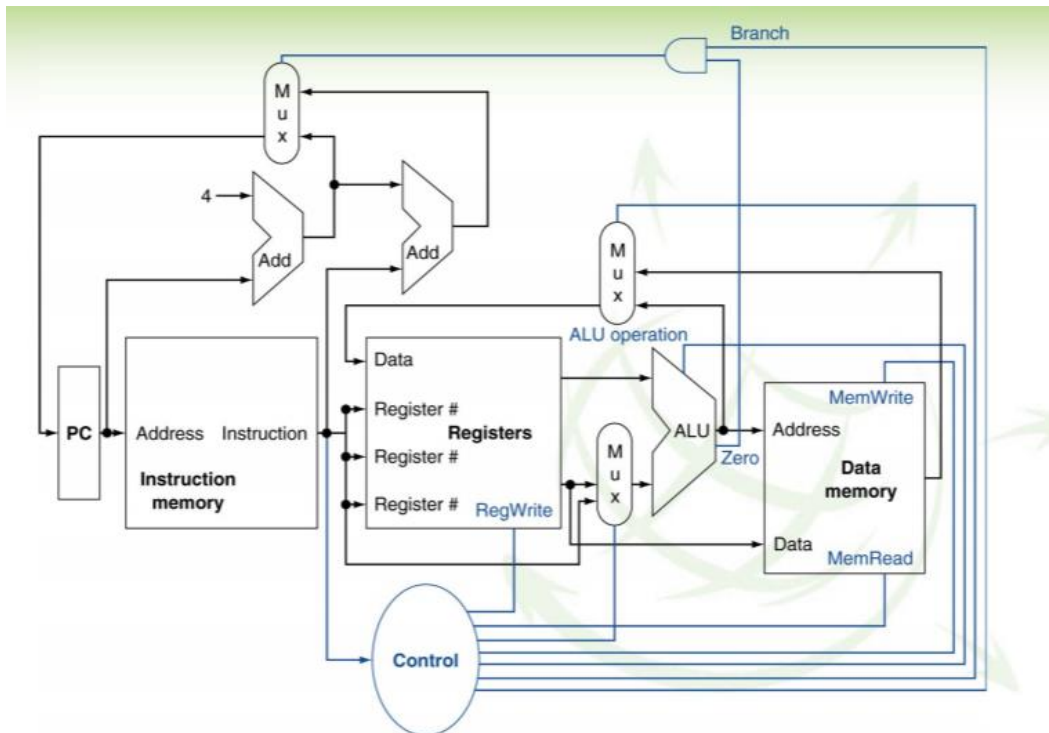
- Registro de instrucción.
- Gestión de instrucción.
- Búsqueda de instrucción en memoria.
- Decodificación de instrucción (los campos son interpretados).
- Leer datos de registros.
- Ejecución en ALU de la propia instrucción.
- Se obtienen valores y pueden necesitar gestionar nuevamente el almacenamiento en memoria o guardarse en los mismos registros.

También se debe tener en consideración las sumas de 4 para pasar a la siguiente instrucción exceptuando cuando se realizan saltos.

Hay estructuras más completas, y se incluyen las líneas de control (en la figura siguiente: marcadas en azul).

Las líneas de control gestionan las diferentes opciones: lectura o escritura en registro, elección de datos, etc.

3. Continuación Tema 2: paralelización a nivel de instrucción



El MIPS trabaja en monociclo, es decir: número de ciclos por instrucción = 1.

Instrucciones MIPS

Instrucciones aritméticas y lógicas

➤ Instrucciones aritméticas: tres registros como operandos.

Instrucción	Ejemplo	Significado	Comentarios
suma	add \$s1, \$s2, \$s3	$\$s1 = \$s2 + \$s3$	Tres operandos, detecta overflow
resta	sub \$s1, \$s2, \$s3	$\$s1 = \$s2 - \$s3$	Tres operandos, detecta overflow
suma inmediata	addi \$s1, \$s2, 100	$\$s1 = \$s2 + 100$	Detecta overflow
suma sin signo.	addu \$s1, \$s2, 100	$\$s1 = \$s2 + \$s3$	Tres operandos, no detecta overflow
resta sin signo.	subu \$s1, \$s2, 100	$\$s1 = \$s2 - \$s3$	Tres operandos, no detecta overflow
multiplicación	mul \$s1, \$s2	$(Hi, Lo) = \$s1 \times \$s2$	Producto 64 bits en (Hi,Lo)
división	div \$s1, \$s2	$Lo = \$s1 / \$s2$ $Hi = \$s1 \text{ mod } \$s2$	Lo = división entera Hi = resto
Mover desde Hi	mfhi \$s1	$\$s1 = Hi$	Recupera valor de Hi
Mover desde Lo	mflo \$s1	$\$s1 = Lo$	Recupera valor de Lo

➤ Instrucciones lógicas

Instrucción	Ejemplo	Significado	Comentarios
and	and \$s1, \$s2, \$s3	$\$s1 = \$s2 \& \$s3$	Tres operandos
or	or \$s1, \$s2, \$s3	$\$s1 = \$s2 \$s3$	Tres operandos
and inmediato	andi \$s1, \$s2, 100	$\$s1 = \$s2 \& 100$	Tres operandos
or inmediato	ori \$s1, \$s2, 100	$\$s1 = \$s2 100$	Tres operandos
desplazamiento lógico izquierda	sll \$s1, \$s2, 10	$\$s1 = \$s2 \ll 10$	Despl. constante
desplazamiento lógico derecha	srl \$s1, \$s2, 10	$\$s1 = \$s2 \gg 10$	Despl. constante

3. Continuación Tema 2: paralelización a nivel de instrucción

Instrucciones de transferencia y saltos

- **Instrucciones de transferencia: direccionamiento base más desplazamiento. El primer operando no siempre es el destino**

Instrucción	Ejemplo	Significado	Comentarios
load word	lw \$s1, 100(\$s2)	\$s1=Memory[\$s2+100]	Word de memoria a registro
store word	sw \$s1, 100(\$s2)	Memory[\$s2+100]=\$s1	Word de registro a memoria
load byte	lb \$s1, 100(\$s2)	\$s1=Memory[\$s2+100]	Byte de memoria a registro
store byte	sb \$s1, 100(\$s2)	Memory[\$s2+100]=\$s1	Byte de registro a memoria
load upper immediate	lui \$s1, 100	\$s1=100*2 ¹⁶	Carga sobre los 16 bits sup.

- **Instrucciones de salto condicionales o incondicionales que modifican el valor del registro contador de programa (PC)**

Instrucción	Ejemplo	Significado	Comentarios
branch on equal	beq \$s1, \$s2, 25	if (\$s1==\$s2) goto PC=PC+4+100	Relativo al PC
branch on not equal	bne \$s1, \$s2, 25	if (\$s1!=\$s2) goto PC=PC+4+100	Relativo al PC
set on less than	slt \$s1, \$s2, \$s3	if (\$s2<\$s3) \$s1=1 else \$s1=0	(instrucción aritmética)
set on less than imm.	slti \$s1, \$s2, 100	if (\$s2<100) \$s1=1 else \$s1=0	Complemento a dos
set on less than imm. unsigned	sltiu \$s1, \$s2, 100	if (\$s2<100) \$s1=1 else \$s1=0	Binario natural
jump	j 2500	PC=2500*4	Absoluto
jump register	j \$ra	PC=\$ra	Absoluto
jump and link	jal 2500	\$ra=PC; PC=2500*4	Absoluto

Formato de instrucción

Cada instrucción debe ser de tipo R, I o J; esto se verifica en los primeros seis bits (de mayor peso).

Teniendo en cuenta el tamaño de palabra de 32 bits se tiene que:

Tipo	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	Observaciones
Tipo R	op	rs	rt	rd	shamt	funct	Formato de instrucción aritmético
Tipo I	op	rs	rt	address/immediate			Formato transferencias, saltos e inmed.
Tipo J	op	dirección destino del salto					Formato instrucción j y jal

A diferencia del año pasado en Fundamentos de Computadores, en esta materia trabajaremos con instrucciones y punto flotante.

Pequeño esquema de resumen

Instrucciones tipo R:

- Aritméticas
- Lógicas

Instrucciones I:

- Saltos condicionales -> inmediato

Instrucciones J:

- Salto incondicional

Etapas por instrucción

Como se mencionó previamente, en el MIPS las instrucciones se ejecutan en cada ciclo de reloj. Sin embargo, dependiendo del tipo de instrucción se requiere pasar por algunas etapas de las indicadas en 1*.

Instrucción	Mem. Instr.	Lectura Reg.	ALU ALU	Mem. Datos	Escritura Reg.	Total
j dir	X					1 etapa
beq \$t0, \$t1, dir	X	X	X			3 etapas
add \$t0, \$t1, \$t2	X	X	X		X	4 etapas
sw \$t0, (\$t1)	X	X	X	X		4 etapas
lw \$t0, (\$t1)	X	X	X	X	X	5 etapas

Como se puede observar, al haber en muchos casos un desaprovechamiento de ese ciclo de reloj, ya que el número de etapas requeridas es muy distinto entre instrucciones; se pierde tiempo (unas instrucciones estarán paradas al sobrar etapas).

Para reducir el ciclo de reloj hay dos opciones: circuitos mejores o cambiar hardware (segmentación).

Segmentación consiste en ejecutar etapas por separado. Es decir, una instrucción puede ir ejecutando su etapa 1 a la vez que otra instrucción ejecuta la 3, otra instrucción la 4 y así sucesivamente. Así, aumenta el número de instrucciones que se pueden ejecutar por segundo y no se cambia el tiempo que se usa para realizar cada operación (se utiliza la misma arquitectura).

**Extra: hay que recordar:

**A frecuencia más baja = más lento todo.

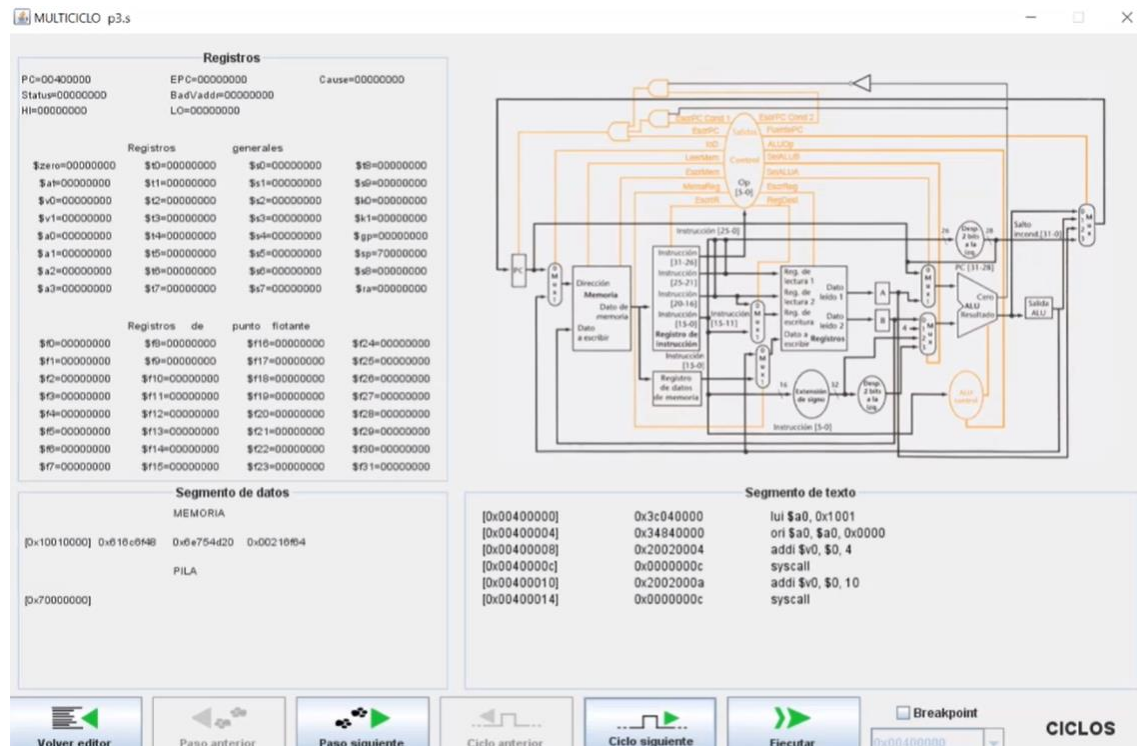
**Frecuencia = 1/ciclo de reloj

Implementación multiciclo y segmentación

- La implementación multiciclo aprovecha mejor los tiempos variables.
- En lugar de ejecutar las instrucciones en una unidad combinacional, es necesario subdividir la unidad combinacional y colocar registros entre etapas.
- Las unidades funcionales ya no están activas continuamente. Podría lanzarse una instrucción y mientras se ejecuta ir lanzando la siguiente: pipelining (segmentación).
- Problemas:
 - No todas las instrucciones terminan al mismo tiempo.
 - Se producen conflictos (hazards) cuando dos instrucciones necesitan usar la misma unidad funcional.
 - Problemas en los saltos...

3. Continuación Tema 2: paralelización a nivel de instrucción

Ejemplo arquitectura multiciclo en simulador



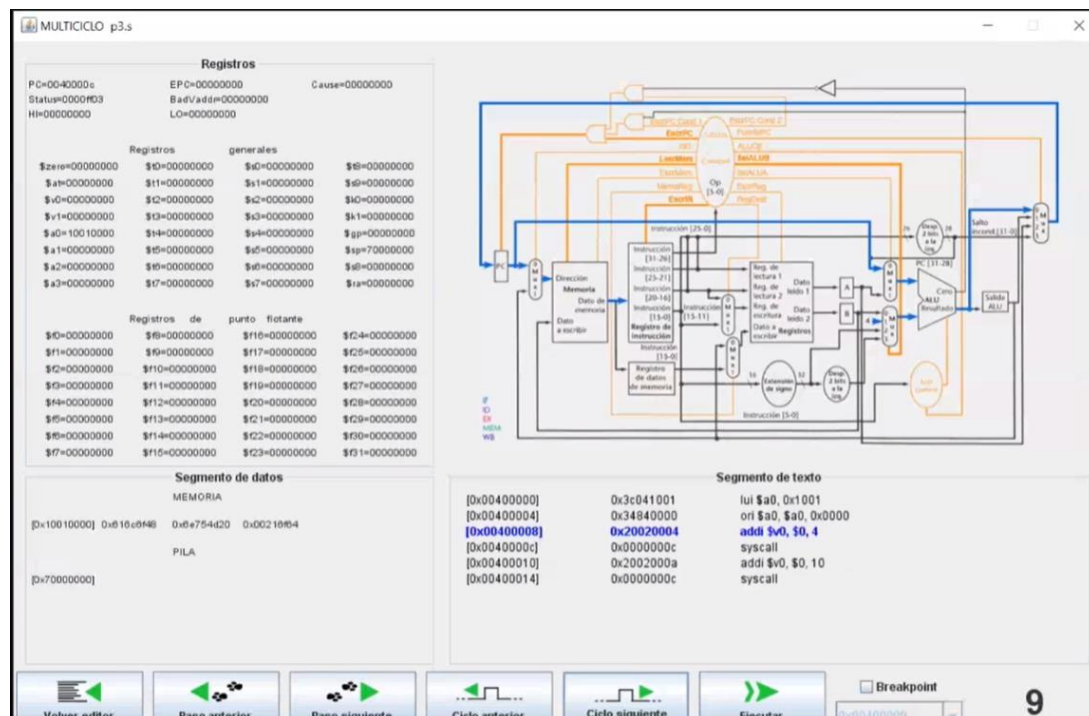
*Se nota parecida, pero para ir pasando de instrucción a instrucción está utilizando más de un ciclo.

Por ejemplo:

Instrucción **addi**:

1. Primer ciclo

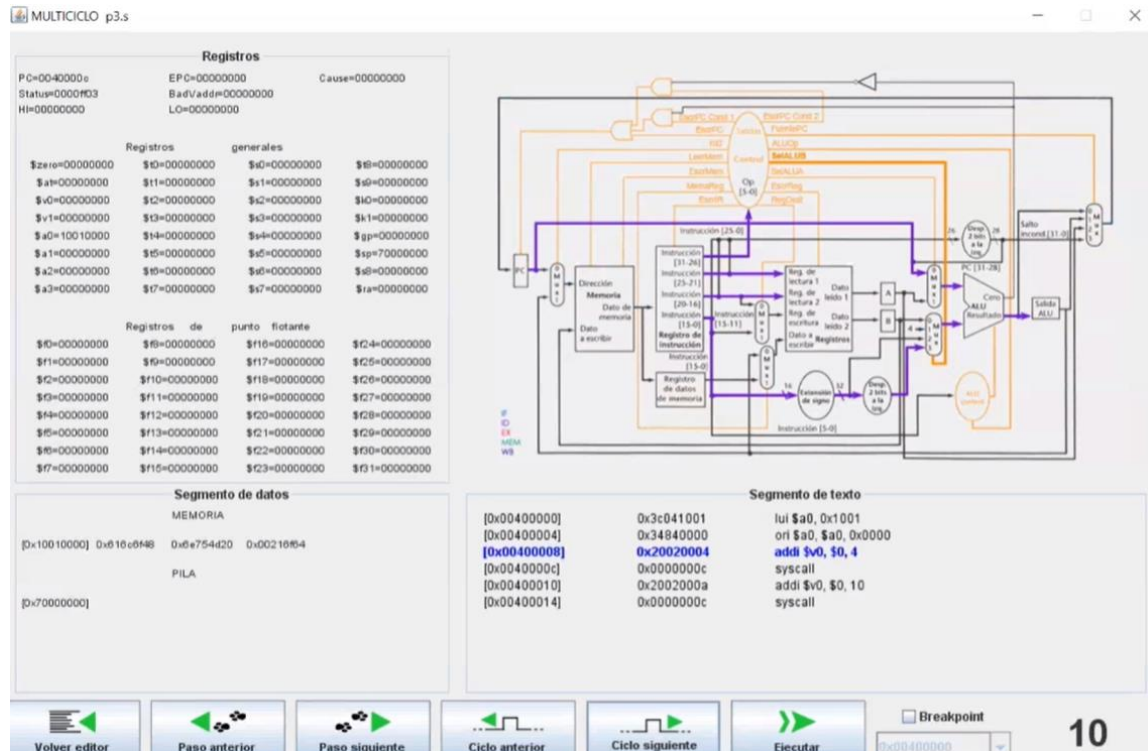
Se busca instrucción y se decodifica.



3. Continuación Tema 2: paralelización a nivel de instrucción

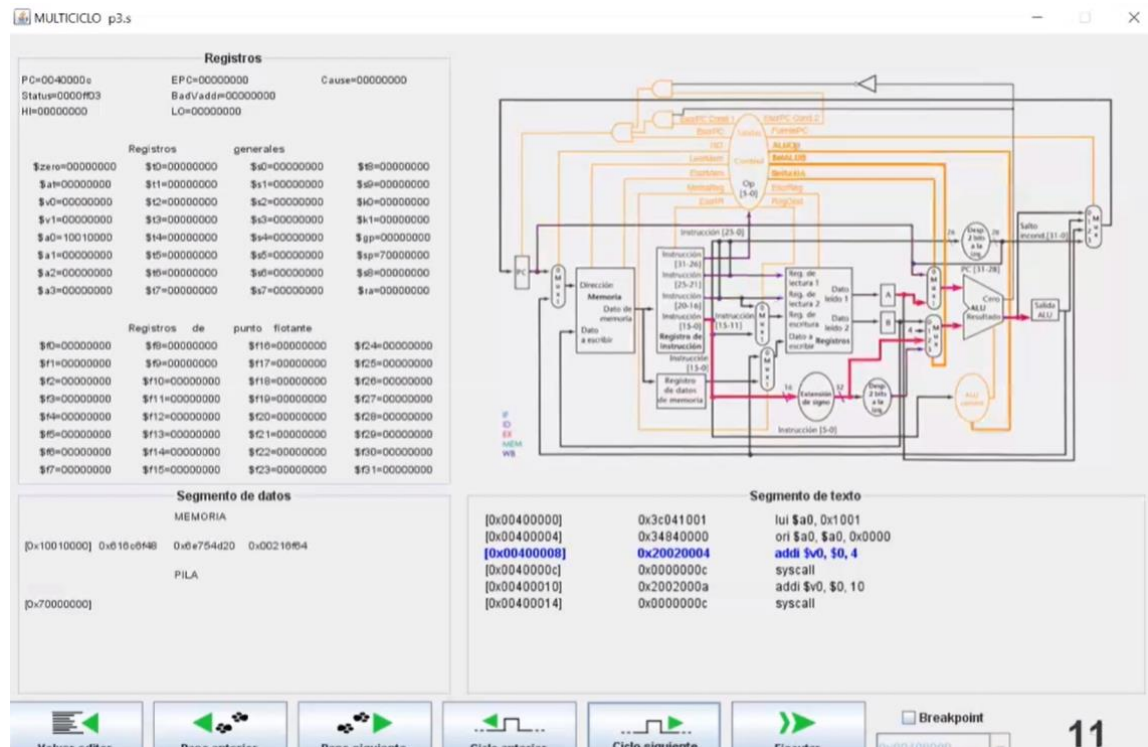
2. Segundo

Lectura de registros y operandos listos para ser ejecutados.



3. Tercero

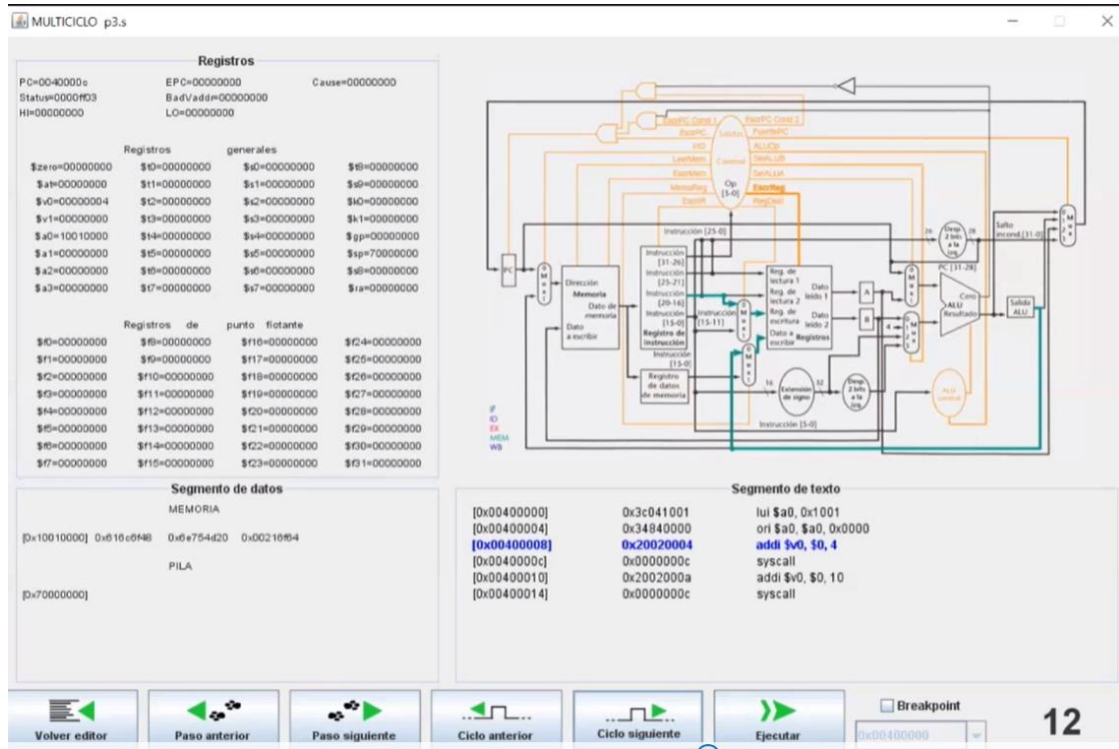
Extensión de signo y ejecución de la ALU.



3. Continuación Tema 2: paralelización a nivel de instrucción

4. Cuarto

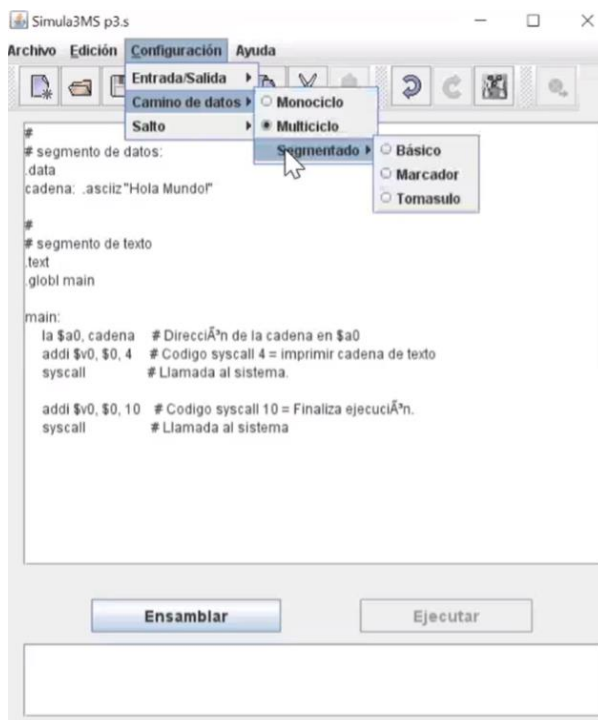
Resultado ALU y guardarlo en los registros.



****pausa de la clase para Dora decir “cachi la mar” porque se le cierra el simulador****

EXTRA:

Para cambiar la configuración del simulador:

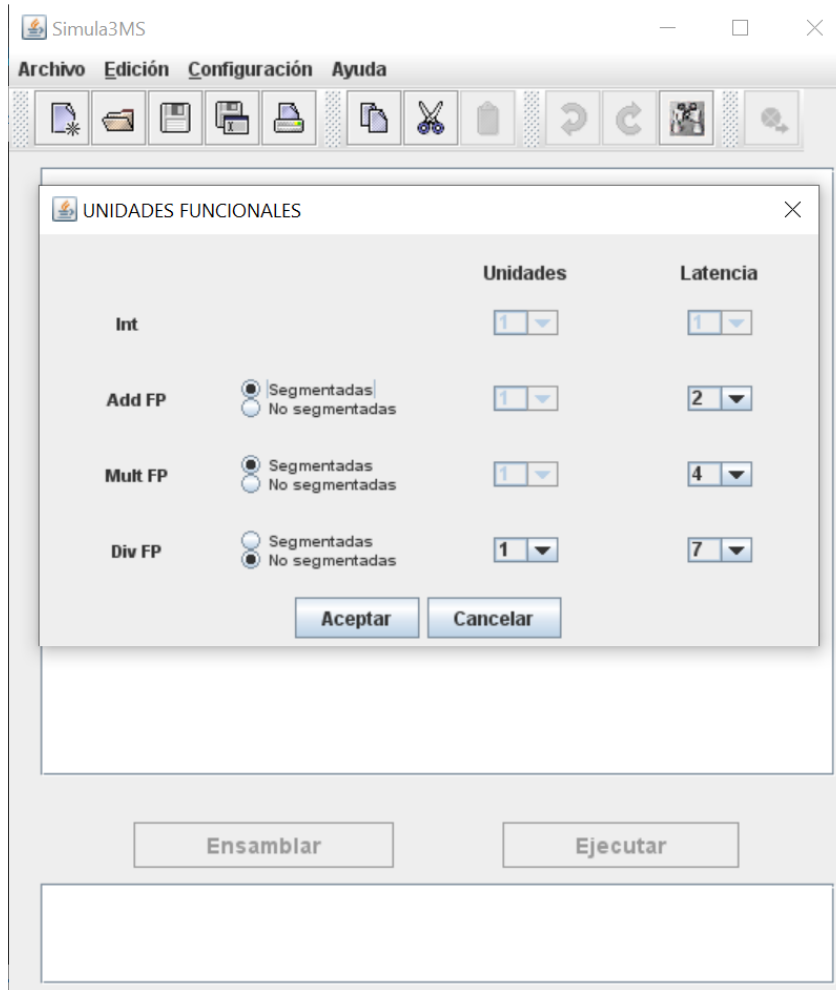


Configuración-Camino de datos-*elegir tipo*

3. Continuación Tema 2: paralelización a nivel de instrucción

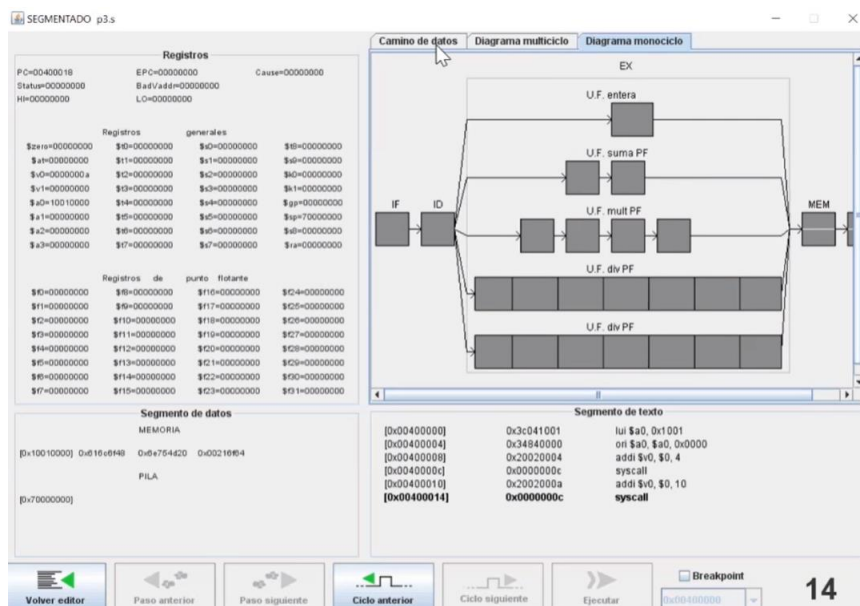
Ejemplo arquitectura segmentación en simulador

Al darle Segmentación-Básica aparece una pantalla como la siguiente:



La que permite diferenciar entre tipos de instrucción y elegir independientemente.

Ref1: Al hacer cambios y ejecutar un código aparecerá algo así:

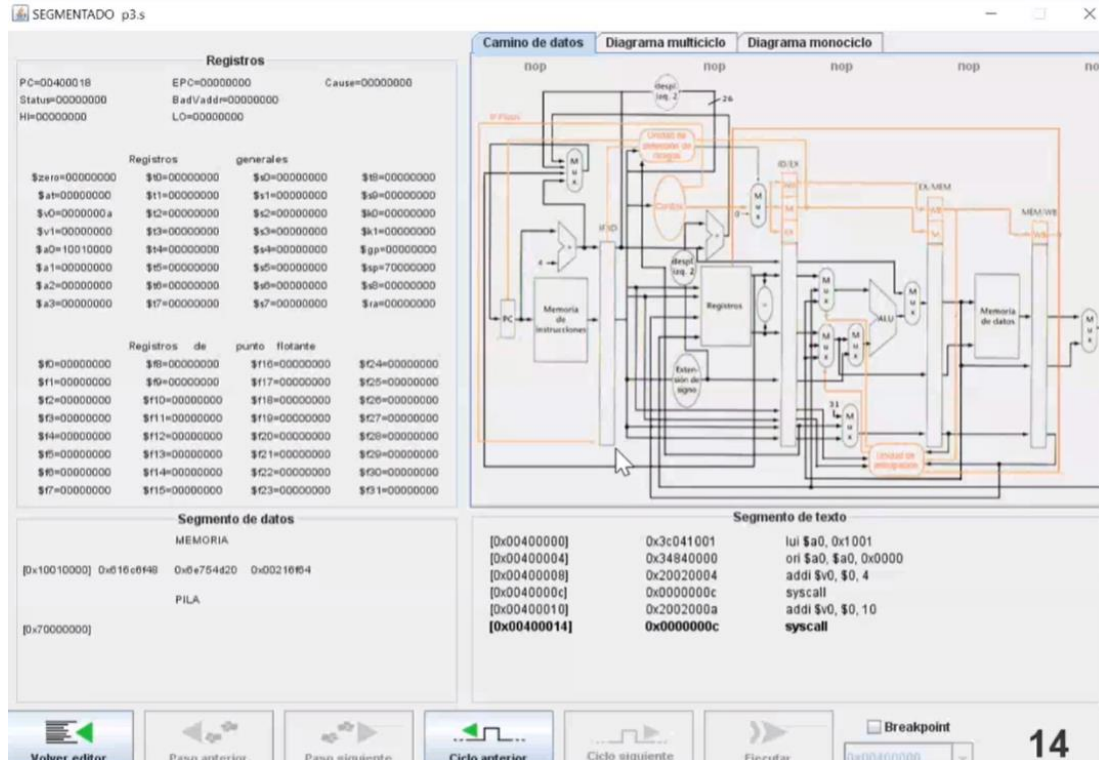


3. Continuación Tema 2: paralelización a nivel de instrucción

Que es un breve esquema de lo cambiado anteriormente y aquí se identifican las distintas etapas.

La ejecución en sí, son las cinco ramificaciones que aparecen, son distintas porque para cada tipo de instrucción tiene un retardo distinto.

En cambio de datos podemos observar nuevamente el camino de datos del código ejecutado.



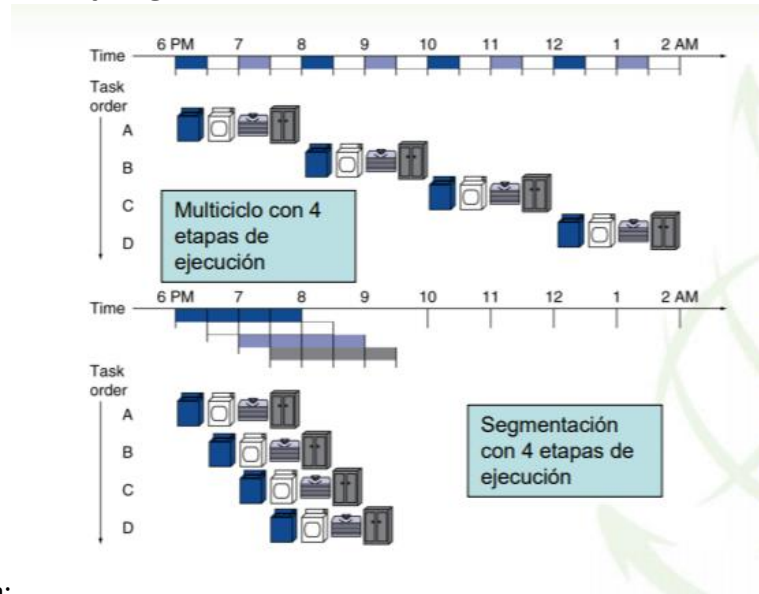
A diferencia de cuando no se había modificado a **segmentado**, ahora aparecen unas barras entre la figura de la ejecución (nótese por ejemplo la señalada con el mouse).

Estas barreras son registros que impiden que pase la información entre una etapa y otra para separar la parte de búsqueda de la instrucción, de la búsqueda de registros, de la ejecución de la memoria de datos y la parte de escritura final.

*Estos segmentos realmente los tenemos en todos los ordenadores.

De esta forma se ahorra tiempo, porque se ejecutan cosas distintas a la vez sin preocupación de que una dependa de la otra.

Diferencia multiciclo y segmentación



Ejemplo Patterson:

$CPI=1$ quiere decir ciclos por instrucción es 1; por esto multiciclo se define como $CPI>1$.

En el caso de la segmentación las instrucciones siguen ejecutándose en más ciclos (en el ejemplo: 4). Pero luego del primer ciclo, la siguiente instrucción puede ejecutar su primer ciclo a la vez que la instrucción anterior ejecuta el segundo suyo; y así sucesivamente.

De esta forma, en el ejemplo se observa que le lleva 16 ciclos (4×4) ejecutar las instrucciones mientras que por segmentación le toma 7 al conseguir una superposición.

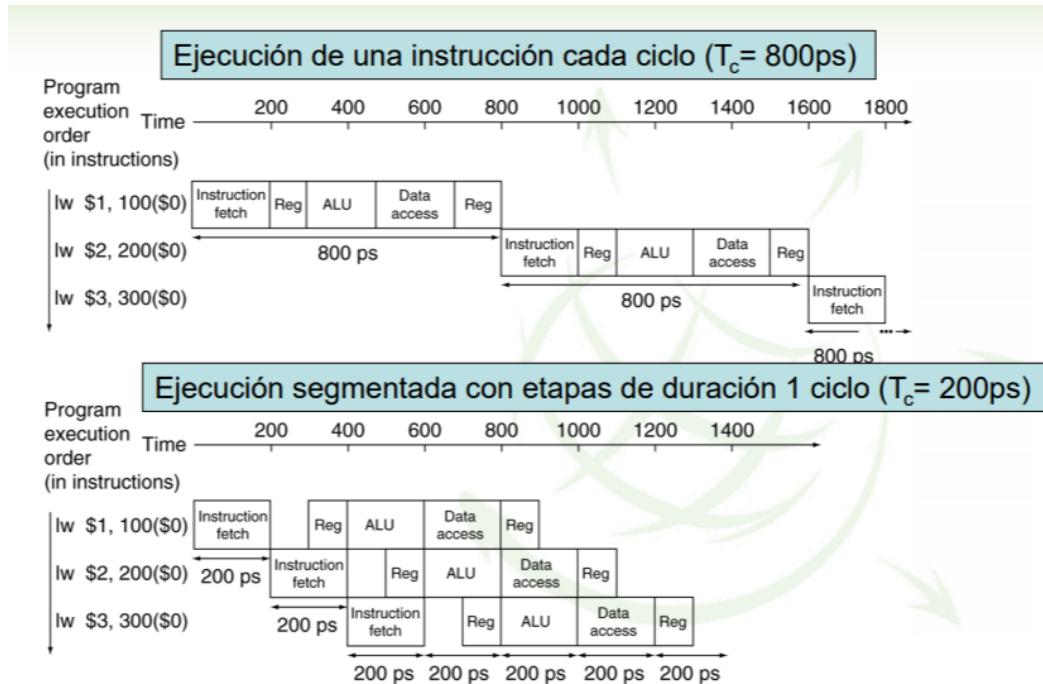
La segmentación demuestra una clara ventaja de aprovechamiento de hardware y el tiempo.

- La ejecución multiciclo con número de ciclos variable permite acelerar la ejecución de instrucciones permitiendo que unas acaben antes que otras.
- La segmentación exige lanzar a ejecución una instrucción antes de que acabe la anterior.
- Se busca aprovechar todas las etapas a la vez para ir progresando en la ejecución de diferentes instrucciones simultáneamente.

****** En un cauce segmentado idealmente se inicia una instrucción en cada ciclo de reloj. No siempre será posible como veremos.

Diferencia monociclo y segmentación

Ejemplo:



Lo que más tarda en ejecutarse en monociclo dura 200 ps, así que para segmentación se asigna el tiempo de ciclo de 200ps y en los casos que dure menos se observa que sobra tiempo (en el caso de los registros).

A los 1400 ps en el caso de monociclo NO se ejecutaron dos instrucciones enteras ya que cada una requiere de 800ps; pero en segmentación acabaron 3 instrucciones enteras, aproximadamente 466 ps por instrucción.

PROBLEMA CON SEGMENTACIÓN: requiere hardware adicional: registros entre etapas.

Segmentación con enteros y con punto flotante

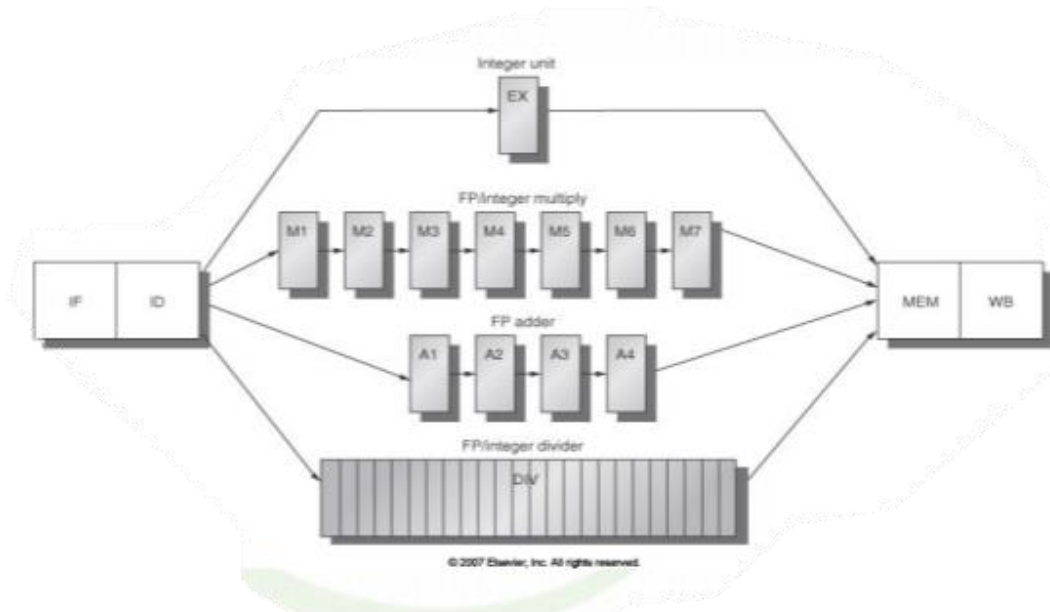
En el caso de segmentación con punto flotante es igual que con enteros, pero con diferencia de que la etapa de ejecución tiene diferentes retardos. Como se ha indicado en * [Ref1](#) por ello hay ALU específicas, aunque hay algunas que se repiten.

- Las instrucciones en punto flotante tienen la misma segmentación que las enteras, pero con las siguientes modificaciones:
 - La etapa EX tiene diferente latencia dependiendo de la instrucción
 - Existen diversas unidades funcionales para cada tipo de operación

En general: la complejidad de instrucción afecta a la complejidad del hardware y las etapas de segmentación.

*****PIPELINE**: conjunto de las diferentes etapas de segmentación.

3. Continuación Tema 2: paralelización a nivel de instrucción



Si todas las etapas tienen la misma duración (correspondiendo a la más grande de las etapas), en general:

$$\text{Tiempo de instrucciones con segmentación} = \frac{\text{tiempo instrucciones sin segmentación}}{\text{número de etapas}}$$

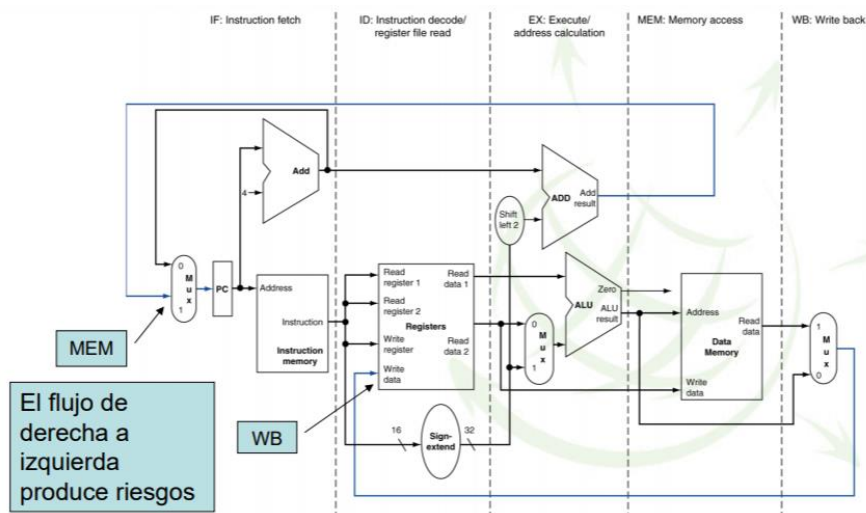
por otro lado, si la duración es distinta, la aceleración será más baja.

La aceleración, la velocidad que se gana al ejecutar con segmentación; se debe al aumento de la llamada **productividad**, es decir; número de instrucciones/unidad de tiempo.

** La latencia (el tiempo total que tarda en ejecutarse cada instrucción por separado) no disminuye.

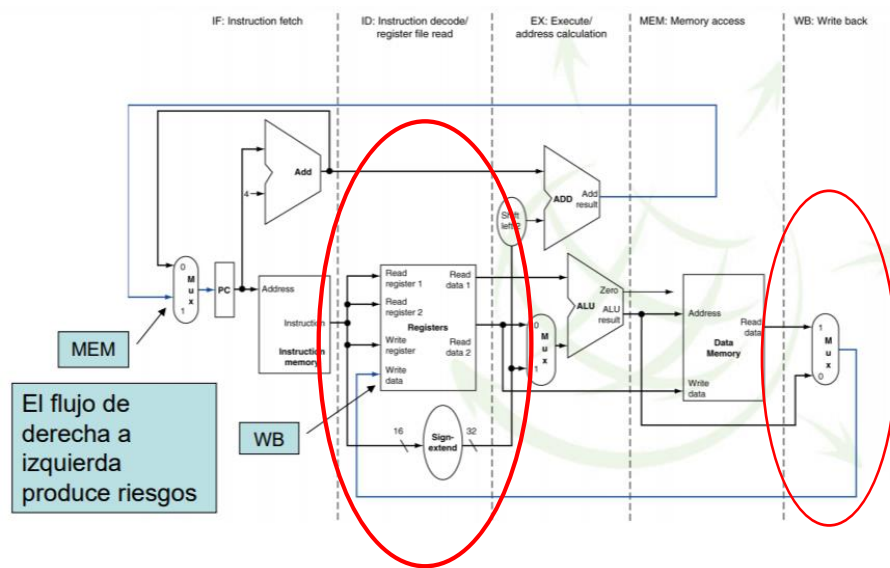
- CPI=1 y se cumple ciclo_no_segmentado > ciclo_segmentado
- Ciclo_no_segmentado = N x ciclo_segmentado

Arquitectura MIPS segmentado



3. Continuación Tema 2: paralelización a nivel de instrucción

Se necesita hardware adicional (frente al multiciclo) para soportar la ejecución de varias etapas simultáneamente.



**Hay que tener cuidado cuando hay etapas de sobrescribir (véase en las etapas resaltadas en la figura); ya que si hay otra instrucción ejecutándose esto no sería posible. Por esto se establecen los registros fijos entre etapas.

Los registros al final de las etapas ayudan a evitar los llamados **riesgos** (posibilidad de que los resultados sean erróneos).

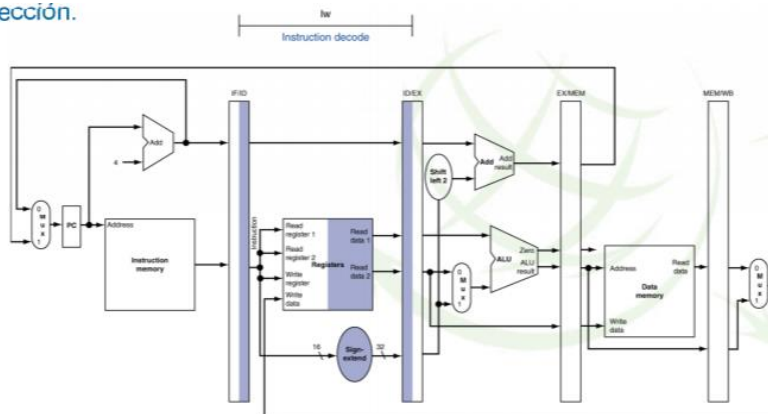
Mientras más etapas haya, más registros. Hay que tener en cuenta que cargar en los registros es un tiempo finito, que en general no es mucho, pero a medida que hayan más registros (etapas) se irá acumulando.

Además, si el pipeline es profundo el número de registros será muy alto y la frecuencia de señal de reloj también tiene que ser muy alta porque cada vez hay trocitos más pequeños, período más pequeño de la señal de reloj porque cada etapa se busca hacer en un ciclo de reloj; así, frecuencia más alta=consumo de potencia más alta.

Conclusión, hay que tener cuidado con la cantidad de etapas en la que se divide y para cada procesador hay que tener cuidado al equilibrar el tamaño de las etapas a la vez para controlar cuánto va a durar cada etapa (por lo tanto, frecuencia y consumo de potencia).

Registros de separación en el MIPS

dirección.



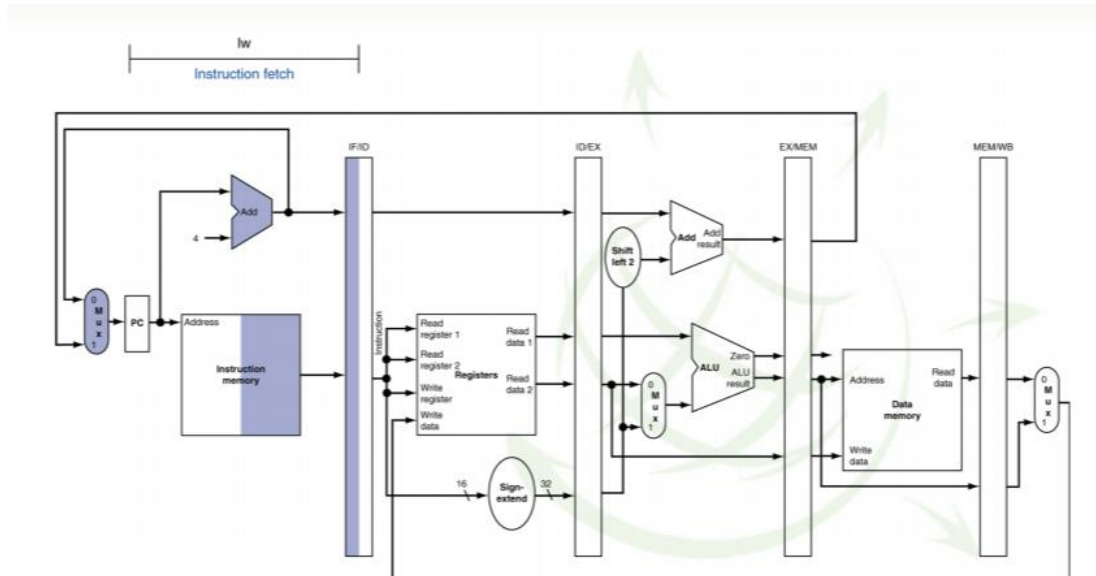
3. Continuación Tema 2: paralelización a nivel de instrucción

Para las instrucciones de carga y almacenamiento hay que leer dos registros.

Decodificación de la instrucción: ID

- Para Store (carga), lectura de dos registros. El inmediato es el desplazamiento para calcular la dirección de almacenamiento y se le extiende el signo para operar en la ALU.
Lw \$t1, 20 (\$t0)
- Para Load (almacenamiento), se hace lectura de un registro (base de la dirección) y se extiende en signo el inmediato, que es el desplazamiento para calcular la dirección.

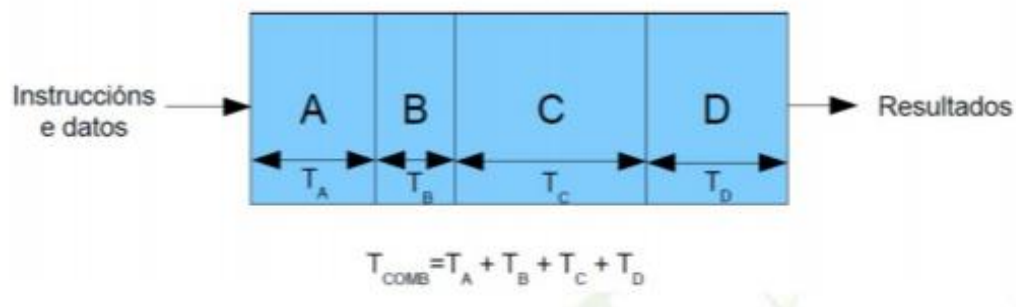
Búsqueda en la instrucción: IF



Mientras se incrementa el valor del PC, se lee la instrucción.

Camino de datos segmentado:

De manera genérica una unidad combinacional de ejecución de 4 etapas tardará un tiempo suma de los tiempos de las 4 etapas en ejecutar una instrucción.

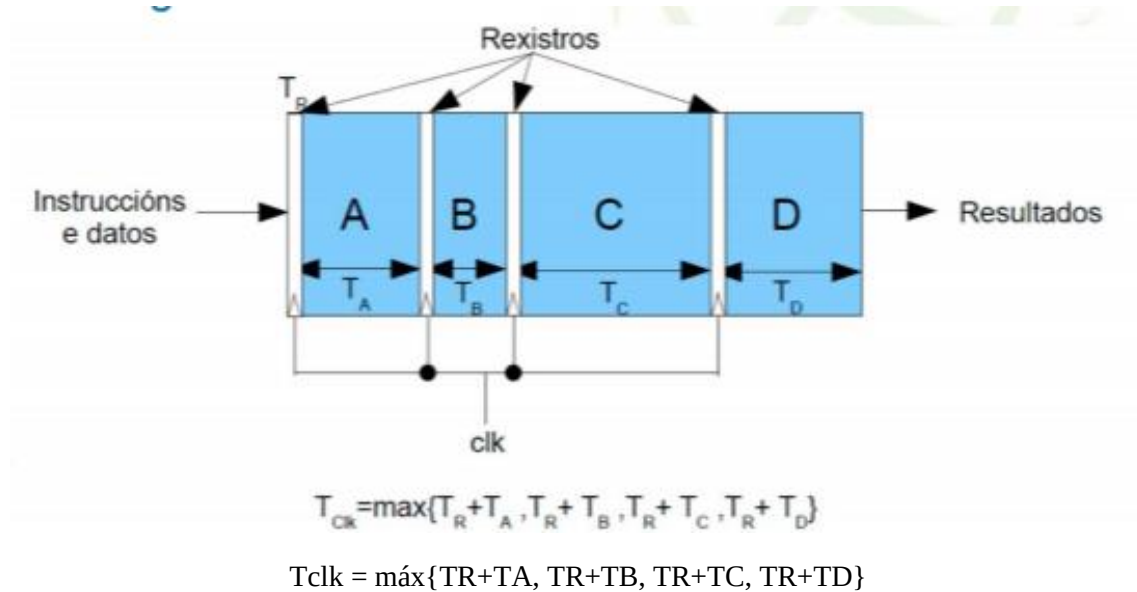


Si hay cuatro etapas (o cinco, es un ejemplo). Supongamos en la figura que es el camino de un procesador, con etapas más cortas que otras y el tiempo total (convencional) será:

$$T_{comb} = T_A + T_B + T_C + T_D$$

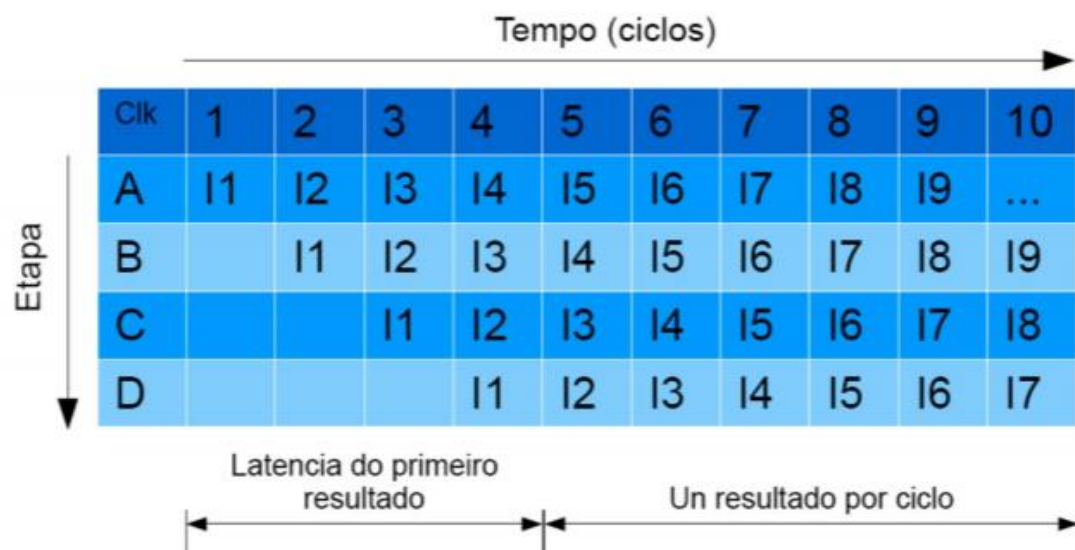
La unidad equivalente segmentada tiene registros por el medio:

3. Continuación Tema 2: paralelización a nivel de instrucción



El tiempo de ciclo es el tiempo de la etapa que más tarde más el tiempo de registro.

- En cada ciclo de reloj se solapa la ejecución de diferentes instrucciones I1, I2, I3,...
- Tras una latencia inicial igual al tiempo de ejecución de la primera instrucción, se obtiene una instrucción por ciclo.
- Conviene que todas las etapas del pipeline tarden un tiempo similar en ser ejecutadas.



Ejercicio de ejemplo

***en cursiva se encuentran explicaciones de Dora del ejercicio, pero NO vienen en el enunciado.*

Procesador no segmentado.

- Ciclo de reloj: 1 ns.
- 40% operaciones ALU → 4 ciclos. *(40% operaciones se ejecutan en la ALU)*
- 20% operaciones de bifurcación → 4 ciclos. *(20% operaciones de salto)*

3. Continuación Tema 2: paralelización a nivel de instrucción

- 40% operaciones de memoria → 5 ciclos.
- Sobrecoste de segmentación → 0.2 ns. (por el uso de los registros)

**¿Qué aceleración se obtiene con la segmentación?

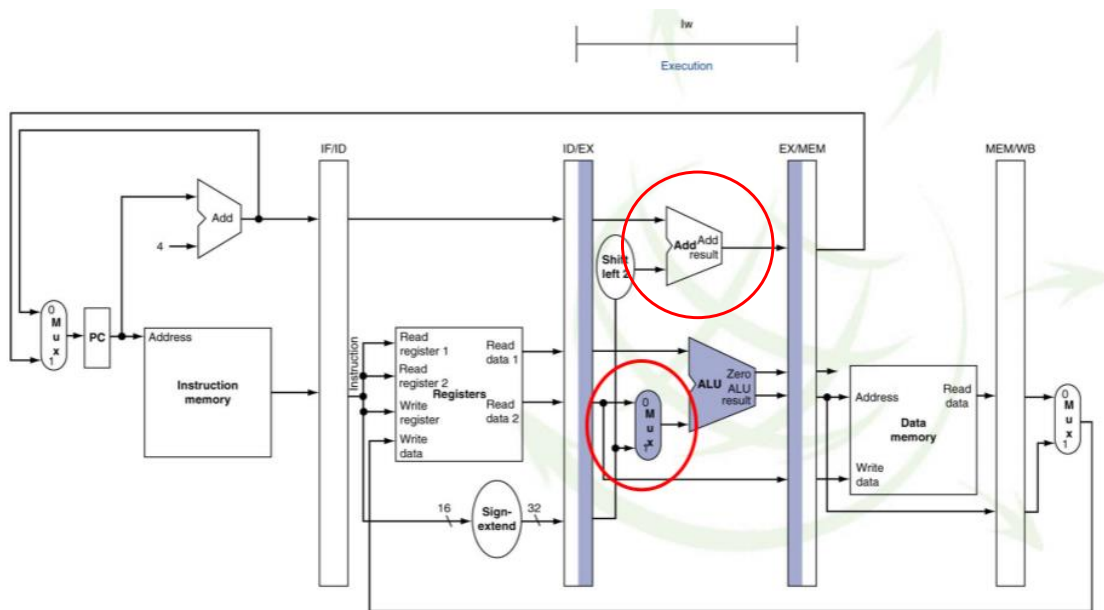
$$t_{orig} = \text{cicloloj} \times \text{CPI} = 1\text{ns} \times (0.6 \times 4 + 0.4 \times 5) = 4.4\text{ns}$$

$\downarrow$ 60% 4 ciclos $\downarrow$ 40% 5 ciclos

$$t_{nuevo} = 1\text{ns} + 0.2\text{ns} = 1.2\text{ns}$$

$$S = 4.4\text{ ns} / 1.2\text{ ns} = 3.7$$

Ejecución de la instrucción: EX

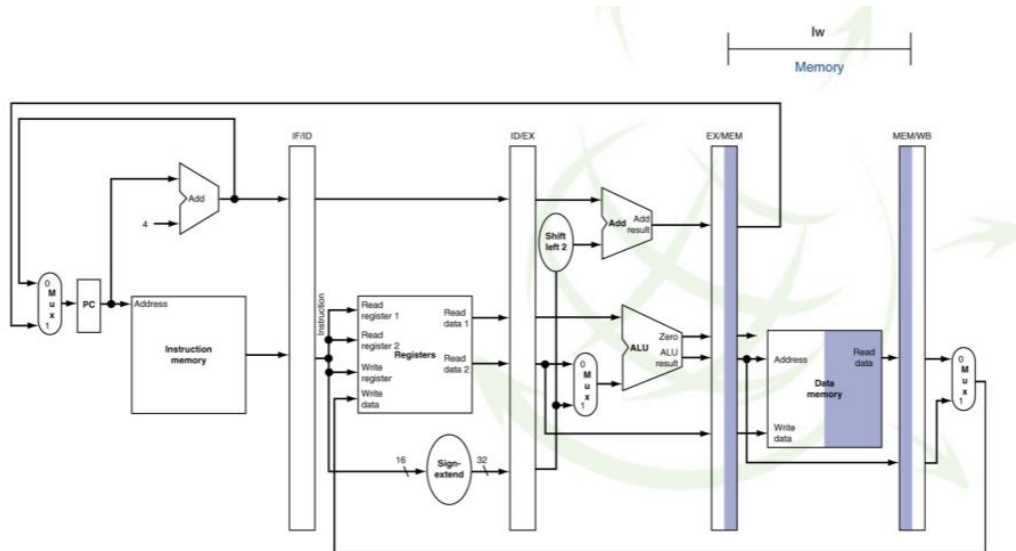


El MUX selecciona el valor inmediato con el signo extendido y la ALU calcula la dirección sumando base más desplazamiento, para poder ir a memoria y coger el resultado correcto.

Memoria de la instrucción: MEM

Se realiza la lectura de la memoria (caché de datos) usando la dirección calculada en la ALU y el contenido del registro que se va a almacenar en memoria que se calculó en la etapa decodificación y que no se modificó en la etapa EX de ejecución.

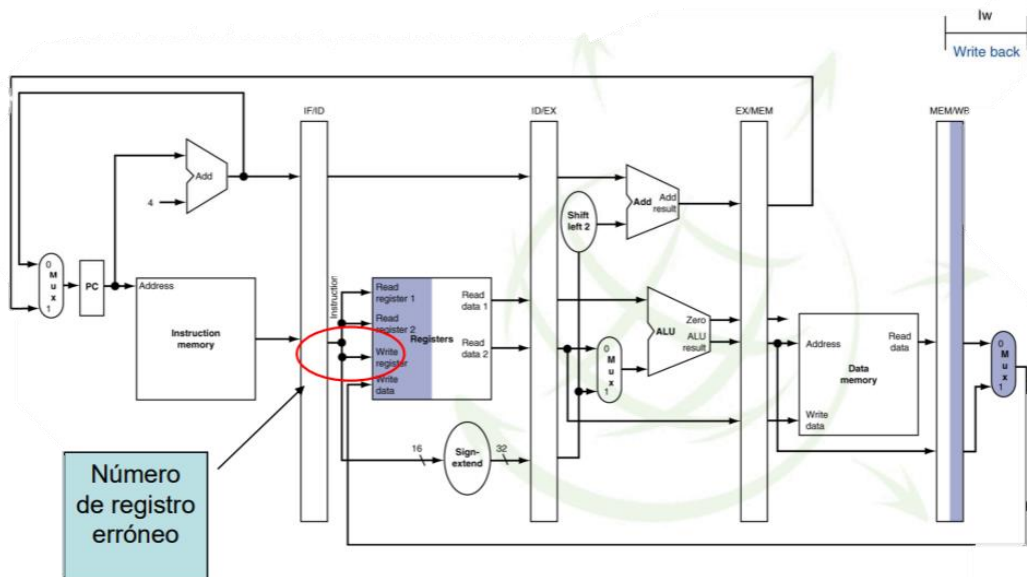
3. Continuación Tema 2: paralelización a nivel de instrucción



Postescritura de la instrucción: WB

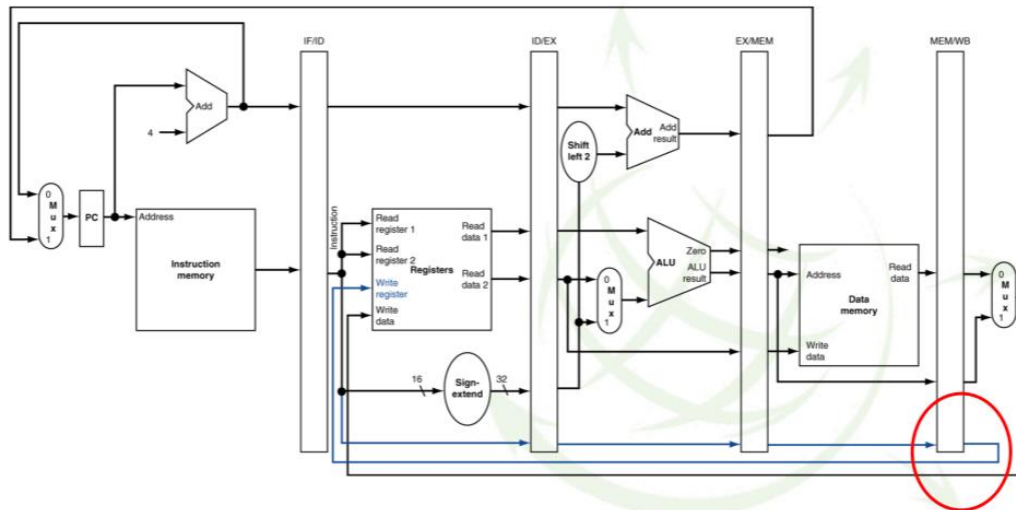
El multiplexor selecciona la salida de memoria y escribe el valor en el banco de registros completando la instrucción Load.

Pero, con el camino de datos que teníamos podemos afectar a la ejecución de otra instrucción que esté en etapa decodificación.



Por ello, se realiza la siguiente modificación:

Camino de datos corregido para hacer WB de Load



****Pregunta de Nico que hizo que Dora se planteara toda su existencia como por 10 minutos****



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura de Computadores

18-03-2021



Clase 06

Temas trabajados:

Tema 2: Acaba camino de datos segmentados

Riegos y tratamiento de los riesgos en la ejecución

Índice

1. Introducción.....	3
2. Sobre la práctica 2.....	3
3. Camino de datos segmentados en el MIPS con registros de segmentación (continuación)	3
3.1 EX para LOAD	3
3.2. MEM para LOAD	4
3.3 WB para Load	4
3.4 Camino de datos corregido para hacer WB de Load	5
3.5 EX para Store.....	5
3.6 MEM para Store.....	6
3.7 WB para Store.....	6
3.8 Control del pipeline (simplificado)	7
4. Riesgos en la ejecución (hazards)	7
5. Riesgos de la estructura.....	8
6. Riesgos de datos.....	10
6.1 Tipo Raw	10
6.2 Tipo WAR	10
6.3 Tipo WAW	11
6.4 Soluciones a los riesgos de datos:	11
6.4.1 Ejemplo de bloqueo por riesgos de datos RAW.....	12
6.5 Técnicas de tratamiento de riesgos de datos	13
6.5.1 Software.....	13
6.5.2 Hardware.....	14
6.5.3 Detectar la necesidad de forwarding.....	15
6.5.4 Anticipación, forwarding o bypassing	15
6.5.5 Control de interbloqueo de instrucciones.....	16
6.5.6 Control de bloqueo de instrucciones.....	16
6.5.7 Forwarding en el camino de datos	17
6.5.8 Ejemplo de parada por bloqueo.....	17
6.5.9 Camino de datos con detección de forwarding y de interbloqueo (hazard detection unit)	18
7. Riesgos de control.....	18

1. Introducción

Como siempre, en esta hora Dora, la profesora se ha encargado de repasar conceptos dados en la clase anterior, los cuales no se tratarán en esta bitácora. Además de hablar de algunas aclaraciones sobre la práctica número dos, lo cual se quedarán reflejados en el siguiente apartado.

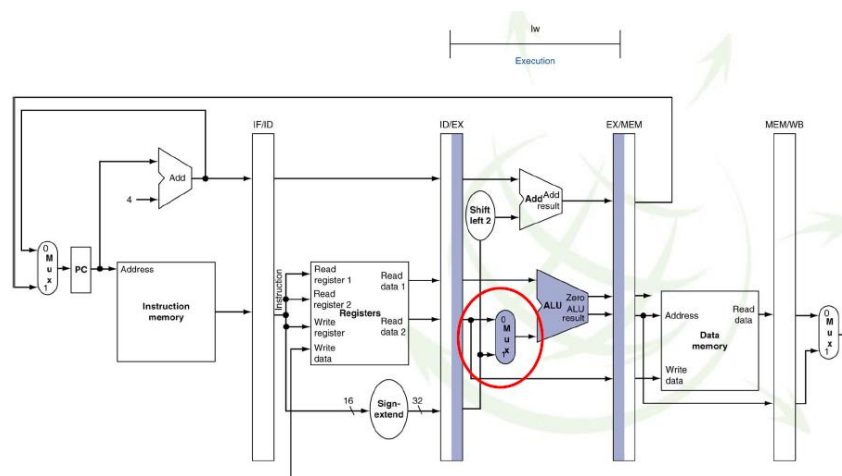
2. Sobre la práctica 2

- La práctica 2, se realizará en 5 sesiones, se va a realizar un programa secuencial el cual será posteriormente mejorado.
- Para mejorarlo se aplicarán cualquiera de las mejoras vistas en la primera sesión de prácticas.
- Cuanto mejor esté hecha la mejora, mejor será la nota.
- Una vez realizada la mejor, se realizará una implementación empleando instrucciones vectoriales en SS3 y en OPEN OMP.
- Consejos de Dora: primero hay que hacer una versión que funcione y que se corresponda al pseudocódigo. Después se hace la versión mejorada, en caso de que nos atascamos con esto, es mejor continuar con la versión SS3 y hacer en otro momento las mejoras.

3. Camino de datos segmentados en el MIPS con registros de segmentación (continuación)

3.1 EX para LOAD

El multiplexor selecciona el inmediato con el signo extendido. La ALU, calcula la dirección sumando base y desplazamiento.

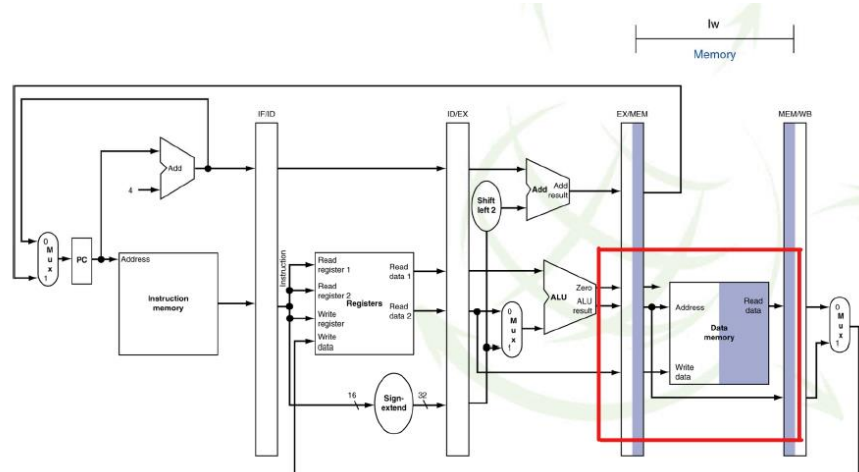


Es una instrucción de carga, tenemos un desplazamiento y un registro (w \$t0, 0(\$s0))

3. Camino de datos segmentados en el MIPS con registros de segmentación (continuación)

3.2. MEM para LOAD

Se realiza la lectura de la memoria (caché de datos) usando la dirección calculada en la ALU y el contenido del registro que se va a almacenar en memoria que se calculó en la etapa ID y que no se modificó en la etapa EX. Es decir, $\$s0 + \text{desplazamiento}$.

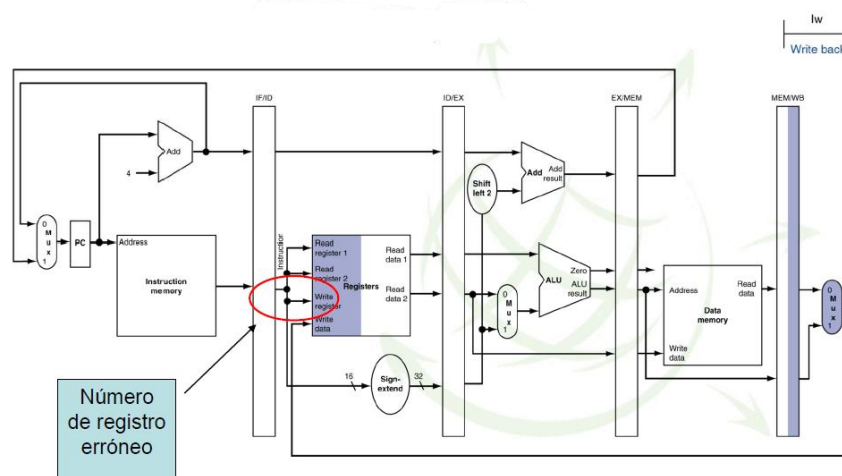


3.3 WB para Load

El multiplexor selecciona la salida de memoria y escribe el valor en el banco de registros complementando la instrucción Load.

Cuidado, ya que con el camino de datos que teníamos podemos afectar a la ejecución de otra instrucción que esté en la etapa ID.

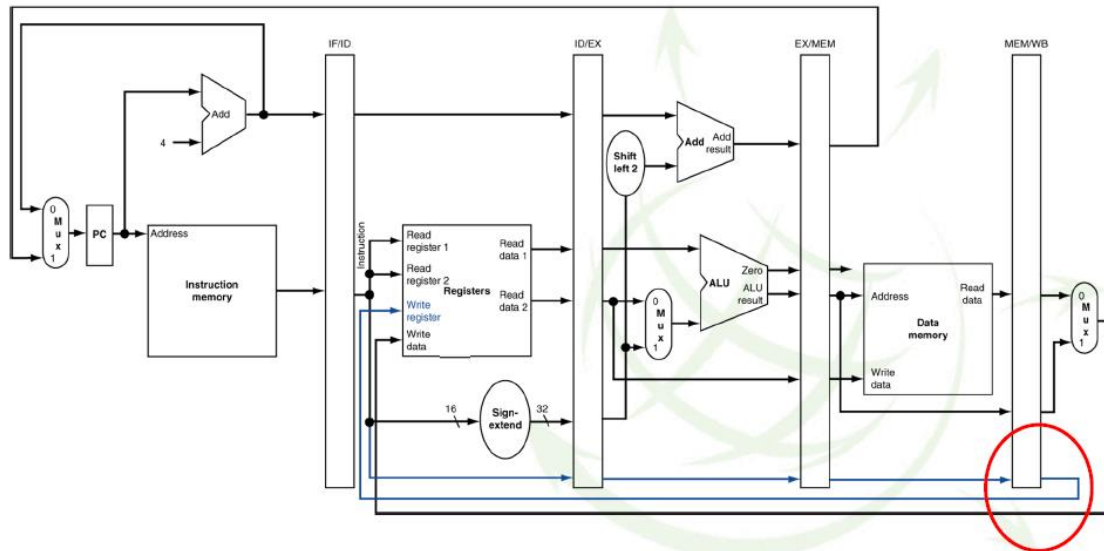
El problema sería el siguiente: si cogemos la información, y la llevamos a la zona marcada, se va a escribir en un registro que tiene se está utilizando en la instrucción que esté en ese momento en fase de decodificación. Por lo que se va a escribir en registro de escritura información sobre información. Por eso la información la llevaremos por otra vía (explicado en el siguiente apartado).



Número de registro erróneo

3. Camino de datos segmentados en el MIPS con registros de segmentación (continuación)

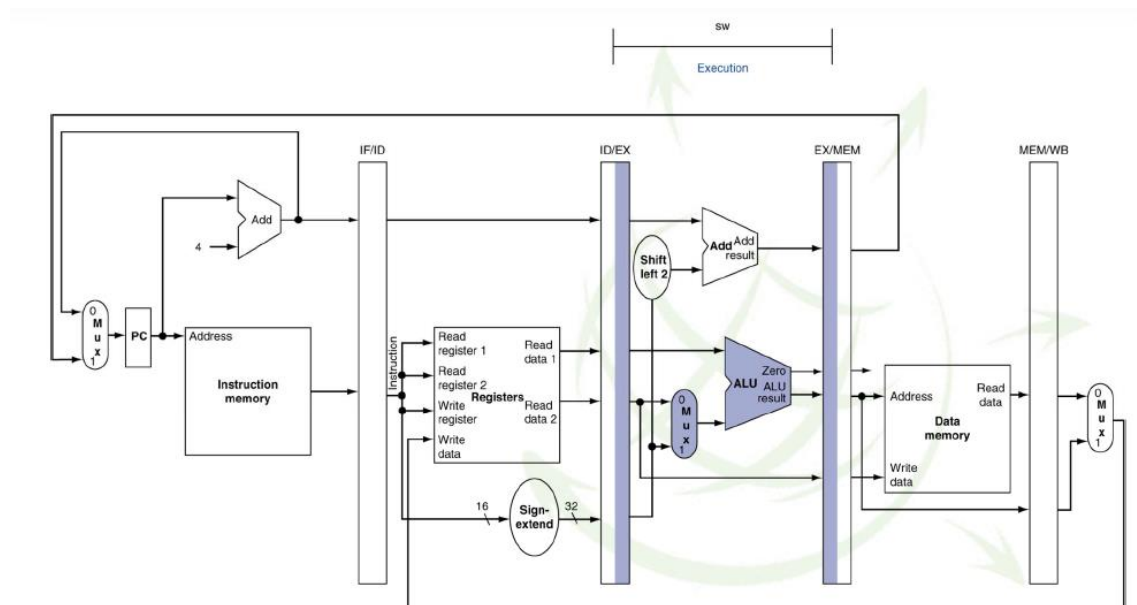
3.4 Camino de datos corregido para hacer WB de Load



El registro que se escribe no se obtiene directamente de la instrucción, sino que se obtiene de la postescritura. Hecho esto, los registros ya nos permiten separar una instrucción de otra.

3.5 EX para Store

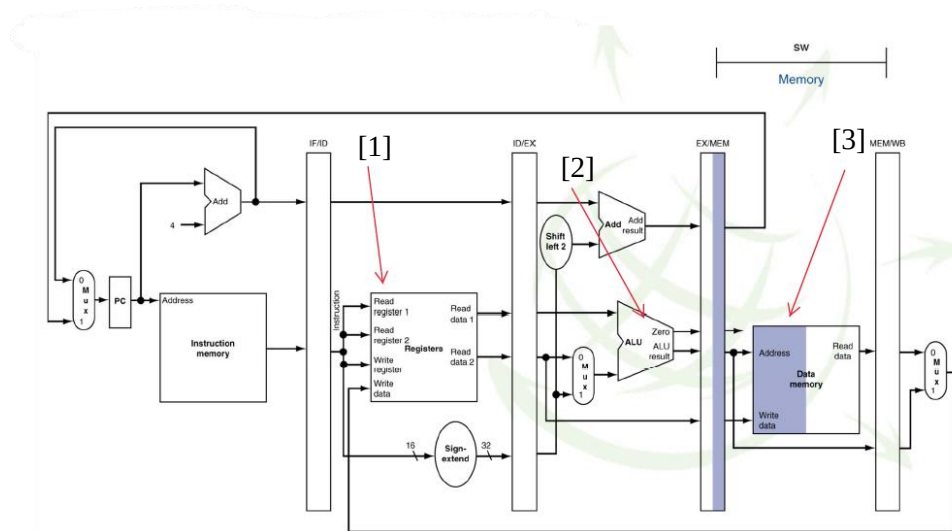
Para Store, el contenido del registro que se va a almacenar en memoria pasa por esta etapa sin ser modificado. Se calcula la dirección base más el desplazamiento.



3. Camino de datos segmentados en el MIPS con registros de segmentación (continuación)

3.6 MEM para Store

Se realiza la escritura de la memoria (caché de datos) usando la dirección calculada en la ALU y el contenido del registro que se va a almacenar en memoria que se calculó en la etapa ID y que no se modificó en la etapa EX. Esta es la última etapa de la instrucción store.



La instrucción sería sw (\$t0, 10(\$s0))

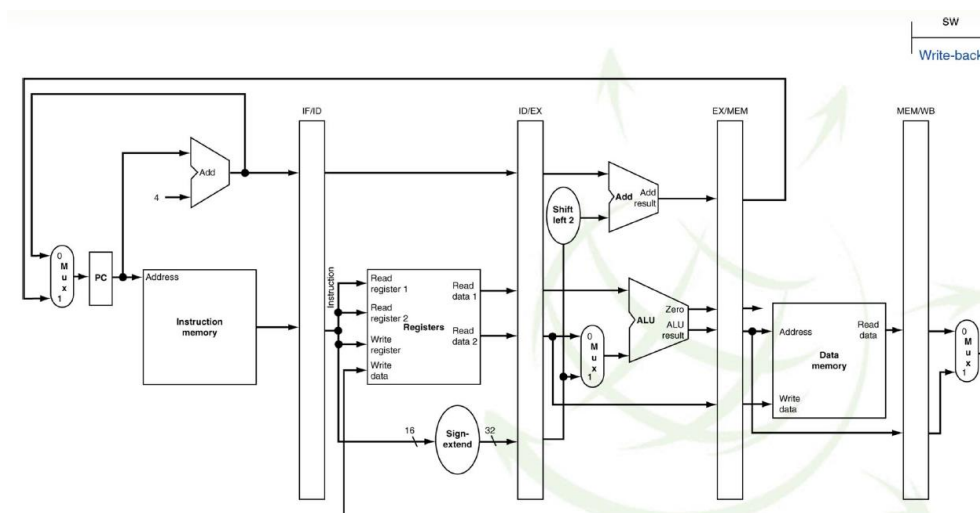
[1] Aquí se lee el contenido del registro \$t0

[2] Aquí se calcula la dirección de memoria

[3] Aquí se lleva la información

3.7 WB para Store

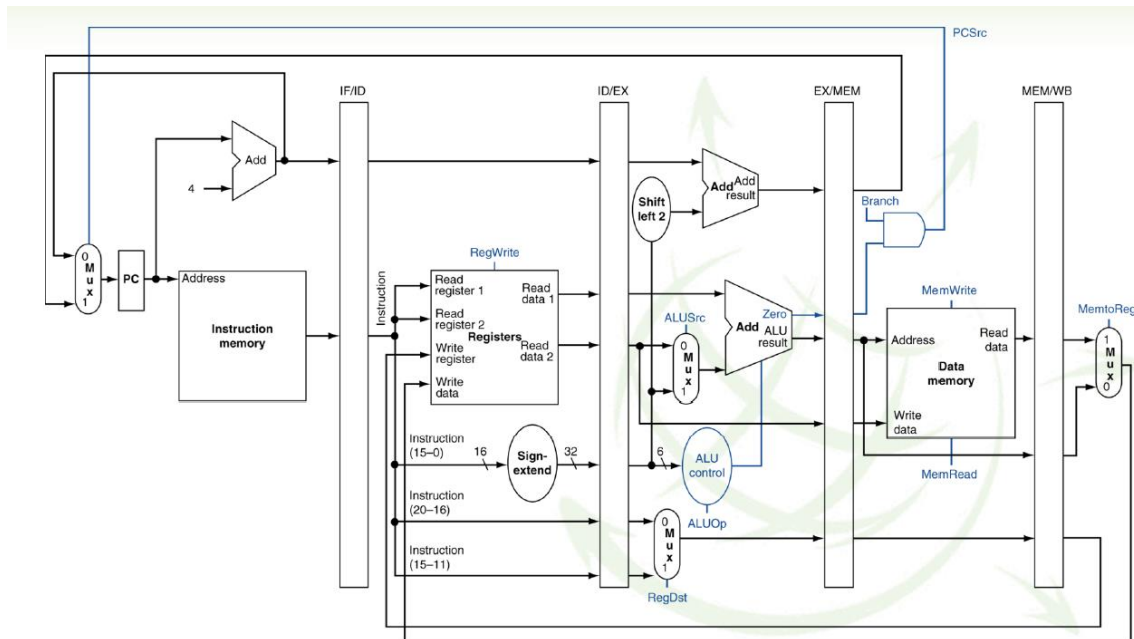
No se hace nada en esta etapa para la instrucción Store.



3.8 Control del pipeline (simplificado)

El control del pipeline (imagen de abajo) tiene registros intermedios, necesario controlarlos mediante líneas de control.

El sistema secuencial síncrono que controla la máquina de ejecución de las instrucciones en un caso de segmentación, es decir, con pipeline, tiene un sistema de control más complejo, ya que necesita más líneas de control para controlar los sistemas de registro.



4. Riesgos en la ejecución (hazards)

El CPI (Ciclo Por Instrucción) ideal de un procesador segmentado es 1.

Existen situaciones llamadas riesgos que impiden que se inicie una instrucción en cada ciclo de reloj.

Razones:

- Datos no disponibles por un fallo caché o dependencia de otra instrucción.
- Instrucciones de salto condicional.

Los ciclos del pipeline en que no se computa se denominan burbujas del pipeline y corresponden al caso en que se introduce en el pipeline un código de no operación (NOP).

El paralelismo a nivel de instrucción (Instruction Level Parallelism-ILP) es la capacidad de procesar instrucciones en paralelo. Viene determinado por el número de instrucciones que pueden solaparse en las etapas de un procesador.

Tipos de riesgos:

- De **estructura**: El hardware no permite la ejecución de determinadas instrucciones simultáneamente.
- De **datos**: Surgen de la necesidad de esperar a que una instrucción anterior lea o escribe un dato.
- De **control**: Surgen del problema de determinar la instrucción correcta que se tiene que ejecutar después de un salto.

5. Riesgos de la estructura

Veremos en primer lugar los riesgos de estructura con ejemplos muy concretos.

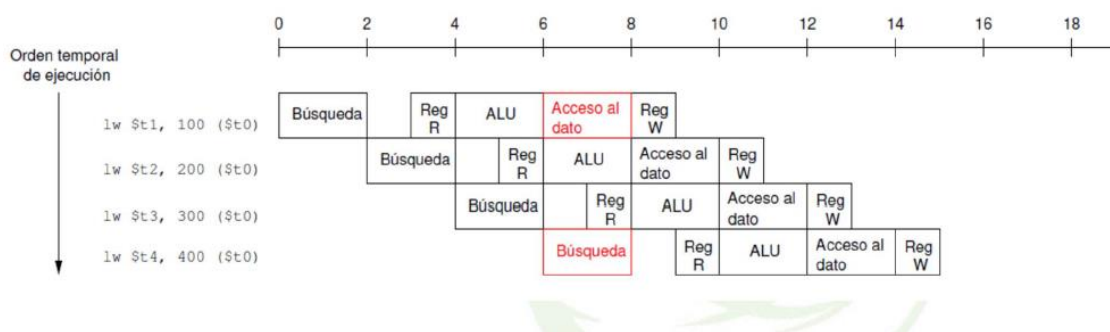
Ejemplo 1: Solo existe una unidad de división. Las instrucciones de la imagen se llaman `div.s` porque están operando con instrucciones en punto flotante, pero son instrucciones de división en el MIPS, muy simples. Tenemos dos instrucciones:

```
div.s $f10, $f12, $f14
div.s $f16, $f18, $f20
```

Si solo tenemos una unidad de división y su ejecución requiere varios ciclos, no se podrá ejecutar estas instrucciones una después de otra, por lo tanto, existe una

dependencia estructural.

Ejemplo 2: supongamos que tenemos el pipeline como el del MIPS, pero tenemos una única memoria para los datos y para las instrucciones. Lanzamos una instrucción de carga con sus 5 etapas (buscar, leer registro, ALU, acceso al dato, escribir registro), acabaría en el ciclo 5 (entre 8 y 10, visible en la imagen).



Si nos fijamos en el segundo ciclo, entre 2 y 4, solo se usa la segunda mitad, y en el quinto, entre 8 y 10, solo la primera mitad, pero se computan todos los ciclos de la misma duración (en este caso 2). Se necesitan por tanto 5 ciclos para cada instrucción, pero en cuanto la primera pasa a la siguiente etapa, se puede comenzar a buscar la siguiente, y así sucesivamente.

Lo que ocurre en este caso es, si nos fijamos en la imagen, en el ciclo 4 (entre 6 y 8) se intenta acceder a un dato a la vez que se busca una instrucción. Si solo tuviera una memoria para datos e

instrucciones, esta instrucción de carga no se podría ejecutar, sería necesario no ejecutar nada a partir del ciclo 4 (meter una burbuja) y ejecutar la búsqueda de la instrucción **lw \$t4, 400(\$t0)** en el ciclo 5, y ejecutar la instrucción a partir de ahí.

***Burbuja:** ejecutar la instrucción NOP ejecutar debajo la de carga. Se produce como consecuencia del riesgo estructural que existiría si tuviera una única memoria para datos e instrucciones.

La solución para este problema sería duplicar las unidades funcionales: la más obvia sería tener una memoria separada para datos e instrucciones. Otra opción es duplicar la unidad de división. Muchos procesadores tienen varias unidades de multiplicación, de división... para evitar ese problema, y que se puedan ejecutar varias a la vez dentro del mismo core.

Ejemplo 3:

Se consideran dos escenarios alternativos:

- A: sin riesgos estructurales
 $T_A (\text{período}) = 1\text{ns}$
- B: con riesgos estructurales que suponen 1 ciclo perdido
 $T_B = 0.9\text{ns}$

30% de instrucciones de acceso a datos con riesgos estructurales

¿Alternativa más rápida?

El tiempo para una instrucción A va a ser el número de ciclos por instrucción por el tiempo del ciclo.

$$t_{\text{inst}}(A) = \text{CPI} \times T_A = 1 \times 1\text{ns}$$

El tiempo para la instrucción de tipo B sería ciclos por instrucción por el tiempo de ciclo. En este caso, el número de ciclos por instrucción es el 70%, que tarda un ciclo y 0,3 (o 30%) que tarda uno más el que pierdo por haber un riesgo estructural.

$$t_{\text{inst}}(B) = \text{CPI} \times T_B = (0,7 \times 1 + 0,3 \times (1 + 1)) \times 0,9\text{ns} = 1,17\text{ns}$$

Aunque el tiempo de ciclo es más pequeño, tenemos el 30% de las instrucciones con riesgos estructurales que obligan a perder cada uno un ciclo.

Para que la segmentación sea la mejor opción, se tiene que tratar bien el asunto de los riesgos, teniendo solución para los estructurales, duplicando unidades para los riesgos de datos y los de control. En ese caso, sí se mejoraría la ejecución con respecto a un procesador donde no haya segmentación.

6. Riesgos de datos

Ocurre cuando la ejecución de una instrucción depende de un dato que aún no está disponible. Existen diversos tipos:

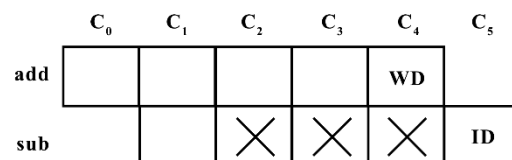
6.1 Tipo Raw

Este tipo de riesgos de datos, RAW (Read After Write) o dependencia verdadera. Una instrucción *j* intenta leer un dato antes de que una instrucción *i* lo escriba. Ocurre que las instrucciones no se pueden ejecutar en paralelo ni solaparse completamente. Lo veremos en el siguiente ejemplo:

i: `add $1, $2, $3`
i+1: `sub $4, $1, $3`

La instrucción *add* almacena su resultado en el registro \$1 en la última etapa (WB). La siguiente instrucción va a necesitar dicho registro en la etapa ID (segunda etapa). Tenemos una instrucción *add* que empieza a ejecutarse en un determinado ciclo *y*, en el ciclo siguiente, empieza a ejecutarse la *sub*, con sus 5 etapas. La primera va a escribir el resultado en la quinta etapa (WB), y se va a necesitar en la etapa de decodificación, donde se leen los registros. En este ciclo de *sub* aún no ha sido calculado, se calculará en el 4.

Por tanto, en el ciclo 2 habrá que meter una espera hasta que se calcule el resultado de la primera instrucción (es decir, habrá 3 ciclos de espera para esta instrucción).



Las posibles soluciones son: detener el hardware (meter burbujas) o meter algún tipo de control por parte del compilador que resuelva este problema.

6.2 Tipo WAR

WAR (Write after read) o escritura después de lectura, consiste en que una instrucción *i+1* modifica un operando antes de que la instrucción *i* lo lea.

En compiladores es conocido como antidependencia.

No puede ocurrir en un MIPS con pipeline de 5 etapas **si consideramos que la etapa de ejecución es única**, ya que se produce la lectura en la etapa 2 y las escrituras en la etapa 5.

i: `sub $4, $1, $3`
i+1: `add $1, $2, $3`
i+2: `mul $6, $1, $7`

Se aprecia un ejemplo de dependencia en la segunda instrucción. La primera lee en el registro uno, y la *add* escribe en este. ¿Qué sucedería si *add* modificase el registro \$1 antes de que lo leyera la primera instrucción? En el MIPS no puede suceder, pero en otros pipelines sí. Esto daría lugar a que la operación de resta (*sub*) diera un valor erróneo.

Se necesita una solución para evitar ese resultado incorrecto.

6.3 Tipo WAW

WAW(Write After Write) o Escritura después de escritura o Falsa dependencia, consiste en que una instrucción $i+1$ intenta escribir un operando antes de que este sea descrito por i . Se corresponde con una dependencia de salida.

En compiladores se conoce como dependencia de salida.

Veamos el siguiente caso:

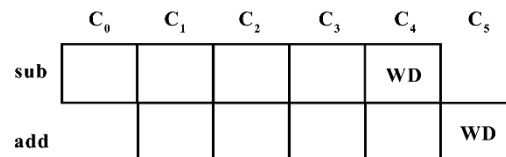
Ejemplo:

i : sub \$1, \$4, \$3
 $i+1$: add \$1, \$2, \$3
 $i+2$: mul \$6, \$1, \$7

Las dos primeras instrucciones intentan escribir en el mismo registro. Deben escribir primero la primera instrucción y luego la segunda, ¿qué sucede si no acaban en el mismo orden? se podría alterar el funcionamiento del programa. En resumen: no se debe permitir que escriba el mismo registro una instrucción posterior antes que la anterior.

No puede suceder en el MIPS de 5 etapas, ya que escribirían las dos en la última etapa.

Llegarían a esta última etapa: primero, la



instrucción sub, de la cual tenemos sus 5 etapas. Esta llegaría a la etapa de escritura y la instrucción add llegaría a la etapa de escritura un ciclo después. Por tanto, nunca habría una dependencia de este tipo, pero en otros pipelines sí.

Tanto las antidependencias como las dependencias de salida se pueden solventar mediante el renombrado de registros.

6.4 Soluciones a los riesgos de datos:

Riesgos de tipo RAW (Read afterWrite): son los más típicos

- Bloqueo: no hacer nada (meter burbujas).
- Reordenar el código para mitigar su efecto: si entre la instrucción que escribe un registro \$t1 y la instrucción que lee dicho registro se mete una o varias instrucciones a mayores, se reordena el código y no se produce el riesgo (dichas instrucciones no pueden depender de las que ya estaban: utilizando otros registros...).
- Envío adelantado (forwarding). También llamado **anticipación** o **bypassing**. Esto es un envío adelantado de información.

Riesgos o dependencias WAR y WAW:

- Renombrado de registros
 - Estáticamente por el compilador
 - Dinámicamente por el hardware

6.4.1 Ejemplo de bloqueo por riesgos de datos RAW

Enunciado del problema: Se considera un pipeline como el del MIPS pero con los siguientes retardos: una carga de datos necesita 3 ciclos de acceso a memoria, un almacenamiento necesita 5 y la carga de cada instrucción 4 ciclos excepto la primera instrucción, que requiere 1 ciclo.

\$r2: I1 -> I2 La primera instrucción escribe \$r2, y la segunda lo lee, es decir, existe una dependencia con el registro \$r2 entre las dos primeras instrucciones.
\$r3: I2 -> I3 Además, la segunda escribe el registro \$r3 y la tercera instrucción lo tiene que leer, de forma que hay otra dependencia. Miramos la tercera y quinta instrucción: la tercera lee el registro \$r1, y la quinta lo escribe → **error en el ejemplo, Dora lo tiene que cambiar porque no existe dependencia RAW, sino WAR.**

Dependencia RAW encontradas: **\$r2: I1 -> I2** y **\$r3: I2 -> I3**

¿Cómo se reflejan estas dependencias a la hora de ejecutar?

Instrucción	IF	ID	EX	M	M	M	WB									
I1: lw \$r2, 0(\$r0)																
I2: addi \$r3, \$r2, 20		IF1	IF2	IF3	IF4	–	ID	EX	M	WB						
I3: sw \$r3, 0(\$r1)							IF1	IF2	IF3	IF4	ID	EX	M	M	M	M
I4: addi \$r0, \$r0, 4											IF1	IF2	IF3	IF4	ID	EX
I5: addi \$r1, \$r1, 4															IF1	IF2

Cargar una instrucción requiere un ciclo la primera vez (primera fila de la tabla), pasa a decodificación, ejecución... después se necesitan 3 ciclos para acceder a memoria, y post escritura. La carga de cada instrucción son 4 ciclos menos la primera, es decir, que para cargarla se necesitan 4 ciclos, y después queremos leer los operandos, por lo que queremos el valor del registro \$r2, pero antes de poder pasar a decodificar para leer los registros, hay que esperar un ciclo.

Para cargar la instrucción de almacenamiento, se necesitan 4 ciclos. Pasa a decodificación, ejecución, y otros 4 ciclos para acceder a memoria.

La cuarta instrucción (addi): 4 ciclos de búsqueda-ID (decodificación)-ejecución. Se empieza a buscar la siguiente instrucción y, cuando se acaba de buscar una, empieza a buscarse la siguiente y así sucesivamente, hasta que hay una parada. Esta se debe a la dependencia que existe entre la instrucción 1 y 2. La primera necesita los operandos, que aún están siendo calculados. También existe una dependencia entre las instrucciones 2 y 3, la segunda escribe un dato que necesita la tercera en la etapa ID, pero ya fue calculado en el momento necesario por lo que no se producen paradas. En resumen: una se resuelve con una parada, y la otra simplemente, no se resuelve, ya que no es necesario hacer ningún tratamiento especial.

6.5 Técnicas de tratamiento de riesgos de datos

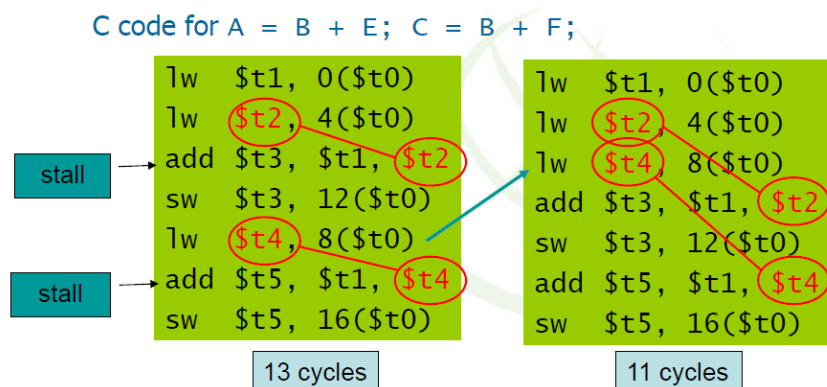
6.5.1 Software

El compilador puede ordenar la secuencia de instrucciones:

- Genera una secuencia de instrucciones independientes, haciendo que los riesgos desaparezcan.
- Inserta instrucciones nop (burbujas) cuando no existen instrucciones independientes.

Veremos un ejemplo donde, en lugar de no hacer nada, trataremos el riesgo de datos tipo RAW, reordenando las instrucciones.

En el ejemplo siguiente, la instrucción lw carga un dato (desde memoria, por lo que hay que esperar a la etapa de memoria) que se necesita como operando, y otra instrucción lo necesita en la etapa de decodificación. Ocurre lo mismo en la siguiente instrucción **lw-add**.

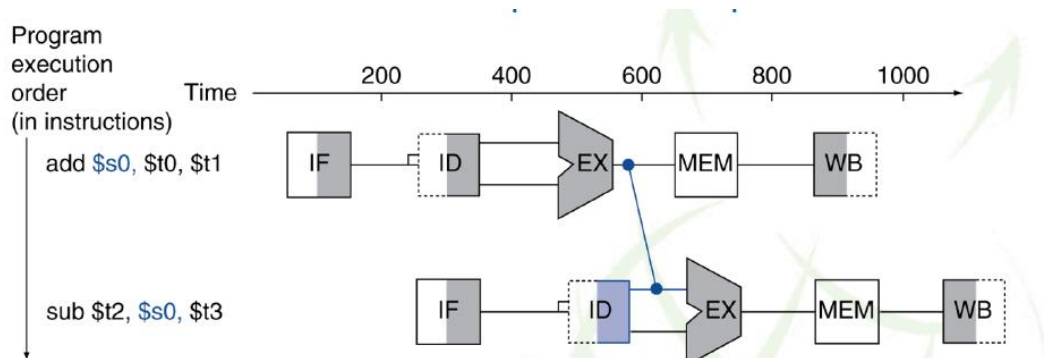


En el primer caso se necesitarán 13 ciclos porque se realizan paradas. Estas se pueden reducir o evitar completamente hasta llegar a un código como el de la derecha, que tarda 11 ciclos en ser ejecutado. Para ello se sube la tercera instrucción lw del primer código. En la dependencia entre add y lw, antes era una instrucción a continuación de otra, cubro la burbuja del tiempo que se tendría que esperar con una instrucción útil, y a la vez separo a esta instrucción de la cual depende lo suficiente para evitar paradas. Por tanto, mediante un reordenamiento de instrucciones, evitamos las paradas y pasamos de una ejecución de 13 ciclos como es este caso, a una de 11.

Esta técnica permite evitar alguna parada puntualmente, pero no va a solucionar el problema de las paradas en código segmentado. Por ello, se suelen utilizar otras técnicas.

6.5.2 Hardware

Forwarding: el hardware adelanta el dato que hace falta una vez éste está calculado, sin necesidad de esperar a la escritura en el registro. Requiere conexiones adicionales. Soluciona dependencias tipo Raw.

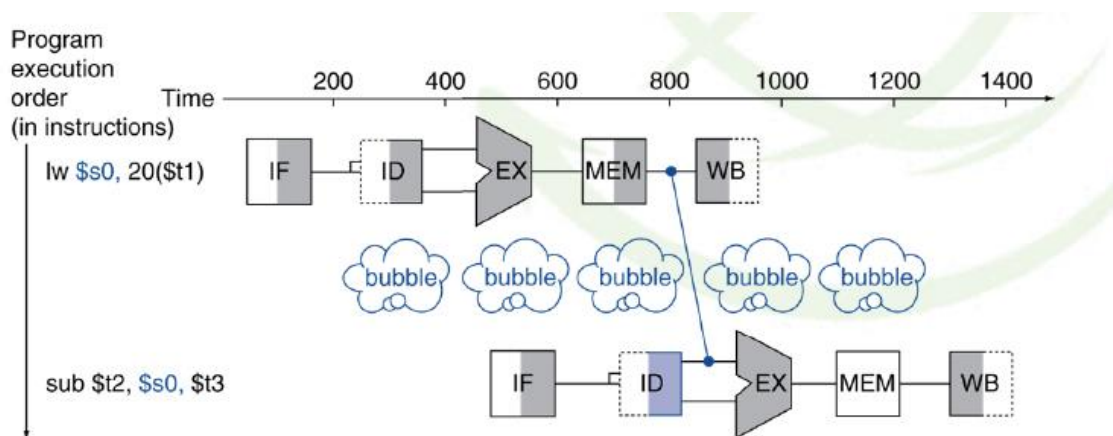


Add \$s0, \$t0, \$t1 : esta instrucción add, suma el contenido de los registros y el resultado lo introduce en un registro que después es operando de la instrucción siguiente.

En este 1º ciclo se busca la instrucción, se decodifica, se busca el valor de los operandos, se ejecuta, siguiente (MEM) no pasa nada, se guarda el resultado en el registro.

Sub \$t2, \$s0, \$t3 : En la siguiente instrucción necesita los datos en la etapa de decodificación, y necesita que lleguen a ejecución antes de que se guarde en memoria, entonces lo que podemos hacer es que desde la etapa de la 1 instrucción de ejecución enviamos directamente a la línea de la siguiente para usar ese dato. Es decir, adelantamos los datos.

Bloqueo: parar por hardware la ejecución de las instrucciones en el cauce esperando a que el dato necesario esté disponible.



Aquí se hace un adelantamiento, pero hacemos un bloqueo para adelantar el dato más tarde, no después de la ejecución.

6.5.3 Detectar la necesidad de forwarding

El forwarding consiste en enviar resultados de las últimas etapas del pipeline a las primeras para evitar esperas.

Requiere:

- 1) Detectar si uno de los datos se puede traer por forwarding desde otra etapa del pipeline
- 2) Generar señales de control adecuadas en los multiplexores para seleccionar la procedencia correcta del operando.

Hay que analizar los códigos de las instrucciones en etapas EX (la que se está ejecutando) y en etapas MEM y WB (porque pueden producir resultados que se pasen por bypass a EX).

Se producen señales de control para los dos MUX que controlen entrada a la ALU.

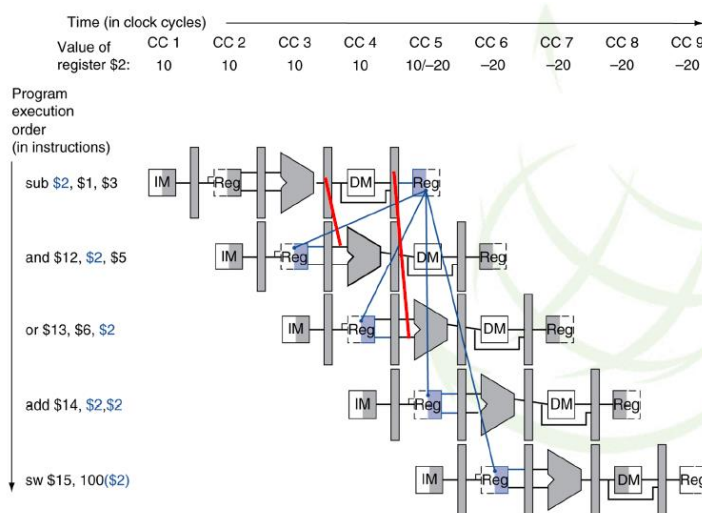
LD R1, 45(R2) Ejemplo: Gracias a forwarding DSUB no sufre penalización en ciclos de espera: Los operandos se pasan desde la etapa MEM de DADD y desde la etapa WB de la instrucción LD.

DADD R5, R6, R7

DSUB R8, R5, R1

6.5.4 Anticipación, forwarding o bypassing

Permite obtener las entradas de la ALU de cualquier registro de segmentación, no solo del ID/EX, así que requiere modificar la ALU.



Mostramos una vista que ilustra el uso de recursos del pipeline en cada ciclo.

En azul: donde se necesita el contenido del registro \$2 calculado por sub.

En rojo: los dos adelantamientos que se producen.

Tenemos la instrucción de resta, el resultado se pone en el registro \$2 que va a ser operando de las instrucciones siguientes.

La instrucción and, lo va a necesitar en ejecución, por lo que podemos hacer el adelantamiento.

En las instrucciones and y or hay que llevar los resultados atrás en el tiempo, por lo que se produce adelantamiento.

6.5.5 Control de interbloqueo de instrucciones

Detectar cuando una instrucción no puede avanzar por dependencia en operandos con otra y parar la emisión de instrucciones hasta que se solucione.

Se detecta en la etapa ID (decodificación) porque ahí se sabe qué registros son operandos de entrada.

Se sabe detectar si en las siguientes etapas del pipeline hay una instrucción que escriba en un registro que se usa como entrada para la ejecución de la instrucción que está en la etapa ID.

No se tendrán en cuenta las instrucciones de las siguientes etapas del pipeline que puedan hacer bypassing del resultado cuando la instrucción en la etapa ID lo necesite.

La detección depende del tipo de instrucciones involucradas.

6.5.6 Control de bloqueo de instrucciones

LD R1, 45(R2)

DADD R5, R1, R7

DSUB R8, R6, R7

OR R9, R6, R7

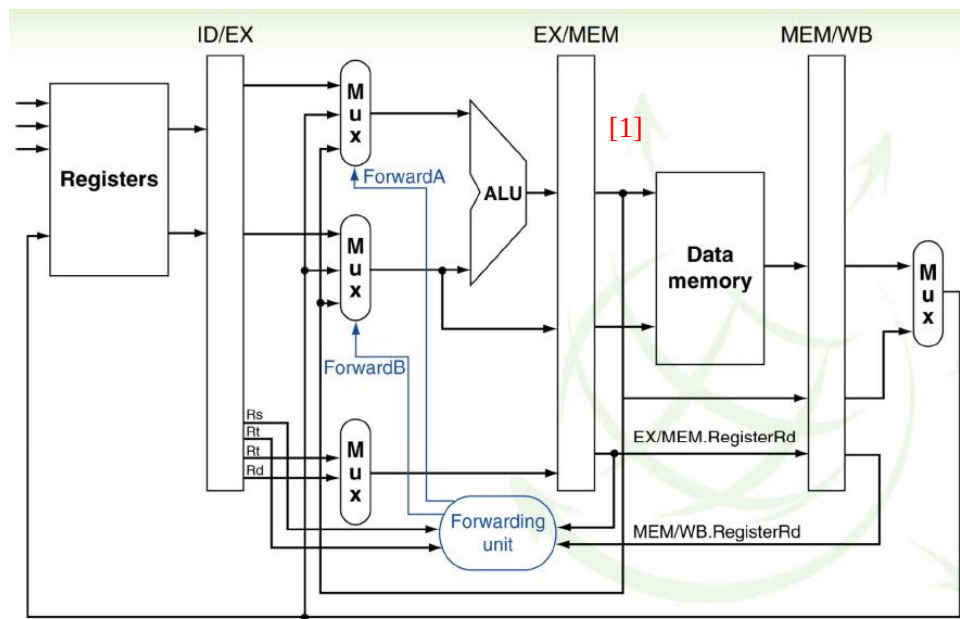
Cuando DADD se decodifica en ID, la instrucción LD está en etapa EX y en R1 no estará listo el resultado que se requiere para DADD.

Se requiere parada en la etapa ID y en el siguiente ciclo se emite una no operación (nop). Los registros en la etapa ID y el PC mantienen su contenido.

Caso ejemplo de detección de interbloqueo. Se activa parada si:

- En etapa ID hay una instrucción tipo ALU o salto (BR) o de almacenamiento (ST).
- Y en etapa EX está ejecutándose una carga (LD).
- Y el registro destino de la LD es operando de instrucción tipo ALU, BR (salto) o ST (almacenamiento).

6.5.7 Forwarding en el camino de datos

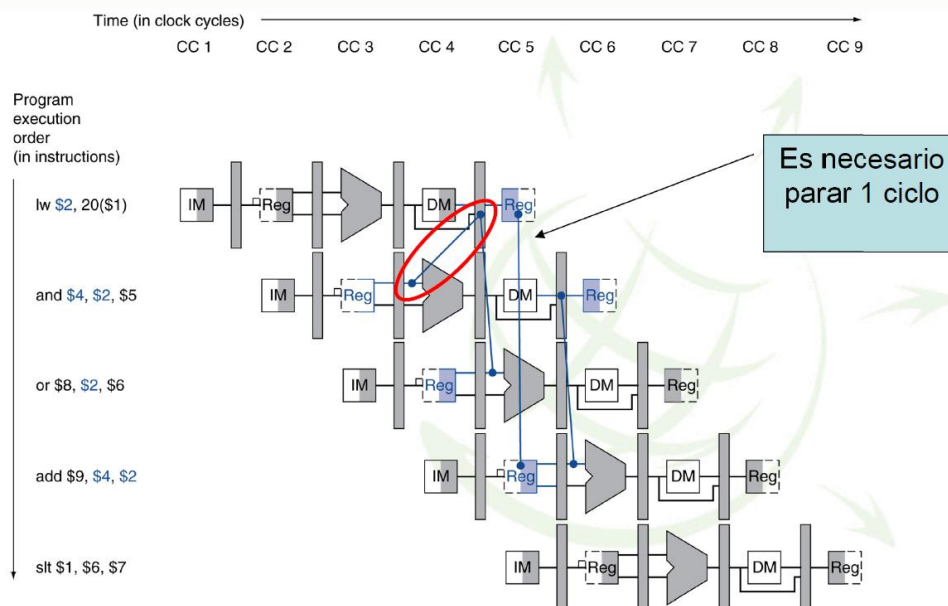


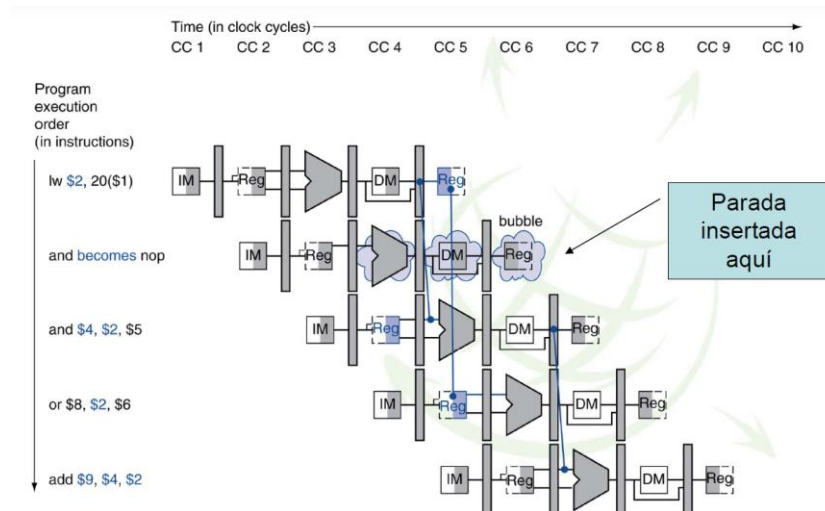
b. With forwarding

Forwarding siempre se produce desde la etapa de ejecución hasta una etapa de decodificación.

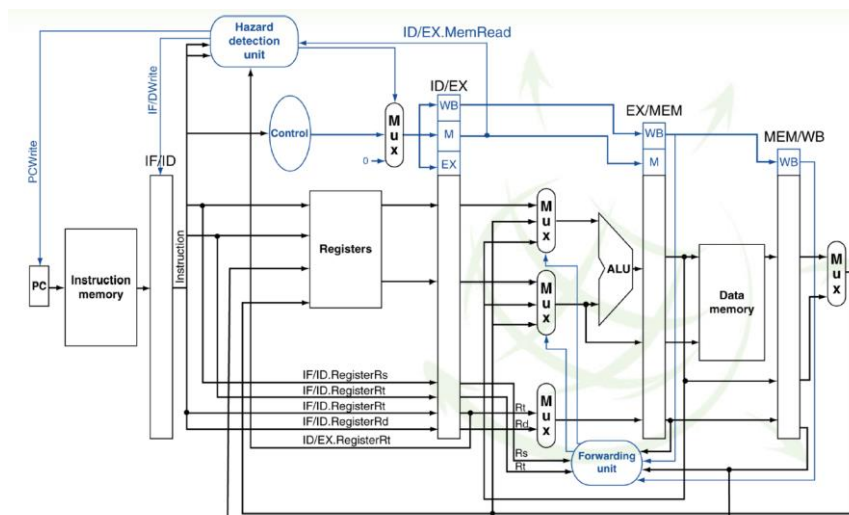
[1] Esta línea sale de la etapa de ejecución, después del registro de ejecución.

6.5.8 Ejemplo de parada por bloqueo





6.5.9 Camino de datos con detección de forwarding y de interbloqueo (hazard detection unit)



7. Riesgos de control

Aparecen cuando surge la necesidad de tomar una decisión basada en los resultados de una instrucción mientras las otras se están ejecutando: instrucciones de salto.

En el pipeline del MIPS para optimizar ejecución de salto es necesario:

- Comparar los registros en una etapa del pipeline anterior a EX.
- Añadir hardware para hacerlo en la etapa ID.

Solución a los riesgos de control

- Primera opción: detener el cauce hasta que se toma la decisión.
- Segunda opción: aplicar una técnica de predicción de salto.
- Tercera opción: aplicar una técnica de salto retardado.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura de Computadores

25-03-2021



Clase 07

Temas trabajados: Riesgos de control y de datos.
Ejercicio 1

Índice

1. Riesgos de control	3
2. Primera opción: Parada asociada a un salto	5
3. Ejemplo de riesgos de datos para saltos(I).....	6
4. Ejemplo de riesgos de datos para saltos (II).....	7
5. Ejemplo de riesgos de datos para saltos (III).....	8
6. Segunda opción: Decisión retardada	8
7. Ejercicio 1 boletín I	10

1. Riesgos de control

Los riesgos de control se producen cuando se ejecuta una instrucción de salto o una instrucción condicional(`beq`, `bne`,...), instrucciones que comparan el contenido de registros y, en función del resultado, se produce, o no, un salto. También hay instrucciones de salto incondicionales que realizan siempre el salto, por ejemplo, el salto a una subrutina(`jal`). ¿Qué sucede cuando se ejecuta una instrucción de salto? Mientras se decide si se realiza o no el salto, ¿cuál es la siguiente instrucción?, ¿qué se hace? Como respuesta a las preguntas anteriores nacen los riesgos de control.

Los riesgos de control aparecen cuando surge la necesidad de tomar una decisión basada en los resultados de una instrucción mientras las otras se están ejecutando.

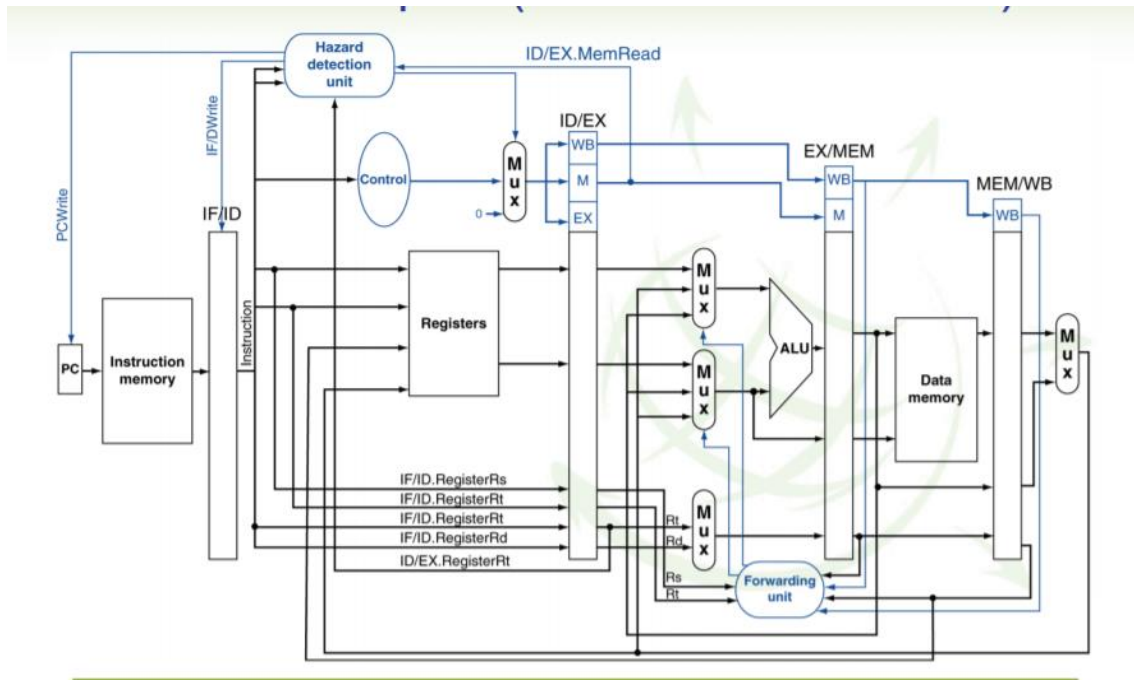
En el pipeline del MIPS, el salto se resuelve en la etapa de ejecución debido a que en esta la ALU compara el resultado de los dos registros. Sin embargo, añadiendo hardware, se puede hacer en la etapa de ID, resolviendo si se produce o no un salto en una etapa interior, reduciendo las paradas que se producen como consecuencia de los saltos.

La solución a los riesgos de control a los riesgos producidos por saltos, pueden ser tres:

- Primera opción: Parar todo hasta que se decida si se salta o no.
- Segunda opción: Aplicar una técnica de salto retardado. Básicamente consiste en meter instrucciones después del salto mientras no se decide. Son instrucciones que se deben ejecutar de todas maneras.
- Tercera opción: Aplicar una técnica que prediga si el salto se va a tomar o no en caso de que el salto sea condicional.

En cada momento el registro del contador del programa (PC) contiene la dirección de la siguiente instrucción que se va a ejecutar. Normalmente en el MIPS apunta 4 bytes más allá que la instrucción en la que se encuentra el PC.

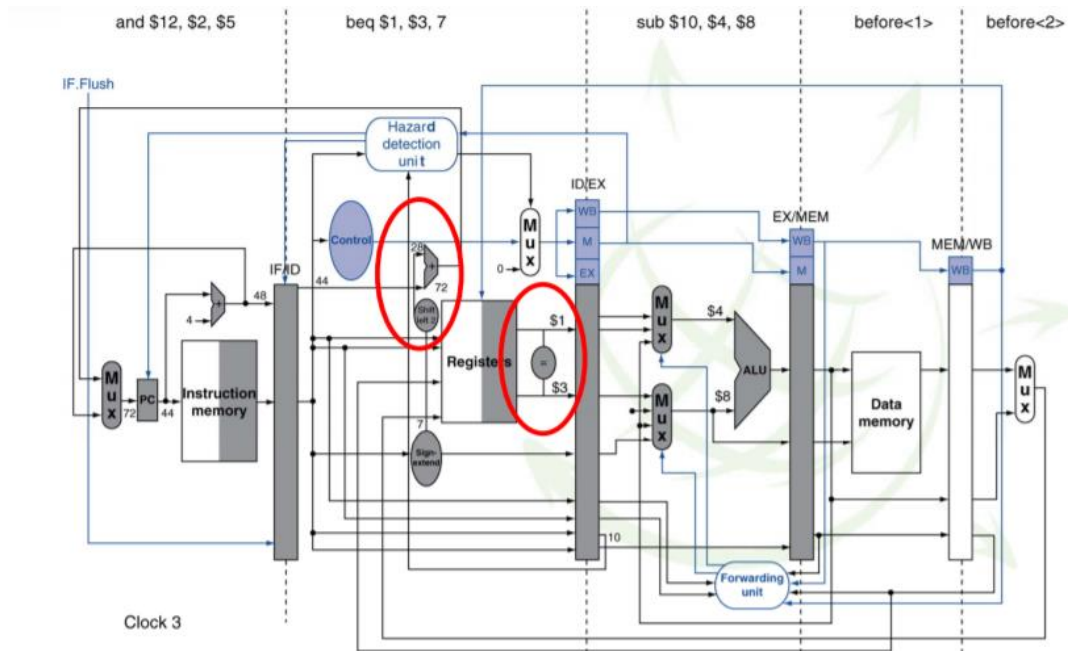
Si el salto se toma, se modifica el contador del programa (PC).



Observando la gráfica anterior, si se decide el salto en la etapa de memoria (MEM) como sucedería normalmente, se observa que la primera instrucción es beq y si se decide al final de la etapa 4, no se puede saber en el ciclo siguiente qué instrucción se va a cargar. No se puede hasta 3 ciclos después (hasta que se vuelva a apuntar al PC), entonces, se están desperdiciando ciclos. Si se resuelve en la etapa de ejecución en lugar de 3 se desperdician solo 2 ciclos. Dependiendo de dónde se solucione el salto, se desperdician más o menos ciclos. En caso de resolverlo en la etapa ID, se ahorra un montón de ciclos.

Los ciclos se desperdician porque si se produce un salto y ya se tiene cargada las instrucciones siguientes y no se desean ejecutar, se va a tener que desperdiciar toda la información cargada y se debe ejecutar la instrucción de salto. Si no se sabe que va a pasar después del salto y se tarda varios ciclos en ejecutarlos, hasta que sepa lo que realmente pasa, no está claro que las instrucciones que se estén ejecutando sean las correctas. Entonces, o se van ejecutando instrucciones sin más y después se vacía el pipeline si no interesan; o se para hasta conocer el valor correcto del PC. Otra opción es hacer lo siguiente:

2. Primera opción: Parada asociada a un salto

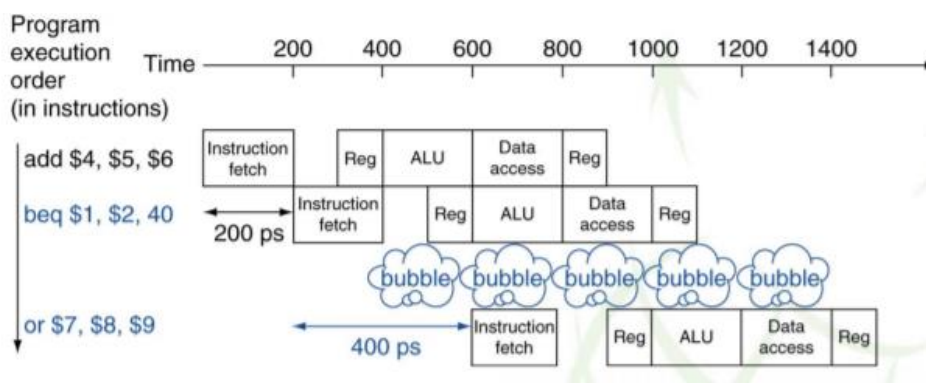


Que es comparar los registros de salto de condición del salto, realizándose con un comparador simple a la salida de un banco de registros (esto solo se activaría si la instrucción es de salto). Y, por otra parte, se desplaza dos bits a la izquierda la etiqueta de salto y se le suma la etiqueta de salto, desplazando 2 bits a la izquierda con extensión del signo al valor del PC.

En la parte izquierda de la imagen anterior, se tiene una ALU pequeña de la memoria de instrucciones que suma 4 al PC y esto se adelanta a la etapa siguiente. En esta etapa, se cogen esos bits del PC y se les suma la etiqueta desplazada dos bits a la izquierda con extensión de signo, con el pequeño sumador que se encuentra dentro de la marca roja, calculando así la dirección de salto y si hay o no salto, resolviendo los dos problemas en la etapa ID. Con esto se ahorra un montón de paradas.

2. Primera opción: Parada asociada a un salto

Detención de la carga de la siguiente instrucción que se ejecutará o de las siguientes que sean necesarias en caso de pipelines más profundos hasta que el resultado de salto se compute.



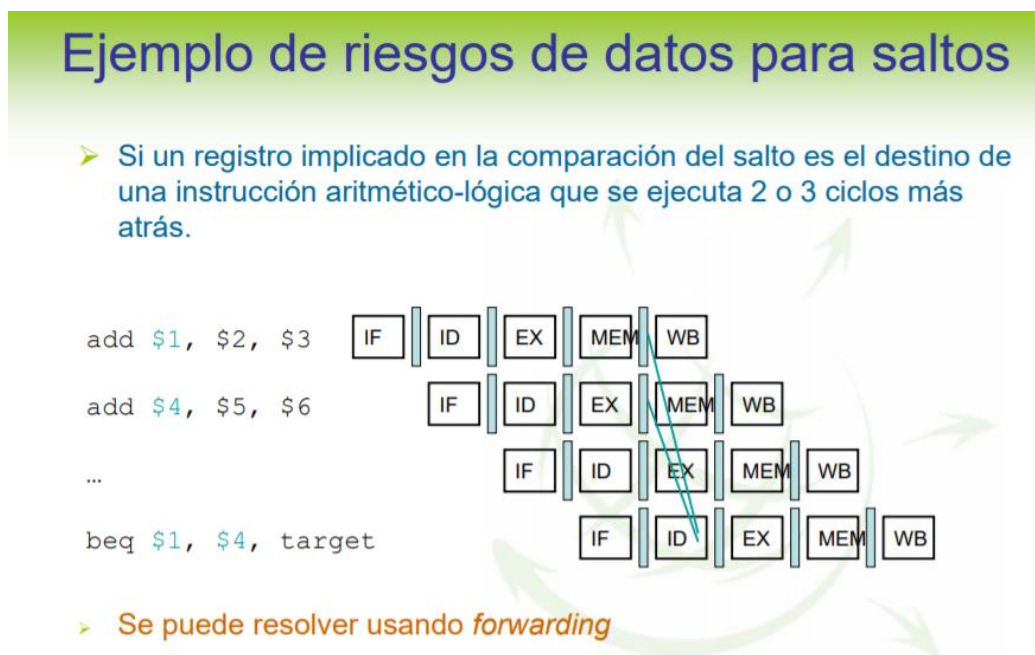
3. Ejemplo de riesgos de datos para saltos(I)

En este ejemplo aparecen las instrucciones beq y add. En la ejecución del beq no se sabe que pasará hasta que se termine el ciclo ID. Si ya se resuelve en este ciclo, solo se pierde un ciclo, indicada como bubble en la imagen anterior. Tras este ciclo, ya se sabe la instrucción que se debe cargar. Evidentemente, si el pipeline tiene muchas etapas, a lo mejor no se consigue resolver en la siguiente etapa y el coste de la parada es más elevado. Si se predice el resultado del salto, se pueden aliviar las paradas, aunque sean pocas, como ocurre en el MIPS, pues resolviendo la etapa ID ya solo se tiene una parada.

En general el pipeline del MIPS, lo que hace es:

- Predecir los saltos como no tomados siempre, es decir, siempre predice que no se va a saltar, por lo que carga las siguientes instrucciones y las va ejecutando. Si luego se salta borra lo que ha cargado.
- Hace la búsqueda de las instrucciones después del salto para que ya estén buscadas.
- Si la predicción es incorrecta se producen paradas.

3. Ejemplo de riesgos de datos para saltos(I)



En este ejemplo se tienen dos instrucciones add, luego un montón de instrucciones y por ultima un beq. Los registros \$1 y \$4 son el destino de dos instrucciones add, las cuáles se encuentran ejecutándose en el pipeline y son los que se necesitan comparar en la instrucción de salto que aparece posteriormente. Entonces se necesita en la etapa ID registro que todavía no están escritos en el banco de registros cuando la instrucción beq llegue a la etapa ID. Esto se resuelve con adelantamiento o forwarding. En el momento en que la primera instrucción de la etapa EX ya tiene el resultado, ya se puede adelantarlo, adelantándole a la etapa ID lo necesario. Entonces desde la etapa de ejecución ya podría adelantar los valores de \$1 y \$4. En el momento en que los necesite beq, se le adelanta a su entrada, teniéndolos así disponibles para calcular en la etapa

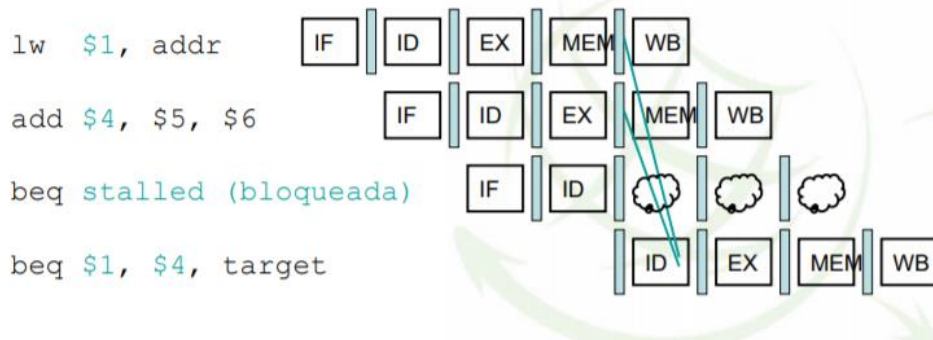
ID, utilizando el hardware que se explicó anteriormente, y a que dirección se produce el salto.

4. Ejemplo de riesgos de datos para saltos (II)

Ejemplo de riesgos de datos para saltos

- Si un registro de comparación es el destino de una instrucción ALU precedente o de una carga que está 2 instrucciones más atrás.

❑ Necesita 1 ciclo de bloqueo

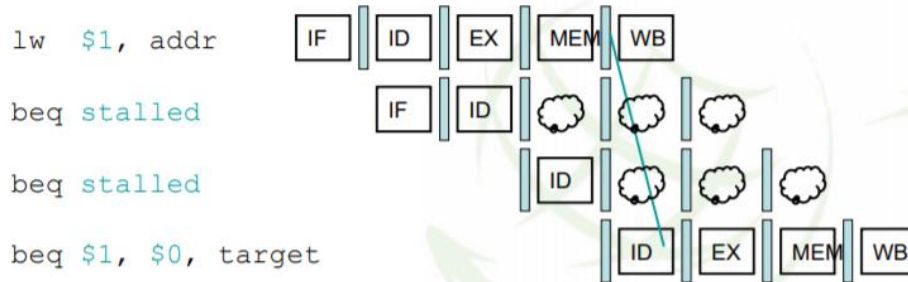


Otro ejemplo de riesgo de datos para saltos relacionados con la instrucción `beq`. Esta instrucción necesita los operandos `$1` y `$4`, que se calculan en las instrucciones de carga y `add`. En la carga va a estar disponible tras la etapa MEM en la `add` después de la etapa de ejecución. Desde ambas se puede adelantar los datos a la instrucción `beq` en este ciclo, es decir, se tiene que utilizar un ciclo, produciéndose un stop o una parada, debido a que el `beq` se iba a ejecutar, pero, se tiene que parar y retomarla más adelante, perdiendo así un ciclo. Fijarse que en este ejemplo ya no se está metiendo una instrucción en cada ciclo, sino que hay un ciclo que se pierde, necesitando 1 ciclo de bloqueo.

5. Ejemplo de riesgos de datos para saltos (III)

Ejemplo de riesgos de datos para saltos

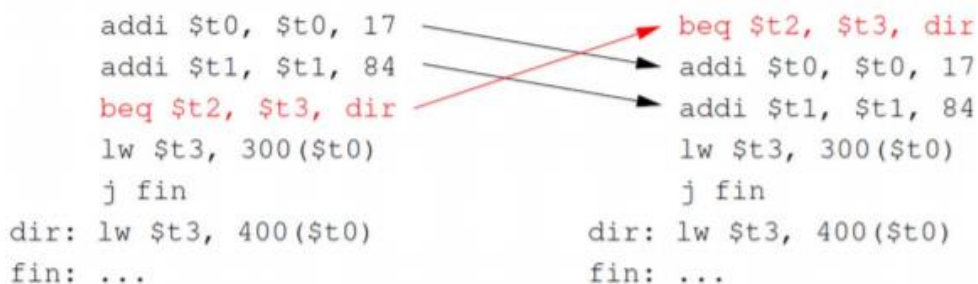
- Si un registro implicado en la comparación del salto es el destino de una instrucción de carga inmediatamente anterior.
 - ❑ Necesita 2 ciclos de parada



En este ejemplo se necesitan 2 ciclos de bloqueo. Se tiene una instrucción de carga al registro \$1, se va a resolver en la etapa MEM, y se tiene la instrucción beq que se va a resolver en la etapa ID, por lo que va a ser necesario esperar 2 ciclos hasta obtener el resultado, puesto que la instrucción va justo después de lw, al contrario del ejemplo anterior, en el que existe una instrucción intermedia que alivia la situación. En este ejemplo la beq se necesita parar 2 ciclos seguidos y luego seguirá su ejecución. Hasta que pueda tener en la ID los resultados que necesite no puede seguir y la instrucción de carga no los tendrá disponibles hasta la etapa MEM, así que se tienen 2 ciclos de parada.

Si se vuelve a la solución de los riesgos, la primera opción de solución de riesgos de control es pararse y, aplicando el forwarding o adelantamiento, las paradas se pueden hacer más pequeñas como se acaban de ver. Otra opción es aplicar la técnica de salto retardado.

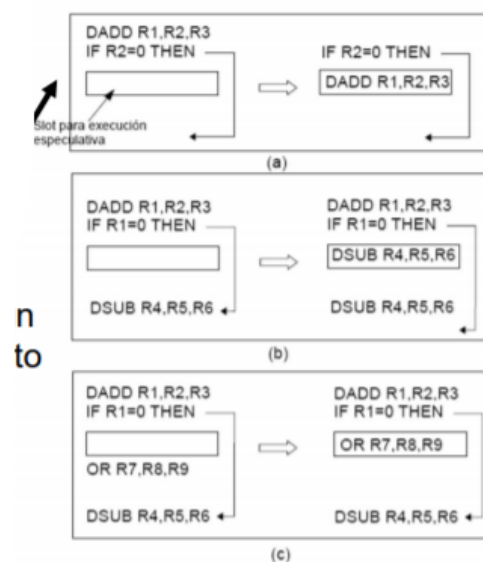
6. Segunda opción: Decisión retardada



En el código de la izquierda se tienen dos instrucciones de suma de un inmediato, luego una instrucción de salto condicional que salta si el contenido del registro \$2 es igual al contenido del registro \$3 a la dirección de memoria construída por la etiqueta dir, desplazándola dos bits, extendiendo el signo y sumándole el contador del programa. Este código se va a ejecutar comparando el contenido de los registros \$2 y \$3. Una posibilidad de evitar las paradas debidas a los saltos es mover varias instrucciones o ranuras de las que están por encima del salto a debajo del salto si son instrucciones que no dependen de este. En lugar de meter paradas, se mueven instrucciones que se tienen que hacer de todos modos pero que no dependen del salto. Puede ocurrir que haya instrucciones que no se puedan mover. Si no existen instrucciones con esta condición, se producirán paradas. En la realidad esto sucede un 50% de las veces. Esta es la solución adoptada por el MIPS, realizar una decisión retardada.

Fijándonos en los códigos de la izquierda y la derecha, las instrucciones `addi` no dependen para nada de la instrucción de salto, ya que estas instrucciones no utilizan los registros \$2 y \$3, por lo que se pueden mover estas dos para después de la ejecución del `beq` y se ejecutan igualmente mientras el salto `beq` no se resuelve. A continuación, se ejecuta la carga. En este caso se mueven 2 ranuras, 2 instrucciones que no dependen de `beq` y que se van a ejecutar de todos modos. El MIPS considera que el salto no se va a tomar, cargando así todas las instrucciones posteriores y, se tome o no el salto, las instrucciones movidas nunca se quitarán del pipeline, ya que, se aprovecharon 2 huecos para que se ejecuten estas instrucciones. Esto, evidentemente, lo hace el compilador de manera automática.

En cuanto a la decisión retardada, dependiendo de la información que esté disponible durante la información, se mueven instrucciones de un sitio o de otro. Es decir, no siempre se mueven instrucciones que estén por arriba, como en el ejemplo anterior. Fijarse en los siguientes ejemplos:



En el primer ejemplo, en lugar del condicional se tiene un `if`, y hay un cuadrado que representa un hueco o ranura disponible para ejecución especulativa. Se puede especular que ejecutar en ese hueco. En el primer caso, se coge la instrucción que se encuentra justo encima del especular y se pone en el hueco, al igual que en el ejemplo

anterior. Esa instrucción se ejecutará siempre, así mientras el salto se va resolviendo se irán ejecutando instrucciones útiles. Otra posibilidad es mover la instrucción de destino del salto, aplicándola, tal y como se muestra en el caso b. La instrucción sub es el destino del salto, es decir, dónde se saltaría. Se coge esa instrucción, se duplica, se inserta en esa ranura y se ejecuta. Evidentemente, si después no se produce salto, esa instrucción no depende de la instrucción de salto, por lo que se ejecuta de todos modos y, si al final se produce el salto, ya se tiene ejecutada, y si no se produce salto, se borra el contenido. Otra opción es mover la instrucción justo después del salto. En el ejemplo c se mueve la instrucción or para ocupar el hueco, pues no depende del salto.

Dependiendo de la información disponible durante la compilación se decide entre mover la instrucción anterior, la de destino del salto o la inmediatamente posterior.

Por ejemplo, si se tuviera una predicción de salto que dijera que el salto se va a tomar, en tiempo de compilación se movería la instrucción de destino del salto y se inserta en la ranura, ejecutándola de todos modos.

7. Ejercicio 1 boletin I

Considere un procesador tipo MIPS con arquitectura segmentada (pipeline) que tiene **dos bancos de registros separados** uno para números enteros y otro para números en coma flotante. El banco de registros enteros dispone de 32 registros. El banco de registros de coma flotante dispone de 16 registros de doble precisión (se cogen de 2 en 2 al ser de doble precisión) (\$f0, \$f2, . . . , \$f31).

Se dispone de suficiente ancho de banda de captación y decodificación como para que no se generen detenciones por esta causa y que se puede iniciar la ejecución de una instrucción en cada ciclo, excepto en los casos de detenciones debidas a dependencias de datos.

La instrucción bnez usa bifurcación retrasada con una ranura de retraso (delay slot). Es decir, la predicción de salto trata de mover una única instrucción.

Bucle:

- I1 ldc1 \$f0, 0(\$t0) (carga de un float)
- I2 ldc1 \$f2, 0(\$t1)
- I3 add.d \$f4, \$f0, \$f2 (suma de floats)
- I4 mul.d \$f4, \$f4, \$f6
- I5 sdc1 \$f4, (\$t2)
- I6 addi \$t0, \$t0, 8
- I7 addi \$t1, \$t1, 8
- I8 subi \$t3, \$t3, 1
- I9 bnez \$t3, bucle
- I10 addi \$t2, \$t2, 8

La siguiente tabla muestra las latencias adicionales de algunas categorías de instrucciones, en caso de que haya dependencia de datos. Si no hay dependencia de datos la latencia no es aplicable. Son los stalls que provoca cada instrucción.

Cuadro 1: Latencias adicionales por instrucción

Instrucción	Latencia adicional	Operación
ldc1	+2	Carga un valor de 64 bits en un registro de coma flotante.
sdcl	+2	Almacena un valor de 64 bits en memoria principal.
add.d	+4	Suma registros de coma flotante de doble precisión.
mul.d	+6	Multiplica registros de coma flotante de doble precisión.
addi	+0	Suma un valor a un registro entero.
subi	+0	Resta un valor a un registro entero.
bnez	+1	Salta si el valor de un registro no es cero.

1. Enumere las dependencias de datos RAW que hay en el código anterior.

Dependencias RAW encontradas:

\$f0: I1 -> I3

\$f2: I2 -> I3

\$f4: I3 -> I4

\$f4: I4 -> I5

\$t3: I8 -> I9

Estamos en MIPS, no puede haber WAW ni WAR.

2. Indique todas las detenciones que se producen al ejecutar una iteración del código anterior, e indique el número total de ciclos por iteración.

I1 ldc1 \$f0, 0(\$t0)

I2 ldc1 \$f2, 0(\$t1)

<Stall> x2 (I3 depende de las dos cargas anteriores-> DETENCION de 2 ciclos (según la tabla)

I3 add.d \$f4, \$f0, \$f2

<Stall> x4 (I4 depende de I3, detención de 4 ciclos según a tabla)

I4 mult.d \$f4, \$f4, \$f6

Stall x6 (I5 depende de I4, detención de 6 ciclos según la tabla)

I5 sdc1 \$f4, (\$t2)

I6 addi \$t0, \$t0, 8

I7 addi \$t1, \$t1, 8

I8 subi \$t3, \$t3, 1

I9 bnez \$t3, bucle

I10 addi \$t2, \$t3, 8

Al usar salto retardado con 1 ranura de retraso, no se produce parada al ejecutar bnez.

Total ciclos: 10 instrucciones + 2 paradas + 4 paradas + 6 paradas= 22 ciclos/iteración del lazo

3. Intente planificar el bucle para reducir el número de detenciones.

Planificar bucle: Reordenar instrucciones para evitar paradas.

Tenemos 2 stalls después de I2 debido a la dependencia de I3. I6 e I7 dependen de \$t0 y \$t1, no dependen de las instrucciones de arriba, así que las puedo mover antes de I3 para evitar los dos stalls.

Además tenemos 4 stalls después de I3. Puedo meter la I8 ahí para comerme un stall (quedan 3 en vez de 4).

Quedaría así:

I1 ldc1 \$f0, 0(\$t0)

I2 ldc1 \$f2, 0(\$t1)

I6 addi \$t0, \$t0, 8

I7 addi \$t1, \$t1, 8

I3 add.d \$f4, \$f0, \$f2

I8 subi \$t3, \$t3, 1

<stall> x3

I4 mul.d \$f4, \$f4, \$f6

<stall> x6

I5 sdc1 \$f4, (\$t2)

I9 bnez \$t3, bucle

I10 addi \$t2, \$t2, 8

Total= 22-2-1=19 ciclos/iteración

4. Desenrolle el bucle de manera que en cada iteración se procesen cuatro posiciones de los arrays y determine el speedup conseguido. Utilice nombres de registros reales (\$f0, \$f2, . . . , \$f30).

Desenrollar el lazo en bloques de 4 iteraciones significa hacer en una iteración el trabajo de 4 iteraciones junto. Cabe mencionar que esto se hace sobre el bucle original.

//I1 e I2 de la primera iteración

I1 ldc1 \$f0, (\$t0)

I2 ldc1 \$f2, (\$t1) // \$f2: 0+2 (al ser doubles los registros van de dos en dos)

//segunda iteracion

I1 ldc1 \$f8, 8(\$t0) //Usamos el \$f8 ya que en el bucle sin desenrollar se usa hasta \$f6

I2 ldc1 \$f10, 8(\$t1) // \$f10: \$f8+2

//desplazamos \$t0 y \$t1 en 8 porque es lo que se hacía en I6 e I7 en el bucle normal

//tercera

I1 ldc1 \$f14, 16(\$t0) //8+6

I2 ldc1 \$f16, 16(\$t1) //14+2

//desplazamos \$t0 y \$t1 en 16 porque se desplazaba en 8 en I6 e I7 en el bucle normal, al ser la tercera iteración se desplaza 16 del original

//cuarta

I1 ldc1 \$f20, 24(\$t0) //14+6

I2 ldc1 \$f22, 24(\$t1) //20+2

//I3 de la primera iteracion

I3 add.d \$f4, \$f0, \$f2

//I3 de la segunda, respetamos la relación de registros utilizados

I3 add.d \$f12 \$f8, \$f10

//tercera

I3 add.d \$f18, \$f14, \$f16

//cuarta

I3 add.d \$f24, \$f20, \$f22

<stall> //de 1 porque era de 4 pero se restan al meter tres instrucciones en el medio

//I4, usamos los registros correspondientes a cada iteracion

I4 mult.d \$f4,\$f4,\$f6

I4 mult.d \$f12,\$f12,\$f6

I4 mult.d \$f18,\$f18,\$f6

I4 mult.d \$f24,\$f4,\$f6 //a partir de aquí acabo la clase pero voy a dejar la solución //igual, sorry si me equivoco en algo

//ahora vendría la I5 pero depende de I4 con 6 stalls, al hacer 4 iteraciones ya se han restado 3 quedando otros 3. I6, I7 e I8 no dependen de I5, por lo que las vamos a desplazar antes de I5 para evitar los 3 stalls que quedan

I6 addi \$t0, \$t0, 32 //sumamos 32 porque se suma 8 en cada iteración, así hacemos //el trabajo de 4 instrucciones en una

I7 addi \$t1, \$t1, 32

I8 subi \$t3, \$t3, 4 //restamos 4 porque restamos 1 en cada iteración

```

I5      sdc1 $f4, ($t2)
I5      sdc1 $f12, 8($t2) //desplazamos 8 porque en I10 se desplazaba 8 este registro
//en cada iteracion
I5      sdc1 $f18, 16($t2)
I5      sdc1 $f24, 24 ($t2)
I9      bnez $t3, bucle
I10     addi $t2, 32 //sumamos 32 porque se sumaba 8 en cada iteracion

```

Se pretende hacer 4 iteraciones juntas en cada iteración del lazo para reducir el número de iteraciones (actual/4), con lo cual las instrucciones se deben repetir 4 veces escogiendo los registros adecuados. Así se reducen las paradas y se reduce el número de ciclos por iteración, ya que al tener más instrucciones que se pueden reordenar y paliar estas paradas. El número medio de ciclos se reduce, reduciendo el tiempo de ejecución. En total son 26 ciclos, es decir, $26/4=6,5$ ciclos por iteración.

Es recomendable reutilizar, en este caso, t0 y t1 siempre porque hay que desplazarlos en cada iteración (I6 e I7), si tuviésemos otros registros tendríamos que desplazarlos a cada uno, tal como está solo hay que desplazar una vez (una instrucción en vez de 4). Lo mismo con \$t2. En general, siempre que se pueda reutilizar, se recomienda.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura de Computadores

08-04-2021



Clase 08

Temas trabajados:

Tema 2

Índice

1. Repaso	3
2. Riesgos de control	4
3. Soluciones a riesgos de control.....	5
4. Extensión a operaciones multiciclo	8
5. Excepciones	9
6. MIPS R4000	9
7. Ejercicio	10

Recomendación: Acompañar la lectura de este documento junto con la presentación del tema 2 de la asignatura, ya que en varias ocasiones se hará referencia a diapositivas de la misma, para evitar aumentar el número de páginas de la bitácora.

1. Repaso

La segmentación permite acelerar la ejecución de los programas, no porque cada instrucción tarde menos tiempo en ejecutarse (la latencia de cada instrucción es la misma), sino porque se pueden ejecutar varias instrucciones de forma simultánea. Esto se debe a que la segmentación consiste en dividir el camino de datos en partes (segmentos), de modo que, mientras una instrucción se está ejecutando en una etapa de dicho camino, otra se encuentra en otra distinta.

Las distintas etapas del camino de datos se separan por registros con unidades de control dirigidas por señales de control específicas para cada etapa, permitiendo mantener las diferentes ejecuciones de las instrucciones aisladas las unas de las otras, aumentando así la productividad.

Lo ideal sería que en cada ciclo de reloj (teniendo en cuenta que cada etapa de la segmentación ocupa 1 ciclo de reloj) entrara una nueva instrucción a ejecutarse en el pipeline, pero esto no siempre es posible, debido a que existen riesgos, ya sean de control, de datos o estructurales. Estos riesgos indican que existe la posibilidad, por dependencias, de que el resultado de la ejecución de las instrucciones no sea correcto:

- Las **dependencias de control** provienen de los saltos. Los saltos, sean del tipo que sean, provocan que el contador de programa (PC) cambie su contenido para ir a otra ubicación a ejecutar código. En concreto los saltos incondicionales producen siempre un salto a una dirección, sin embargo, los condicionales dependen de la evaluación de una condición. En el caso de estos últimos hay que esperar a que se decida si hay salto o no para tener el valor correcto del contador del programa. Esto supone, a veces, esperar varios ciclos en la segmentación y, por lo tanto, aumentar el número de retardos.
- Los **riesgos de datos** ocurren porque existe la probabilidad de que una instrucción tenga que leer el contenido de un registro en el que otra tiene que escribir, pero que, debido a la segmentación, estas no se produzcan en el orden correcto y por consiguiente no se llegue a actualizar el valor correcto en el registro. Se concluye entonces que las dependencias entre datos suponen dependencias entre los registros que usan las instrucciones.
- Los **riesgos estructurales** se producen porque varias instrucciones quieren usar la misma unidad de ejecución (por ejemplo, una unidad de suma o una unidad de multiplicación).

De esta manera, los riesgos de control y de datos producen la pérdida de ciclos en la segmentación, ocasionando que esta no sea tan eficiente como podría desearse.

2. Riesgos de control

Los riesgos de control ocurren cuando surge la necesidad de tomar una decisión basada en los resultados de una instrucción mientras otras se están ejecutando. Están asociados a las instrucciones de salto, que tienen que hacer un cálculo y tomar una decisión basada en este (si se salta o no y, en caso afirmativo, a dónde).

En el pipeline del MIPS, para optimizar la ejecución de estas instrucciones, lo que se hace es calcular si hay salto y a dónde, todo en la misma etapa ID. Normalmente, se hace en la etapa de ejecución, porque es donde se encuentra la ALU, unidad donde se realiza la comprobación del salto en una instrucción de salto, por ejemplo, *bne*, donde se salta a la dirección construida en la etiqueta mediante direccionamiento relativo al PC* siempre que los primeros registros especificados posean contenidos de diferente valor.

* direccionamiento relativo al PC: Se desplaza la etiqueta dos posiciones a la izquierda (para pasarla a bytes, ya que está en palabras), se le extiende el signo (como esta etiqueta puede ser positiva o negativa, se escribe en C_2 .) hasta llegar a 32 bits y se suma con la dirección donde está la instrucción de salto + 4. Este direccionamiento requiere una comprobación de si se produce el salto y el cálculo de la dirección a la que se saltaría (que sería la cuenta previamente realizada). La primera comprobación se realiza normalmente en la etapa de ejecución de la instrucción en el MIPS, ya que es donde se encuentra la ALU, que realiza la diferencia del contenido de los registros implicados en la condición de salto y compara si esta diferencia es 0 o no. El cálculo de la dirección de salto puede realizarse antes de saber si hay salto. Se modifica el hardware del MIPS para que se pueda hacer todo en la etapa de decodificación (ID) de la instrucción. En general, aun así, se van a producir retardos, porque no es en la primera etapa donde se sabe si hay salto, sino en la segunda, en la etapa ID (Recordatorio: La primera etapa consiste en la búsqueda de la instrucción (IF) y la segunda, en la decodificación (ID)), por lo tanto va a haber riesgos asociados a los saltos, que se pueden solucionar de diferentes formas, como se verá más adelante.

Cabe mencionar que cuando se produce un salto (salto tomado), el PC se modifica siguiendo el direccionamiento relativo a PC. Por otro lado, un salto no tomado implica que se accede a la instrucción escrita a continuación de la del salto, (PC + 4) ya que cada instrucción en el MIPS tiene 4 bytes.

Los riesgos de control tienen el efecto siguiente:

Se tiene una instrucción de salto como *beq*, seguida de las instrucciones *and*, *or*, suma y carga, como se observa en la diapositiva 54 de la presentación del tema 2. Entonces, si se decide el salto en la etapa de memoria (como sucedería normalmente) y suponemos que no se produce el salto (es decir, seguimos cargando las instrucciones que hay detrás del salto como si este no se fuera a producir), en el momento en el que se resuelve, que sería cuando se produce la carga (cuando se puede calcular el valor correcto del PC), se habrán perdido 3 ciclos, porque al haber hecho una predicción de que no va a haber salto y realmente lo hay, las acciones realizadas por las 3 instrucciones *and*, *or* y suma tienen que ser eliminadas (realizar un *flush*), perdiendo, por lo tanto, 3 ciclos de la señal de reloj.

3. Soluciones a riesgos de control

Para la solución de los saltos en la etapa ID, se pueden comparar los registros en la zona señalada en la diapositiva 55, porque hacer una comparación de si el contenido de un registro es igual al de otro es simple. Por esta razón, estas operaciones se pueden realizar en la etapa ID, así como calcular el desplazamiento, extender el signo, sumar con el PC y, por tanto, se puede calcular también la dirección de salto en dicha etapa. Es decir, se puede resolver todo el salto en la segunda etapa.

En general, existen tres soluciones asociadas a evitar los riesgos producidos por los saltos.

- **Primera opción, parada asociada a un salto:** Consiste en parar la carga de la siguiente instrucción (o de las siguientes) hasta que se resuelva el salto. En el caso del pipeline del MIPS, sería parar 1 ciclo, pero puede haber pipelines con más etapas (más profundos) y que el salto se resuelva en la etapa 4, por lo que habría que parar 4 ciclos de emisión de instrucciones. En la diapositiva 56 se puede observar la aplicación de esta primera solución, donde se para la emisión de instrucciones hasta que se resuelve el salto, de manera que ya se puede calcular la instrucción correcta. Se habrá perdido por tanto 1 slot, es decir 1 ciclo, en el que se podría haber insertado una instrucción y no se inserta, representado por las burbujas/stall/NOP (no operación, instrucción fantasma que no hace nada). Si el pipeline es profundo, el coste de la parada es muy alto, entonces una forma de evitar este coste es mediante la predicción del salto (tercera opción que se estudiará), debido a que algunas veces se acertará lo que va a hacer el salto y eso permitirá ahorrar ciclos. Si la predicción fuera incorrecta, habrá que producir alguna parada.

En algunas ocasiones, los riesgos de datos están asociados a saltos, por ejemplo, en la diapositiva 57, donde los contenidos de los dos registros que se comparan mediante la instrucción *beq*, se calculan en instrucciones aritméticas anteriores. Por lo tanto, lo que se hace es adelantar hasta la etapa ID los valores desde la etapa de memoria en que se obtienen tanto para el registro \$1 como para el registro \$4 (realmente el valor del registro \$1 se podría adelantar desde la etapa de ejecución), mediante adelantamiento *forwarding*, pudiendo realizar la comparación del contenido de los dos registros. De esta manera, las instrucciones de salto a veces tienen que detenerse porque dependen de instrucciones anteriores, por ejemplo, en el caso de la diapositiva 58: si la instrucción que escribe el registro que usa la instrucción de salto para comparar es una suma no hay ningún problema, pero si es una carga, como esta tiene que ir a memoria, hasta la etapa de memoria no se tiene un valor correcto en el registro, con lo cual la instrucción de salto en la etapa ID no tendría los valores correctos y tendría que retrasarse un ciclo (una burbuja de un ciclo perdida hasta que en la etapa ID se puede realizar la comparación de los 2 registros).

En el caso de la diapositiva 59, se tiene otro ejemplo distinto. En este caso, hay que producir dos paradas, porque por un lado se tiene el registro \$0 del cual se conoce el contenido, pero la instrucción *beq* depende de un registro en el cual se produce una carga en la instrucción anterior, y la carga se produce en la etapa de memoria, de tal forma que hasta ese punto no se tiene el valor correcto. Como para la instrucción *beq* se necesita el valor correcto del registro en la etapa ID, se pierden 2 ciclos.

- **Segunda opción, decisión retardada:** Cuando hay retardos que dependen de la instrucción de salto, el compilador trata de mover varias instrucciones que están por encima del salto y que se van a ejecutar de todos modos, colocándolas después del salto,

de forma que se vayan ejecutando mientras se realizan los cálculos del salto sin que se produzcan paradas. El problema es que en algunas ocasiones no hay instrucciones independientes del salto para introducir en las ranuras existentes. Esta solución la adopta el MIPS. En la diapositiva 60 se observa un ejemplo que adopta esta solución donde se reubica la instrucción *addi*, que sería independiente del salto.

Dependiendo de cuantas paradas suponga un salto habrá que buscar más o menos instrucciones útiles para insertar en los huecos, lo que depende de cuántas etapas tenga el pipeline y de en qué etapas se calcula la decisión de salto.

Esta solución puede dar lugar a diferentes opciones, es decir, que el compilador, dependiendo de la implementación que se haga, mueve diferentes instrucciones a las ranuras disponibles: en la diapositiva 61 se presenta un ejemplo, con una sola ranura para ejecución. Se intenta mover una única instrucción para evitar la parada que se produciría en el MIPS (se mueve una instrucción localizada antes del salto que se va a ejecutar de todos modos y se sitúa después del mismo, evitando la parada de si se salta o no se salta). Otra posibilidad sería mover a la ranura la instrucción a la que se saltaría si se produjese el salto, duplicándola y ejecutándola mientras se decide dicho salto. Si posteriormente no se salta, el contenido de los registros de dicha instrucción debe ser borrado. También existe la opción de colocar en la ranura la instrucción que está inmediatamente después del salto.

- **Tercera opción, predicción de salto.** La predicción se puede hacer de dos maneras:

- **Estática:** Se basa en el comportamiento típico del salto, por ejemplo, si un salto está asociado a un lazo, como el salto al lazo casi siempre se va a tomar (excepto cuando se acaba), parece conveniente apostar siempre por la predicción de que el salto será tomado. Cuando no se toma, se tendrá que corregir. Es una predicción fija.
- **Dinámica:** Se mide el comportamiento real del salto, guardando la historia reciente en un registro del salto para finalmente decidir la predicción en base a esta. Va cambiando en función del histórico, no es fija.

Dependiendo de cómo sea la máquina se tomará una u otra decisión.

En las diapositivas 68 y 69, se muestra el funcionamiento del MIPS en función de si toma una predicción de salto fija (estática) no tomado y tomado, respectivamente. Una predicción de salto tomado no supone una gran ventaja frente a no tomado, debido a que la resolución del salto se calcula en la segunda etapa (ID) y solo se perdería un ciclo en caso de fallar la predicción. En pipelines más profundos sí supondría una gran ventaja establecer una predicción de salto tomado.

Los pipelines más profundos que el del MIPS y superescalares, que se verán en el tema 3, poseen una penalización por saltos más grande, además de que suele haber varios núcleos trabajando a la vez (se ejecutan instrucciones de varios cores juntas), por lo que se tiende a usar penalización dinámica del salto. Normalmente hay, por tanto, un buffer para la predicción de salto (tabla de memoria que almacena la historia del salto). Dicho buffer está indexado mediante las direcciones de la instrucción de salto: almacena el resultado del salto y si se toma o no se toma, de manera que para ejecutar un salto se busca en la tabla, suponiendo que el salto va a hacer lo mismo que fue haciendo históricamente. En una instrucción de salto se inicia la búsqueda de la siguiente instrucción, si se cree que no se va a saltar, o la del destino del salto, si se cree que se va a saltar (según la tabla), y si la predicción es errónea se debe vaciar el pipeline (perdiendo ciclos) y cambiar la predicción en la tabla. Esta predicción dinámica, aunque se pueden llegar a

producir burbujas, ahorra pérdida de ciclos, porque se va adaptando a lo que vaya haciendo el salto.

Es necesario calcular la dirección de salto independientemente de la predicción (en el MIPS se perdería 1 ciclo para calcular el salto), es decir, si se determina salto tomado, se debe calcular la dirección del salto en la etapa ID. En cuanto al buffer de destino del salto, en algunas ocasiones, se tendrá una caché con las direcciones de destino de los saltos, pudiendo ir a buscarlas ahí para ahorrarse la penalización previamente comentada. Si la instrucción se ejecuta muchas veces seguidas, si se salta puede saberse a dónde, guardándose en la caché de saltos. Con esta técnica se evitaría el tener que esperar si se toma la decisión de salto tomado que requiere saber la dirección destino del salto, evitando tardar un ciclo en el MIPS para calcularlo. Así, por ejemplo, en un bucle que tiene al final una instrucción *bnez*, siempre que se ejecute esa instrucción, se saltará de nuevo al inicio del bucle, cuya dirección se almacenaría en la caché, asociada a la instrucción *bnez*.

Los predictores de salto pueden ser de diferentes tipos:

- De 1 bit. La historia de un salto se guarda en 1 solo bit. Es un sistema secuencial síncrono de 2 estados, como se puede observar en el diagrama de estados de la diapositiva 73, donde cada instrucción se encuentra en un estado de salto no tomado (0): si el salto no se toma, la siguiente vez que se ejecute se mantendrá la misma predicción, pero si se toma el salto se pasa al estado de salto tomado (1) para la siguiente ejecución, donde si efectivamente se toma el salto se permanece en la misma predicción, y si no se toma, se cambia de nuevo al estado 0, y así sucesivamente. Es decir, si se cumple la predicción establecida, se mantiene esa predicción, si no se acierta, se establece la otra. Esta opción resulta de interés si se tiene un salto que casi siempre hace lo mismo, como en un lazo, pero cuando se observan otros tipos de comportamientos no asociados a bucles, entonces este predictor no es muy adecuado, ya que, por ejemplo, los saltos de los lazos internos se van a predecir erróneamente: en la última iteración del lazo más interno se tiene una predicción errónea que ocasiona cambiar la predicción a salto no tomado y en la nueva primera iteración del lazo interno, al haber cambiado dicha predicción también se fallará (ocurriendo, por tanto, 2 errores de predicción por iteración del lazo interno).
- De 2 bits: Es más estable, solo cambia la predicción después de 2 predicciones erróneas, al pasar por estados intermedios: Si se tiene la predicción de que el salto se toma y se produce, se mantiene la predicción actual. Sino se toma una vez, se pasa al estado intermedio donde se predice que se toma “débilmente”, y si se vuelve a tomar se pasará de nuevo al estado en que se toma “fuertemente”. En cambio, si la predicción volviese a ser errónea, se pasaría a no tomado “débilmente”. Esto permite acertar muchas veces, puesto que solo fallará una predicción en el lazo interno comentado anteriormente, que sería en la última iteración (la nueva primera iteración se acertaría debido a que se requieren dos predicciones erróneas para cambiar la predicción). Además, normalmente, se usa una caché para almacenar las predicciones, indexada mediante la etiqueta de salto, obteniendo a partir de esta la predicción asociada.

Los predictores también pueden estar basados en información local o global:

- Basados en información local: Tiene en cuenta la historia de un salto para tomar la decisión. Evita el 50% de fallos cuando se tienen secuencias del tipo de la

diapositiva 77. Para una etiqueta de s bits se obtiene la predicción en base a las últimas k veces que se ejecutó el salto. Una vez conocido el resultado real del salto, se actualiza el contenido de la historia.

- Basados en información global: Tiene en cuenta la historia de las últimas N instrucciones de salto, no solo las de una instrucción. Se usa un registro, como el de la diapositiva 78, al que llega el resultado de la última instrucción de salto. Se trata de un registro de desplazamiento con N posiciones, de manera cada vez que llega el resultado de un salto, el del más antiguo desaparece en caso de estar lleno. Siempre se tiene la historia de las N últimas instrucciones de salto, sea la misma repetida o diferentes instrucciones.

Se pueden combinar ambos tipos de predicciones, como se observa en la diapositiva 79, donde se tiene un predictor global con una memoria de 4K entradas que almacena 2 bits de estado por entrada y, por otro lado, un predictor local con memoria de 1K donde para cada entrada se asocia un estado de 3 bits. La unión se realiza mediante un multiplexor que escogerá si la predicción será local o global de manera adaptativa, dependiendo de la dirección del salto, que generará 2 bits que determinarán dicha decisión.

La tasa de acierto en procesadores reales depende del programa. Generalmente con técnicas híbridas como estas suele estar entre 85-99% de los saltos. En procesadores avanzados se suele emplear predicción dinámica combinando información global y local, usando en torno a 30 Kbits para almacenar la historia relacionada con los saltos (Cuanta más información se guarde, mejor será la predicción. Esta cantidad fue aumentando a medida que mejoró la tecnología de fabricación).

4. Extensión a operaciones multiciclo

En el caso del MIPS, se asume que las instrucciones se ejecutan en el mismo tiempo (tardan lo mismo). Sin embargo, aunque la parte de ejecución de las instrucciones, especialmente las de punto flotante, tienen la misma segmentación que las enteras, realmente la etapa de ejecución tiene diferente latencia dependiendo de la instrucción: en punto flotante la suma tiene 4 ciclos, la multiplicación, 7 y la división, 24.

Esto se puede representar como una única unidad hardware en la que hay que iterar un número determinado de veces, de manera que el número de ciclos total será distinto dependiendo de la instrucción. Por lo tanto, en el pipeline de la diapositiva 80 no se puede ejecutar una instrucción por ciclo en la etapa de ejecución, lo cual complica las dependencias entre instrucciones.

Resulta necesario detectar los conflictos estructurales en la unidad de divisiones y en el acceso al banco de registros, es decir, en la etapa de post-escritura (WB) de la ejecución. Para detectar dichos conflictos se para el paso de la instrucción a la etapa de ejecución si es una división y hay otra en ejecución o se para la división si se detecta que van a llegar a la vez a post-escritura más instrucciones que puertos de acceso se tiene en el banco de registros, porque en el momento en que unas instrucciones son más largas que otras, si están en el pipeline, aunque estén ordenadas, puede resultar que la que se introdujo antes tarde más en acabar que las que hay después, afectando al resultado de las instrucciones siguientes. En el caso de conflictos por dependencias tipo RAW es similar al pipeline del MIPS, aunque existen más casos posibles.

Los conflictos de tipo escritura después de escritura, son poco comunes. En este tipo de conflictos se va a tener una instrucción que escribe en un registro en el que también escribe una instrucción anterior, entonces lo que hay que decidir es si hay alguna instrucción en la etapa posterior a la decodificación que tiene como destino el mismo registro que la instrucción que está en la etapa de decodificación, en cuyo caso se debe hacer una parada para evitar que empiece la ejecución de la segunda instrucción antes de que acabe la primera. Entonces, cuando hay instrucciones multiciclo el número posible de conflictos es mayor.

5. Excepciones

Se producen excepciones en la ejecución cuando se producen syscall, desbordamientos... Todo ese tipo de ejecuciones anómalas se denominan excepciones, las cuales se producen por diferentes eventos que pueden suceder en la CPU. Requieren que el programa se ejecute de nuevo desde el punto en el que se produjo la excepción. Para poder reestablecer el estado, es necesario poder volver al estado anterior al que se produjo la excepción. Las excepciones que permiten recuperar dicho estado por completo se llaman **excepciones precisas** (no todos los procesadores soportan excepciones precisas, algunos solo recuperan una parte del estado en el que estaban). Es complicado gestionarlas sin que haya un problema de rendimiento, ya que volver a reestablecer el estado del sistema significa tener que almacenar información y perder ciclos, por lo que normalmente se suele intentar resolver la excepción más antigua en caso de haber varias, incorporando para ello un banco de registros que guarda los valores que produce cada instrucción. Para recuperar valores de una instrucción que fue ejecutada, se lee de esa posición de ese banco de registros (de manera similar a cuando se llama a una subrutina, donde se guarda información en la pila para poder volver al punto desde el que se llamó). Otra opción para recuperar los valores antiguos consiste en utilizar una rutina de software que permite repetir la ejecución de las instrucciones desde las que se produjo la excepción. También se puede detener la emisión de instrucciones si en la etapa de ejecución se detecta un riesgo de excepción.

Una excepción sería por ejemplo la que se produce en la diapositiva 83: Teniendo las instrucciones de división, suma y resta en doble precisión, independientes entre sí, es decir, se pueden ejecutar a la vez, la división terminará después de la suma y la resta en condiciones normales. Si se produjese una excepción en la ejecución de la división, por ejemplo, un desbordamiento, y la suma y la resta ya han acabado su ejecución, por lo que han cambiado el estado del programa pese a que no deberían haberse ejecutado, debería poder recuperarse el punto del programa en el que se produjo la excepción, ya que no deberían haberse actualizado los valores de *F10* y de *F12*. Para volver a dicho estado inicial se recuperan los valores de *F10* y *F12* que se tenía hasta ese momento usando cualquiera de las técnicas comentadas previamente. Normalmente se guardan los valores de los registros en una especie de caché, a modo de tabla de información, que permita recuperar los valores anteriores a la excepción (es la solución más eficiente).

6. MIPS R4000

El MIPS R4000, presentado en la diapositiva 84, es un procesador como el MIPS que usamos habitualmente, pero más complejo. Es un procesador con 8 etapas de pipeline en vez de

4, por lo que es más profundo que el del MIPS convencional. Se van a tener por tanto más estados al aumentar el número de etapas: La lectura de instrucciones de la caché se hace en dos etapas y media, seguida de la lectura de los registros. RF chequea la dirección de la línea caché para comprobar que la lectura se corresponde con una línea caché correcta (coincide con la etapa ID, además de esta comprobación). A continuación, se llevaría a cabo la etapa de ejecución, y posteriormente se empezarían a leer los datos (3 ciclos para dicha lectura). La etapa TC chequea la etiqueta de la dirección de la línea caché para determinar si la lectura fue un acierto.

En este pipeline hay más caminos de adelantamiento. Por ejemplo, se pueden adelantar datos desde DS, TC y WB (post-escritura) a la etapa de ejecución, porque en las primeras ya están leídos los datos (aunque no sean correctos, en cuyo caso se pueden corregir) y en la última ya están cargados correctamente.

Las instrucciones que dependen de una carga van a tener más latencia (van a esperar más ciclos) porque la carga tiene que ir a memoria y no se van a tener los datos hasta entonces, por lo que se tarda más ciclos que en el MIPS normal en tener el resultado correcto. También se tarda más en resolver los saltos, porque se necesita pasar 3 etapas de lectura (la decodificación no se hace hasta la etapa 3). Aquí se muestra un ejemplo de la instrucción de carga donde se pueden adelantar los datos desde la etapa DS hasta la de ejecución para una instrucción de suma. Si la instrucción DADD o DSUB no dependieran de R1 (si no estuvieran las paradas), también se podría hacer un adelantamiento de LD a la instrucción OR. Al haber dividido la parte de lectura de datos en varias etapas se tarda en tener un dato correcto hasta TC (6 ciclos).

En el pipeline del MIPS R4000 hay 4 etapas antes de conocer el destino del salto que se calcula en la etapa EX (La condición de salto y la dirección). Se utiliza salto retardado con 1 slot. En dicha la diapositiva 84 se ve un ejemplo de una predicción de salto no tomado (siendo BR la instrucción de salto), y en la ranura se introduce una instrucción útil. Si esa instrucción no debió ser ejecutada, en función de lo que determine el estado del salto, se deberá eliminar, sino (si es independiente del salto, por ejemplo), se deja.

En el caso de salto tomado, como se decide el salto en la etapa de ejecución, si se insertaron instrucciones que no debieron ser ejecutadas, porque no son coherentes con la decisión tomada, no se puede saber hasta que hayan pasado 3 ciclos, que es cuando se conoce cuál es el destino del salto. Si la predicción fue incorrecta, se pierden 3 ciclos.

Relacionado con todo lo anterior, en el caso del MIPS R4000, teniendo un porcentaje de saltos condicionales (tomados y no tomados) e incondicionales, en función de diferentes esquemas de predicción, se verá el *speedup* que se produce cuando se predicen saltos tomados y cuando se predicen saltos no tomados con respecto a no predecir nada.

7. Ejercicio

El enunciado del ejercicio comienza en la diapositiva 62. En él se especifica una máquina de 16 registros de doble precisión, es decir 32 registros en punto flotante juntándose el 0 con el 1, para crear un registro de doble precisión, el 2 con el 3, y así sucesivamente. El enunciado también expresa que se pueden obtener instrucciones en cada ciclo de reloj sin ningún problema, excepto cuando haya paradas por dependencias.

Se quiere ejecutar el código de la diapositiva 63. También se proporciona una tabla explicando la funcionalidad de cada una de las instrucciones presentes en el código, así como el

número de ciclos perdidos que se producen para cada una de estas cuando se suceden paradas por dependencia. Dados los valores iniciales de los registros, se pide lo expresado en la diapositiva 64.

NOTA: La instrucción *add.d* actúa como *add* pero trabaja con valores de doble precisión. Ídem con la multiplicación y la carga.

Si se enumeran las instrucciones del código como en el primer bucle de la diapositiva 65, dicho bucle tendrá diferentes paradas:

- Primero, debido a que la instrucción *add.d* necesita leer el registro *f2*, que es escrito por la instrucción de carga (habiendo por tanto 2 paradas, ya que se resuelve en el pipeline en la etapa de memoria la carga y el *add.d* lo necesita en la etapa de decodificación).
- En el caso de la multiplicación, se necesita el valor de *f4* sobre el que escribe la instrucción *add.d* (cuando se producen paradas por dependencia, se debe recurrir a la tabla, observándose que en este caso se producirían 4 ciclos de retardo). Cuando se carga un *double* en el registro *f4*, lo que se tiene es que se va a una dirección de memoria a almacenar el valor de lo que hay en *f4*, pero este se escribe en la de multiplicación, por lo que se tendrá una dependencia (según la tabla de 6 ciclos de parada).

Las dependencias resumidas que provocan paradas son las indicadas en la diapositiva 65. Como estamos en un pipeline tipo MIPS solo hay dependencias de tipo RAW (ni escritura después de escritura, ni escritura después de lectura). Las dependencias son las del registro *f0*, entre las instrucciones *I1* e *I3* (en *I1* se guarda en el registro *f0* un valor que se debe leer en *I3*); en el registro *f2*, entre la *I2* y la *I3*, que es la que provoca las 2 paradas; ...; en cuanto a la *I8*, esta escribe un valor en el registro *t0* que la instrucción *bnez* necesita, sin embargo, al usar un salto retardado con una ranura de retardo no se produce parada al ejecutar dicha instrucción de salto, porque la instrucción *addi* se introduce después que la *bnez*.

Por otro lado, se solicita la planificación del bucle. Planificar el bucle implica reordenar la ejecución de las instrucciones para reducir el número de paradas. Por lo que hay que mirar qué instrucciones se tienen que se podrían colocar donde se producen paradas. Por ejemplo, para evitar las paradas de las instrucciones *I1* e *I2* se insertan las instrucciones *I6* e *I7* a continuación de *I2*, ya que son independientes de las instrucciones previas y permiten cubrir las paradas. A continuación, se producirían 4 paradas entre *add.d* y la *mul.d*, por lo que se tendría que comprobar si existen 4 instrucciones adecuadas para insertar a continuación de *I3*. Como solo habría una instrucción que se pueda introducir, que sería la instrucción *subi*, se reduce el retardo a 3 paradas. Por otro lado, como la instrucción *mul.d* produce 6 paradas, pero no hay más instrucciones que se puedan insertar tras ella, no se puede reducir más, quedando el bucle finalmente como se muestra al final de la diapositiva 65: en el primer bucle (sin planificar) cada iteración tardaba 22 ciclos, mientras que en el siguiente cada iteración tarda 19 ciclos. Es decir, mediante planificación se consigue reducir 3 ciclos de ejecución en cada iteración del bucle.

Por último, se debe planificar el bucle para hacer 4 veces más trabajo en cada iteración. Entonces intenta colocar el mínimo número de instrucciones posibles que hagan 4 veces más trabajo y que eviten así posibles paradas, lo cual permite el desenrollamiento de lazo: aprovechar huecos y mejorar la planificación, a costa de tener más instrucciones, razón por la cual no siempre se consigue un beneficio. En la diapositiva 66 se muestra el bucle desenrollado. En el bucle original lo que se hace es mover 8 posiciones el contenido de *\$t0*, *\$t1* y *\$t2* en cada iteración. Al realizar 4 iteraciones juntas, se deben mover esos registros 32 posiciones, de 8 en 8. Por lo tanto, en vez de 2 cargas por iteración se realizarán 8: 4 para obtener información de las diferentes posiciones (0, 8, 16 y 24) de *\$t0* y las otras 4 de *\$t1*. También se realizarán 4 sumas en vez de 2,

con los registros pertinentes, almacenando en registros distintos el resultado. Como antes entre la suma y la multiplicación se producían 4 paradas y ahora se introdujeron 3 instrucciones de suma en el medio, solo se produce una parada. En las multiplicaciones, se multiplican los registros donde se almacenaron los resultados de las sumas por el registro *\$f6*. En este caso las instrucciones *addi*, como ya se explicó se moverán, 32 bytes de una sola tacada. A continuación, se realiza una carga (distanciada 5 instrucciones desde la multiplicación que operaba con el dato necesitado, por lo que las 6 paradas que se producían previamente se reducen a 1). La instrucción *subi* ahora actualiza el registro *\$t3* restándole 4 en vez de restarle 1, ya que ahora se realizan 4 iteraciones juntas, y en *addi* se suma 32 en *\$t2*, cuando antes se sumaban 8, por el mismo motivo.

Como resultado se obtienen 27 ciclos para 4 iteraciones, lo que equivale a 6.75 ciclos por iteración, obteniendo una reducción de ciclos por iteración mayor con respecto a la planificación anterior.



Grado en Ingeniería Informática

2º CURSO – 2º CUATRIMESTRE

ARQUITECTURA DE COMPUTADORES

15-04-2021

Autores:



Clase 09

Temas trabajados:

Tema 2: página 85-92 – Tema 3: página 0-12

Índice

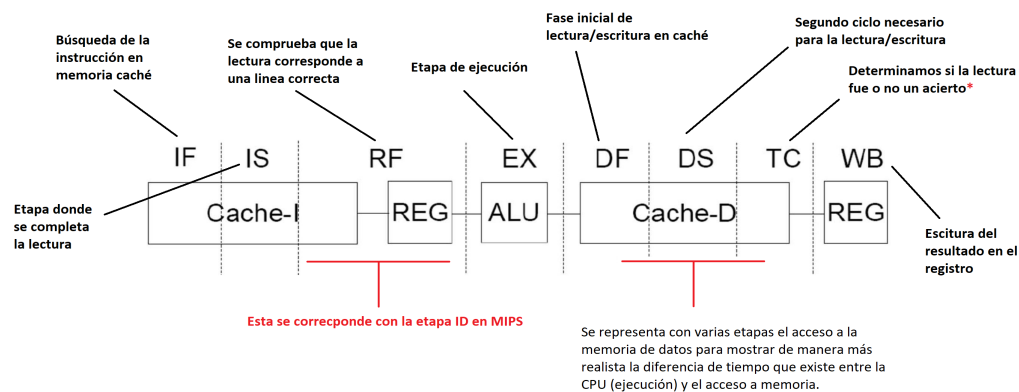
1. Caso práctico: MIPS R4000	2
2. Caso práctico: MIPS R4000	2
3. Caso práctico: MIPS R4000. Saltos	3
4. Caso de predicción de salto tomado	3
5. MIPS R4000 (pipeline más profundo que MIPS).	3
6. Speedup de predecir tomado sobre vaciar pipeline.	5
7. Speedup de predecir no tomado sobre vaciar pipeline.	5
8. Importante:	5
9. Conclusiones	5
1. Índice Tema 3	6
2. Instruction-Level Parallelism (ILP)	6
3. Dependencia de nombre	7
4. Ejemplos	7
5. Emisión múltiple	7
6. Emisión múltiple estática	8
7. Planificación de la emisión múltiple estática	8
8. MIPS con emisión dual estática	8
9. MIPS con emisión dual estática	9
10. Riesgos en la emisión dual del MIPS	9
11. Ejemplo de planificación	10

Tema 2

1. Caso práctico: MIPS R4000

MIPS R4000 (pipeline más profundo que MIPS)

1. 8 etapas de pipeline.
2. Las instrucciones se leen de memoria caché y los datos se escriben a memoria caché → se segmentan los accesos a caché en varias etapas.
3. RF: se chequea la etiqueta de la dirección de línea caché para comprobar que la lectura se corresponde con la línea correcta. También se hacen la decodificación de la instrucción, lectura de registros y detección de conflictos como en la etapa ID del pipeline del MIPS de 5 etapas.
4. TC: chequeo de la etiqueta de la dirección de la línea caché para determinar si la lectura/escritura fue un acierto.



*Hay que tener en cuenta que estos ciclos son solo en caso de acierto. Si se produce un fallo en la etapa TC serían necesarios más ciclos para realizar el acceso a memoria.

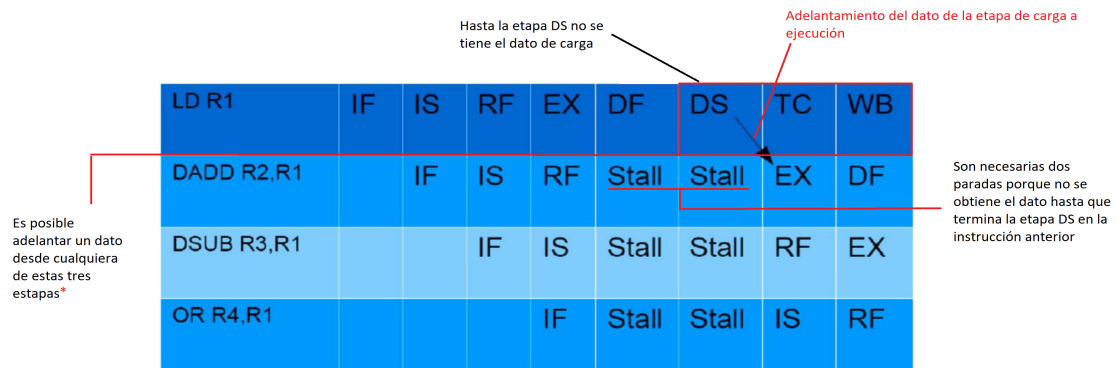
2. Caso práctico: MIPS R4000

Diferencias con el pipeline MIPS de 5 etapas:

- Más caminos de adelantamiento por ejemplo, desde DS, TC y WB a EX.
- Más latencia para las instrucciones dependientes de un Load.
- Más latencia en la resolución de saltos.

Ejemplo: se produce adelantamiento de LD a DADD.

- Si DADD y DSUB no dependiesen de LD, se produciría adelantamiento de LD a OR.



Hay que tener en cuenta que es posible adelantar el dato desde la etapa DS por lo que se adelanta **antes** de determinar que el acceso fue un acierto. De este modo, en caso de que el acceso no fuese un acierto sería necesario abortar todas las instrucciones a las que se les adelantó el dato.

3. Caso práctico: MIPS R4000. Saltos

MIPS R4000 (pipeline más profundo que MIPS):

- 4 etapas antes de conocer destino de bifurcación (computado en EX).
- En la etapa EX se calculan la condición de salto y la dirección de salto
- Se usa un slot para salto retardado y se predice para decidir qué se hace en los otros dos.

Caso de predicción de salto no tomado.

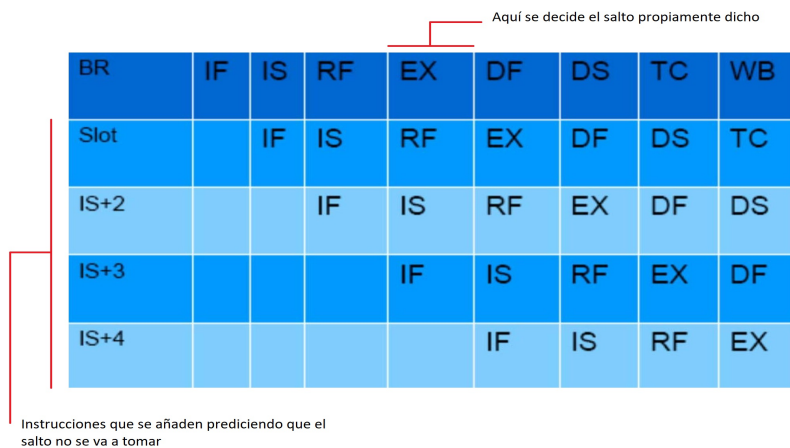
- Si la instrucción del slot no es coherente deberá anularse antes de que cambie el estado del sistema.

4. Caso de predicción de salto tomado

- Si la instrucción del slot no es coherente deberá anularse antes de que cambie el estado del sistema.
- Para los otros dos ciclos al ser predicción de salto no tomado se anularán las dos instrucciones emitidas cuando se resuelve el salto en la etapa EX.

5. MIPS R4000 (pipeline más profundo que MIPS).

- 4 etapas antes de conocer destino de bifurcación (computado en EX).
- Asumiendo que no hay detenciones de datos en comparaciones.



BR	IF	IS	RF	EX	DF	DS	TC	WB
Slot		IF	IS	RF	EX	DF	DS	TC
IS+2			IF	IS	NOP	NOP	NOP	NOP
IS+3				IF	NOP	NOP	NOP	NOP
Destino Salto					IF	IS	RF	EX

■ Frecuencia de bifurcaciones: 9 Bifurcación incondicional: 4 %.

- Bifurcación incondicional: 4 %
- Bifurcación condicional, no-tomada: 6 %
- Bifurcación condicional, tomada: 10 %

Esquema bifurcación	Penalización		
	incondicional	no-tomada	tomada
Vaciar pipeline	2	3	3
Predecir tomada	2	3	2
Predecir no-tomada	2	0	3

Esquema bifurcación	Bifurcación			Total
	incondicional	no-tomada	tomada	
Frecuencia	4%	6%	10%	20%
Vaciar pipeline	$0.04 \times 2 = 0.08$	$0.06 \times 3 = 0.18$	$0.10 \times 3 = 0.30$	0.56
Predecir tomada	$0.04 \times 2 = 0.08$	$0.06 \times 3 = 0.18$	$0.10 \times 2 = 0.20$	0.46
Predecir no-tomada	$0.04 \times 2 = 0.08$	$0.06 \times 0 = 0.00$	$0.10 \times 3 = 0.30$	0.38

6. Speedup de predecir tomado sobre vaciar pipeline.

$$S = (1 + 0.56) / (1 + 0.46) = 1.068$$

Ciclos por predecir siempre salto tomado

Ciclos de vaciar el pipeline haría que tardase $1 + 0.56$ ciclos

Mejora en predecir todos los saltos como tomados en comparación con vaciar el pipeline

7. Speedup de predecir no tomado sobre vaciar pipeline.

$$S = (1 + 0.56) / (1 + 0.038) = 1.130$$

8. Importante:

En cualquier caso, siempre es mejor predecir si el salto es tomado o no tomado que vaciar el pipeline

9. Conclusiones

- El hardware del camino de datos y la lógica de control se complican con la segmentación:
 - Caminos de anticipación (forwarding)
 - Lógica para detección de conflictos de datos
 - Lógica para predicción de saltos
- La segmentación mejora la productividad, pero no el tiempo de ejecución de cada instrucción. Incrementar la longitud del cauce segmentado no tienen por qué incrementar el rendimiento:
 - Los conflictos de datos hacen que incrementando la longitud se incremente el tiempo que se invierte en cada instrucción

- Los conflictos de control provocan que un incremento en la longitud ralentice los saltos
- La sobrecarga por los registros de segmentación (requieren un tiempo de acceso) puede limitar el aumento de la frecuencia de reloj.

Tema 3: Núcleos de procesamiento. Paralelismo a nivel de instrucción

1. Índice Tema 3

- Emisión múltiple estática y dinámica
 - MIPS con emisión dual estática
- Emisión múltiple dinámica
- Emisión múltiple y planificación estática
 - VLIW processors
- Planificación dinámica
 - Renombrado de registros
 - Algoritmo de Tomasulo
- Especulación basada en hardware
 - Buffers de reordenamiento
- Eficiencia energética

2. Instruction-Level Parallelism (ILP)

- Segmentación: permite ejecutar varias instrucciones en paralelo
- Cuando se explota el ILP, el objetivo es optimizar el CPI
 - $\text{Pipeline CPI} = \text{Pipeline ideal CPI} + \text{Structural stalls} + \text{Data hazard stalls} + \text{Control stalls}$
 - La predicción de saltos realizada por el compilador junto con unrolling y otras técnicas ayuda a exponer más paralelismo.
- Para mejorar el paralelismo a nivel de instrucción:
 - **Pipeline más profundo**
 - Menos trabajo por etapa $\Rightarrow$ ciclo de reloj más corto
 - **Emisión múltiple**
 - Replicar etapas del pipeline $\Rightarrow$ múltiples pipelines
 - Comenzar múltiples instrucciones en cada ciclo de reloj
 - $\text{CPI} < 1$, así que se usa como medida Instrucciones Por Ciclo (IPC = instrucciones/ciclos)

3. Dependencia de nombre

- Las dependencias pueden producir riesgos que limitan el paralelismo que puede ser explotado, como vimos en el tema 2.
- **Dependencias de nombre:** Dos instrucciones usan el mismo nombre de registro pero no hay flujo real de información.
 - No es una dependencia de datos real pero supone un problema cuando se reordenan instrucciones.
 - **(WAR)Antidependencia:** la instrucción j escribe un registro o posición de memoria que la instrucción i lee.
 - El orden inicial(i antes de j) se debe preservar
 - **(WAW)Dependencia de salida:** las instrucciones i y j escriben el mismo registro o la misma posición de memoria
 - El orden se debe preservar
- Para resolverlas se usa **renombrado de registros**

4. Ejemplos

Ejemplo 1:

```
add x1, x2, x3
beq x4, x0, L
sub x1, x1, x6
L: ...
or x7, x1, x8
```

Instrucción **or** dependiente de **add** y **sub**

Ejemplo 2:

```
add x1, x2, x3
beq x12, x0, skip
sub x4, x5, x6
add x5, x4, x9
skip:
or x7,x8,x9
```

Asumir que x4 no se usa después de "skip"

Es posible mover **sub** a antes del salto teniendo en cuenta que si finalmente se salta, es necesario cancelar la suma

5. Emisión múltiple

- Emisión múltiple estática
 - La realiza el compilador

- El compilador agrupa instrucciones para emitirlas juntas (normalmente 3 o 4 instrucciones)
- Las empaqueta en "issue packets"
- El compilador detecta y evita los riesgos que puede provocar este empaquetamiento.
- No es eficiente para aplicaciones que no sean científicas
- Emisión múltiple dinámica
 - La CPU examina el flujo de instrucciones y selecciona las que se deben emitir en cada ciclo.
 - El compilador puede ayudar **reordenamiento las instrucciones**
 - La CPU resuelve riesgos utilizando técnicas avanzadas en tiempo de ejecución
 - Usada sobre todo en servidores y procesadores de escritorio

6. Emisión múltiple estática

- El compilador agrupa las instrucciones en "issue packets" ("paquetes de emisión"):
 - Grupo de instrucciones que pueden ser emitidas en un solo ciclo
 - Determinado por los recursos del pipeline que requieran.
- Un paquete de emisión se puede ver como una instrucción muy larga:
 - Que especifica múltiples operaciones concurrentes
 - $\Rightarrow$ arquitecturas Very Long Instruction Word (VLIW)

7. Planificación de la emisión múltiple estática

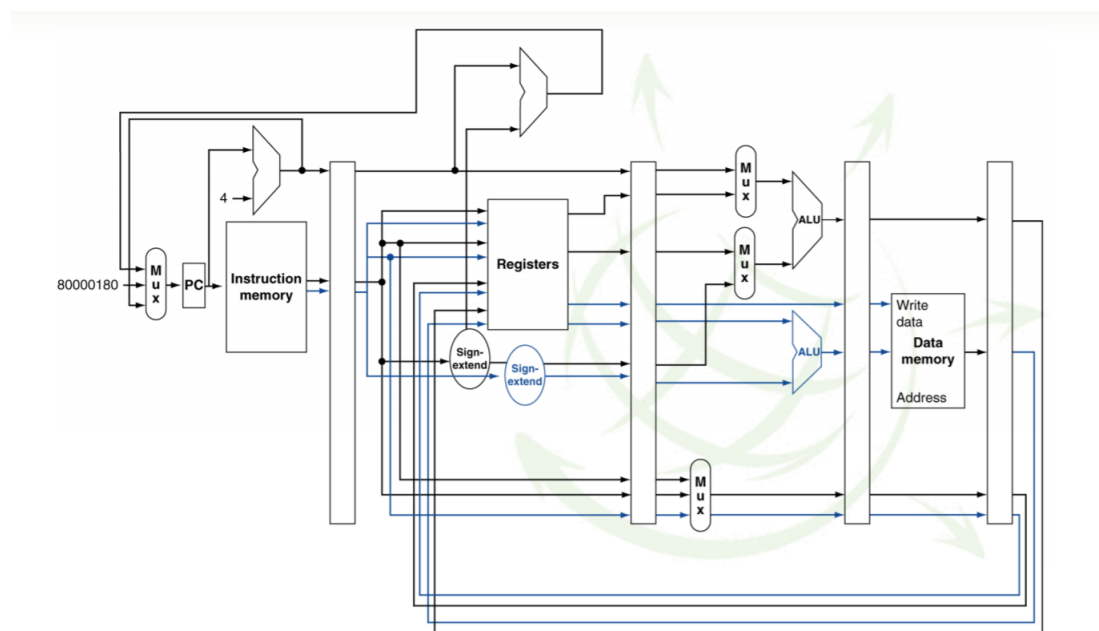
- El compilador debe eliminar algunos/todos los riesgos
 - Reordenar las instrucciones para conseguir paquete de emisión
 - Dentro de un paquete van instrucciones sin dependencias
 - Las instrucciones con dependencias no se pueden ejecutar simultáneamente
 - Posiblemente haya dependencias entre paquetes
 - Varía entre ISAs; el compilador debe soberlo!
 - Rellenar con "nop" (no operación, ciclo perdido, stalls, burbujas...) si es necesario.

8. MIPS con emisión dual estática

- Paquete de dos instrucciones emitidas:
 - Una instrucción ALU/salto
 - Una instrucción load/store
 - Alineación de 64-bits (2 instrucciones máximo)
 - ALU/salto, después load/store
 - Rellenar una instrucción no utilizada con "nop"

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

9. MIPS con emisión dual estática



Las líneas de color azul representa una instrucción y las de color negro otra. Como se puede observar, es necesario duplicar mucho hardware.

10. Riesgos en la emisión dual del MIPS

Se ejecutan más instrucciones en paralelo pero

- Surgen nuevos casos de riesgos de datos en EX:
 - El forwarding evita paradas que se producirían con emisión simple
 - Ahora no se puede usar el resultado de una instrucción ALU en una load/store en el mismo paquete de dos instrucciones:
 - add \$t0, \$s0, \$s1
 - load \$s2, 0(\$t0)

- Hay que dividirlo en dos paquetes o se producirá parada
- Riesgo de carga-usa(se carga un dato que se usa en otra instrucción):
 - Sigue habiendo un ciclo de latencia uso, pero ahora se ejecutan dos instrucciones.

Se requiere una planificación más agresiva

11. Ejemplo de planificación

- Planificación esto para el MIPS de emisión dual:

```

Loop: lw $t0, 0($s1)    # $t0=array el ement
      addu $t0, $t0,$s2  #add scalar in $s2
      sw $t0, 0($s1)    #store result
      addi $s1, $s1,-4  #decrement pointer
      bne $s1, $zero, Loop #branch $s1!=0
  
```

	ALU/branch	Load/store	cycle
Loop:	nop	lw \$t0 , 0(\$s1)	1
	addi \$s1 , \$s1,-4	nop	2
	addu \$t0 , \$t0 , \$s2	nop	3
	bne \$s1 , \$zero, Loop	sw \$t0 , 4(\$s1)	4

$$IPC = \frac{5}{4} = 1,25 \text{ (el máxima sería IPC=2)}$$



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura de Computadores

22-04-2021



Clase 10

Temas trabajados:

Unrolling

Emisión múltiple y planificación estática

Especulación

Índice

1. Introducción	3
2. Unrolling (desenrollamiento) de lazos	3
2.1 Ejemplo de unrolling	3
3. Emisión múltiple y planificación estática	4
3.1 Procesadores VLIW.....	4
4. Emisión múltiple dinámica	5
4.1 CPU planificada dinámicamente	6
4.2 Renombrado de registros.....	7
4.2.1 Cómo se realiza el renombrado	7
4.2.2 Estaciones de reserva (RS)	8
4.2.3 Algoritmo de Tomasulo	8
5. Especulación.....	9
5.1 Tipos de especulación	10
5.1.1 Especulación basada en hardware. Buffers de reordenamiento (ROB).....	10

1. Introducción

Otra hora Dora, en la que estuvimos los primeros 20 minutos de clase, rememorando lo larga que fue la clase anterior. Repasamos otra vez lo explicado, lo cuál se encuentra en la bitácora previa a esta.

2. Unrolling (desenrollamiento) de lazos

El **unrolling** consiste en replicar el cuerpo del lazo para exponer más paralelismo. Esto reduce la sobrecarga por control del lazo. Por ejemplo, si yo tengo un lazo de 4 instrucciones que hace 500 interacciones, que sucede si hago 250 interacciones, pero me las apaño para hacer el mismo trabajo. Pues la solución es aumentar la carga de trabajo de las instrucciones.

Se utilizan diferentes registros para la replicación, a esto se le llama “renombrado de registros”, evita “antidependencias” asociadas a los lazos”. Por ejemplo, un store seguido por load en el mismo registro (WAR). También se le conoce como “dependencia de nombre”, porque reusa el nombre de un registro.

2.1 Ejemplo de unrolling

Vamos emitiendo las instrucciones duales del mips como emisión estática dual. El compilador decide que dos instrucciones emiten, no pueden tomar dependencias, se emiten dos en cada ciclo.

Si tenemos una ALU/Branch o una Load/store, tenemos la posibilidad de tener las dos que no dependan entre sí.

En la tabla, podemos ver un ejemplo de desenrollamiento, estamos ejecutando 14 instrucciones en 8 ciclos. Se podrían ejecutar 16, pero solo se han encontrado 14 que no tengan dependencias.

En el 1 ciclo, tenemos el control del lazo. Tenemos en el lazo de la diapositiva de ejemplo de planificación, carga, suma, almacenamiento, índice de lazo y salto. El índice le resta un 4. Pero en este caso, se le resta 16, es decir se han hecho 4 interacciones del lazo juntas.

Tenemos también una única instrucción de salto, que es la del ciclo 8 a la izquierda.

	ALU/branch	Load/store	cycle
Loop:	addi \$s1, \$s1, -16	lw \$t0, 0(\$s1)	1
	nop	lw \$t1, 12(\$s1)	2
	addu \$t0, \$t0, \$s2	lw \$t2, 8(\$s1)	3
	addu \$t1, \$t1, \$s2	lw \$t3, 4(\$s1)	4
	addu \$t2, \$t2, \$s2	sw \$t0, 16(\$s1)	5
	addu \$t3, \$t3, \$s2	sw \$t1, 12(\$s1)	6
	nop	sw \$t2, 8(\$s1)	7
	bne \$s1, \$zero, Loop	sw \$t3, 4(\$s1)	8

Como conseguimos 14 instrucciones en 8 ciclos, el valor de las instrucciones por ciclo es 1.75.

$$IPC = 14/8 = 1.75$$

Es más próximo a 2, pero a coste de usar más registros y construir un código más largo.

3. Emisión múltiple y planificación estática

Para tener $CPI < 1$, hay que completar varias instrucciones por ciclo de reloj.

Soluciones:

- **Procesadores superescalares con planificación estática**, por ejemplo, los multicore.
- **Procesadores VLIW** (very long instruction Word), consiste en juntar instrucciones dentro de otra.
- **Procesadores superescalares con planificación dinámica**, también pueden ser los multicore. La planificación dinámica es más agresiva, podemos cambiar sobre la marcha el orden de ejecución de las instrucciones.

3.1 Procesadores VLIW

Empaquetan múltiples operaciones en una instrucción.

Ejemplo de tipo de instrucción en procesador VLIW:

- Una instrucción con enteros o de salto.
- Dos operaciones independientes en punto flotante.
- Dos referencias a memoria independientes.

Debe haber suficiente paralelismo en el código como para llenar todos los slots disponibles.

En el siguiente ejemplo hay muchos huecos sin cubrir, ya que no todas las instrucciones se pueden ejecutar en paralelo. Por lo que realizar múltiples operaciones en una misma instrucción no es tan sencillo.

Memory reference 1	Memory reference 2	FP operation 1	FP operation 2	Integer operation/branch
f1d f0,0(x1)	f1d f6,-8(x1)			
f1d f10,-16(x1)	f1d f14,-24(x1)			
f1d f18,-32(x1)	f1d f22,-40(x1)	fadd.d f4,f0,f2	fadd.d f8,f6,f2	
f1d f26,-48(x1)		fadd.d f12,f0,f2	fadd.d f16,f14,f2	
		fadd.d f20,f18,f2	fadd.d f24,f22,f2	
f1d f4,0(x1)	f1d f8,-8(x1)	fadd.d f28,f26,f24		
f1d f12,-16(x1)	f1d f16,-24(x1)			addi x1,x1,-56
f1d f20,24(x1)	f1d f24,16(x1)			
f1d f28,8(x1)				bne x1,x2,Loop

Desventajas de esto:

- Encontrar paralelismo estáticamente.
- Tamaño del código.
- No hay hardware de detección de riesgos.

4. Emisión múltiple dinámica

Los procesadores que la realizan se llaman procesadores “superescalares”.

En ella la CPU decide si emitir 0,1,2... instrucciones en cada ciclo, evitando riesgos estructurales y de datos.

También evita la necesidad de planificación realizada por el compilador. Hacer esta planificación podría ayudar. La semántica del código la asegura la CPU.

Esta emisión permite a la CPU ejecutar instrucciones fuera de orden para evitar paradas. Esto llevar el resultado a los registros en orden. Todas las cosas que dependen del resultado de una instrucción se le llama “commit” de una instrucción, que es la fase final. Esta sería la fase en la que se modifican los registros y saltos con valores correctos finales o direcciones de memoria.

Ejemplo:

<pre>lw \$t0, 20(\$s2) addu \$t1, \$t0, \$t2 sub \$s4, \$s4, \$t3 slti \$t5, \$s4, 20</pre>	Aquí se podría comenzar a realizar sub, mientras addu está esperando por lw.
--	---

Se reordenan las instrucciones para reducir paradas a la vez que se mantiene el flujo de datos del programa. Se realiza mediante hardware, no lo hace el compilador.

Ventajas de la planificación dinámica:

- El compilador no necesita conocer la infraestructura hardware.
- Puede gestionar casos en los que las dependencias no se conocen en tiempo de compilación.

Desventajas:

- Hay un incremento sustancial de complejidad hardware.
- Complica la gestión de excepciones.

La planificación dinámica implica:

- Ejecución fuera de orden.
- Terminación fuera de orden.

Se tiene que hacer el commit de forma que no se cambia la semántica del programa.

Ejemplo 1:

<pre>fdiv.d f0,f2,f4 fadd.d f10,f0,f8 fsub.d f12,f8,f14</pre>	fsub.d no es dependiente, se emite después de fadd.d y eso no causará problema porque ambas tardarán el mismo número de ciclos en ejecutarse.
---	--

¿Por qué hacer planificación dinámica y no dejar que el compilador planifique directamente el código?

- Porque **no todas las paradas son predecibles**, ejemplo: fallos caché.
- Porque **no siempre se puede planificar cuando hay un salto**, el resultado del salto se determina dinámicamente.
- Porque **diferentes implementaciones de un ISA pueden tener diferentes latencias** y dar lugar a diferentes riesgos.

Ejemplo 2:

fdiv.d f0,f2,f4

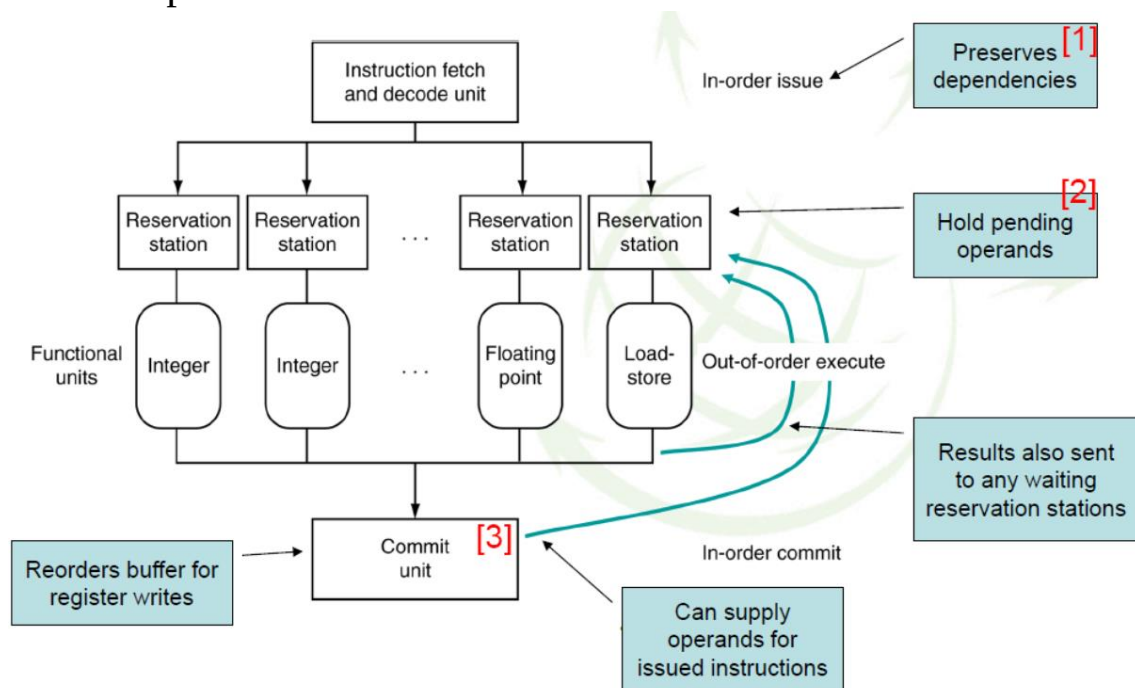
fmul.d f6,f0,f8

fadd.d f0,f10,f14

fadd.d no depende de otras, pero la antidependencia (WAR en f0) hace imposible que pueda emitirse tras **fmul.d** si no se aplica renombrado de registros.

La planificación dinámica adecuada requiere de nuevos instrumentos.

4.1 CPU planificada dinámicamente



1. Las **instrucciones se emiten en orden** y se van almacenando en las llamadas “estaciones de reserva”.
2. Aquí **se almacenan operandos que están pendientes**, cuando es posible se ejecutan operaciones de enteros, de punto flotante, de carga...
3. Aquí **se reordena el buffer** para las escrituras en registros, ya que es importante que las instrucciones se terminen en orden.

Para la planificación dinámica hay dos técnicas básicas:

- **Marcador:** Retener las instrucciones emitidas hasta que haya recursos suficientes y no haya riesgos de datos.
- **Tomasulo:** Elimina dependencias de datos WAR y WAW realizando renombrado de registros.

4.2 Renombrado de registros

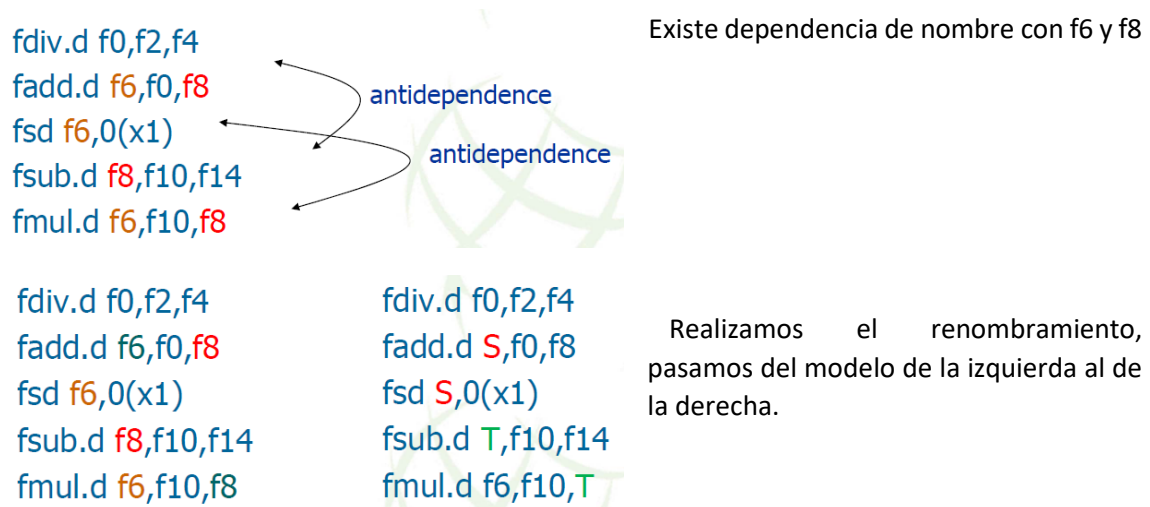
Las estaciones de reserva y los buffers de reordenamiento proporcionan renombrado de registros.

El renombrado de registros resuelve las dependencias de nombre.

Cuando una instrucción se emite a una estación de reserva:

- **Si el operando está disponible** en el almacén de registros o en un buffer de reordenamiento.
 - Es copiado a la estación de reserva.
 - Si no se requiere en el registro para más adelante, se puede sobrescribir.
- **Si el operando NO está disponible.**
 - Se proporcionará a la estación de reserva.
 - Podría no ser necesario actualizar el registro.

Ejemplo 3:



Ahora solo nos quedan los riesgos RAW, que pueden ser ordenados.

4.2.1 Cómo se realiza el renombrado

- **Aproximación de Tomasulo**
 - Rastrea cuando los operandos están disponibles.
 - Introduce renombrado de registros en el hardware. Esto minimiza riesgos WAW and WAR.
- El cambio de nombre de los registros se realiza mediante las **estaciones de reserva (RS)**.
 - La estación de reserva contiene:
 - La instrucción.

- Valores del operando en el buffer (cuando están disponibles) y validez de cada uno de ellos.
- Número de estación de reserva de la instrucción que proporciona los valores del operando.

4.2.2 Estaciones de reserva (RS)

Una **estación de reserva** es una **entrada por cada instrucción en ejecución**.

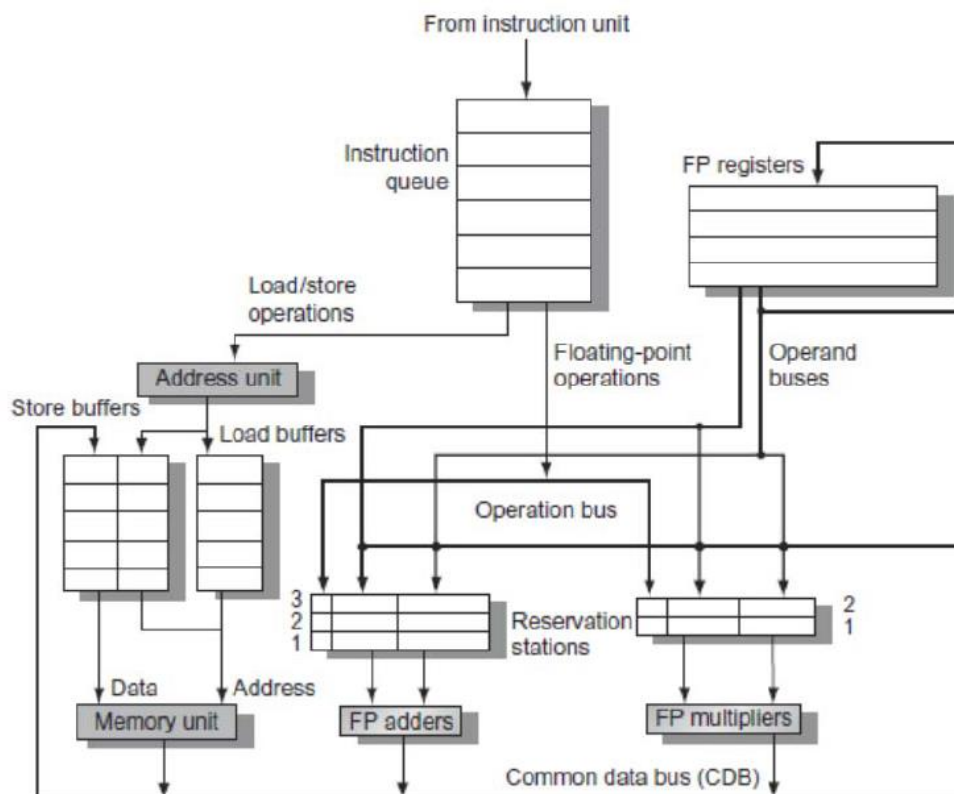
Las instrucciones entran a las RS por orden, pero pueden ejecutarse fuera de orden dependiendo de la disponibilidad de los operandos de entrada de las instrucciones.

Campos de una entrada:

- **B**: indica si la entrada está ocupada por instrucción.
- **I**: campo código de la instrucción.
- **OP1 y OP2**, operandos de entrada de la instrucción: proceden de registros o de los caminos de bypass de las unidades de ejecución. Alguno puede ser un identificador de registro renombrado, ósea, que su valor está siendo calculado.
- **V1 y V2** indica si el operando es válido.
- **R** indica si la instrucción está lista para emisión.



4.2.3 Algoritmo de Tomasulo



Elementos:

- **La estación de reserva (RS)** obtiene y almacena en la memoria un operando tan pronto como está disponible.
- Las instrucciones pendientes designan el RS al que enviarán su salida.
 - Los valores de los resultados se difunden en un bus de resultados, llamado **bus de datos común (CDB)**.
- Sólo la **última salida actualiza el archivo de registros**.
- A medida que se emiten instrucciones, los **especificadores de registro** se renombran con la estación de reserva.
- Puede haber más estaciones de reserva que registros.
- **Buffers de carga y almacenamiento:** contienen datos y direcciones, actúan como estaciones de reserva.

Pasos del algoritmo:

- **Emitir:**
 - Obtener la siguiente instrucción de la cola FIFO.
 - Si la RS está disponible, emite la instrucción a la RS con los valores del operando si están disponibles.
 - Si los valores del operando no están disponibles, detiene la instrucción.
- **Ejecutar:**
 - Cuando el operando está disponible, lo almacena en cualquier estación de reserva que lo esté esperando.
 - Cuando todos los operandos estén listos, emite la instrucción.
 - Las caras y el almacenamiento se mantienen en el orden del programa a través de la dirección efectiva.
 - No se permite indicar la ejecución de ninguna instrucción hasta que se hayan completado todas las ramas que la preceden en el orden del programa.
- **Escribir el resultado:**
 - Escribir el resultado en el CDB en las estaciones de reserva y en los buffers de almacenamiento (Los almacenamientos deben esperar hasta que la dirección y los valores estén disponibles).

5. Especulación

Consiste en especular acerca de lo que hará una instrucción:

- Comenzar la operación tan pronto como sea posible.
- Chequear si se acertó:
 - Si es que sí, completar la operación.
 - Si no, retroceder y hacer lo correcto.
- Es común a la emisión múltiple estática y dinámica.
- Ejemplos:
 - **Especular acerca del resultado del salto:**
 - Se hace para no actualizar (commit) un resultado en los registros hasta que se determine el resultado del salto.
 - Se vio en el tema anterior técnicas para la predicción de salto.

- **Especular acerca de la carga:**
 - Evitar retado por carga y fallo caché.
 - Predecir la dirección efectiva.
 - Predecir el valor cargado.
 - Pasar los valores almacenados a la unidad de carga.
 - No asignar la carga hasta que la especulación se resuelva.

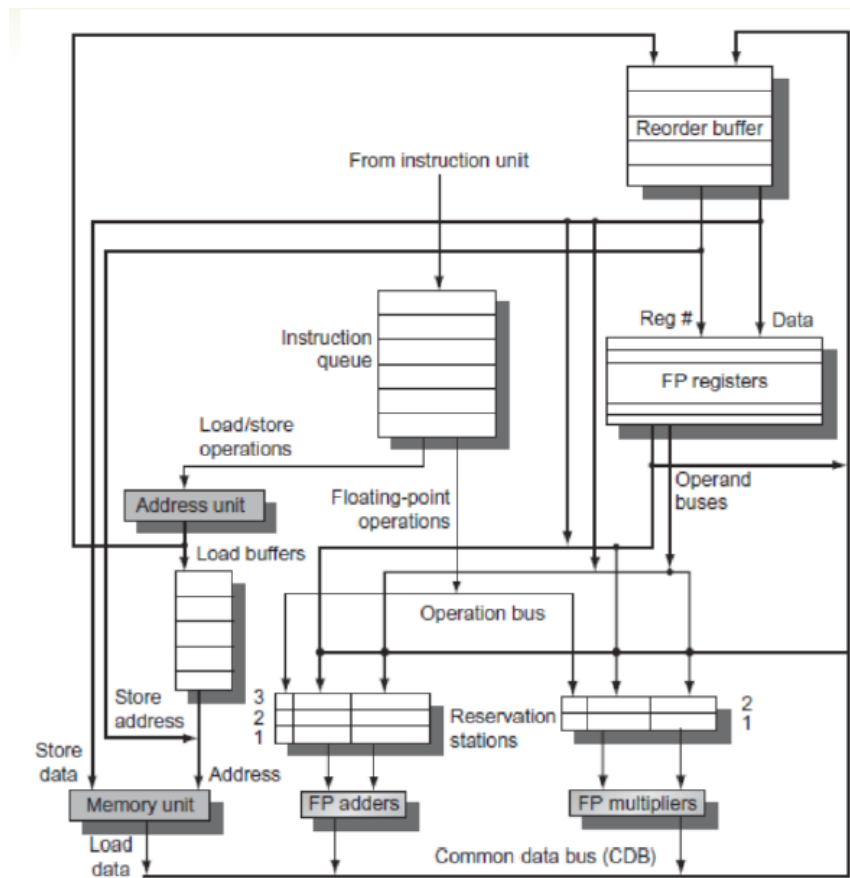
5.1 Tipos de especulación

- **Realizada por el compilador**, que puede reordenar instrucciones.
 - Por ejemplo: mover la carga a antes de un salto.
 - Puede incluir instrucciones de “arreglo” para recuperarse de un error de una especulación incorrecta.
- **Realizada por hardware**, que puede buscar por adelantado instrucciones para ejecutar.
 - Meter los resultados de un buffer hasta que se determina si se necesitan realmente.
 - Vaciar los buffers si la especulación fue incorrecta.

5.1.1 Especulación basada en hardware. Buffers de reordenamiento (ROB)

- **ROB** – almacena el resultado de la instrucción entre que se completa y se produce el commit (finalización o retirada de la instrucción).
- Se retiran del ROB las instrucciones en el orden del programa secuencial.
- **Campos básicos** de cada entrada:
 - **Tipo de instrucción:** Branch/store/register.
 - **Dirección de la instrucción.**
 - **Campo de destino:** número de registro renombrado.
 - **Campo de valor:** valor de salida.
 - **Campo de estado** que indica el estado de la instrucción.
 - **Campo que indica si la instrucción está en estado especulativo.**
- Exige modificar las estaciones de reserva: la fuente de un operando es ahora el ROB en lugar de una unidad funcional.
- Posibles estados para una instrucción:
 - **En ejecución:** Si la instrucción no acabó de ejecutarse.
 - **Finalizado:** Al finalizar la ejecución se escribe el resultado en un registro renombrado (si se produce un resultado), o en un buffer previo a escribir en la memoria si se trata de una instrucción de escritura en memoria.
 - **Completada:** Cuando la instrucción modifica los registros del núcleo, o sea, su estado, es decir, cuando ya se hizo el commit.
 - **Retirada:** si el destino es un registro que coincide con completada, si el destino es memoria, se retira cuando se escribe en la caché de datos.

Representación gráfica de los buffers de reordenamiento:





Escola
Técnica
Superior
de Enxeñaría



Grao en Enxeñaría Informática

2º curso – 2º cuadrimestre

Arquitectura de computadores

29-04-2021



Clase 11

Temas traballados: Tema 3

Índice

1. Planificación estática e dinámica (repaso)	3
Introducción.....	3
Paralelismo a nivel de instrucción	3
Emisión múltiple dinámica	3
Planificación dinámica.....	4
Especulación basada en hardware. Buffers de reordenamiento (ROB).....	8
2. Arquitectura de alto nivel de un núcleo.....	9
Canto especular?	10
Funciona a emisión múltiple?	11
Planificación dinámica, emisión múltiple e especulación	11
Eficiencia energética.....	11
ARM Cortex-A8 Pipeline	12
Core i7 Pipeline	13
Falacias	13
3. Curiosidades	14
4. Conclusiones.....	14

1. Planificación estática e dinámica (repaso)

Dora volveu meter nesta clase un repaso de 47 minutos. Neste apartado trátanse temas que xa foron recollidos na bitácora anterior, mais o repaso recolleuse nesta para que non quedara nada atrás e realizar algunhas aclaracións. A pesar deste feito, como cabe esperar dun simple repaso, a precisión non é a mesma que a da clase anterior.

Introdución

A emisión dinámica de instrucións (mediante a cal o compilador escolle o número de instrucións que se emiten en cada ciclo en función de distintas circunstancias) pode aumentar a eficiencia e o IPC de forma considerable, mais ó emitir varias instrucións por ciclo aumentan as dependencias entre elas, polo que tamén se vai necesitar unha planificación dinámica. A planificación estática tamén se pode levar a cabo (mediante reordenacións de instrucións por parte do planificador) pero ten moitas limitacións e é máis práctico e eficiente realizar a planificación de xeito dinámico. Esta lévese a cabo no hardware e fai uso principalmente de dous elementos: estacións de reserva e buffers de reordenamento.

Este feito vese reforzado se as distintas instrucións (no caso dun ISA de tipo CISC) se poden dividir en instrucións máis pequenas, polo que é máis eficiente e pódense conseguir uns maiores beneficios se as ordenamos unha vez divididas.

Se imos un paso máis alá da planificación dinámica chegamos á especulación. As predicións á hora de tomar saltos ou de realizar cargas de datos dende memoria son moi útiles, pois poden minimizar paradas e aumentar a eficiencia e velocidade da execución.

Paralelismo a nivel de instrución

O obxectivo de explotar o paralelismo a nivel de instrución é optimizar o CPI, mais esta optimización é entorpecida polas paradas estruturais, os riscos de datos e polas paradas de control debidas ós saltos.

$$\text{Pipeline CPI} = \text{Ideal pipeline CPI} + \text{Structural stalls} + \text{Data hazard stalls} + \text{Control stalls}$$

Coa finalidade de mellorar o paralelismo a nivel de instrución pódese pensar en utilizar un pipeline máis profundo, o que nos levaría a ter un ciclo máis curto, maior frecuencia e un consumo de enerxía máis alto, emitindo máis calor e tendo máis necesidades de disipación. Por este motivo é que o tamaño do pipeline está limitado. Outra posibilidade é duplicar etapas do pipeline para así poder emitir varias instrucións en cada ciclo de reloxo. Deste xeito conseguimos reducir o CPI por debaixo de 1 e comezamos a utilizar como medida o IPC (instrucións por ciclo) que é a inversa do CPI.

Víronse os riscos deste tipo de emisión, o exemplo concreto do MIPS que emite dúas instrucións por ciclo e o funcionamento dos procesadores VLIW, que poseen algunhas desvantaxes fronte a emisión múltiple dinámica de instrucións como a necesidade de atopar o paralelismo estaticamente á hora de ordear as instrucións para emitilas, o tamaño do código neste tipo de arquitecturas e a ausencia de hardware de detección de riscos.

Emisión múltiple dinámica

É a realizada polos procesadores denominados como “superescalares”, podendo estes realizar emisión múltiple estática ou dinámica. Deciden se emitir 0, 1, 2, ... en cada ciclo, dependendo das circunstancias e de como sexa a semántica do código, coa finalidade de evitar os riscos estruturais e de datos de forma que o compilador non sexa o encargado da planificación.

Permite a CPU executar instrucións fóra de orde para evitar paradas e aproveitar mellor o hardware que temos dispoñible.

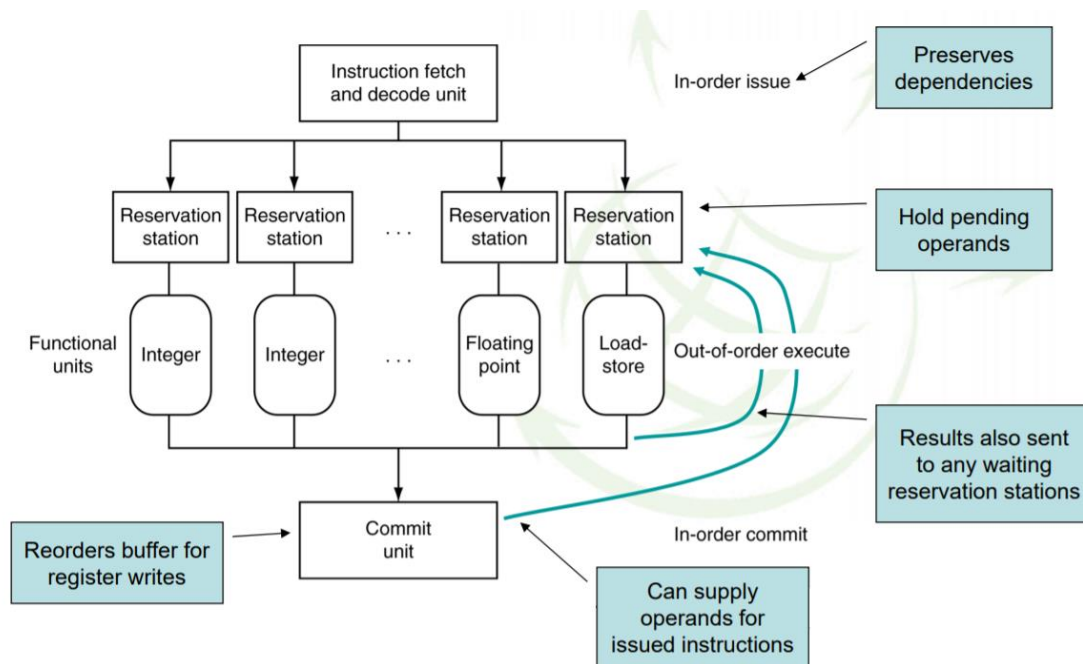
Planificación dinámica

Consiste en reordenar as instrucións para reducir paradas á vez que se mantén o fluxo de datos do programa. Realízase mediante hardware, non o fai o compilador.

- Vantaxes:
 - O compilador non necesita coñecer a infraestrutura hardware.
 - Pode xestionar casos nas que as dependencias non se coñecen en tempo de compilación, como fallos caché ou saltos.
- Desvantaxes:
 - Aumenta a complexidade do hardware
 - Complica a xestión de excepcións, pois cando se detectan o sistema ten que ser capaz de volver ó seu estado anterior, mais complicase se se están a executar operacións desordenadas.

A execución das instrucións é fora de orde e terminan fora de orde, mais a forma da que se modifican os rexistros é ordenada. As instrucións non fan commit ata que se poida cumprir a orde secuencial do código.

Un exemplo de CPU planificada dinamicamente é o seguinte:

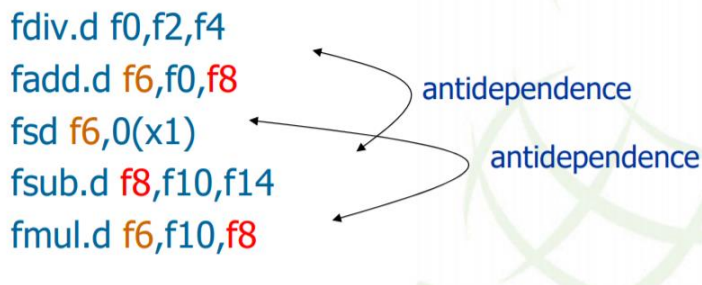


Pódese ver como existe unha unidade de busca e decodificación de instrucións, que o fai en orde para preservar as dependencias. Estas transmítense a unhas unidades chamadas estacións de reserva. En cada unha destas hai entradas nas que se atopan distintas instrucións xunto cunha serie de datos a maiores. Aquí as operacións esperan a ter todos os operandos necesarios (que pode conseguir a través dos camiños procedentes de unidades posteriores a través do *bypassing*). Debido a que estas execucións se producen fora de orde, de forma paralela e non todas as instrucións tardan o mesmo número de ciclos en ser resoltas, os riscos de datos de tipo WAR e WAW si que poden ser moi importantes. Unha vez as instrucións saen da estación de reserva, execútanse na unidade funcional correspondente e por último a unidade de Commit vai asegurar que os distintos commits se fagan en orde, sen alterar o efecto dunha execución secuencial da instrución.

En xeral existen dúas técnicas de planificación dinámica:

- **Marcador:** Detén as instrucións emitidas ata que hai os recursos adecuados e non hai dependencias de datos
- **Tomasulo:** Elimina as dependencias WAW e WAR mediante o renomeado de rexistros.

Renomeado de rexistros



Nesta imaxe pódese ver como existen riscos de datos entre as instrucións sinaladas, mais estes pódense evitar mediante o renomeado de rexistros da seguinte maneira:



Tras esta pequena modificación, as operacións xa non interfírense entre sí.

Esta operación ten lugar nas estacións de reserva, que ademais da instrución conteñen os valores dos operandos no buffer no caso de que estean dispoñibles e a validez de cada un deles. Tamén se recolle o número da estación de reserva da instrución que proporciona os valores para o operando.

A aproximación de Tomasulo rastrea cando os operandos están dispoñibles e introduce a técnica comentada de renomeado de rexistros por hardware.

Estacións de reserva

As entradas das estacións de reserva seguen o seguinte esquema:

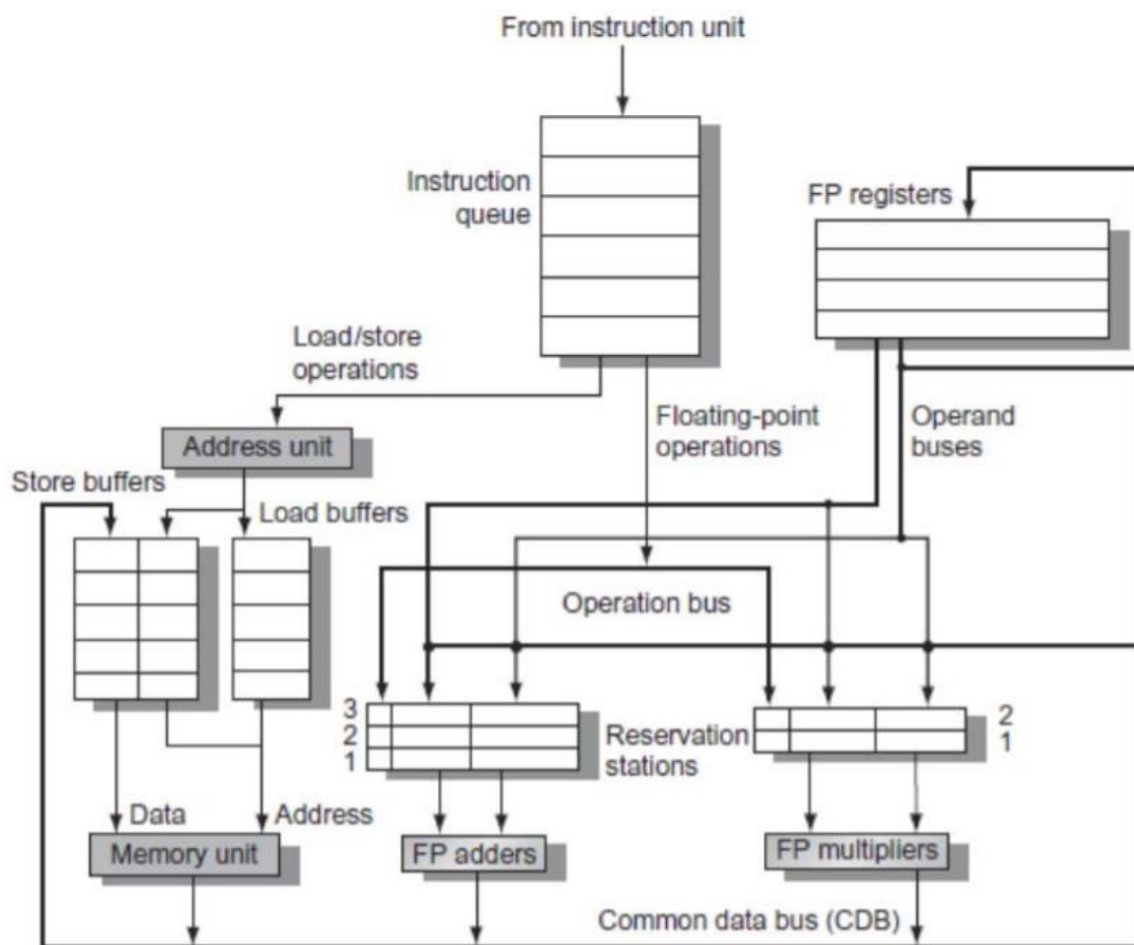
➤ Campos de una entrada:

- ❑ B indica si entrada ocupada por instrucción
- ❑ I campo Código de instrucción
- ❑ OP1 y OP2 operandos de entrada de la instrucción: proceden de registros o de los caminos de bypass de las unidades de ejecución. Alguno puede ser un identificador de registro de renombrado, o sea, que su valor está siendo calculado.
- ❑ V1 y V2 indica si el operando es válido
- ❑ R indica si la instrucción está lista para emisión

INST.	B	OP1	V1	OP2	V2	R
-------	---	-----	----	-----	----	---

Algoritmo de Tomasulo

Unha implementación do algoritmo de Tomasulo pode ter unha estrutura como a da seguinte imaxe:



Cabe destacar que existe un bus común de datos que permite comunicar entre sí los buffers de almacenaxe, la unidad de memoria, o resultado das operacións en punto flotante e o resultado dos multiplicadores e sumadores en punto flotante (que a pesar de que só aparece un na imaxe, isto é unha simplificación e en realidade están replicados). Por outro lado existe outro bus que leva as operacións dende o seu fluxo único ás estacións de reserva (que teñen distinto número de entradas neste caso), e tamén chama á atención como se almacenan en buffers separados as instrucións de carga e as de almacenaxe.

Voltando á operación de renomeado de rexistros, as estacións de reserva van obter e almacenar os operando tan cedo como estean dispoñibles. As instrucións pendentes van designar a estación de reserva á cal van enviar o seu dato de saída, que farán a través do bus de datos común (CDB). Só a última saída se vai actualizar nos rexistros, pois para as operacións os datos pódense obter das estacións de reserva, por exemplo, no caso do renomeado de rexistros producido no conxunto de instrucións de [1], o valor T non se vai que actualizar no rexistro ata que se modifique por última vez, e é importante saber que se corresponde co rexistro f8 para realizar esta actualización. A medida que se van emitindo instrucións, os especificadores de rexistro renoméanse coa estación de saída.

Os buffers de carga e almacenaxe actúan como estacións de reserva pero só para este tipo de instrucións. Ó final están almacenando cargas e almacenamentos pendentes de ser executados.

Os pasos da execución do algoritmo de Tomasulo son 3. Por un lado está a emisión de instrucións:

□ Emitir

- Obter a seguinte instrución de la cola FIFO
- Si la RS está disponible, emite la instrucción a la RS con los valores del operando si están disponibles
- Si los valores del operando no están disponibles, detener la instrucción

Por outro está a súa execución:

□ Ejecutar

- Cuando el operando está disponible, lo almacena en cualquier estación de reserva que lo esté esperando
- Cuando todos los operandos estén listos, emite la instrucción
- Las cargas y el almacenamiento se mantienen en el orden del programa a través de la dirección efectiva
- No se permite iniciar la ejecución de ninguna instrucción hasta que se hayan completado todas las ramas que la preceden en el orden del programa

E por último está a escritura de resultados:

□ Escribir el resultado

- Escribir el resultado en el CDB en las estaciones de reserva y en los buffers de almacenamiento
 - (Los almacenamientos deben esperar hasta que la dirección y los valores están disponibles)

Especulación

O obxectivo da especulación é predecir o que fará unha instrución de forma que se poidan ir executando as instrucións que se predicen que van ser as seguintes sen ter a certeza de que o sexan. Se se acerta a predicción, o fluxo de instrucións segue o seu curso, mentres que en caso contrario é necesario desfacer as operacións froito da predicción e executar as que realmente corresponden. Esta técnica é común a emisión múltiple estática e dinámica. Algunhas operacións nas que se pode levar a cabo é nos saltos ou na carga de datos.

- **Salto:** Comézanse a lanzar instrucións asumindo que son as que corresponde executar despois do salto, sen saber se este se toma ou non. O commit destas instrucións non

se leva a cabo ata que se determine o resultado do salto. Para predecir o resultado do salto existen técnicas das que xa se falou no tema anterior.

- **Carga:** Especular nunha operación de carga permite evitar o retardo da propia carga e do posible fallo caché. Consisten en predecir a dirección efectiva do dato que se vai cargar ou predecir o dato que se vai cargar, pasando estes valores á unidade de carga pero sen completar a asignación ata que se resolva a especulación.

Por outro lado estas especulacións poden ou ben ser realizadas polo compilador, que pode reordenar instrucións, ou ben pode estar realizada polo hardware, que pode buscar por adiantado instrucións para executar. Ata saber se eses resultados se precisan realmente, estes quedan almacenados nun buffer. No caso de que a especulación sexa incorrecta, baléiranse os buffers.

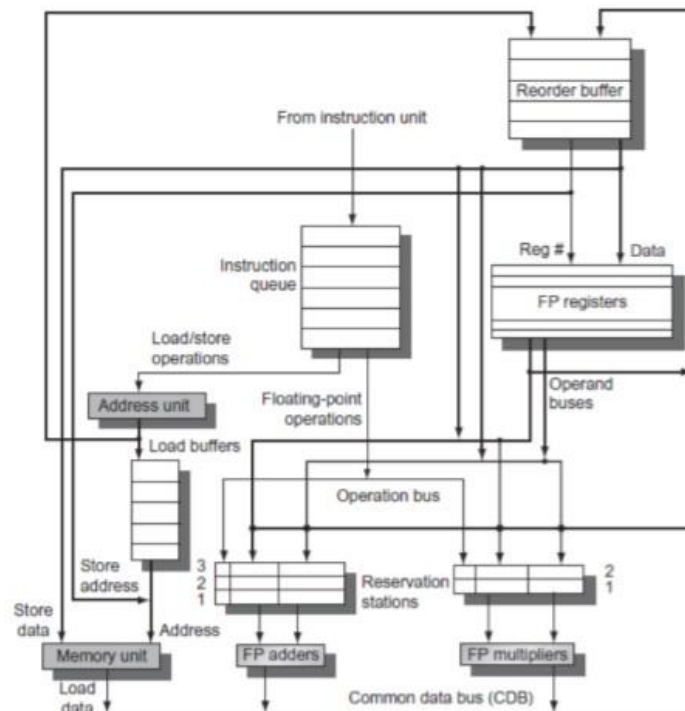
Especulación baseada en hardware. Buffers de reordeamento (ROB)

ROB – almacena o resultado das instrucións entre que se completa e se produce o commit (finalización ou retirada da instrución).

Retíranse do ROB aquelas instrucións na orde da programa secuencial. A continuación móstranse os campos básicos de cada entrada:

- Tipo de instrución.
- Dirección da instrución.
- Campo de destino: número do rexistro renombrado
- Campo de valor: valor de saída.
- Campo de estado que indica o estado da instrución
- Campo que indica si a instrución está nun estado especulativo

Esixe modificar as estacións de reserva. A fonte dun operando é agora o ROB no lugar dunha unidade funcional.



Posibles estados para unha instrución:

- **En execución:** Se a instrución non acabou de executarse.
- **Finalizado:** O finalizar a execución escríbese o resultado nun rexistro renomeado (si se produce un resultado), ou nun buffer previo a escribir en memoria se se trata dunha instrución de escritura en memoria.
- **Completada:** Cando a instrución modifica os rexistros do núcleo, o seu estado.
- **Retirada:** Se o destino é un rexistro que coincide con completada. Se o destino é memoria ,retírase cando se escribe na caché de datos.

Cando o hardware utiliza buffers de reordenamento, nas distintas etapas da execución realízanse as seguintes operacións:

- **Emisión:**
 - Asígnaselle á instrución unha estación de reserva e un buffer de reordenamento. Lense os operandos dispoñibles.
- **Execución:**
 - Comézase a execución cando os valores dos operandos estean dispoñibles.
- **Escritura de resultados:**
 - Escríbese o resultado e a etiqueta do ROB no buffer CBD.
- **Finalizar instrución (commit):**
 - Cando o ROB alcanza a súa cabeceira, actualízase o rexistro.
 - Se un salto con predicción errónea alcanza a cabeceira do ROB, descártanse todas as entradas.

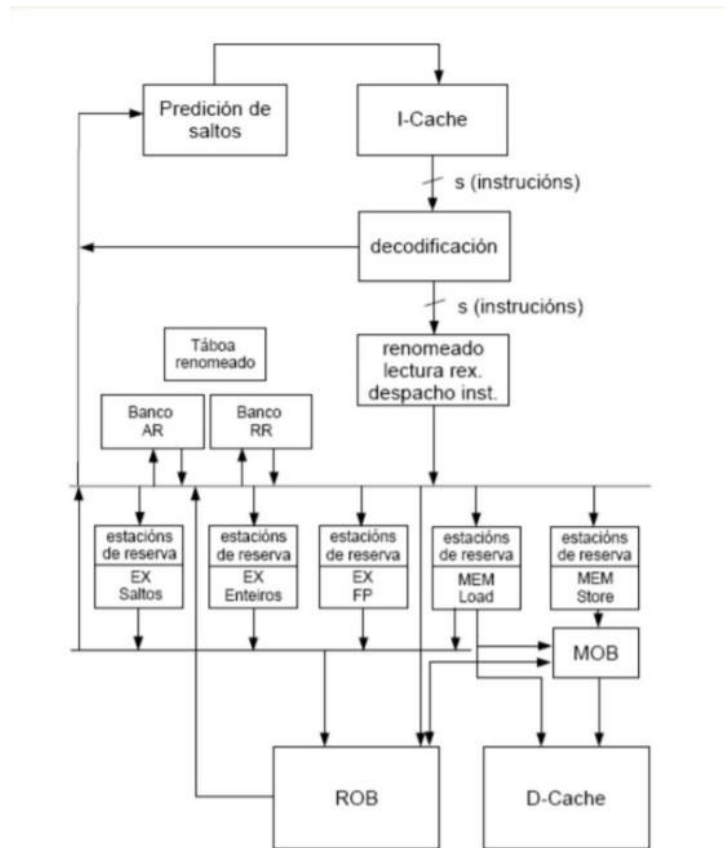
2. Arquitectura de alto nivel de un núcleo

Cada caixa da estrutura (da imaxe da páxina seguinte) pode corresponder a varias etapas do pipeline (visión esquemática).

Para a implementación dun procesador “superescalar”, en cada ciclo tense un conxunto de instrucións que se obteñen da memoria caché (instrucións). A lóxica de predicións de saltos detecta con gran probabilidade os grupos nos que existen instrucións de saltos que se toman e cal é a dirección de destino. Isto permite especulativamente acceder o seguinte grupo de instrucións que se vai executar . O deseño da memoria caché non é trivial, xa que ten que permitir a lectura de moitos bytes por ciclo, ten que ser unha memoria caché moi optimizada.

Na etapa de decodificación as instrucións son decodificadas en paralelo. Esta etapa tamén pode ser moi complexa en caso de repertorio de instrucións tipo CISC, xa que estas son irregulares e estas son divididas en micro-operacións RISC elementais, permitindo unha decodificación máis eficiente.

A continuación, execútase a etapa de renomeado de rexistros, despachando cada instrución a súa estación de reserva. As estación de reserva en este caso repártanse por cada pipeline de execución. Nelas as instrucións esperan a ter listos os operandos de entrada. O ROB (Buffer de reordeamento) recibe información dos diferentes cambios de estado das instrucións para almacenalo na entrada correspondente.



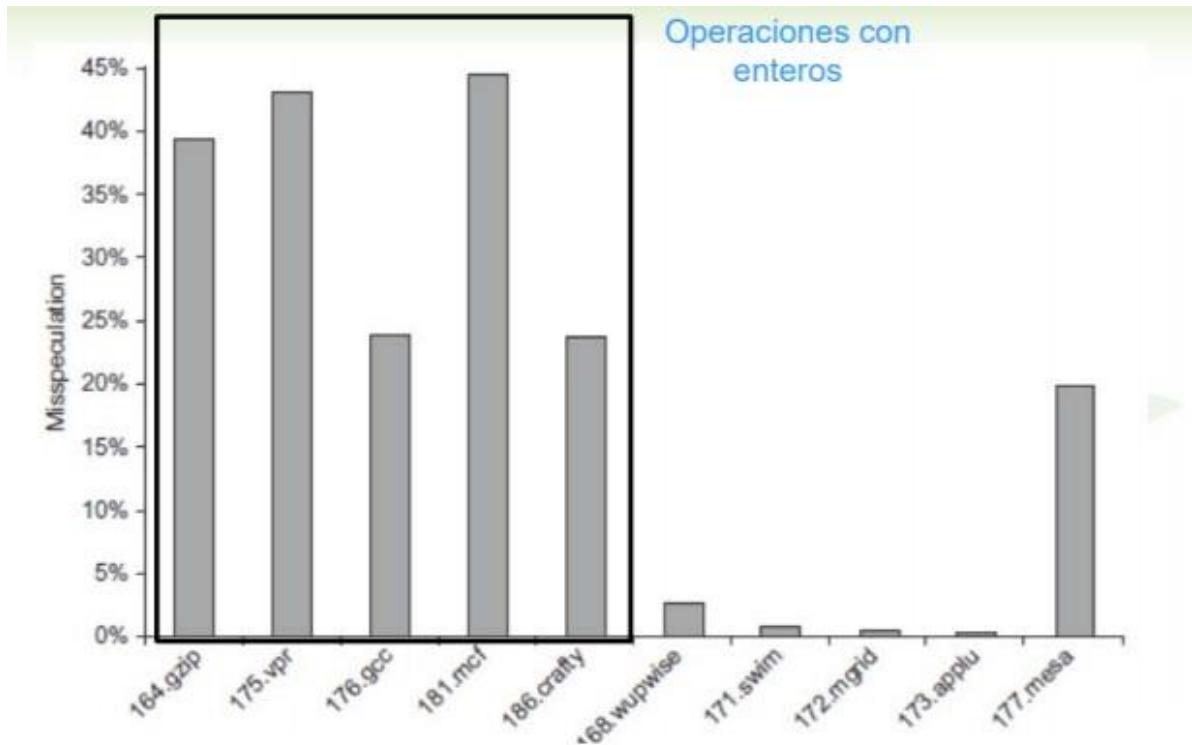
Canto especular?

A especulación errónea degrada o rendemento e o consumo de potencia respecto a non especular. Pode tamén provocar erros adicionais na caché, TLB...

Débese evitar que o código especulativo cause erros caché de alto custo.

Hai que especular a través de múltiples ramas o que complica a recuperación da especulación, xa que no caso de especular moito vanse ter moitas ramas como consecuencia de decisións de salto tendo que retroceder demasiado cando se produza unha especulación errónea.

A especulación só é eficiente cando se mellora significativamente o rendemento. Na seguinte imaxe representase unha gráfica composta por diversos programas de probas nos estudos do rendemento situados no eixo X, mentres que no eixo Y se representa a porcentaxe de especulación na que se errou. En operacións con enteiros os erros son maiores (caixa da esquerda). En códigos con puntos flotantes (parte dereita) apreciase como a especulación soe ser máis eficiente.



Funciona a emisión múltiple?

Sí, pero non tanto como gustaría.

Os programas teñen dependencias reais que limitan o paralelismo a nivel de instrución. Algunhas dependencias son difíciles de eliminar como as asociadas a punteiros.

Algún paralelismo é difícil de atopar no código xa que se usa unha ventá de tamaño limitado durante a emisión. Ademais, hai retardos de memoria e ancho de banda limitados (dificultade para manter o pipeline cheo). Finalmente, a especulación pode axudar si se fai ben e se realiza un conxunto de instrucións pequeno.

Planificación dinámica, emisión múltiple e especulación

Nas microarquitecturas modernas realízase planificación dinámica, emisión múltiple e especulación. Atópanse dous enfoques distintos:

- Asignar estacións de reserva e actualizar a táboa de control do pipeline no medio do ciclo de reloxo (executar dúas instrucións/ciclo).
- Engadir lóxica de deseño para manexar calquera posible dependencia entre as instrucións.

A lóxica de emisión é o pescozo de botella nos “superescalares” planificados dinámicamente. Isto é o máis caro e principal reductor de rendemento.

Eficiencia enerxética

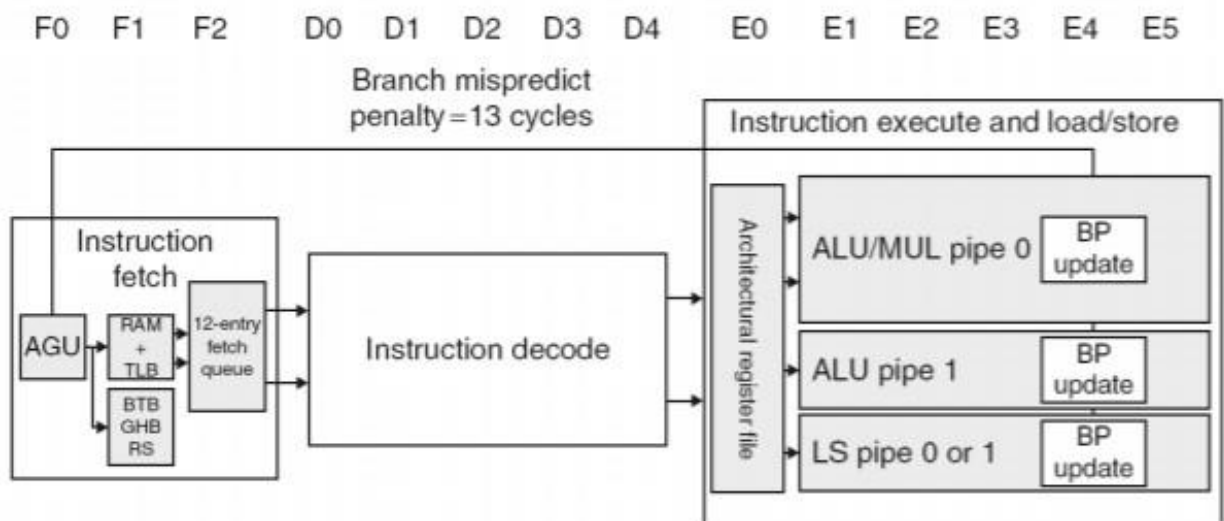
A complexidade da planificación dinámica e as especulacións requiren enerxía. Moitos núcleos máis simples poden resultar mellor que un só núcleo máis complexo. Na seguinte táboa pódese observar as características de diferentes microprocesadores.

2. Arquitectura de alto nivel de un núcleo

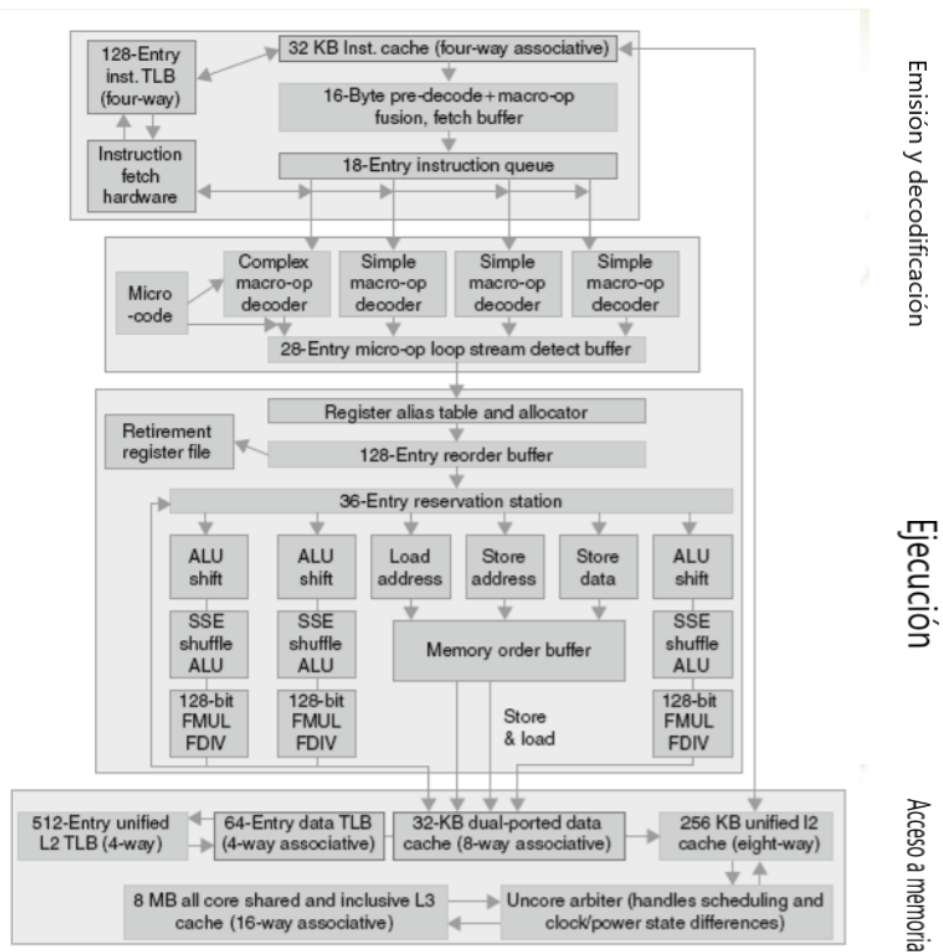
Microprocessor	Year	Clock Rate	Pipeline Stages	Issue width	Out-of-order/ Speculation	Cores	Power
i486	1989	25MHz	5	1	No	1	5W
Pentium	1993	66MHz	5	2	No	1	10W
Pentium Pro	1997	200MHz	10	3	Yes	1	29W
P4 Willamette	2001	2000MHz	22	3	Yes	1	75W
P4 Prescott	2004	3600MHz	31	3	Yes	1	103W
Core	2006	2930MHz	14	4	Yes	2	75W
UltraSparc III	2003	1950MHz	14	4	No	1	90W
UltraSparc T1	2005	1200MHz	6	1	No	8	70W

ARM Cortex-A8 Pipeline

Conta con 14 etapas no pipeline. Por cada predición errónea nos saltos obtense un custo de 13 ciclos. 3 etapas para a búsqueda de instrución (F), 5 etapas para a decodificación (D) e 6 etapas para a execución (E).



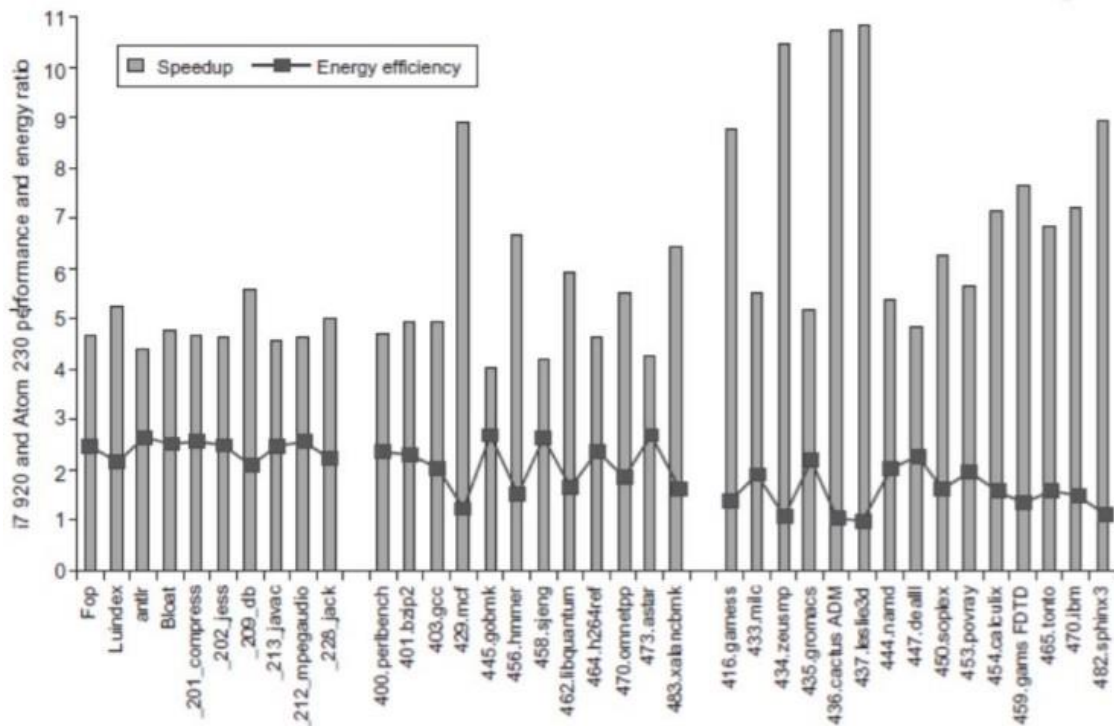
Core i7 Pipeline



Falacias

Falacia 1: É fácil predecir o rendemento/eficiencia enerxética de dúas versións diferentes do mesmo ISA se mantemos a tecnoloxía constante.

O rendemento en execución en canto a tempo de execución non sempre ten que ver coa eficiencia enerxética.



Falacia 2: Crer que un procesador con un CPI máis baixo ou unha frecuencia de reloxo máis alta vai ser máis rápido.

Non sempre será o máis rápido, pois depende da súa arquitectura.

Processor	Implementation technology	Clock rate	Power	SPECint2006 base	SPECfp2006 baseline
Intel Pentium 4 670	90 nm	3.8 GHz	115 W	11.5	12.2
Intel Itanium 2	90 nm	1.66 GHz	104 W approx. 70 W one core	14.5	17.3
Intel i7 920	45 nm	3.3 GHz	130 W total approx. 80 W one core	35.5	38.4

3. Curiosidades

As veces solucións de maior tamaño e deseño simple son as máis beneficiosas, e as veces deseños máis intelixentes son mellores que solucións de maior tamaño con deseño simple (sucede con algúns predictores de saltos).

4. Conclusións

ISA influencia o deseño do camiño de datos e control (viuse en Fundamentos de Computadores).

Pipelining mellora a produtividade en número de instrucións acabadas por unidade de tempo usando paralelismo.

- Máis instrucións completadas por segundo
- A latencia (tempo requerido) por cada instrución non cambia.

Riscos: estruturais, de datos e de control.

Emisión múltiple e planificación dinámica (ILP).

- As dependencias limitan a cantidade de paralelismo explotable.
- A complexidade leva a fronteira de potencia.



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Arquitectura de Computadores

06-05-2021

Clase 12

Temas trabajados:

Tema 4. Introducción, protocolos de coherencia
caché: snooping, basados en directorio.

ÍNDICE

1. Introducción.....	2
Cuestiones sobre la asignatura.....	2
Cuestiones sobre la segunda práctica	2
Cuestiones sobre las últimas clases.....	3
2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo	4
Introducción.....	4
Protocolos de coherencia caché	10
Protocolo de snooping. Invalidación.....	12
Protocolos basados en directorio	18

1. Introducción

Cuestiones sobre la asignatura

Antes de empezar la clase, Dora hace una serie de explicaciones sobre diferentes temas.

En primer lugar, hace hincapié en que se están dando menos detalles tecnológicos de lo que normalmente se veía en esta asignatura, centrándonos más en las bases. Luego, para ver cómo se aplican a cada procesador, podemos mirar ejemplos, aunque va evolucionando tecnológicamente. A lo que se le dará más importancia es a que, dentro de un tiempo, cuando veamos cosas relacionadas con un procesador, entendamos qué ocurre “debajo”, qué es todo esto: planificación dinámica, segmentación, qué limitaciones tienen (ya que la frecuencia de un procesador no se puede aumentar indefinidamente). También es importante saber (que veremos en la clase de hoy) qué es coherencia y consistencia de memoria: en qué consisten, cómo se aplican, cómo se consiguen, en qué procesadores... todo este tipo de cosas que nos encontraremos en nuestro trabajo.

Dora pondrá en el campus virtual alguna lectura relacionada con este tema de protocolos, junto con las diapositivas, y el tema correspondiente del libro. Debemos tener cuidado, porque a veces, la forma de plantear los protocolos, de escribirlos, es muy distinta en función de dónde lo miremos. No significa que sean formas incorrectas, sino distintas, para explicar un mismo protocolo.

Cuestiones sobre la segunda práctica

Por otra parte, respecto a la **entrega** de las **prácticas**, Dora recuerda que las fechas límites se acercan. Comenta que intenta responder a todas las dudas que le llegan sobre las mismas. En el campus hay dos entregas distintas en función del grupo de prácticas al que se pertenezca.

Una cuestión que le parece importante aclarar respecto a esta segunda práctica es que, la base del código es el producto de dos matrices. El anidamiento hijo, que emplea los lazos del producto matriz por matriz, no es el mejor método para hacer la operación vectorial (pero se puede hacer). También se pueden emplear otras técnicas como reordenar los lazos. Esto lo dice porque: si cogemos un reordenamiento de los lazos en la versión secuencial, y ahora cogemos la versión vectorial (aunque esté bien hecha y dé el resultado correcto), puede que no haya mucha mejora en velocidad, porque no es una operación especialmente indicada para operaciones vectoriales. Más bien, es muy común en álgebra, de la que nos podemos encontrar muchas versiones en ensamblador o en AVX vectorizada. El problema es que no es tan inmediata como la práctica que se venía haciendo años anteriores (productos y sumas con vectores), por ello conviene tener en

cuenta que el speedUp no va a ser muy alto. Si conseguimos una aceleración de 1,5 o 2, ya sería considerado como alto.

Dora quería hacer esta aclaración porque le han llegado mensajes de que el código vectorial les sale más lento. Recuerda que no pasa nada por ello, si el código está bien construido y se explica lo que se ha hecho, destacando cuáles son las partes más pesadas del mismo, no habrá ningún problema.

Cuestiones sobre las últimas clases

Respecto a la semana que viene, al ser festivo el jueves 13, si faltara algo que ver de teoría la semana que viene, se haría una última clase en otro momento, en un horario compatible para todos. El martes se hará la clase de ejercicios.

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

Dora destaca que, para entender este tema, es necesario estudiarlo o intentar hacer un ejercicio, ya que está basado en la descripción de protocolos (reglas para llevar a cabo procesos que consigan una visión coherente de la memoria). Conviene, por ello, estudiarlos para entender el tema.

Introducción

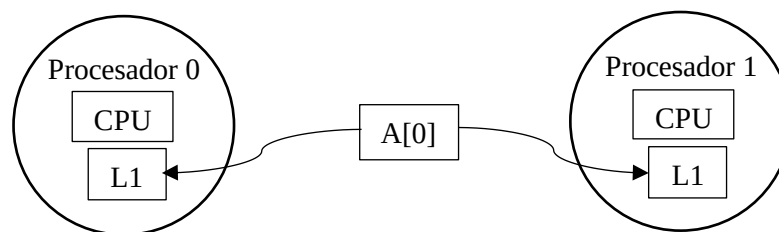
Tipos de datos en memoria caché:

- Datos privados → Datos usados por un único procesador
- Datos compartidos → Datos usados por varios procesadores

En mi memoria caché va a haber datos que use solo yo durante toda la ejecución del código, y datos de los que va a tener otro procesador copia en su caché, porque los seguirá utilizando (datos compartidos).

Problema con datos compartidos:

- El dato puede replicarse en varias cachés.
- Reduce contención (tiempo de espera) porque cada procesador accede a su copia local.
- Si dos procesadores modifican sus copias es necesario aplicar protocolos de coherencia caché.



Si hay una copia de un dato A[0], que son modificadas por los procesadores, y cada uno los copia “a su aire”, va a llegar un momento en que el contenido de A[0] será distinto en función de dónde lo lea. Es decir, se producirá un problema, que se suele arreglar aplicando protocolos de coherencia caché.

Por tanto, Un sistema es coherente (incluye coherencia y consistencia) si cualquier lectura de un dato devuelve el valor más recientemente escrito de ese dato.

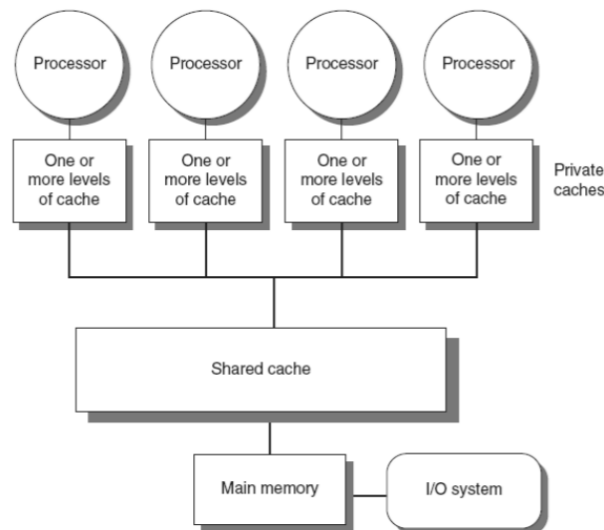
- Coherencia: define el comportamiento de lecturas y escrituras de la misma posición de memoria (conseguir que A[0] esté actualizado en todas las memorias caché).

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

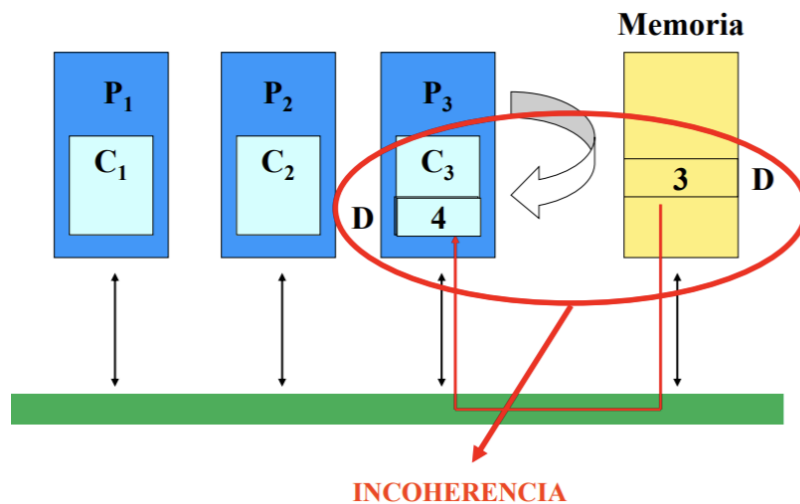
- Consistencia: define el comportamiento de lecturas y escrituras con respecto a accesos a otras posiciones de memoria.
- Sistemas monoprocesador: memoria principal y caché. Debe haber la misma información en ambas. Cuando se actualiza un dato, o se pasa una copia a memoria principal (protocolo de ED o escritura directa: write-through), o podemos dejar que la caché actualice el dato las veces que sea necesario, y solo se actualiza en memoria principal cuando esa línea caché vaya a ser reemplazada (protocolo PE o post-escritura: write-back).
- Sistemas multiprocesador: además hay un problema de coherencia entre cachés, porque puedo estar actualizando un dato en una caché y que otra tenga copia, sin tener el valor actualizado en dicha caché.
- ¿Cuándo debe un procesador ver un valor que ha sido escrito por otro procesador?
- ¿Qué orden se establece entre lecturas y escrituras a diferentes posiciones por distintos procesadores?

Multiprocesadores UMA:

- También llamados multiprocesadores simétricos (SMP)
- Número de Cores pequeño ≤ 8
- Comparten una memoria única con latencia de acceso a memoria uniforme.

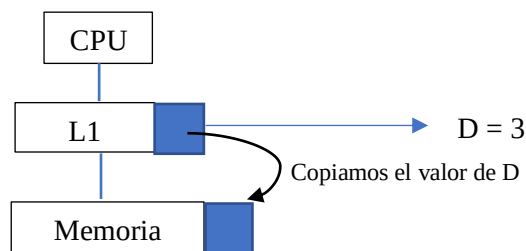


Problema de incoherencia cuando tengamos diferentes memorias caché: multiprocesadores UMA



Hay tres procesadores, cada uno con su memoria caché. En la memoria principal tenemos un valor de una posición de memoria D, que es el número 3. Cuando el procesador 3 necesite dicho valor, se va a copiar en su caché (en el dibujo, bajo C₃). El problema es que el procesador 3 lo necesitaba, pero para escribirlo, así que: escribirá el valor 3, y en su lugar colocará el valor 4 que se ve en la imagen (escribir en la posición de la variable D, que tengo copiada en memoria caché, actualizando la variable). Ahora el valor que hay en la memoria caché del procesador 3 y en la posición de memoria correspondiente para la variable, no coinciden: se produce una incoherencia.

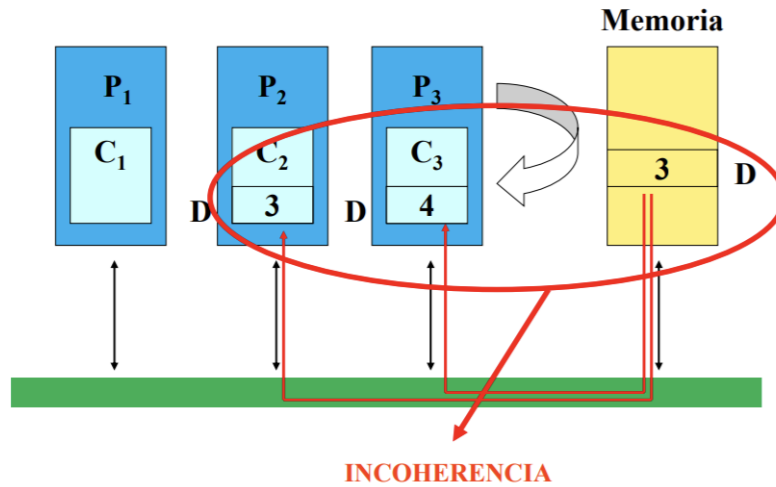
Si estuviéramos en un sistema monoprocesador, y tuviéramos la CPU, la L1 y la Memoria principal:



Cuando la variable D tenga el valor de 3, lo que haremos será, directamente, copiar ese valor en memoria para que D = 3. Con cualquier política de copia de datos en la memoria principal (PE, ED) nos aseguramos de que caché y memoria principal en el sistema monoprocesador tengan el problema de coherencia resuelto.

Volviendo al caso de varias cachés, que es un sistema multiprocesador, vemos que habrá un problema de coherencia si hay un dato que, desde la memoria se lleva a la caché (que es lo normal), y es actualizado por un procesador, de forma que en la memoria tenga un valor distinto. Con PE lo puedo actualizar, aunque surge el problema de que también haya una copia en otro procesador, que le dé a la variable D, el valor 5. ¿Cómo sabemos el valor que debe quedar

finalmente en memoria? ¿Qué sucede cuando tenemos distintas actualizaciones en diferentes lugares? ¿Qué protocolo debo utilizar para que haya coherencia?



En esta imagen, si la D está en el segundo procesador y, se modifica su valor, habrá un problema de incoherencia. Este se aprecia sobre la información contenida en la variable D entre memoria, una caché, y la otra.

Resumidamente, un **sistema de memoria es coherente** si se cumple que cualquier lectura de una dirección devuelve el valor más reciente que se haya escrito para esa dirección.

CONDICIONES PARA LA COHERENCIA:

1. **Preservación del orden del programa.** Un procesador P escribe la posición X y luego P lee la posición X, y no hay escrituras por parte de otro procesador entre la escritura y la lectura de P: Entonces el valor devuelto es siempre el valor escrito por P (preserva el orden del programa, también en sistemas monoprocesador).
2. **Vista coherente de la memoria.** Un procesador Q escribe la posición X y luego otro procesador P lee X, entonces el valor devuelto por la lectura es el valor escrito por Q. Esto se cumplirá si la escritura y la lectura están suficientemente separadas y entre los dos accesos no hay ninguna escritura de X.
3. **Serialización de escrituras:** dos escrituras de la misma posición por dos procesadores cualesquiera son vistas en el mismo orden por todos los procesadores.

Si se cumplen estas tres condiciones, tendremos un sistema de memoria coherente.

MÉTODOS DE ACTUALIZACIÓN DE MEMORIA PRINCIPAL UTILIZADOS EN CACHÉS:

1. **Escritura directa** (ED o write-through/WT): Siempre que se modifica una dirección en la caché, se modifica la memoria principal.

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

- a. Cada escritura supone utilización de la red. Poco eficiente en sistemas multiprocesadores (estaremos yendo a memoria todo el tiempo) → varios procesadores escribiendo simultáneamente.
 - b. No evita incoherencia entre cachés porque, si actualizamos un dato en la memoria principal cada vez que escriba, ¿qué ocurre si mientras tanto se está modificando su valor en la caché de otro procesador? Realmente, no se termina evitando el problema de la incoherencia.
2. **Post-escritura (PE o write-back):** Cuando un procesador modifica una dirección de memoria sólo se escribe en la caché. El dato no se transfiere a memoria principal, por lo que se puede escribir varias veces en un bloque sin acceder a memoria principal.
- a. Escritura del dato solo cuando se elimina de la caché.
 - b. Se produce incoherencia entre cachés, y entre caché y memoria.

Es decir, si trabajamos con un sistema multiprocesador, ni ED ni PE resolverán los problemas de incoherencia, solo evitan los problemas entre una caché y su memoria. Se necesitarán métodos adicionales

OBTENEMOS COMO CONCLUSIÓN:

Falta de coherencia entre cachés y procesadores en sistemas multiprocesador, o entre cachés y memoria principal, en el sistema monoprocesador, tanto utilizando escritura directa como post-escritura → necesitamos Hardware específico.

Existen otras soluciones como la declaración de zonas de memoria “no *cacheables*” o con escritura inmediata, por lo que no irán a la caché y, cuando todos los procesadores escriban, dejar una zona de la memoria que solo sirva para que actualicen los procesadores, y sea esta la que se encargue de modificar conjuntamente los datos. Se utiliza mucho en los dispositivos E/S pero no es eficiente para la computación normal.

Solución → Protocolos de coherencia caché

Hacen que cada escritura sea visible a todos los procesadores, y propaga de forma fiable un nuevo valor escrito. Cuando escribamos un valor, esta escritura será propagada eficientemente a los otros procesadores y a memoria principal, a través del bus que los comunica. El protocolo indicará ciertas normas de actuación para hacerlo de forma eficiente.

PROPAGACIÓN DE ESCRITURAS EN TODAS LAS CACHÉS DE LOS DIFERENTES PROCESADORES:

- **Escritura con actualización (WU: write-update):** comprueba que el dato es compartido. Si no es así, no es necesario actualizar otras cachés. Actúa como caché de ED para datos compartidos, y de PE para datos privados.

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

- **Escritura con invalidación** (WI: write-invalidate): el más frecuente. En los protocolos de snooping (espionaje) se distribuye la señal de invalidación por el bus, y las cachés comprueban si tienen una copia. Si es así invalidan (poner a 0 bit de validez) la línea que contiene el dato.

V (bit de validez) Directorio (etiquetas) Datos

➔ **Estructura línea caché**

Ejemplo de invalidación: si tenemos dos cachés que actualizan la variable D (podemos comprobar los dibujos anteriores), cuando el procesador 3 escriba el valor 4, inmediatamente el bit de validez de la línea caché del procesador 2 tendrá un 0. Es decir, no se considerará válido el valor de la variable D que tenía el procesador 2 (D valía 3 en este procesador). Esto se hace con el resto de procesadores. La única copia válida será la que se encuentra en el tercer procesador, que después se actualizará en memoria principal.

DIFERENCIAS ENTRE WU Y WI: (WU: mucha comunicación, mayor uso de buses. WI: más ligero)

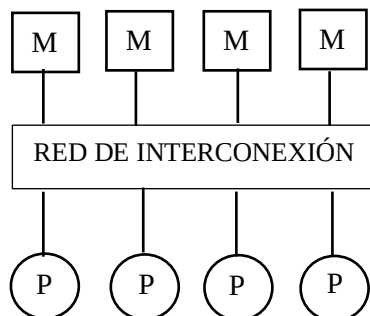
- **Múltiples escrituras a la misma palabra** (sin lecturas por el medio) requieren múltiples difusiones de escritura en WU, pero sólo una invalidación en WI. Con WU, si yo modifico una línea caché, y la tienen otras cachés, voy a mandar múltiples difusiones (mismo dato) a todas las cachés para que se actualicen. Con WI, empiezo a escribir el dato, mando una invalidación, y ya queda invalidado en las otras cachés, pudiendo así modificarlo como quiera.
- Con **líneas caché de más de una palabra**, cada palabra escrita en una línea requiere una difusión en WU, mientras que sólo la primera escritura a cualquier palabra de la línea genera una invalidación en WI → WU actúa a nivel de palabra, y WI a nivel de línea. En cuanto se toca la línea, se invalida solo una vez, sin embargo, con la actualización, si una línea tiene cuatro palabras y hay que actualizar todas, cada vez que actualice una, le mando copia a todas las cachés, por lo que el coste de WU es mucho mayor. Elegimos la opción WI.
- El retardo entre **escribir una palabra en un procesador y leer el valor escrito en otro**, es menor en WU, ya que el dato escrito es actualizado inmediatamente en la caché del procesador lector. En WI el lector es invalidado y no puede leer hasta que reciba copia actualizada.

Protocolos de coherencia caché

Se basan en seguir la pista al estado de un bloque de datos compartido. Existen dos tipos:

- **Snooping** (o Espionaje): cada caché tiene información sobre el estado de compartición de los bloques (líneas) que contiene
 - o Típico de sistemas de memoria compartida con un bus común. Los controladores caché espían el bus para determinar si tienen o no una copia del bloque compartido solicitado. Así, cuando alguien necesite un bloque, no va solamente a memoria principal a buscarlo, sino que, a través del bus, hace un broadcast (envío a todos) de que necesita esa línea, y el que la tenga más recientemente actualizada, la mandará (no tiene por qué ser la memoria principal).
 - o La información de coherencia es proporcional al número de bloques de la caché. Es decir, hay información por cada una de las líneas caché.
 - o Problema: no escalan bien porque generar mucho tráfico en el bus, porque cada vez que se hace una modificación, hay que estar continuamente mandando mensajes a las otras cachés y a la memoria principal a través de este bus. Si en un momento dado yo quiero escribir algo en mi caché, y no tengo el bloque que quiero escribir, tengo que leer a través del bus el bloque que necesito.

Este tipo de protocolo se emplea en arquitecturas con memoria centralizada (UMA: acceso uniforme a memoria) con red escalable:

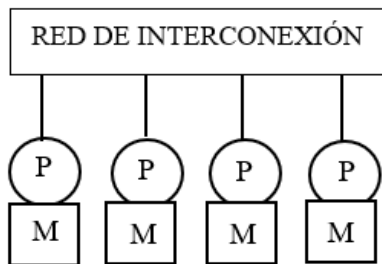


Tenemos una serie de procesadores conectados a una red de interconexión, que a su vez están conectados, mediante buses, por ejemplo, a memorias (tantas como procesadores tenga). La gestión de esta memoria se realiza como si hubiera una única memoria (como cuando utilizamos OpenMP), y son sistemas con una red escalable.

Costará lo mismo, desde un procesador, acceder a cualquiera de los distintos módulos de la memoria, ya que hay que pasar por la red de interconexión.

- **Basados en directorio:** el estado de un bloque de memoria física se mantiene en una única localización (directorio).
 - o Típico de sistemas escalables.
 - o No necesitan que todas las cachés estén comunicadas por un bus común. Comunicaciones sólo entre las cachés implicadas.
 - o Información que contiene el directorio: cachés que tienen copia del bloque y si ha sido modificado. Tamaño del directorio proporcional al número de líneas de MP. Los sistemas basados en espionaje, cada caché, en su línea, tiene información sobre el estado de la línea (compartido, modificado, invalidado). Sin embargo, en los basados en directorios, la información se refiere a la memoria principal. Para cada pequeña parte de memoria principal, hay información de: en qué cachés está, en qué estado está en cada una de ellas.
 - o Es una aproximación centralizada: el estado compartido de un bloque está siempre en una única posición (cuello de botella). Pero las entradas del directorio pueden estar distribuidas (reducción de la contención).

Este tipo de protocolo se emplea en arquitecturas de multiprocesador de memoria físicamente distribuida (CC-NUMA: coherencia caché-acceso no uniforme a memoria).



Tenemos una red de interconexión y los procesadores. Cada uno de ellos tendrá su memoria asociada (un único chip para memoria y procesador). Esto implica que el tiempo de acceso a memoria no será uniforme. Para un procesador, acceder a su memoria resulta más barato que acceder a otra, ya que hay que atravesar la red de interconexión.

Sin embargo, en nuestros sistemas, va a interesar que, cuando tenga una copia de un dato en una memoria o en una caché del procesador, esté esa copia del dato actualizada en todos las cachés de los procesadores. Es decir, que haya una visión coherente de la memoria.

Protocolo de snooping. Invalidación.

La invalidación de escrituras es una estrategia que garantiza que un procesador tiene acceso exclusivo a un bloque antes de realizar una escritura. Invalida el resto de copias que puedan tener otros procesadores usando el bit de validez V de cada línea caché.

- **Uso del bus de memoria para invalidación:** el procesador adquiere el bus y difunde la dirección a invalidar. Todos los procesadores espían el bus. Cada procesador comprueba si tienen en caché la dirección difundida y la invalida.
- No puede haber dos escrituras simultáneas: el uso exclusivo del bus serializa las escrituras.
- **Escrituras:** necesidad de saber si hay otras copias en caché. Si no hay otras copias no hay que difundir escritura. Se añade bit de compartición (S) asociado a cada bloque. Cuando hay escritura:
 - o Se genera invalidación en bus.
 - o Se pasa de **estado compartido** a **estado exclusivo** si existe en el protocolo o a **estado modificado** en otro caso.

Cuando hay fallo caché en otro procesador se pasa de estado exclusivo/modificado a estado compartido.

Cambios de estado de la línea caché:

- Primero, estaba compartida con otros procesadores (Estado Shared)
- Escribí en ella, para lo que la pasamos a estado exclusivo e invalidarla en el resto de las cachés.
- Cuando otro procesador la necesita, se pasa a estado compartido de nuevo y se envía el valor a la otra caché.

Si el estado es compartido, es porque varias están leyéndola. Cuando uno la escribe, pasa a estado modificado y todas las demás la invalidan (incluyendo la de memoria principal).

PROTOCOLO DE INVALIDACIÓN DE TRES ESTADOS (MSI):

Utiliza post-escritura como política de actualización de memoria principal, y escritura con invalidación para mantener la coherencia entre cachés.

- Basado en un **sistema secuencial síncrono** que cambia de estado para cada bloque de la caché en función de peticiones del procesador y peticiones del bus.
- Acciones: cambios de estado y acciones sobre el bus.
- Un bloque (línea caché) se puede encontrar en **tres estados** M, S o I.
 - o Modificado (M): El bloque ha sido modificado. Es el **único componente en el sistema con una copia válida del bloque**, el resto de cachés y la memoria tienen una copia no

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

actualizada. La caché debe proporcionar el bloque si observa al espiar el bus que algún componente lo solicita, y debe invalidarlo si algún otro solicita una copia para modificarlo.

- Compartido (S de Shared): El bloque está compartido. Todas las copias del bloque en el sistema están actualizadas. La caché debe invalidar la copia si algún otro nodo modifica el bloque.
- Inválido (I): El bloque ha sido invalidado. Supone que el bloque no está físicamente en la caché o, si se encuentra en la caché, ha sido invalidado como consecuencia de la escritura en el bloque de otra caché.

Peticiones posibles:

- Petición de **lectura de un bloque de caché** (PtLec): se genera como consecuencia de la lectura del procesador (PrLec) de una dirección que no se encuentra en la caché.
 - El controlador de la caché pone en el bus la dirección a la que desea acceder.
 - El sistema proporcionará el bloque donde se encuentra la dirección solicitada.
- Petición de **acceso exclusivo a un bloque** (PtLecEx): se genera como consecuencia de la escritura del procesador (PrEsc) en una dirección, cuyo bloque en la caché se encuentra en estado compartido o inválido (incluida la no presencia).
 - El controlador genera el paquete que indicará cual es la dirección en la que se quiere escribir.
 - El resto de las cachés invalidarán sus copias.
 - También se invalida el bloque en memoria si se encuentra en estado válido.
- Petición de **postescritura** (PtPEsc): lo genera el controlador de caché cuando reemplaza un bloque modificado en la caché como consecuencia del acceso del procesador a un bloque que no se encuentra en la caché (estamos suponiendo política de actualización de post escritura por parte de la caché).
 - El bloque a reemplazar debe transferirse a memoria principal si está en estado modificado.
 - El controlador de la caché genera la transferencia poniendo en el bus la dirección del bloque a escribir en memoria y el contenido del propio bloque.
- Paquete de **Reconocimiento con el bloque solicitado** (RpBloque) por una caché: la genera un nodo que observa en el bus una petición del bloque que incluye su lectura si el nodo comprueba que tiene la única copia válida del bloque en el sistema (estado modificado).

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

En esta tabla, se recoge un resumen de todo esto:

Estado inicial	Evento	Acción	Estado sig.
Modificado (M)	Procesador lee (PrLec)	Acierto	Modificado
	Procesador escribe (PrEsc)		Modificado
	Petición de lectura (PtLec)	RpBloque	Compartido
	Petición de acceso exclusivo (PtLecEx)	RpBloque Invalida bloq.	Inválido
	Reemplazo	PtPEsc	Inválido
Compartido (S, Shared)	Procesador lee (PrLec)		Compartido
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Compartido
	Petición de acceso exclusivo (PtLecEx)	Invalida bloq.	Inválido
Inválido (I)	Procesador lee (PrLec)	PtLec	Compartido
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Inválido
	Petición de acceso exclusivo (PtLecEx)		Inválido

Posibles fallos y aciertos:

- Fallo de lectura:

- El procesador lee (PrLec) y el bloque no está en la caché.
- El controlador difunde la petición de lectura (PtLec); el estado del bloque después de la lectura será de compartido. La copia en otras cachés también será compartida y en la memoria, válida.
- La petición PtLec provoca los siguientes efectos en otras cachés:
 - Bloque en otra caché está modificado → deposita bloque en bus (RpBloque) y pasa a compartido. Memoria también recoge bloque y pasa a estado válido.
 - Bloque en otra caché está compartido → memoria proporciona bloque a caché que lo solicita. El bloque sigue como compartido.

- Fallo de escritura:

- El procesador escribe (PrEsc) y el bloque no está en la caché.
- El controlador difunde petición de acceso exclusivo al bloque (PtLecEx); el estado del bloque después de la escritura pasa a modificado.
- La petición PtLecEx provoca:
 - Si la memoria tiene un bloque válido, lo deposita en el bus y este pasa a inválido.
 - Si otra caché tiene el bloque como modificado, deposita el bloque en el bus y pasa a inválido.
 - Si otra caché posee el bloque en estado compartido, pasa a estado inválido.

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

- Acierto de **escritura en bloque compartido**:
 - o El proc. escribe (PtEsc) y el bloque está como compartido.
 - o Se debe obtener acceso exclusivo al bloque (PtLecEx); el estado del bloque después de la escritura pasa a modificado (no hay distinción entre acierto y fallo de escritura).
 - o La petición PtLecEx provoca:
 - Memoria deposita bloque en bus, ya que lo tiene válido (aunque se ignora), y pasa a inválido.
 - Solución para evitar bloque en bus → Petición de acceso exclusivo sin lectura (PtEx).
 - Si una caché tiene el bloque en estado compartido, pasa a inválido.
- Acierto de **escritura en bloque modificado**:
 - o El proc. escribe (PrEsc), y el bloque está en la caché en estado modificado.
 - o El bloque se mantiene como modificado, y no se genera ninguna petición.
- Acierto de **Lectura**:
 - o El bloque se encuentra en las cachés y es una copia válida.
 - o Se mantiene el estado y no hay petición.
- **Reemplazo**:
 - o Se produce un fallo, y se debe reemplazar un bloque para hacer sitio al nuevo.
 - o Si el bloque reemplazado se encuentra en estado modificado, el controlador genera un paquete de post-escritura (PtPEsc), y el bloque pasa a inválido (es decir, no está en la caché).
 - o Se utiliza post-escritura como política de actualización de la memoria principal. Esta petición provoca que el estado del bloque en memoria pase de inválido a válido.

Problemas asociados a la invalidación que degradan el rendimiento del protocolo:

- Fallos de **compartición verdadera** (true sharing):
 - o Un procesador escribe en bloque compartido e invalida y luego otro procesador lee del bloque compartido.

Tengo una línea caché con cuatro palabras, que almacenan los elementos del A[0] al A[3] de un array A. El procesador 0 va a estar modificando los elementos A[0] y A[1] y el procesador 1, los elementos A[2] y A[3].

¿Qué ocurrirá? Como son dos procesadores distintos, el procesador 0 va a estar modificando el valor de A[0], e invalidará las demás. Luego, el procesador 1 va a coger la línea, va a modificar A[2], e invalidará la línea en todos los demás. Después, el procesador 0, escribirá en A[1], con lo cual, lo que estaba en estado modificado, lo cogerá e invalidará. Resumidamente: cuando comparto una línea caché entre procesadores, los protocolos de

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

snooping son un desastre, porque estarán pasándose la línea modificada de un lado a otro y generando tráfico a través del bus.

- **Falsa compartición** (False Sharing):

- Se produce cuando dos variables sin relación entre ellas se encuentran dentro del mismo bloque caché (línea).
- Todo el bloque se transmite entre procesadores, aunque se esté accediendo a diferentes variables.
- Se producen fallos caché llamados de falsa compartición: un procesador escribe en una línea compartida y la invalida, luego otro procesador lee una palabra diferente de la línea.

Dentro de la misma línea caché, tenga A[0], A[1], y otro array: B[0], B[1]. Tenemos dos arrays distintos, cada uno almacena dos palabras. ¿Qué ocurrirá?

Tenemos dos variables distintas sin relación en la misma línea caché. Como van a tener localidad distinta los accesos a A y a B, nos ocurrirá lo mismo que en los fallos de compartición verdadera.

(*)EJERCICIOS POSIBLES SOBRE ESTE TEMA: partir de una secuencia de direcciones de memoria y de dónde están almacenadas, y ver por qué estados tienen que ir pasando las líneas caché, y qué transacciones se generan a través del bus.

El rendimiento de los protocolos MSI se ve degradado cuando hay aplicaciones secuenciales (no se comparten variables entre procesos). Por ello, se define un nuevo estado: **exclusivo**.

PROTOCOLO DE INVALIDACIÓN DE CUATRO ESTADOS (MESI):

Igual que el anterior, utiliza post-escritura como política de actualización de memoria principal, y escritura con invalidación para mantener la coherencia entre cachés.

Se añade un estado:

Exclusivo (E): Única caché con copia válida. La memoria tiene también copia actualizada. La caché debe invalidar su copia si observa al espiar el bus que algún otro nodo solicita una copia exclusiva del bloque, pero... No hace falta invalidar cuando se escribe un bloque en estado E.

El resto se mantienen de forma parecida:

Modificado (M): Es el único componente en el sistema con una copia válida del bloque, el resto de cachés y la memoria tienen una copia no actualizada. La caché debe proporcionar el bloque si observa al espiar el bus que algún componente lo solicita, y debe invalidarlo si algún otro solicita una copia **exclusiva** del bloque para modificarlo.

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo

Compartido (C, Shared): El bloque es válido en esa caché, en memoria y en al menos otra caché. La caché debe invalidar la copia si algún otro nodo solicita una copia **exclusiva** del bloque

Inválido (I): El bloque no está físicamente en la caché, o si se encuentra ha sido invalidado como consecuencia de la escritura en la copia del bloque en otra caché.

Estado inicial	Evento	Acción	Estado sig.
Modificado (M)	Procesador lee (PrLec)		Modificado
	Procesador escribe (PrEsc)		Modificado
	Petición de lectura (PtLec)	RpBloque	Compartido
	Petición de acceso exclusivo (PtLecEx)	RpBloque Invalida bloq.	Inválido
	Reemplazo	PtPEsc	Inválido
Exclusivo (E)	Procesador lee (PrLec)		Exclusivo
	Procesador escribe (PrEsc)		Modificado
	Petición de lectura (PtLec)		Compartido
	Petición de acceso exclusivo (PtLecEx)	Invalida copia local	Inválido
Estado inicial	Evento	Acción	Estado sig.
Compartido (S, Shared)	Procesador lee (PrLec)		Compartido
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Compartido
	Petición de acceso exclusivo (PtLecEx)	Invalida copia local	Inválido
Inválido (I)	Procesador lee (PrLec) Hay copias en otras cachés	PtLec	Compartido
	Procesador lee (PrLec) No hay copias	PtLec	Exclusivo
	Procesador escribe (PrEsc)	PtLecEx	Modificado
	Petición de lectura (PtLec)		Inválido
	Petición de acceso exclusivo (PtLecEx)		Inválido

() NO SE PROFUNDIZARÁ EN EL PROTOCOLO MESI. LOS EJERCICIOS QUE REALIZAREMOS SERÁN SOBRE EL MSI.**

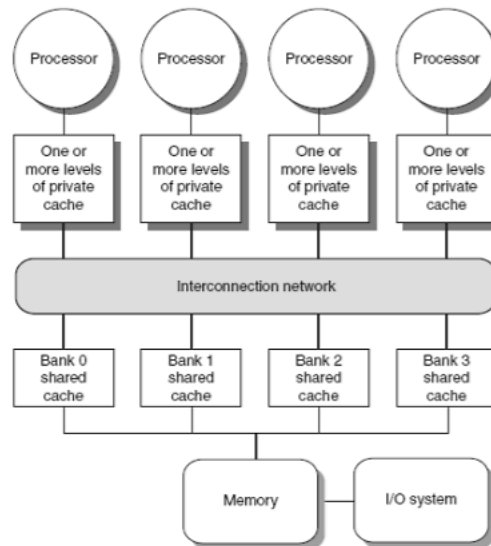
Resumen de otras extensiones de MSI:

- MESI: Añade estado exclusivo (E) que indica que el bloque reside en una única caché, pero no está modificado. Escritura de un bloque E no genera invalidaciones.
- MESIF: Añade estado forward (F): Alternativa a S que indica que un nodo debe responder a una petición. Usado en Intel Core i7.
- MOESI: Añade estado poseído (O) que indica que el bloque está desactualizado en memoria. Evita escrituras a memoria. Usado en AMD Opteron.

Problemas de snooping:

El bus de memoria compartido y el ancho de banda que requiere el snooping es un cuello de botella para escalar los SMPs.

Soluciones: usar redes de interconexión punto a punto con memoria en bancos (mayor ancho de banda) o emplear protocolo basado en directorio.



Protocolos basados en directorio

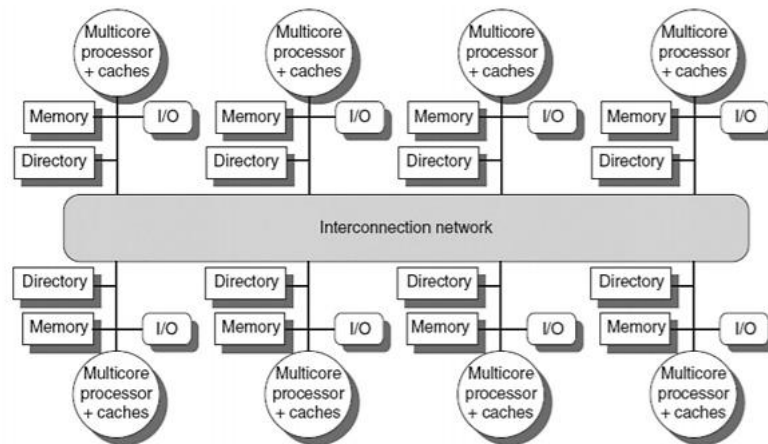
Mantiene el estado de cada bloque de la memoria caché en memoria principal. Se usa por ejemplo en multicores con caché externa compartida. Almacena un vector de bits (bits de estado del bloque) de longitud igual al número de Cores/procesadores.

¿Qué cachés tienen copia del dato? En los sistemas multicore, coge un vector de bits, encargado de indicar qué cachés privadas pueden tener la copia del bloque, y envía invalidaciones solo a cachés marcadas en el mapa de bits. Esto se emplea en Intel Core i7 en muchas ocasiones.

- Se usan si la difusión es costosa, o se necesita mayor escalabilidad.
- Para multiprocesadores de memoria físicamente distribuida (CC-NUMA), o en sistemas con memoria centralizada (UMA) con red escalable como los de la diapositiva anterior.
- Reducen tráfico red ya que:
 - o Se hace envío selectivo de órdenes
- Se requiere un directorio de memoria que contiene y actualiza información sobre el estado de compartición de cada bloque de memoria.

Sistemas CC-NUMA como el de la figura. El directorio local solo almacena información de coherencia de los bloques de cache en la memoria local.

2. Tema 4: Subsistema de Memoria Compartida en Procesadores Multinúcleo



Estados de un bloque en cachés y memoria: **el próximo día. También se harán ejercicios.**

- **Cachés:** Consideramos una implementación de cuatros estados MESI.
- **Directorio** (de memoria): Tres posibles estados:
 1. Estado local: El bloque está actualizado en la memoria principal y no hay copias en ninguna caché.
 2. Estado compartido: Hay múltiples copias válidas del bloque, y la memoria está actualizada. La entrada del directorio informará de donde está las copias válidas.
 3. Estado exclusivo:
 - o Hay una copia del bloque en una caché en estado modificado o exclusivo; la memoria puede no estar actualizada.
 - o El directorio informa de cuál es la caché con copia del bloque.
 - o Algunos estados de transición (pendientes de algún paquete para estabilizar su estado) ...



Escola
Técnica
Superior
de Enxeñaría



Grado en Ingeniería Informática

2º curso – 2º cuatrimestre

Asignatura: Arquitectura de Computadores

20-04-2021



Clase: Ejercicios Prácticos1

Temas trabajados: Tema 2, desenrollamiento

Índice

1. Ejercicio 1	3
Enunciado	3
Detenciones.....	5
Planificación del bucle.....	5
Bucle desenrollado con factor 4.....	6
2. Ejercicio 2	8
Enunciado.....	8
Posibles dependencias de datos	9
Apartado 1:.....	9
Apartado 2.....	10
Apartado 3.....	11
Apartado 4.....	12
Apartado 5.....	12

1. Ejercicio 1

Enunciado

- Considere un procesador tipo MIPS con arquitectura segmentada (pipeline) que tiene dos bancos de registros separados uno para números enteros y otro para números en coma flotante. El banco de registros enteros dispone de 32 registros.
- El banco de registros de coma flotante dispone de 16 registros de doble precisión (\$f0, \$f2, ..., \$f31).
- Se dispone de suficiente ancho de banda de captación y decodificación como para que no se generen detenciones por esta causa y que se puede iniciar la ejecución de una instrucción en cada ciclo, excepto en los casos de detenciones debidas a dependencias de datos.
- La instrucción bnez usa bifurcación retrasada con una ranura de retraso (delay slot)
- En esta máquina se desea ejecutar el siguiente código:

```
Bucle: ldc1 $f0, 0($t0)
        ldc1 $f2, 0($t1)
        add.d $f4, $f0, $f2
        mul.d $f4, $f4, $f6
        sdc1 $f4, ($t2)
        addi $t0, $t0, 8
        addi $t1, $t1, 8
        subi $t3, $t3, 1
        bnez $t3, bucle
        addi $t2, $t2, 8
```

***anexo 1**

- Inicialmente los valores de los registros son:

```
$t0: 0x00100000
$t1: 0x00140000
$t2: 0x00180000
$t3: 0x00000100
```

***anexo 2**

****EXTRA, Dora dijo:**

- Se supone que cada instrucción se ejecuta en cada ciclo de reloj excepto al haber detenciones por dependencias entre instrucciones (tercer párrafo).
- bnez: (palabras textuales de Dori) “ranura de retraso, forma de tratar los saltos poniendo alguna instrucción justo después de la instrucción de salto para que se ejecute una instrucción que haya que ejecutar de todos modos, podría ser mover una que está encima del salto a después del salto de manera que dejemos que se ejecute siempre y poder aprovechar ese hueco para cosas que habría que ejecutar se salte o no se salte y que no dependan del salto. No siempre hay instrucciones de este tipo: que se puedan ejecutar, o pueda mover, a después del salto, que no dependan de él y se ejecute siempre. Por lo tanto, si el salto se produce no afecte a la ejecución de esa instrucción porque se tiene que ejecutar de todos modos.”

****FIN EXTRA**

En el bucle de **anexo 1**, se indican los valores iniciales de los registros pero realmente en este enunciado no se utilizan para nada. → OJO → INFORMACIÓN IRRELEVANTE.

Se pide:

- Enumere las dependencias de datos RAW que hay en el código anterior.
- Indique todas las detenciones que se producen al ejecutar una iteración del código anterior, e indique el número total de ciclos por iteración.
- Intente planificar el bucle para reducir el número de detenciones.
- Desenrolle el bucle de manera que en cada iteración se procesen cuatro posiciones de los arrays y determine el speedup conseguido. Utilice nombres de registros reales (\$f0, \$f2, . . . , \$f30).
- La siguiente tabla muestra las latencias adicionales de algunas categorías de instrucciones, en caso de que haya dependencia de datos. Si no hay dependencia de datos la latencia no es aplicable.

Cuadro 1: Latencias adicionales por instrucción

Instrucción	Latencia adicional	Operación
ldc1	+2	Carga un valor de 64 bits en un registro de coma flotante.
sdc1	+2	Almacena un valor de 64 bits en memoria principal.
add.d	+4	Suma registros de coma flotante de doble precisión.
mul.d	+6	Multiplica registros de coma flotante de doble precisión.
addi	+0	Suma un valor a un registro entero.
subi	+0	Resta un valor a un registro entero.
bnez	+1	Salta si el valor de un registro no es cero.

****EXTRA, recordar en RAW:**

- En el MIPS solamente hay dependencias en el MIPS de cinco etapas normal que nosotros usamos, el MIPS R2000 y R3000.
- Las únicas dependencias de datos que producen detenciones son estas: de una instrucción que quiera leer un registro después de que otra lo escribió.
- RAW: (read after write).
- Da lugar a paradas, además de dependencias de este tipo; también hay de tipo WAR, WAW (en el tema 3 se ve que se solucionan mediante renombrado de registros). Estas no se producen nunca en el pipeline del MIPS de cinco etapas → solo la RAW.

Tabla: lo que va a tardar en ejecutarse cada instrucción.

- Latencia adicional: que depende de la carga tenemos que aumentar x ciclos de latencia

****FIN EXTRA**

Detenciones

```

Bucle:  ldc1 $f0, 0($t0)           #I1
        ldc1 $f2, 0($t1)           #I2
        <stall> x 2
        add.d $f4, $f0, $f2        #I3
        <stall> x 4
        mul.d $f4, $f4, $f6        #I4
        sdc1 $f4, ($t2)            #I5
        <stall> x 6
        addi $t0, $t0, 8           #I6
        addi $t1, $t1, 8           #I7
        subi $t3, $t3, 1           #I8
        bnez $t3, bucle            #I9
        addi $t2, $t2, 8           #I10

```

***anexo 3**

Al usar salto retardado con 1 ranura de retraso, no se produce parada al ejecutar bnez.

Por ejemplo, se podría mover la I7 para debajo de la I10 sin problema porque entre o no en el salto se va a ejecutar.

En este código hay DOS paradas porque necesito realizar una carga y NO sé el resultado de la carga hasta la etapa de memoria pero la instrucción add la necesita en la etapa de decodificación, cuando lee los registros.

Si hay dudas en dónde se realizan las paradas, es útil recurrir a una descripción gráfica.

Las paradas son equivalentes a las de la tabla, en el caso de suma son 4, en la multiplicación son 6, etc.

Planificación del bucle

```

Bucle:  ldc1 $f0, ($t0)           #I2
        ldc1 $f2, ($t1)           #I1
        addi $t0, $t0, 8           #I6
        addi $t1, $t1, 8           #I7
        add.d $f4, $f0, $f2        #I3
        subi $t3, $t3, 1           #I8
        <stall> x 3
        mul.d $f4, $f4, $f6        #I4
        <stall> x 6
        sdc1 $f4, ($t2)            #I5
        bnez $t3, bucle            #I9
        addi $t2, $t2, 8           #I10

```

***anexo 4**

Dependencias RAW:

```

$f0: I1 -> I3
$f2: I2 -> I3
$f4: I3 -> I4

```

```
$f4: I4 -> I5
```

```
$t3: I8 -> I9
```

****EXTRA:** estas son dependencias tipo RAW, ya que en este MIPS se ejecuta instrucción por ciclo de reloj

****FIN EXTRA**

Como se explica anteriormente, la I6 e I7 son independientes, por lo que se pueden ordenar en cualquier parte del código sabiendo que se van a ejecutar.

Para ahorrar las paradas de carga en I1 e I2 se pueden subir la I6 e I7 y colocarlas justo debajo de ellas.

De igual forma, para ahorrar una parada entre la suma y la multiplicación se puede subir la I8 ya que se aprovecha para ejecutar una instrucción

¿Cuántos ciclos se requieren para ejecutar cada uno de los códigos?

-22 en **anexo 3** y 19 ciclos en **anexo 4**.

Bucle desenrollado con factor 4

****EXTRA**

- Desenrollar un bucle NO consiste en copiar el bucle un montón de veces, mas en tratar de repetir (algunas instrucciones).
- Se siguen manteniendo las paradas y la estructura del código.
-

****FIN EXTRA**

```
Bucle:      ldc1 $f0, ($t0)          #notar como va sumando
            ldc1 $f2, ($t1)          #aquí en 0
            ldc1 $f8, 8($t0)         #aquí en 8
            ldc1 $f10, 8($t1)
            ldc1 $f14, 16($t0)       #aquí 16
            ldc1 $f16, 16($t1)
            ldc1 $f20, 24($t0)       #aquí 24 y así por el factor 4
            ldc1 $f22, 24($t1)
            add.d $f4, $f0, $f2
            add.d $f12, $f8, $f10
            add.d $f18, $f14, $f16
```

```

add.d $f24, $f20, $f22
<stall>
mul.d $f4, $f4, $f6
mul.d $f12, $f12, $f6
mul.d $f18, $f18, $f6
mul.d $f24, $f24, $f6
addi $t0, 32                #este addi se cambió del
addi $t1, 32                #otro código *
subi $t3, 4
sdc1 $f4, ($t2)
sdc1 $f12, 8( $t2)
sdc1 $f18, 16( $t2)
sdc1 $f24, 24( $t2)
bnez $t3, bucle
addi $t2, 32                #ranura de retraso (slot)
                                #para el salto

```

* **anexo 5**

El **anexo 5** con respecto al **anexo 4**:

- Se cambió un addi que se colocó luego de las multiplicaciones para reducir el número de paradas entre multiplicar y almacenar.
- Con las instrucciones de almacenamiento se consigue bastante eficiencia ya que no hay detenciones por las instrucciones que hay en medio.
- NO hay instrucciones adicionales para controlar el lazo

El desenrollamiento de bucles puede ser de varias formas, por ejemplo en **anexo 5** se observan que algunas instrucciones se separaron en 8 y otras en 4.

En la condición del lazo NO se puede hacer nada.

¿Cuántos ciclos se requieren para las 4 iteraciones?

- 4 iteraciones juntas → 27 ciclos en total

¿Cuántos ciclos se requieren por iteración?

- $27/4 = 6,75$ ciclos por iteración

2. Ejercicio 2

Enunciado

Sea el siguiente fragmento de código:

```
Bucle:    lw $f0, 0( $r1 )
          lw $f2, 0( $r2 )
          mul.f $f4, $f0, $f2
          add.d $f6, $f6, $f4
          addi $r1, $r1, 4
          addi $r2, $r2, 4
          sub $r3, $r3, 1
          bnez $r3, bucle
```

***anexo 6**

Se pide:

- 1- Haga una lista con todas las posibles dependencias de datos RAW y WAR. Para cada dependencia debe indicar, registro, instrucción de origen, instrucción de destino y tipo de dependencia.
- 2- Elabore un diagrama de tiempos para una arquitectura MIPS con un pipeline en 5 etapas, con las siguientes consideraciones:
 - No hay hardware de forwarding.
 - La arquitectura permite que una instrucción escriba en un registro y otra instrucción lea ese mismo registro sin problemas. *
 - Las bifurcaciones (saltos) se tratan vaciando el pipeline.
 - Las referencias a memoria requieren un ciclo de reloj.
 - La dirección efectiva de salto se calcula en la etapa de ejecución
- 3- Determine cuantos ciclos se necesitan para ejecutar N iteraciones.
- 4- Elabore un diagrama de tiempos para una arquitectura MIPS con un pipeline en 5 etapas con las siguientes consideraciones:
 - Hay hardware completo de forwarding.
 - Asuma que las bifurcaciones se tratan prediciendo todos los saltos como tomados.
- 5- Determine cuantos ciclos se necesitan para ejecutar N iteraciones del bucle en las condiciones del apartado 4 y compare con el resultado del apartado 3.

****EXTRA:** aclaraciones Dora enunciado

- En el apartado 2: que no haya adelantamiento.
 - *En el mismo ciclo: si está en la etapa de postescritura (WB → writing back). Puede una instrucción escribir un registro y otra leer del banco de registros en el mismo ciclo.
- Bifurcaciones == saltos.
- Voy cargando datos como si no se saltara si después se salta: se vacía lo que hay en el pipeline y se pierden una serie de ciclos. Si no se salta, pues ya he ido ejecutando cosas que necesitaba.
- No hay predicción de saltos.
- La etapa de ejecución es donde se calcula el salto.

- Es el MIPS básico. → lo indica el enunciado en “La dirección efectiva de salto se calcula en la etapa de ejecución.” Por lo que no tiene todo optimizado para que se haga en la etapa ID.

****FIN EXTRA**

Posibles dependencias de datos

****EXTRA**

-- SIEMPRE COMENZAR ESTOS EJERCICIOS ENUMERANDO LAS INSTRUCCIONES--

****FINEXTRA**

```

Bucle:      lw $f0, 0( $r1 )           #I1
            lw $f2, 0( $r2 )           #I2
            mul.f $f4, $f0, $f2         #I3
            add.d $f6, $f6, $f4         #I4
            addi $r1, $r1, 4            #I5
            addi $r2, $r2, 4            #I6
            sub $r3, $r3, 1             #I7
            bnez $r3, bucle             #I8

```

Apartado 1:

```

$f0 = I1 → I3
$f2 = I2 → I3   #I2 escribe, I3 lee
$f4 = I3 → I4   #I3 escribe, I4 lee
$r3 = I7 → I8   # una escribe y otra lee

```

Todas son dependencias RAW.

****PREGUNTA A DORA:** si no hay más dependencias:

- La respuesta es sí, pero como tienen muchos ciclos de por medio y es un pipeline de 5 etapas no terminan influyendo en las detenciones. Como entre I2 → I6, I1 → I5

Por lo que el resultado sería OFICIAL:

```

$f0 = I1 → I3      RAW
$f2 = I2 → I3      RAW
$f4 = I3 → I4      RAW
$r3 = I7 → I8      RAW
$r1 = I1 → I5      WAR
$r2 = I2 → I6      WAR

```

****Las dependencias tipo WAR o WAW no afecta al pipeline de 5 etapas del MIPS.**

Apartado 2

Diagrama de tiempos:

\$f0: I1 → I3 (RAW)

\$f2: I2 → I3 (RAW) -> producirá en el pipeline del MIPS detención

\$f4: I3 → I4 (RAW) -> detención

\$r3: I7 → I8 (RAW) -> detención

- No hay forwarding pero I4 puede leer en el mismo ciclo en que I3 escriba en el banco de registros *importante jeje
- I5 no puede comenzar hasta que se haya liberado IF
- Ciclo 9: se lee en el mismo ciclo en el que se escribe en el banco de registros
- No puede comentar captación de I9 hasta que se libera IF
- Para salto se conoce si se salta en ID pero el destino del salto no hasta EX

Coste de una iteración diferente de la última: número de ciclos desde que comienza a ejecutarse I1 hasta que vuelve a ejecutarse (16 ciclos en este caso).

Coste de la última iteración: número de ciclos hasta que finaliza la ejecución de I8 (18 ciclos).

Coste = $16 \cdot n + 2$ considerando n iteraciones (cada una 16 ciclos, la última 18 ciclos)

Instrucción	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
I1: lw \$f0, 0(\$r1)	IF	ID	EX	M	WB																
I2: lw \$f2, 0(\$r2)		IF	ID	EX	M	WB															
I3: mul.f \$f4, \$f0, \$f2			IF	-	-	ID	EX	M	WB												
I4: add.d \$f6, \$f6, \$f4						IF	-	-	ID	EX	M	WB									
I5: addi \$r1, \$r1, 4									IF	ID	EX	M	WB								
I6: addi \$r2, \$r2, 4										IF	ID	EX	M	WB							
I7: sub \$r3, \$r3, 1											IF	ID	EX	M	WB						
I8: bnez \$r3, bucle												IF	-	-	ID	EX	M	WB			
I9: (sig a bnez)															IF						
I1: lw \$f0, 0(\$r1)																	IF	ID	EX	M	WB

Podemos fijarnos que en cada caso que exista una dependencia con respecto a una instrucción anterior, hay una espera de que cargue en memoria el dato para poder usarlo.

Instrucción	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
I1: lw \$f0, 0(\$r1)	IF	ID	EX	M	WB																
I2: lw \$f2, 0(\$r2)		IF	ID	EX	M	WB															
I3: mul.f \$f4, \$f0, \$f2			IF	-	-	ID	EX	M	WB												
I4: add.d \$f6, \$f6, \$f4						IF	-	-	ID	EX	M	WB									
I5: addi \$r1, \$r1, 4									IF	ID	EX	M	WB								
I6: addi \$r2, \$r2, 4										IF	ID	EX	M	WB							
I7: sub \$r3, \$r3, 1											IF	ID	EX	M	WB						
I8: bnez \$r3, bucle												IF	-	-	ID	EX	M	WB			
I9: (sig a bnez)															IF						
I1: lw \$f0, 0(\$r1)																	IF	ID	EX	M	WB

En este cuadro se señalan en azul las esperas dependientes del cargado en memoria, que están en verde

2. Ejercicio 2

Instrucción	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
I1: lw \$f0, 0(\$r1)	IF	ID	EX	M	WB																
I2: lw \$f2, 0(\$r2)		IF	ID	EX	M	WB															
I3: mul.f \$f4, \$f0, \$f2			IF	-	-	ID	EX	M	WB												
I4: add.d \$f6, \$f6, \$f4						IF	-	-	ID	EX	M	WB									
I5: addi \$r1, \$r1, 4									IF	ID	EX	M	WB								
I6: addi \$r2, \$r2, 4										IF	ID	EX	M	WB							
I7: sub \$r3, \$r3, 1											IF	ID	EX	M							
I8: bnez \$r3, bucle												IF	-	-	ID	EX	M	W			
I9: (sig a bnez)															IF						
I1: lw \$f0, 0(\$r1)																	IF	ID	EX	M	WB

*En este cuadro se muestra la ejecución del salto. Se salta en **ID** PERO NO SE SABE A DÓNDE HASTA QUE PASE **EX**, en este caso el destino es la I1 (lw \$f0, 0(\$r1))*

El enunciado indica que las bifurcaciones se tratan vaciando el pipeline, lo que quiere decir es que en el momento en el que se sepa si hay un salto se elimina lo que hay en el pipeline si hay que saltar, y si no se siguen cargando las instrucciones por orden ((sig a bnez)).

Apartado 3

Número de ciclos que requeriría para ejecutar n iteraciones: para esto hace falta saber el número de ciclos para las iteraciones distintas de la última y el de la última.

Primero: ver la cantidad de ciclos desde que se inicia I1 hasta que se vuelve a iniciar I1

➔ 16 CICLOS: duración de ejecución del bucle

Segundo: ver la cantidad de ciclos desde que inicia I1 hasta que acaba I8 (salto)

➔ 18 CICLOS

Para saber el coste de n iteraciones: $16(n-1) + 18$

*Para que quede más bonito vamos a despejar, pipol:

$$16(n-1) + 18$$

$$16(n-1) + 16 + 2$$

$$16(n-1+1) + 2$$

$$16n + 2$$

*

**EXTRA: RECORDAR: aproximadamente diapositiva 16, Tema 2

Etapas de una instrucción:

- IF: búsqueda de instrucciones
- ID: decodificación de instrucciones y lectura del banco de registros
- EX: ejecución o cálculo de direcciones
- MEM: acceso a memoria para acceder a un dato
- WB: escritura de resultados

**FINEXTRA

Apartado 4

Diagrama de tiempos (con forwarding (adelantamiento de datos) y predicción estática de saltos tomados):

- Pipeline 5 etapas sin forwarding
- Los saltos se tratan prediciendo como tomados

\$f0: I1 → I3 (RAW)

\$f2: I2 → I3 (RAW) → forwarding

\$f4: I3 → I4 (RAW) → forwarding

\$r3: I7 → I8 (RAW) → forwarding

Instrucción	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
I1: lw \$f0, 0(\$r1)	ID	ID	EX	M	WB																
I2: lw \$f2, 0(\$r2)		IF	ID	EX	M	WB															
I3: mul.f \$f4, \$f0, \$f2			IF	ID	-	EX	M	WB													
I4: add.d \$f6, \$f6, \$f4				IF	-	ID	EX	M	WB												
I5: addi \$r1, \$r1, 4						IF	ID	EX	M	WB											
I6: addi \$r2, \$r2, 4							IF	ID	EX	M	WB										
I7: sub \$r3, \$r3, 1								IF	ID	EX	M	WB									
I8: bnez \$r3, bucle									IF	ID	EX	M	WB								
I1: lw \$f0, 0(\$r1)											IF	ID	EX	M	WB						

- Directamente se pasa a la ejecución del salto.

Apartado 5

El coste para las iteraciones distintas de la última sería de 10 ciclos, sin embargo para la última es de 13 ciclos. De esta forma para n iteraciones la respuesta es **10n+3**.



Grado en Ingeniería Informática

2º CURSO – 2º CUATRIMESTRE

ARQUITECTURA DE COMPUTADORES

27-05-2021

Autores:



Ejercicios 2

Temas trabajados:

Ejer-I (3 y 4) & Ejer-II (1)

Índice

1. Ejer-I: Ej 3	2
1.1. Determinar los riesgos de datos RAW que presenta el código y que tienen impacto en su ejecución.	2
1.2. Muestre un diagrama de tiempos con las fases de ejecución de cada instrucción en una iteración.	3
1.3. Proponga un desenrollamiento de lazos del bucle asumiendo que se ejecuta 1000 iteraciones. Desenrolle con un factor de cuatro iteraciones.	4
1.3.1. Instrucciones de carga	5
1.3.2. Instrucción de suma	5
1.3.3. Instrucción de almacenamiento	5
1.4. Fragmento final	5
1.5. Determine la aceleración o speedup obtenido mediante el desenrollamiento del apartado anterior.	6
2. Ejer-I: Ej 4	7
2.1. Calcular el número de ciclos necesarios para ejecutar una iteración del bucle exterior y tres del bucle interior.	8
2.2. Desenrollar tres iteraciones del bucle interior, no desenrollar el bucle exterior y volver a realizar el cálculo anterior.	9
3. Ejer-II: Ej 1	9
3.1. Calcule el CPI si mejoramos el predictor de saltos para que tenga una tasa de aciertos del 90 %.	10
3.2. Calcular el CPI si además se aplica la técnica de salto retardado suponiendo que, en el código, el compilador siempre encuentra una instrucción para aplicarla, y consigue el máximo beneficio que esta permite.	10

1. Ejer-I: Ej 3

En un procesador se pretende ejecutar el siguiente segmento de código:

```
I0:lw$r4, 0($r1)
I1:lw$r5, 0($r2)
I2:add$r4, $r4,$r5
I3:sw$r4, 0($r3)
I4:addi$r1, $r1,4
I5:addi$r2, $r2,4
I6:addi$r3, $r3,4
I7:bne$r3, $r0,i0
```

Asumimos que el procesador tiene una arquitectura segmentada de 5 etapas como la del MIPS estándar sin envío adelantado (forwarding)¹. Todas las operaciones se ejecutan en un ciclo por etapa, excepto:

- Las operaciones de carga y almacenamiento que requieren un total de dos ciclos para la etapa de memoria (un ciclo adicional)².
- Las instrucciones de salto, que requieren un ciclo adicional en la etapa de ejecución.³ Considerar que estas instrucciones no cuentan con ningún tipo de predicción de saltos.

Apartados:

- Determinar los riesgos de datos RAW que presenta el código y que tienen impacto en su ejecución.
- Muestre un diagrama de tiempos con las fases de ejecución de cada instrucción en una iteración.
- Proponga un desenrollamiento de lazos del bucle asumiendo que se ejecuta 1000 iteraciones. Desenrolle con un factor de cuatro iteraciones⁴.
- Determine la aceleración o speedup obtenido mediante el desenrollamiento del apartado anterior.

1.1. Determinar los riesgos de datos RAW que presenta el código y que tienen impacto en su ejecución.

Info:

- En el código todas las instrucciones solo pueden tener dependencias únicamente con la inmediatamente siguiente **salvo**, sw, lw y bne que pueden tener dependencias con las **dos** instrucciones inmediatamente siguientes.
- Los riesgos tipo RAW son dependencias de **lectura después de escritura**.

¹No poseer forwarding implica, en un pipeline tipo MIPS estándar de 5 etapas, que no se va a poseer el resultado correcto hasta **postescritura** (WB), es decir cuando el resultado es escrito en un registro

²Estas instrucciones no liberarán la etapa **M** para que otra pueda ejecutarse en ella en **2 ciclos** completos

³De nuevo, no liberarán la etapa **EX** en dos ciclos completos

⁴Se realizan las operaciones de 4 iteraciones en una.

I0:lw \$r4, 0(\$r1)	Dependencias (riesgos):
I1:lw \$r5, 0(\$r2)	\$r4: I0 -> I2
I2:add \$r4, \$r4,\$r5	\$r5: I1 -> I2
I3:sw \$r4, 0(\$r3)	\$r4: I2 -> I3
I4:addi \$r1, \$r1,4	\$r3: I6 -> I7
I5:addi \$r2, \$r2,4	
I6:addi \$r3, \$r3,4	
I7:bne \$r3, \$r0,i0	

1.2. Muestre un diagrama de tiempos con las fases de ejecución de cada instrucción en una iteración.

En la etapa **ID** se calcula la dirección de salto, en un pipeline normal, y la decisión de si se realiza o no el salto se calcula en la etapa **EX**. En estos pipelines, en **ID**, se sabe la dirección de salto, pero no se sabe si se salta o no hasta la etapa **EX**. Sin embargo, existe una modificación del MIPS en la que en la etapa **ID** se incluye un comparador y la decisión de salto también se determina en **ID** de este modo, la siguiente instrucción a la instrucción de salto puede comenzar después de que acabe la etapa **ID** de la instrucción de salto. En el pipeline normal, como la decisión de salto se realiza en la etapa **ID**, hay que esperar a que termine la etapa **EX**.

Así los ciclos por iteración son:

- Pipeline normal: 20 ciclos por iteración. Dado que la instrucción de salto I7, en un pipeline normal, no determina si se va a saltar o no a la dirección hasta después de la etapa EX y dado que esta etapa ocupa dos ciclos en estas instrucciones como explica el enunciado, la siguiente instrucción a I7 no puede empezar en la etapa IF hasta el fin de X2, es decir hasta que la instrucción I7 llega a la etapa M.
- Pipeline modificado: 18 ciclos por iteración. Si la etapa ID incluye un comparador, entonces la instrucción I7 sabrá si se salta o no y a donde una vez terminada la etapa ID por lo que la siguiente instrucción podrá ser ejecutada cuando I7 llegue a X1.

Instrucción	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
I0	IF	ID	EX	M1	M2	WB																
I1		IF	ID	EX	-	M1	M2	WB														
I2			IF	-	-	-	-	ID	EX	M	WB											
I3								IF	-	-	ID	EX	M1	M2	WB							
I4											IF	ID	EX	-	M	WB						
I5												IF	ID	-	EX	M	WB					
I6													IF	-	ID	EX	M	WB				
I7															IF	-	-	ID	X1	X2	M	WB

Explicación de ciclos perdidos por dependencias:

- I0-I1: son instrucciones independientes pero existe un ciclo de espera en el ciclo 5 para la instrucción I1 porque, la instrucción I0 es de tipo *lw* y ocupa dos ciclos la etapa de memoria.
- I0-I2: la instrucción I2 no puede entrar en la etapa de búsqueda de datos (ID) hasta que I0 haya llegado a WB, es decir hasta que I0 escriba la información en el registro \$r4.
- I1-I2: la instrucción I2 tampoco puede entrar en la etapa de búsqueda hasta que la instrucción I1 escriba la información en el registro \$r5.

- I2-I3: La instrucción I3 no puede comenzar en la etapa IF hasta que I2 pasa a ID además, I3 depende de I2, no puede acceder al contenido del registro \$r4 y, por lo tanto no puede pasar a la etapa ID hasta que la instrucción I2 haya llegado a WB y por lo tanto haya escrito el contenido en el registro.
- I3-I4: de nuevo la instrucción I4 no puede entrar en la etapa IF hasta que la instrucción I3 pasa a la etapa ID y, además, como la instrucción I3 es una instrucción *sw* ocupa dos ciclos en la etapa M por lo que se retrasa también el acceso en I3 a la etapa M un ciclo.
- I4-I5: como se retrasa el acceso a memoria de la instrucción I4 un ciclo por culpa de la instrucción I3, I4 suelta la etapa EX un ciclo más tarde por lo que, en la instrucción I5 se produce un retraso de un ciclo a la hora de pasar de ID a EX.
- I5-I6: del mismo modo, por lo de antes I5 suelta la etapa ID con un ciclo de retraso por lo que se produce un retraso de un ciclo en el paso de IF a ID en la instrucción I6.
- I6-I7: la instrucción I7 solo puede entrar en IF cuando I6 “suelta” la etapa IF y pasa a la etapa ID. Además, la instrucción I7 es dependiente de I6 dado que tiene que esperar a que I6 escriba el registro \$r3 por lo que no puede entrar en la etapa ID para buscar los datos hasta que I6 llega a la etapa WD.
Importante: hay que fijarse que I7 dado que es una instrucción de salto necesita dos ciclos en la etapa EX, representados en el diagrama como X1 y X2.

1.3. Proponga un desenrollamiento de lazos del bucle asumiendo que se ejecuta 1000 iteraciones. Desenrolle con un factor de cuatro iteraciones.

Original:

```
I0: lw $r4, 0($r1)
I1: lw $r5, 0($r2)
I2: add $r4, $r4,$r5
I3: sw $r4, 0($r3)
I4: addi $r1, $r1,4
I5: addi $r2, $r2,4
I6: addi $r3, $r3,4
I7: bne $r3, $r0,i0
```

Desenrollado:

```
I0: lw $r4, 0($r1)*
I1: lw $r5, 0($r2)*
I2: lw $r6, 4($r1)
I3: lw $r7, 4($r2)
I4: lw $r8, 8($r1)
I5: lw $r9, 8($r2)
I6: lw $r10, 12($r1)
I7: lw $r11, 12($r2)
I8: add $r4, $r4, $r5*
I9: add $r6, $r6, $r7
I10: add $r8, $r8, $r9
I11: add $r10, $r10, $r11
I12: sw $r4, 0($r3)
I13: sw $r6, 4($r3)
I14: sw $r8, 8($r3)
I15: sw $r10, 12($r3)
I16: addi $r3, $r3, 16*
I17: addi $r2, $r2, 16*
I18: addi $r1, $r1, 16*
I19: bne $r3, $r0, i 0*
```

Las instrucciones con asteriscos son las que ya existían en el código original. Dado que se pide un factor de 4 iteraciones es necesario que en cada iteración del bucle se ejecuten las instrucciones equivalentes a cuatro iteraciones. Si vamos cuadruplicando las instrucciones en orden queda del siguiente modo:

1.3.1. Instrucciones de carga

Vemos como las instrucciones de carga, traen de memoria el contenido que se encuentra en la dirección contenida en \$r1 y \$r2 a la que se le suma una constante de 4 en cada iteración, de esta forma al desenrollar hay que ir sumando 0, 4, 8 y 12 al contenido de \$r1 y \$r2 para simular 4 iteraciones en una:

```
I0: lw $r4, 0($r1)*  
I1: lw $r5, 0($r2)*  
I2: lw $r6, 4($r1)  
I3: lw $r7, 4($r2)  
I4: lw $r8, 8($r1)  
I5: lw $r9, 8($r2)  
I6: lw $r10, 12($r1)  
I7: lw $r11, 12($r2)
```

1.3.2. Instrucción de suma

De nuevo se cuadruplica la instrucción add, **teniendo en cuenta** que los registros a sumar tienen que coincidir con la forma en la que se realizaron las cargas.

```
I8: add $r4, $r4, $r5*  
I9: add $r6, $r6, $r7  
I10: add $r8, $r8, $r9  
I11: add $r10, $r10, $r11
```

1.3.3. Instrucción de almacenamiento

Una vez más se repite cuatro veces teniendo en cuenta los registros en los que se almacenaron las operaciones anteriores y el desplazamiento en el registro \$r3.

```
I12: sw $r4, 0($r3)*  
I13: sw $r6, 4($r3)  
I14: sw $r8, 8($r3)  
I15: sw $r10, 12($r3)
```

1.4. Fragmento final

No es necesario repetir ninguna de las instrucciones de este fragmento dado que son operaciones de suma y podemos sumar directamente el valor que se resultante después de 4 iteraciones: $1iteracion = +4$ por lo que $4iteraciones = +16$, también se puede pensar como que la última posición leída/almacenada fue el valor de los registro $+12$ por lo que, dado que son operaciones sobre ints la siguiente dirección será $12 + 4 = +16$:

```

I16: addi$r3, $r3, 16
I17: addi$r2, $r2, 16
I18: addi$r1, $r1, 16
I19: bne$r3, $r0, i 0

```

Importante: Hay que tener en cuenta que esta es una de las posibles soluciones, sería posible realizarlo de forma que sean necesarios menos registros auxiliares o directamente copiando y pegando el fragmento:

```

I0: lw$r4, 0($r1)
I1: lw$r5, 0($r2)
I2: add$r4, $r4,$r5
I3: sw$r4, 0($r3)
I4: addi$r1, $r1,4
I5: addi$r2, $r2,4
I6: addi$r3, $r3,4

```

cuatro veces. Esta última opción no es recomendable porque se realizan operaciones innecesarias y a la Dori no le gustan.

1.5. Determine la aceleración o speedup obtenido mediante el desenrollameintodel apartado anterior.

Si hacemos de nuevo el diagrama de tiempos observamos lo siguiente:

Instrucción	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
I0	IF	ID	EX	M1	M2	WB																
I1		IF	ID	EX	-	M1	M2	WB														
I2			IF	ID	-	EX	-	M1	M2	WB												
I3				IF	-	ID	-	EX	-	M1	M2	WB										
I4						IF	-	ID	-	EX	-	M1	M2	WB								
I5								IF	-	ID	-	EX	-	M1	M2	WB						
I6										IF	-	ID	-	EX	-	M1	M2	WB				
I7												IF	-	ID	-	EX	-	M1	M2	WB		
I8														IF	-	ID	-	EX	-	M	WB	
I9																IF	-	ID	-	EX	M	WB
I10																		IF	-	ID	M	WB
I11																				IF	ID	EX
I12																					F	D
I13																						F

Instrucción	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37
I10	WB														
I11	M	WB													
I12	EX	M1	M2	WB											
I13	ID	EX	-	M1	M2	WB									
I14	IF	ID	-	EX	-	M1	M2	WB							
I15		IF	-	ID	-	EX	-	M1	M2	WB					
I16				IF	-	ID	-	EX	-	M	WB				
I17						IF	-	ID	-	EX	M	WB			
I18								IF	-	ID	EX	M	WB		
I19										IF	ID	EX	EX	M	WB

De esta forma y de nuevo dependiendo de qué pipeline usemos obtenemos:

- Pipeline normal: 35 ciclos.
- Pipeline optimizado: 33 ciclos.

Como son 4 iteraciones de la versión no desenrollada en cada iteración, tenemos que el número de ciclos equivalente a una iteración del bucle original es:

- Pipeline normal: $\frac{35}{4} = 8,75 \text{ ciclos}$
- Pipeline optimizado: $\frac{33}{4} = 8,25 \text{ ciclos}$

Así obtenemos un speedUp:

- Pipeline normal: $\frac{18}{8,25} \approx 2,18 \text{ ciclos}$
- Pipeline optimizado: $\frac{20}{8,75} \approx 2,2857 \text{ ciclos}$

2. Ejer-I: Ej 4

Consideremos el siguiente código donde cada instrucción tiene como coste asociado un ciclo además de los indicados en la tabla. Supongamos además que la máquina en la que se ejecuta el código es capaz de emitir una instrucción por ciclo, salvo las esperas debidas a detenciones y que es un procesador con un único camino de datos (un único pipeline). Debido a riesgos estructurales, las detenciones de una instrucción (stalls) se realizan siempre, independientemente de que haya o no dependencia de datos con las instrucciones siguientes. Inicialmente se tiene que R1=0, R2=24, R3=16.

```

Loop1: LD F2 , 0(R1)
        ADDD F2, F0, F2
Loop2: LD F4 , 0 ( R3)
        MULTD F4 , F0 , F4
        DIVD F10 , F4 , F0
        ADDD F12 , F10 , F4
        ADDI R1 , R1 , #8
        SUB R18 , R2 , R1
        BNZ R18 , Loop2
        SD F2 , 0 ( R3)
        ADDI R3 , R3 , #8
        SUB R20 , R2 , R3
        BNZ R20 , Loop1
    
```

Instrucción	Latencia
LD	3
SD	1
ADD	2
MULTD	4
DIVD	10
ADDI	0
SUB	0
BNZ	1

Apartados:

- Calcular el número de ciclos necesarios para ejecutar una iteración del bucle exterior y treinta del bucle interior.
- Desenrollar tres iteraciones del bucle interior, no desenrollar el bucle exterior y volver a realizar el cálculo anterior.
- Comparar los resultados de ambos apartados y justificar la respuesta.

2.1. Calcular el número de ciclos necesarios para ejecutar una iteración del bucle exterior y tres del bucle interior.

Loop1:	LD F2 , 0(R1)	1+3
	ADDD F2, F0, F2	1+2
Loop2:	LD F4 , 0 (R3)	1+3
	MULTD F4 , F0 , F4	1+4
	DIVD F10 , F4 , F0	1+10
	ADDD F12 , F10 , F4	1+2
	ADDI R1 , R1 , #8	1
	SUB R18 , R2 , R1	1
	BNZ R18 , Loop2	1+1
Iteración interior		27
	SD F2 , 0 (R3)	1+1
	ADDI R3 , R3 , #8	1
	SUB R20 , R2 , R3	1
	BNZ R20 , Loop1	1+1
Iteración exterior		13

Cada una de las instrucciones tiene un coste de **1** más la latencia indicada en la tabla.
Costes en ciclos de los bucles:

- Coste del bucle interior: comienza en “LD F4, 0(R3)” y finaliza en “BNZ R18, Loop2” por lo que se ejecuta en **27** ciclos.
- Coste del bucle exterior: eliminando el coste del bucle interior resulta en **13** ciclos.

El coste de una iteración del bucle exterior y 3 del bucle interior es, por lo tanto: $1 \cdot 13 + 27 \cdot 3 = 94$ *ciclos*

2.2. Desenrollar tres iteraciones del bucle interior, no desenrollar el bucle exterior y volver a realizar el cálculo anterior.

2. Análisis desenrollamiento:

```

Loop1: LD F2 , 0(R1)      1+3
      ADDD F2, F0, F2      1+2
Loop2: LD F4 , 0 ( R3)     1+3
      LD F6 , 0 ( R3)      1+3
      LD F8 , 0 ( R3)      1+3
      MULTD F4 , F0 , F4    1+4
      MULTD F6 , F0 , F6    1+4
      MULTD F8 , F0 , F8    1+4
      DIVD F10 , F4 , F0    1+10
      DIVD F12 , F6 , F0    1+10
      DIVD F14 , F8 , F0    1+10
      ADDD F16 , F10 , F4    1+2
      ADDD F18 , F12 , F6    1+2
      ADDD F20 , F14 , F8    1+2
      ADDI R1 , R1 , 24      1
      SUB R18 , R2 , R1      1
      BNZ R18 , Loop2        1+1
3 iteraciones interiores 73
      SD F2 , 0 ( R3)        1+1
      ADDI R3 , R3 , #8      1
      SUB R20 , R2 , R3      1
      BNZ R20 , Loop1        1+1

```

```

Loop1: LD F2 , 0(R1)
      ADDD F2, F0, F2
Loop2: LD F4 , 0 ( R3)
      MULTD F4 , F0 , F4
      DIVD F10 , F4 , F0
      ADDD F12 , F10 , F4
      ADDI R1 , R1 , #8
      SUB R18 , R2 , R1
      BNZ R18 , Loop2
      SD F2 , 0 ( R3)
      ADDI R3 , R3 , #8
      SUB R20 , R2 , R3
      BNZ R20 , Loop1

```

Costes en ciclos de los bucles:

- Coste del bucle interior: comienza en “LD F4, 0(R3)” y finaliza en “BNZ R18, Loop2” por lo que se ejecuta en **73** ciclos.
- Coste del bucle exterior: eliminando el coste del bucle interior resulta en **13** ciclos.

El coste de una iteración del bucle exterior y 1 iteración del bucle interior (que equivale a 3 iteraciones en el bucle original) es, por lo tanto: $1 \cdot 13 + 73 \cdot 1 = 83$ *ciclos*

El ahorro entre el desenrollamiento del bucle no supone una mejora considerable con respecto a la versión no desenrollada.

3. Ejer-II: Ej 1

Un programa corre en un procesador segmentado presentando un CPI de 1.6. El sistema admite hasta una instrucción por ciclo. El programa ejecuta un 20% de instrucciones de salto y la tasa de acierto en la predicción⁵ es del 75%. El coste de un salto mal predicho es de 6 ciclos, y el coste de los saltos bien predichos es cero.

Apartados:

- Calcular el CPI si mejoramos el predictor de saltos para que tenga una tasa de aciertos del 90%.
- Calcular el CPI si además se aplica la técnica de salto retardado suponiendo que, en el código, el compilador siempre encuentra una instrucción para aplicarla, y consigue el máximo beneficio que esta permite.

⁵Decidir si se va a tomar o no el salto y aplicar esa decisión para cargar las siguientes instrucciones

3.1. Calcula el CPI si mejoramos el predictor de saltos para que tenga una tasa de aciertos del 90 %.

Datos:

- Ciclos por instrucción mínimo para todas las instrucciones: 1
- CPI = 1,6
- Instrucciones de salto: 20 %
 - Fallos de predicción: 25 %
 - Ciclos perdidos: 6
 - Aciertos de predicción: 75 %
 - Ciclos perdidos: 0

Como la fórmula para calcular el CPI es:

$$CPI = \frac{\sum_{i=1}^M CPI_i \cdot I_i}{I}$$

Donde:

- CPI_i : número de ciclos de las instrucciones tipo i.
- I_i : número total de instrucciones tipo I.
- I : número total de instrucciones.

Así, podemos despejar el producto $I_{otras} \cdot CPI_{otras} = otras$ no especificado. En caso de que no existan otras instrucciones el resultado será 0.

$$CPI = \frac{\sum_{i=1}^M CPI_i \cdot I_i}{I} \Rightarrow 1,6 = \frac{1 \cdot I + (0,2 \cdot 0,25 \cdot 6) \cdot I + otras \cdot I}{I}$$
$$otras = 1,6 - (1 \cdot I + (0,2 \cdot 0,25 \cdot 6) \cdot I) = 0,3$$

Por lo tanto si mejoramos la predicción de saltos para que se acierte un 90 % de los saltos el CPI resultante será :

$$CPI = \frac{1 \cdot I + (0,2 \cdot 0,10 \cdot 6) \cdot I + 0,3 \cdot I}{I} = 1,42$$

3.2. Calcular el CPI si además se aplica la técnica de salto retardado suponiendo que, en el código, el compilador siempre encuentra una instrucción para aplicarla, y consigue el máximo beneficio que esta permite.

El compilador consigue mover **siempre una instrucción** por debajo de la instrucción de salto, de modo que la instrucción de salto produce una pérdida de un ciclo menos. Así obtenemos:

$$CPI = \frac{1 \cdot I + (0,2 \cdot 0,10 \cdot 5) \cdot I + 0,3 \cdot I}{I} = 1,40$$